Rockchip RKStudio_Tool 算法控件说明

文件标识：RK-XX-XX-nnn

发布版本：V1.3.0

日期：2025-10-27

文件密级：□绝密 □秘密 □内部资料 ■公开

免责声明

本文档按“现状”提供，瑞芯微电子股份有限公司（“本公司”，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

商标声明

“Rockchip”、“瑞芯微”、“瑞芯”均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

版权所有 © 2025 瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址： 福建省福州市铜盘路软件园A区18号

网址： www.rock-chips.com

客户服务电话： +86-4007-700-590

客户服务传真： +86-0591-83951833

客户服务邮箱： fae@rock-chips.com

前言

概述

本文为RKStudio_Tool算法控件参考文档，描述了算法件的在工具中的位置，功能，参数设置，实际效果演示等。

产品版本

芯片名称	内核版本
RK2118/RK2116/RK2108/RK3308/RK3506/RK356X	本文与内核无关，仅作为RKStudio_Tool算法控件参考文档

读者对象

本文档（本指南）主要适用于以下工程师：

技术支持工程师

修订记录

版本号	作者	修改日期	修改说明
V1.0.0	廖华平	2024-02-22	初始版本
V1.1.0	廖华平	2024-08-15	更新算法和图示
V1.2.0	江迪	2025-01-20	更新算法和图示
V1.3.0	廖华平	2025-10-27	更新部分算法和图示
V1.4.0	廖华平	2026-02-02	更新部分算法和图示

目录

- Rockchip RKStudio_Tool 算法控件说明
  - 1 RK_Algorithms
    - 1.1 Loudness
      - 1.1.1 Loudness (Low & High)
      - 1.1.2 Loudness (Low End)
      - 1.1.3 Loudness (High End)
      - 1.1.4 Loudness (Low & High) External Ctrl
      - 1.1.5 Loudness (Low & High) Dynamic
    - 1.2 shift
      - 1.2.1 Phat Shift (FIR)
      - 1.2.2 Phat Shift (IIR)
      - 1.2.3 Frequency Shift (FIR)
      - 1.2.4 Frequency Shift (IIR)
    - 1.3 Surround Sound
    - 1.4 Pitch Shifting
      - 1.4.1 Phase Vocoder
      - 1.4.2 Phase Vocoder (Formant)
      - 1.4.3 WSOLA
      - 1.4.4 WSOLA (Formant)
    - 1.5 Phat Stereo
    - 1.6 DTS
      - 1.6.1 DTS Virtual:X
      - 1.6.2 DTS:X
    - 1.7 Dolby
    - 1.8 BF_ECNR
    - 1.9 AVAS
      - 1.9.1 AVAS
      - 1.9.2 AVAS External Source
    - 1.10 Voice Separators
      - 1.10.1 Voice Separator (MS)
      - 1.10.2 Voice Separator (LRM)
    - 1.11 CrossTalk Suppressor
  - 2 Basic DSP
    - 2.1 Arithmetic Operations
      - 2.1.1 Absolute Value
      - 2.1.2 Inverse Square Root
      - 2.1.3 Log2
      - 2.1.4 Log10
      - 2.1.5 Exp2
      - 2.1.6 Square Root
      - 2.1.7 Signal Add
      - 2.1.8 Signal Subtract
      - 2.1.9 Divide
      - 2.1.10 Multiply
      - 2.1.11 Signal Invert
      - 2.1.12 Multi Signal Invert
    - 2.2 Adjustable Gain
      - 2.2.1 Gain
      - 2.2.2 Gain (Slew)
      - 2.2.3 Gain (Slew Double)
      - 2.2.4 Gain External Mute (Slew Double)
    - 2.3 Dsp Function
      - 2.3.1 Delay

- 2.3.2 Multi-Delay
- 2.3.3 Multi Delay (Index Selectable)
- 2.3.4 Multi Delay Slew (Index Selectable)
- 2.3.5 DSP Feedback

▼ 2.4 Logic
- 2.4.1 AbsMax
- 2.4.2 Max
- 2.4.3 Min
- 2.4.4 Logic AB
- 2.4.5 Logic ABCD
- 2.4.6 Zero Comparator
- 2.4.7 Value Hold
- 2.4.8 AbsMax Value Hold
- 2.4.9 Max Value Hold
- 2.4.10 Min Value Hold

▼ 3 Dynamics Processors

▼ 3.1 Envelope
- 3.1.1 Envelope RMS
- 3.1.2 Envelope Peak

▼ 3.2 Compressors
- 3.2.1 Compressor Rms
- 3.2.2 Compressor Rms External Input
- 3.2.3 Compressor Peak
- 3.2.4 Compressor Peak External Input

▼ 3.3 Limiter
- 3.3.1 Limiter RMS
- 3.3.2 Limiter Peak
- 3.3.3 Limiter Ratio
- 3.3.4 Limiter Clip

▼ 3.4 Noise Gate
- 3.4.1 Noise Gate
- 3.4.2 Noise Gate (Ext)

- 3.5 Agc

▼ 4 Filters
- 4.1 FIR
- 4.2 DC-Blocking

▼ 4.3 Emphasis
- 4.3.1 Pre-Emphasis
- 4.3.2 De-Emphasis

- 4.4 IIR - 1st order

▼ 4.5 IIR - 2nd order
- 4.5.1 IIR - 2nd order
- 4.5.2 IIR - 2nd order (Index Selectable)
- 4.5.3 IIR - 2nd order (Double Precision)

- 4.6 IIR Nth Order(Single Precision)
- 4.7 IIR Nth Order(Double Precision)
- 4.8 IIR - Coeff

▼ 4.9 Graphical EQ
- 4.9.1 Graphical EQ
- 4.9.2 Graphic EQ (High Precision)

▼ 4.10 Graphic EQ Group
- 4.10.1 Graphic EQ Group
- 4.10.2 Graphic EQ Group (Index Selectable)

▼ 4.11 Crossover (Single Precision)
- 4.11.1 2-Way Crossover (Single Precision)
- 4.11.2 3-Way Crossover(Single Precision)
- 4.11.3 4-Way Crossover(Single Precision)

- 4.12 Crossover (Double Precision)
- 4.13 Dynamaic EQ
- 4.14 Text-In EQ
- 4.15 Resample
  - 4.15.1 Up-sampler
  - 4.15.2 Down-sampler
- 4.16 Crossover EQ
- 4.17 Allpass Delay

5 IO
- 5.1 Audio Input
- 5.2 Audio Output
- 5.3 Trigger Signal IO
- 5.4 Debug IO

6 Mixers/Splitters
- 6.1 Mixers
  - 6.1.1 Signal Merge
  - 6.1.2 Single Control Mixer
  - 6.1.3 Single Control Mixer (Slew)
  - 6.1.4 Multiple Control Mixer
  - 6.1.5 Multiple Control Mixer (Slew)
  - 6.1.6 Cross Mixer
  - 6.1.7 Cross Mixer (Slew)
  - 6.1.8 Cross Mixer Fast
  - 6.1.9 Cross Mixer Fast (Slew)
  - 6.1.10 Cross Mixer Slew (Index)
- 6.2 Splitters
  - 6.2.1 Single Control Splitter
  - 6.2.2 Single Control Splitter (Slew)
  - 6.2.3 Multiple Control Splitter
  - 6.2.4 Multiple Control Splitter (Slew)
- 6.3 Fader Balance
- 6.4 Matrix Mixer
- 6.5 Matrix Mixer Custom插件

7 Muxes/DeMuxes
- 7.1 Multiplexers Switch
  - 7.1.1 Mono Switch Mux
  - 7.1.2 Stereo Switch Mux
  - 7.1.3 Multi Switch Nx4
  - 7.1.4 Multi Switch Nx6
  - 7.1.5 Multi Switch Nx8
  - 7.1.6 Multi Switch Nx12
  - 7.1.7 Multi Switch NxM
- 7.1_ Multiplexers Switch (Slew)
  - 7.1.8 Mono Switch Mux (Slew)
  - 7.1.9 Stereo Switch Mux (Slew)
  - 7.1.10 Multi Switch Nx4 (Slew)
  - 7.1.11 Multi Switch Nx6 (Slew)
  - 7.1.12 Multi Switch Nx8 (Slew)
  - 7.1.13 Multi Switch Nx12 (Slew)
  - 7.1.14 NxM Switch Mux (Slew)
- 7.2 Multiplexers Index Selectable
  - 7.2.1 Mono Index Selectable Mux
  - 7.2.2 Stereo Index Selectable Mux
  - 7.2.3 Multi Index Selectable Nx4
  - 7.2.4 Multi Index Selectable Nx8
- 7.2_ Multiplexers Index Selectable (Slew)
  - 7.2.5 Mono Index Selectable Mux （Slew）

- 7.2.6 Stereo Index Selectable Mux Slew)
- 7.2.7 Multi Index Selectable Nx4 （Slew)
- 7.2.8 Multi Index Selectable Nx8（Slew)

▼ 7.3 DeMultiplexers Switch

- 7.3.1 Mono Switch Demux
- 7.3.2 Stereo Switch Demux
- 7.3.3 NxM Switch Demux
- 7.3.4 Mono Switch Demux（Slew)
- 7.3.5 Stereo Switch Demux（Slew)
- 7.3.6 NxM Switch Demux（Slew)

▼ 7.4 DeMultiplexers Index Selectable

- 7.4.1 Mono Index Selectable Demux
- 7.4.2 Stereo Index Selectable Demux
- 7.4.3 Mono Index Selectable Demux (Slew)
- 7.4.4 Stereo Index Selectable Demux (Slew)

▼ 8 Source

- 8.1 DC
- 8.2 Index
- 8.3 Beeq
- 8.4 Sawtooth
- 8.5 Sine
- 8.6 Square
- 8.7 Triangle
- 8.8 Chime
- 8.9 Switch(0,1)
- 8.10 Voltage Controlled
- 8.11 White Noise
- 8.12 Pink Noise
- 8.13 Wav Player
- 8.14 LFO
- 8.15 Ctrl Rate Noise

▼ 9 Volume Controls

- 9.1 Generic Gain Envelope

▼ 9.2 Mute

- 9.2.1 Single Mute
- 9.2.2 Single Mute(Slew)
- 9.2.3 Multiple Mute
- 9.2.4 Multiple Mute(Slew)

▼ 9.3 Volume

- 9.3.1 Single Volume
- 9.3.2 Single Volume(Slew)
- 9.3.3 Multiple Volume
- 9.3.4 Multiple Volume(Slew)
- 9.3.5 Single Volume(Slew Linear)
- 9.3.6 Multiple Volume(Slew Linear)
- 9.3.7 Single Volume(Slew Ext)

▼ 10 Detectors/Lookup Tables

- 10.1 Multiple Band Level Detector
- 10.2 Single Band Level Detector
- 10.3 1D Index Lookup Table
- 10.4 2D Index Lookup Table

▼ 11 Audio Effects

▼ 11.1 Wah-Wah

- 11.1.1 Auto Wah

- 11.2 Flanger

▼ 11.3 Echo

- 11.3.1 Echo

- 11.3.2 Ping-Pong Echo (Mono)
- 11.4 Virtual Bass
- 11.5 Reverbs
  - 11.5.1 Simple Reverb
  - 11.5.2 Dattorr Reverb
  - 11.5.3 Advance Reverb
  - 11.5.4 Early Reflection
  - 11.5.5 Room Reverb
- 11.6 Chorus
- 11.7 Transient Shaper
- 11.8 Vibrato
- 11.9 Robot Voice

▼ 12 Cores
- 12.1 Core0 Input
- 12.2 Core0 Output

▼ 13 Custom Algorithms
- 13.1 CustomHowlingSupporess
- 13.2 CustomVocalSeparation
- 13.3 CustomAlgo_V1
- 13.4 CustomAlgo_V2

▼ 14 Counters
- 14.1 Counter
- 14.2 Counter_Ext

▼ 15 Non Linear Process
- 15.1 Hard Clip
- 15.2 Soft Clip
- 15.3 Soft Clip Bilateral
- 15.4 Non Linear

▼ 附录1 Schematic Design
- 附录1.1 T Connection
- 附录1.2 User Comment
- 附录1.3 Hierarchy Board
- 附录1.4 Hierarchy Input
- 附录1.5 Hierarchy Output
- 附录1.6 Alias

▼ 附录2 转换公式
- 附录2.1 参数转换说明
- 附录2.2 IIR1st计算公式及说明
- 附录2.3 IIR2nd计算公式及说明
- 附录2.4 IIR_Nnd计算公式及说明
- 附录2.5 Graphical EQ 滤波器参数计算公式及说明
- 附录2.6 Crossover 控件计算公式及说明
- 附录2.7 使用python库函数计算fir滤波器参数
- 附录2.8 Ratio增益转换为线性增益
- 附录2.9 SlewRate参数转换为平滑系数
- 附录2.10 与延时和控制时间相关的参数转换
- 附录2.11 与平滑时间系数相关的参数转换
- 附录2.12 与增益变化系数相关的参数转换
- 附录2.13 增益相关的参数转换

▼ 附录3 术语说明
- 附录3.1 biquads :
- 附录3.2 attack_time :
- 附录3.3 release_time :
- 附录3.4 品质因数Q :
- 附录3.5 Boost（滤波器增益） 与 Gain（输出整体增益） :
- 附录3. 6 day & wet 干声和湿声 :
- 附录3.7 响度（Loudness）和声音强度（Sound intensity）

---

1 RK_Algorithms

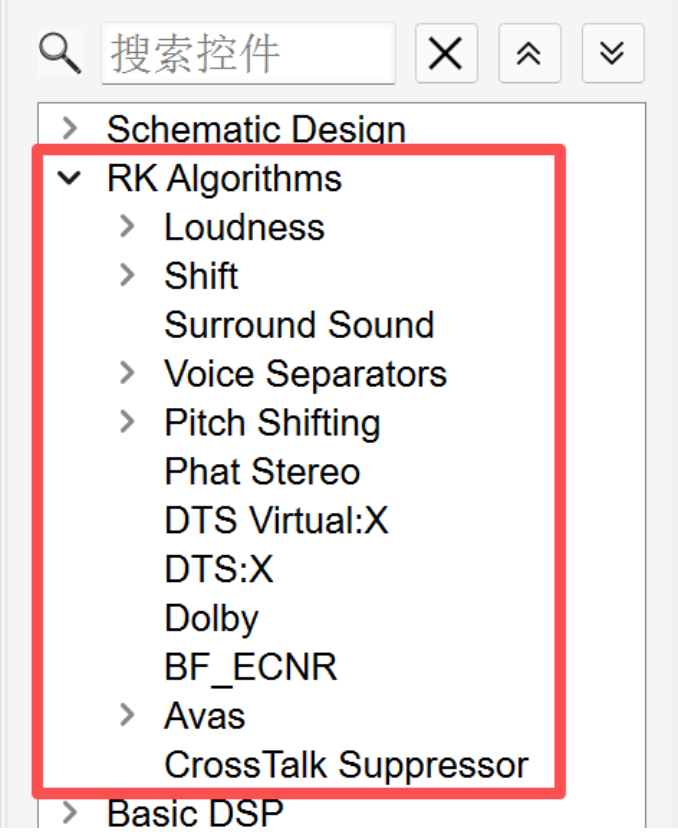


图 1-1 RK_Algorithrms位于算法列表的位置

1.1 Loudness

1.1.1 Loudness (Low & High)

功能：响度控制，利用低通和高通组合分别对低频和高频施加增益。(响度*是人主观感觉的声音大小，声音强度是物理客观的声音大小，该控件通过控制不同频率声音强度的增益，给人呈现不同的响度效果)

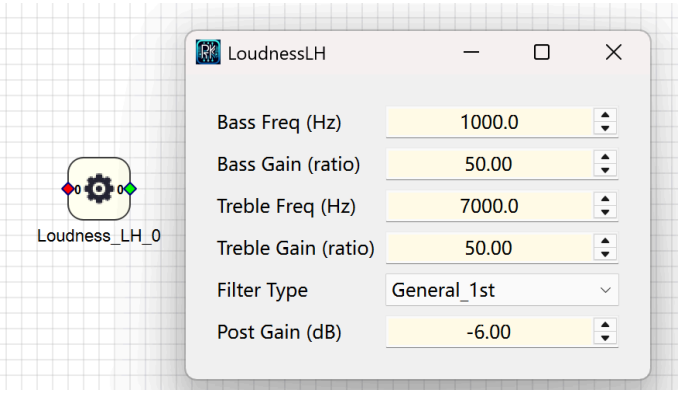


图 1.1.1-1 Loudness (Low & High) 控件和设置界面

参数数量：6

参数名	参数位	类型	功能	单位	建议取值	默认取值
BassFreq (HZ)	0	float	低通截至频率	Hz	<1000	1000
BassGain (ratio)*	1	float	低频施加百分比，越大低频增益越大	%	0 ~ 100	50

参数名	参数位	类型	功能	单位	建议取值	默认取值
TrebleFreq (HZ)	2	float	高通截至频率	Hz	> 7000	7000
TrebleGain (ratio)*	3	float	高频施加百分比，越大高频增益越大	%	0 ~ 100	50
FilterType	4	float	滤波器类型		0/1	0
PostGain (dB)	5	float	总增益下降数	dB	0 ~ -60	-6

注：参数FilterType的值0对应General 1st 普通一阶滤波器；值1对应Butterworth 巴特沃斯滤波器

实际效果展示：
输入白噪声源获取输出，列出典型工况：

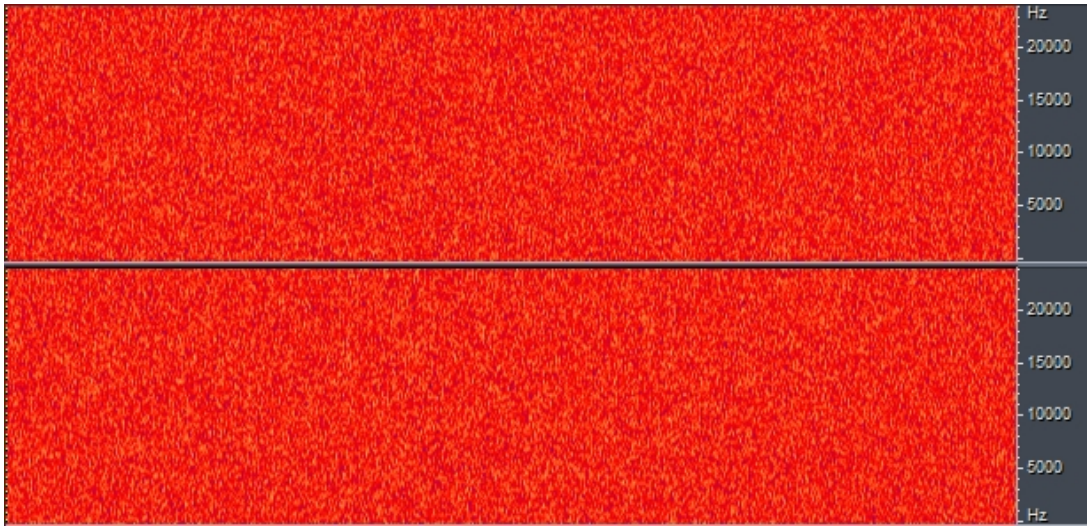


图 1.1.1-2 输入白噪频谱图

通道1：参数按顺序配置为：1000, 50,7000, 50, 0, -6,即LP截止频率1000Hz，LP使能50%，HP截至频率7000Hz，HP使能50%，采用一阶普通滤波器，前置增益-6dB。

通道2：参数按顺序配置为：1000, 100,7000, 100, 0, -6, 即LP截止频率1000Hz，LP使能100%，HP截至频率7000Hz，HP使能100%，采用一阶普通滤波器，前置增益-6dB。

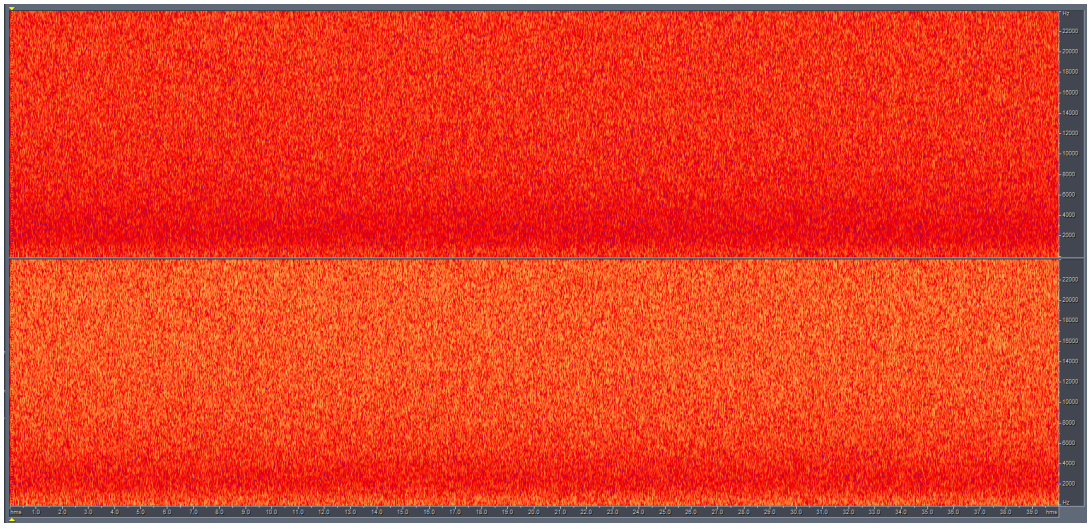


图 1.1.1-3 RKStudio输出频谱（图片横线上方为通道1输出，下方为通道2输出）

1.1.2 Loudness (Low End)

功能：响度控制，利用低通滤波器对低频信号施加增益。

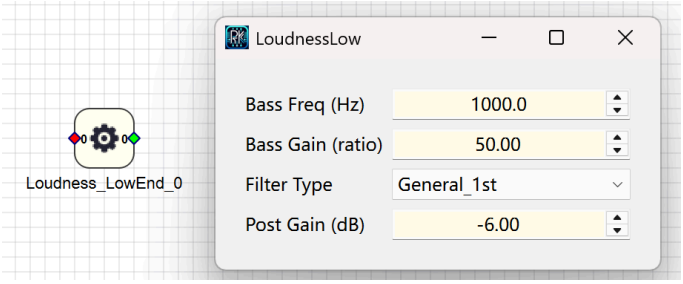


图 1.1.2-1 Loudness (Low End) 控件和设置界面

参数数量：4

参数名	参数位	类型	功能	单位	建议取值	默认取值
BassFreq （HZ）	0	float	低通截至频率	HZ	<1000	1000
BassGain (ratio)	1	float	低频施加百分比，越大低频增益越大	%	0~100	50
FilterType	2	float	滤波器类型		0/1	0
PostGain （dB）	3	float	增益	dB	0 ~ -60	-6

注：参数FilterType的值0对应General 1st 普通一阶滤波器；值1对应Butterworth 巴特沃斯滤波器

1.1.3 Loudness (High End)

功能：响度控制，利用高通滤波器对高频信号施加增益。

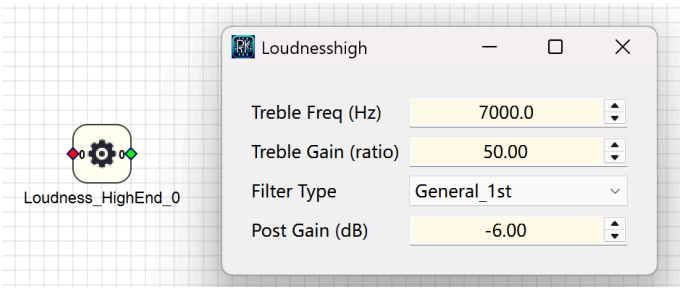


图 1.1.3-1 Loudness (High End) 控件和设置界面

参数数量：4

参数名	参数位	类型	功能	单位	建议取值	默认取值
TrebleFreq （HZ）	0	float	高通截至频率	HZ	>7000	7000
TrebleGain (ratio)*	1	float	高频施加百分比，越大高频增益越大	%	0~100	50
FilterType	2	float	滤波器类型		0/1	0
PostGain (dB)	3	float	增益	dB	0 ~ -60	-6

注：参数FilterType的值0对应General 1st 普通一阶滤波器；值1对应Butterworth 巴特沃斯滤波器

1.1.4 Loudness (Low & High) External Ctrl

功能：响度控制，利用低通和高通组合分别对低频和高频施加增益，EXT模块外部传入Post Gain用于整体增益下降，范围在0-1之间，0.5为-3dB。

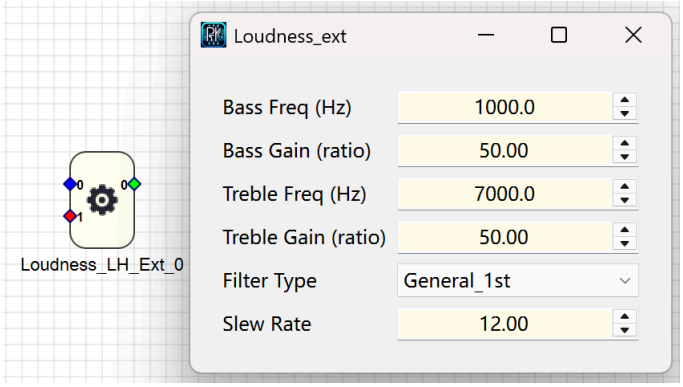


图 1.1.4-1 Loudness (Low & High) External Ctrl 控件和设置界面

参数数量：6

参数名	参数位	类型	功能	单位	建议取值	默认取值
BassFreq (HZ)	0	float	低通截至频率	Hz	<1000	1000
BassGain (ratio)*	1	float	低频施加百分比，越大低频增益越大	%	0 ~ 100	50
TrebleFreq (HZ)	2	float	高通截至频率	Hz	> 7000	7000
TrebleGain (ratio)*	3	float	高频施加百分比，越大高频增益越大	%	0 ~ 100	50
FilterType	4	float	滤波器类型		0/1	0
SlewRate*	5	float	平滑速度，slew越大变化越慢		1-23	12

注：Filter Type = 0 对应配置框的General 1st，1为Butterworth

该模块和上述模块类似，区别在于post_gain由EXT输入，因此不做额外效果说明。

1.1.5 Loudness (Low & High) Dynamic

功能：动态响度控制，在音量较小时，抬升低频和高频；音频较大时，降低中频，使得总体的响度维持在Keep Level附近。

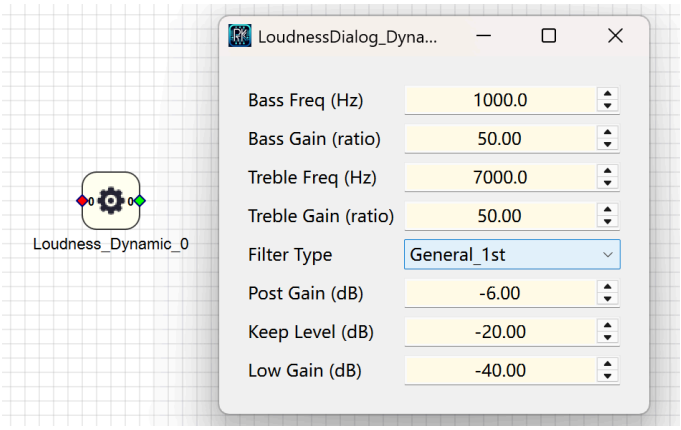


图 1.1.5-1 Loudness (Low & High) Dynamic 控件和设置界面

参数数量：8

参数名	参数位	类型	功能	单位	建议取值	默认取值
BassFreq (HZ)	0	float	低通截至频率	Hz	<1000	1000
BassGain (ratio)*	1	float	低频施加百分比，越大低频增益越大	%	0 ~ 100	50
TrebleFreq (HZ)	2	float	高通截至频率	Hz	> 7000	7000
TrebleGain (ratio)*	3	float	高频施加百分比，越大高频增益越大	%	0 ~ 100	50

参数名	参数位	类型	功能	单位	建议取值	默认取值
FilterType	4	float	滤波器类型		0/1	0
PostGain (dB)	5	float	高电平计算上限，电平高于此阈值将保持当前计算能量	dB	0 ~ -60	-6
KeepLevel (dB)	6	float	保持电平（启动阈值）：高于此电平响度衰减，低于此电平响度增强	dB	-60 ~ 0	-20
Low Gain (dB)	7	float	低电平计算下限，电平低于此阈值将保持当前计算能量	dB	-100 ~ -20	-40

注： 参数FilterType的值0对应General 1st普通一阶滤波器；值1为Butterworth巴特沃斯滤波器

实际效果展示(正弦扫频信号在时域上的波形对比)

工况一：低电平扫频输入测试

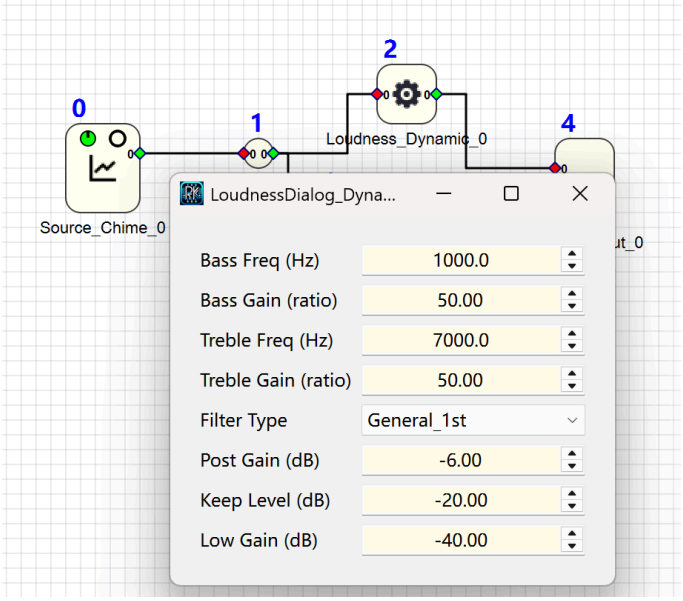


图 1.1.5-2 工况1控件参数

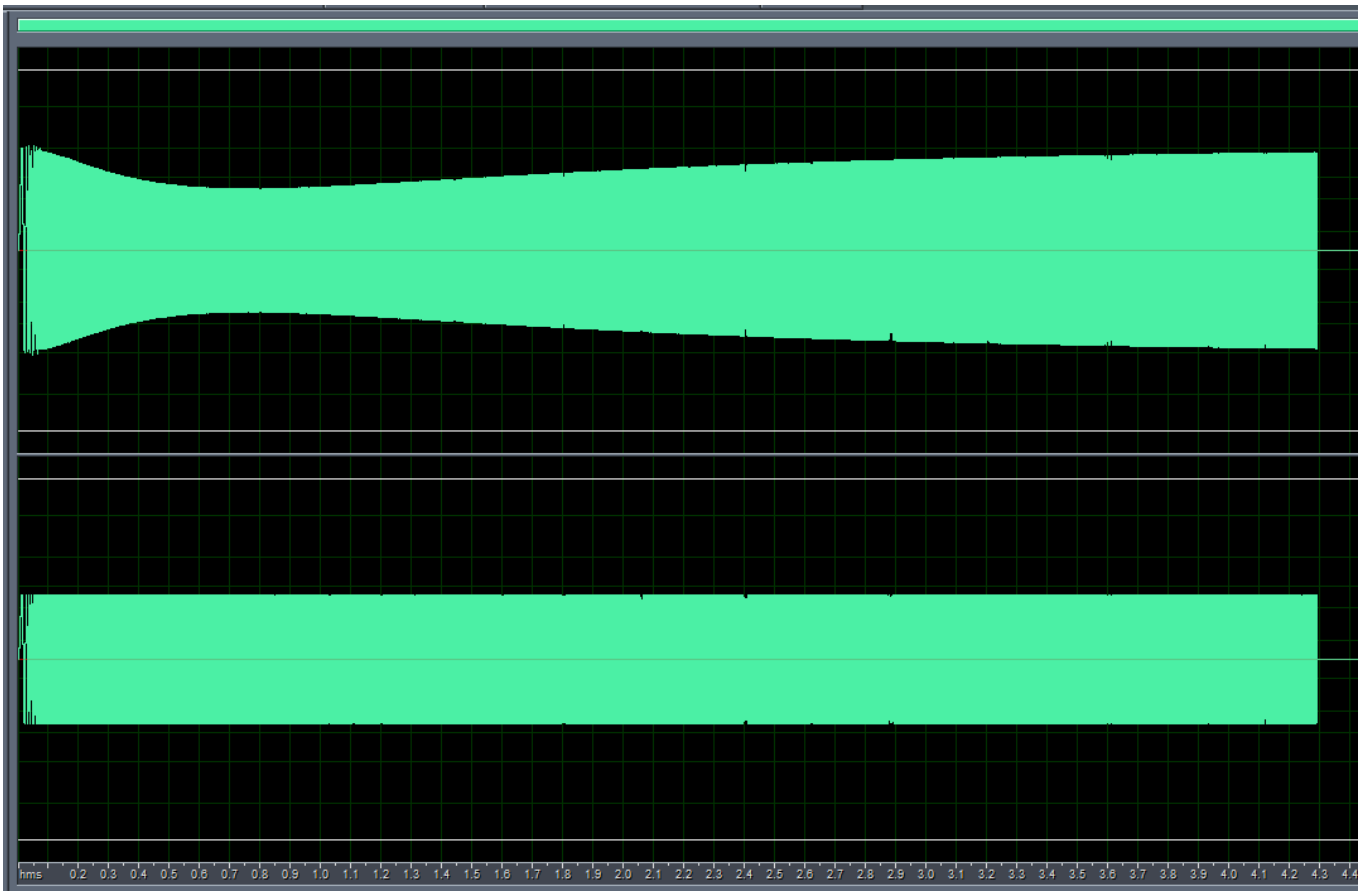


图 1.1.5-3 工况1波形对比图

结果分析： 由于输入电平低于设定的保持阈值，算法激活了响度补偿。从波形图可见，输出信号（上轨）相较于输入信号（下轨）在1000Hz以下及7000Hz以上频段有明显的增益。

工况二：高电平扫频输入测试

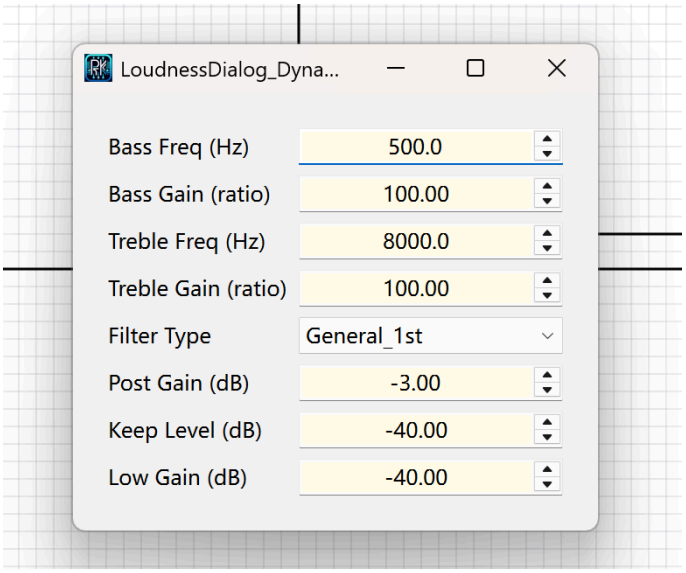


图 1.1.5-4 工况2控件参数

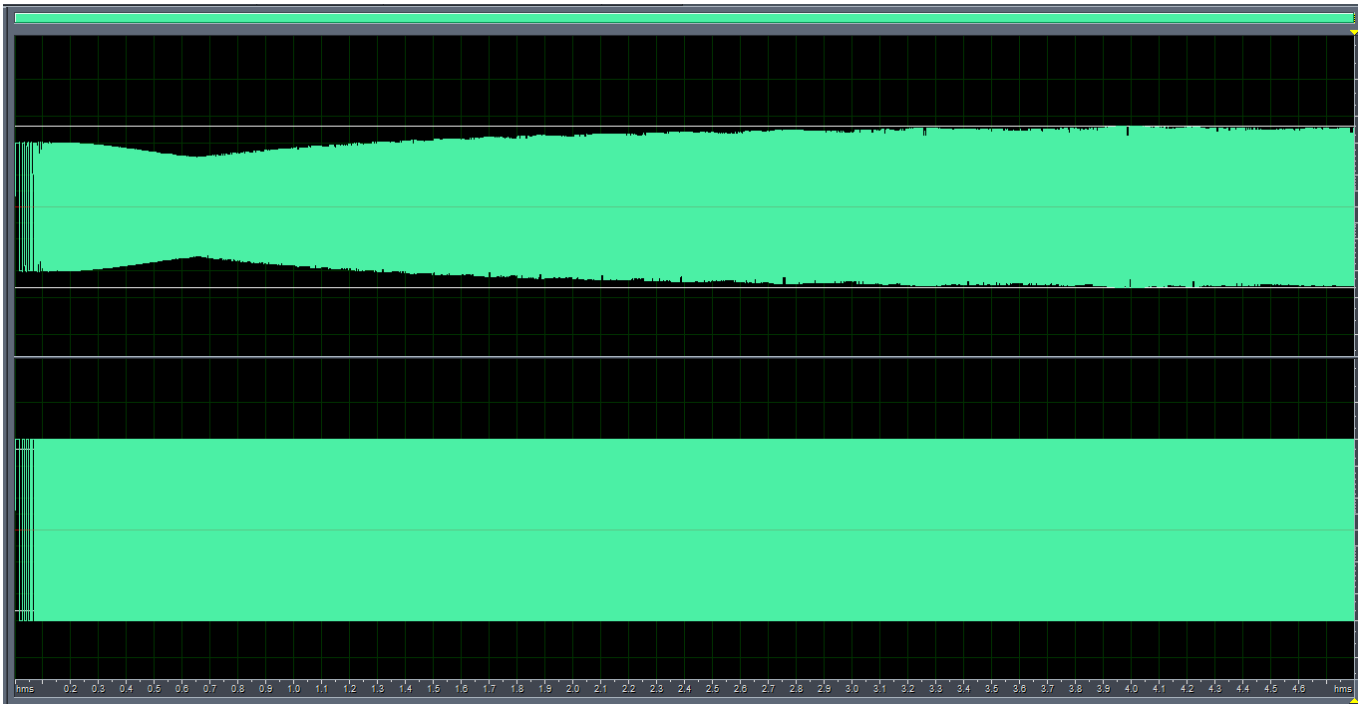


图 1.1.5-5 工况2波形对比图

结果分析： 由于输入电平高于设定的保持阈值，算法识别到当前响度充足，触发了高电平工况下的“中间衰减”机制。从波形图可见，输出信号（上轨）相较于平直的输入信号（下轨）在扫频中段呈现出衰减。

1.2 shift

1.2.1 Phat Shift (FIR)

功能：进行FIR进行相位移动。

这个模块使用fir进行移相，低阶时，对低频信号的效果较差，会产生噪声；高阶时，会增加延时和计算量。建议使用Phat Shift (IIR)*

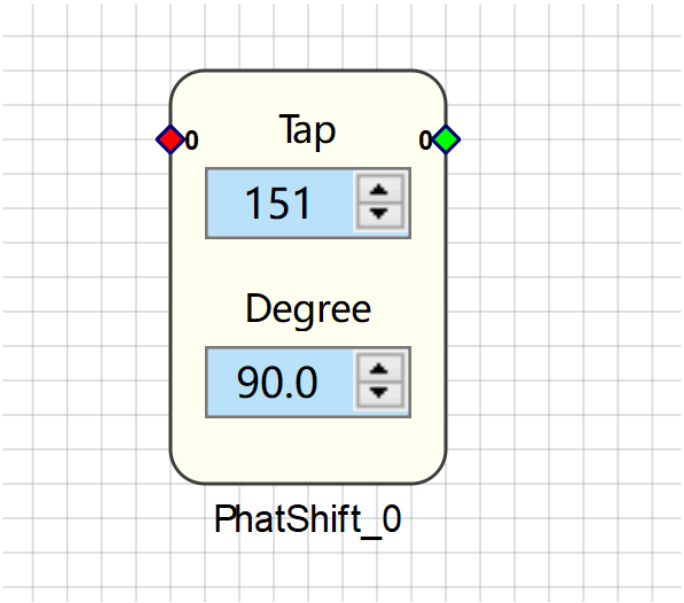


图 1.2.1-1 Phat Shift控件和设置界面

参数数量：2

参数名	参数位	类型	功能	单位	建议取值	默认取值
Tap	0	float	相移滤波器长度	无	151	151

参数名	参数位	类型	功能	单位	建议取值	默认取值
Degree	1	float	相位偏移角度	度	0 ~ 359	90

实际效果展示：
输入1kHz正弦波声源获取输出，参数配置位151，0和151，90，则输出会相差90度相位差：

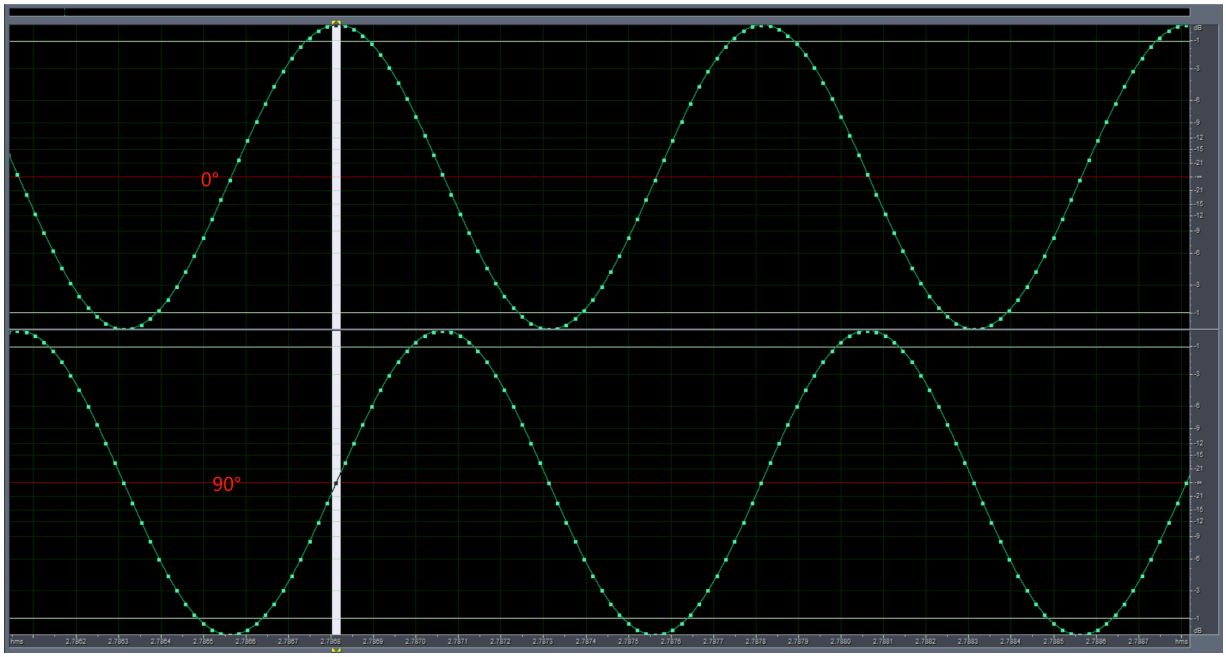


图 1.2.1 正弦时域相移0°（通道1）和相移90°（通道2）输出图

1.2.2 Phat Shift (IIR)

功能：使用IIR进行相位移动。

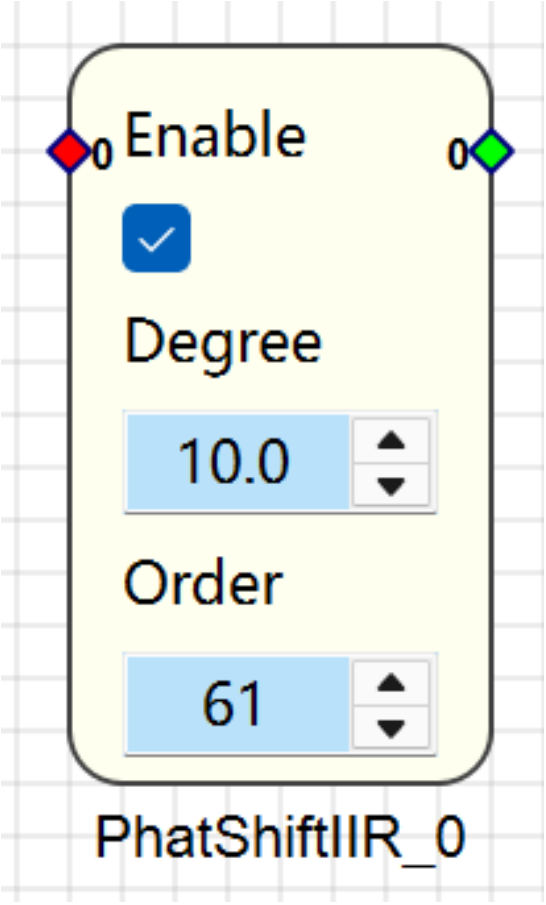


图 1.2.2-1 Phat Shift IIR控件和设置界面

参数数量：2

参数名	参数位	类型	功能	单位	建议取值	默认取值
Degree	0	float	相位偏移角度	度	0 ~ 359	90
Order	1	float	相移滤波器阶数	无	5 ~ 61	61

1.2.3 Frequency Shift (FIR)

功能：使用FIR进行频率移动。

这个模块使用fir进行移相，低阶时，对低频信号的效果较差，会产生噪声；高阶时，会增加延时和计算量。建议使用Frequency Shift (IIR)*

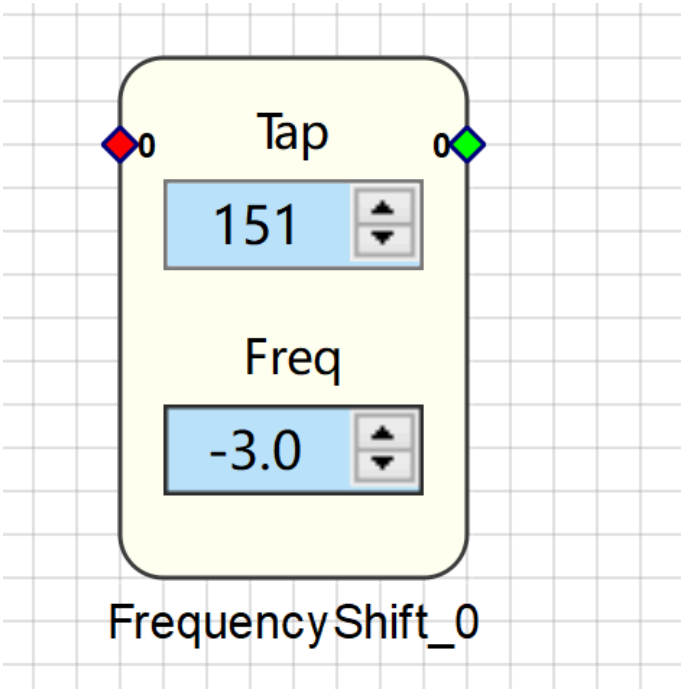


图 1.2.3-1 Frequency Shift控件和设置界面

参数数量：2

参数名	参数位	类型	功能	单位	建议取值	默认取值
Tap	0	float	移频滤波器长度	无	151	151
Freq	1	float	移动的频率	Hz	-10 ~ 10	-3

实际效果展示：
输入1kHz正弦波声源获取输出，参数配置位151，0和151，10，则输出会相差10Hz：
时域结果分析：

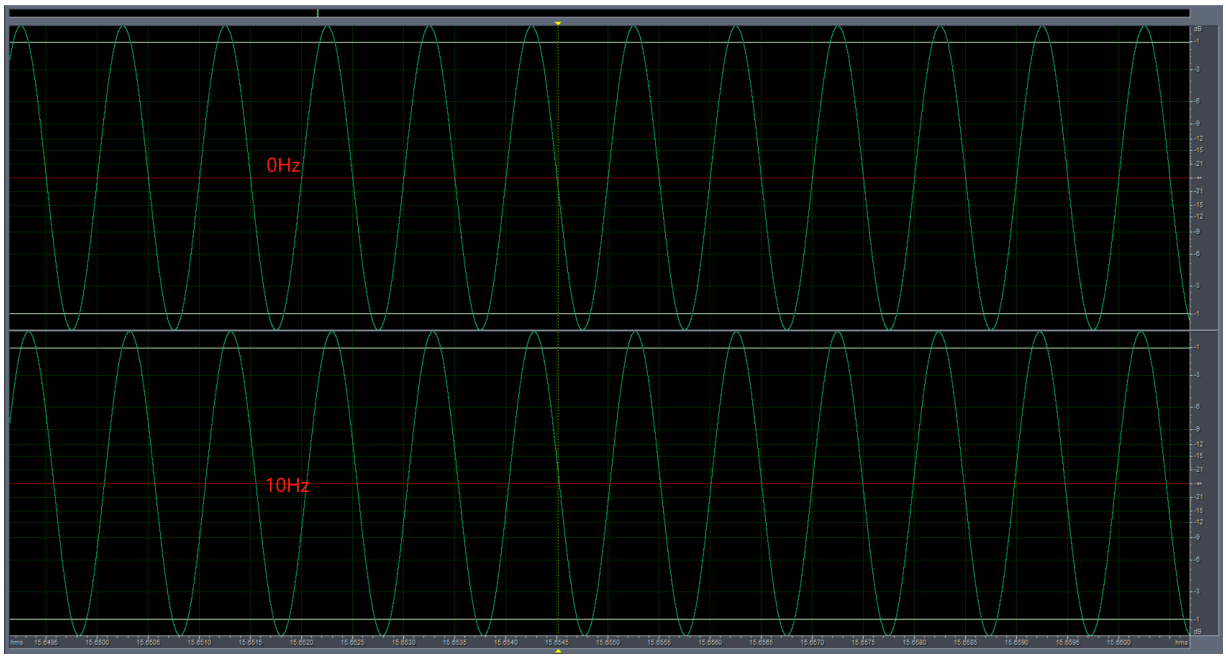


图 1.2.3-2 正弦频域频移0Hz（通道1）和频移10Hz（通道2）输出图

对应的频率分析结果如图所示:

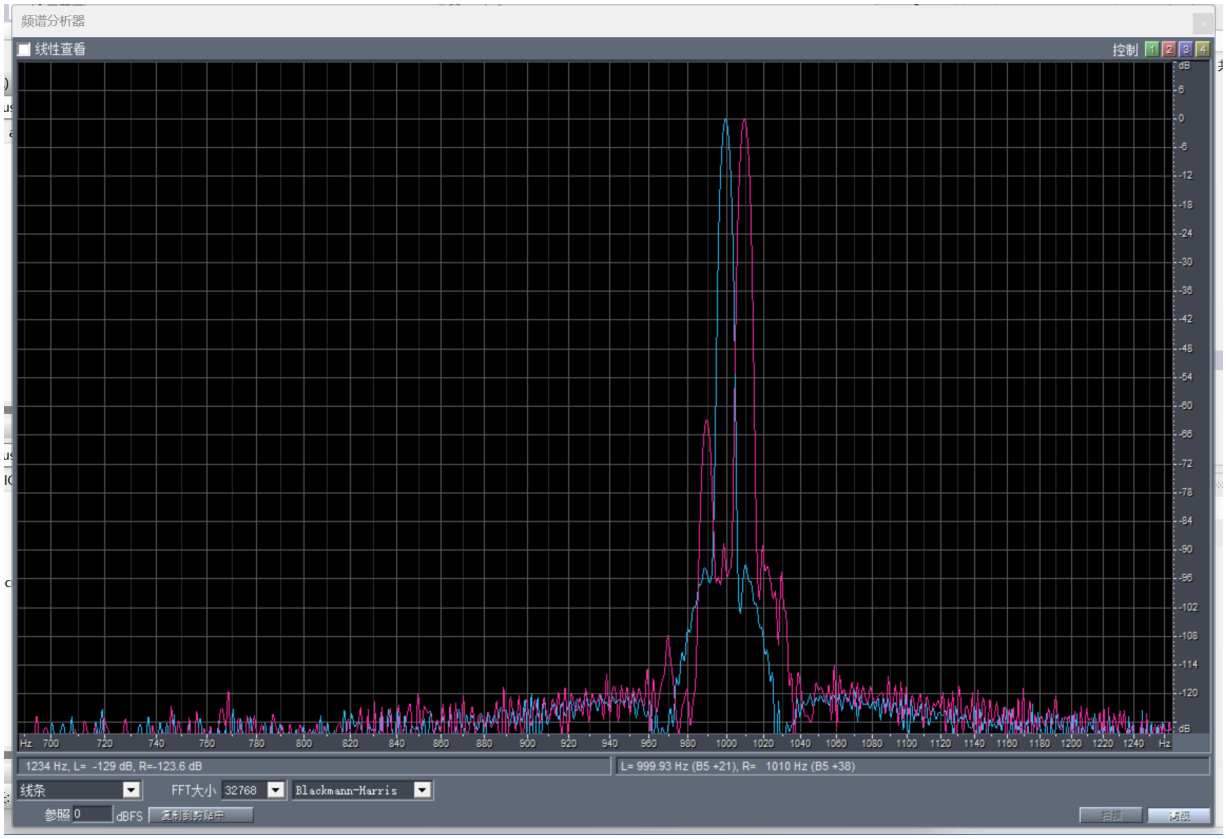


图 1.2.3-3 正弦频域频移0Hz（蓝线）和频移10Hz（紫线）频谱分析图

1.2.4 Frequency Shift (IIR)

功能：使用IIR进行频率移动。

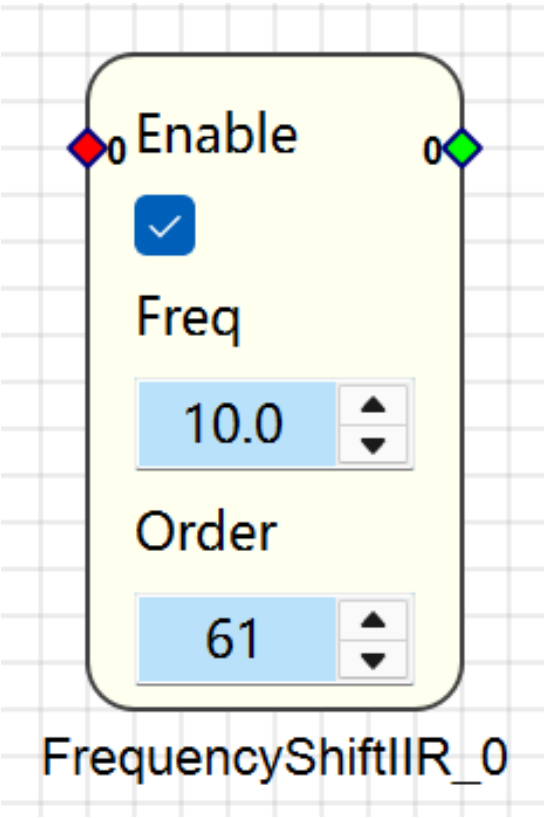


图 1.2.4-1 Frequency Shift IIR控件和设置界面

参数数量：2

参数名	参数位	类型	功能	单位	建议取值	默认取值
Freq	0	float	移动的频率	Hz	-10 ~ 10	-3
Tap	1	float	移频滤波器阶数	无	5~61	61

1.3 Surround Sound

功能：传统环绕声模块，支持立体声upmix成5通道。输出5通道分别为FL（左前）、FR（右前）、RL（左后环绕）、RR（右后环绕）、C（中置低音）

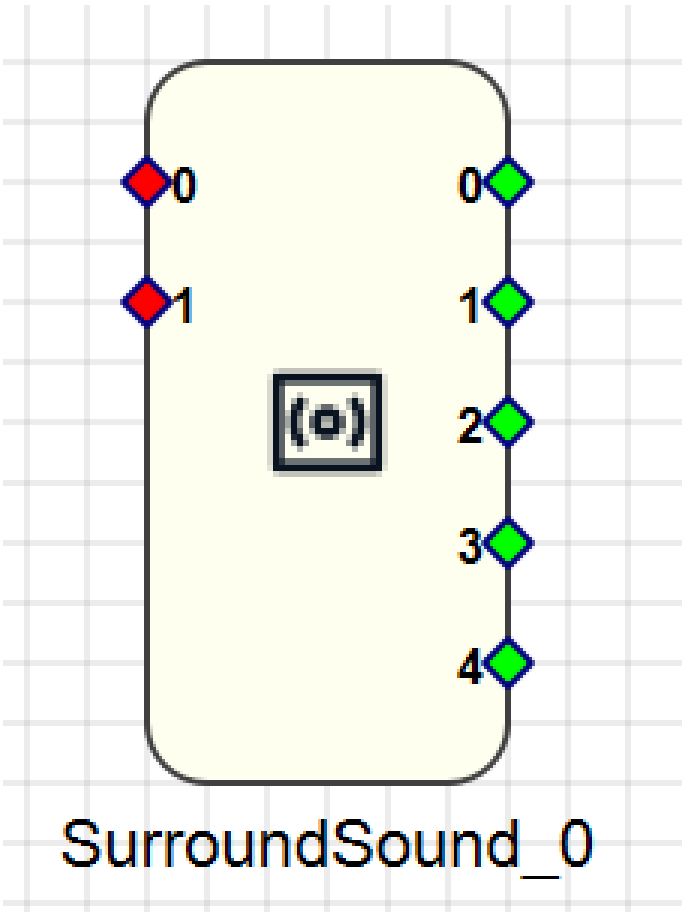


图 1.3-1 Surround Sound控件

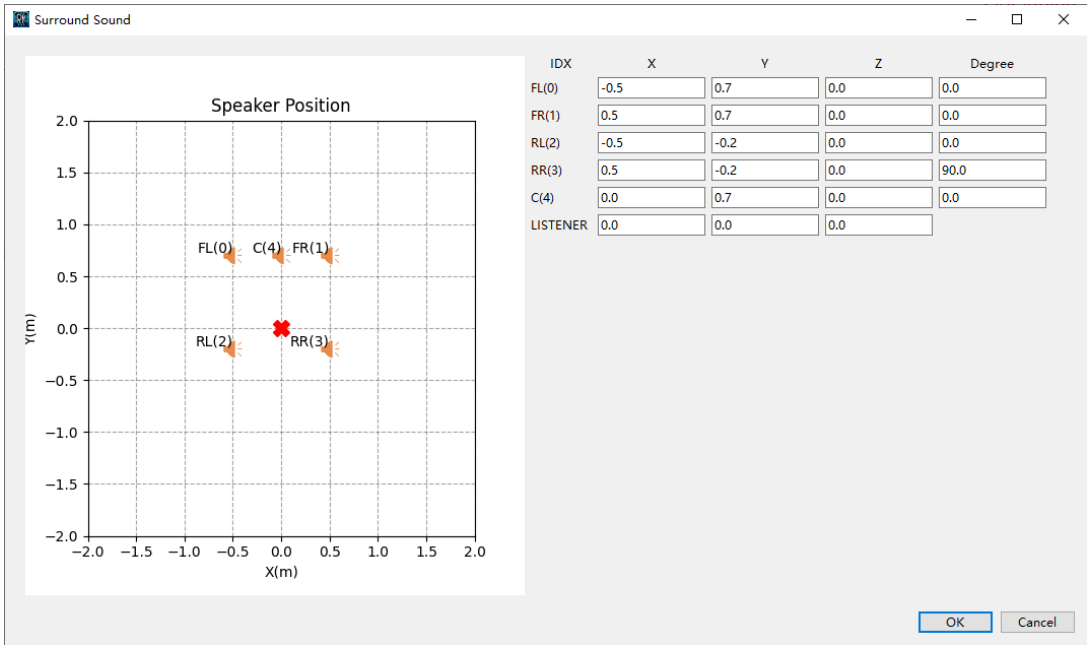


图 1.3-2 Surround Sound参数界面

参数数量：3+4*5=23

参数名	参数位	类型	功能	单位	建议取值	默认取值
X(listener)	0	float	收音位置的X轴坐标	米	无	0
Y(listener)	1	float	收音位置的Y轴坐标	米	无	0
Z(listener)	2	float	收音位置的Z轴坐标	米	无	0
X(speaker)	1*n+2	float	扬声器布局的X轴坐标	米	无	如上图
Y(speaker)	2*n+2	float	扬声器布局的Y轴坐标	米	无	如上图
Z(speaker)	3*n+2	float	扬声器布局的Z轴坐标	米	无	如上图
Degree(speaker)	4*n+2	float	输出相位偏移角度	度	无	0

1.4 Pitch Shifting

1.4.1 Phase Vocoder

功能 :基于相位声码器实现变调，该模块可用于人声变调以及音乐变调，对帧长要求较高，帧长最好在384以上。

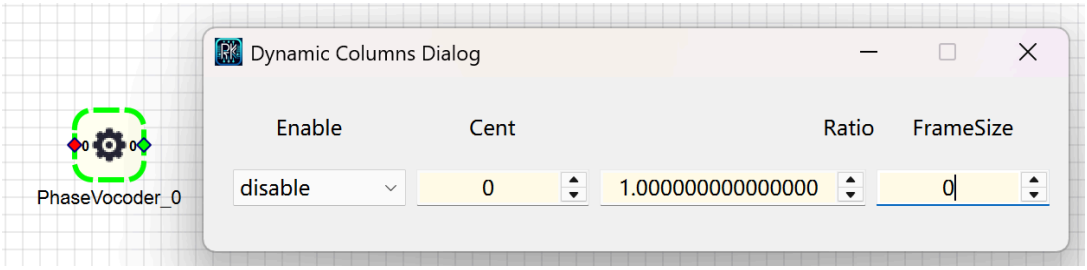


图 1.4.1-1 Phase Vocoder控件和设置界面

参数数量：4

参数名	参数位	类型	功能	单位	建议取值	默认取值
Enable	0	float	使能	无	0/1	0
Cent	1	float	变调大小（单位：音分）	Cent	-1200 ~ 1200	0
Ratio	2	float	变调系数，小于1，音调变低，大于1，音调变高		0.5 ~ 2	1
FrameSize	3	int	内部处理帧长：配置为0时跟随系统；配置大于0时开启拼帧处理	Samples	512 / 1024	0

注1： Cent和Ratio实际上是同一个参数在不同单位下的表示，两者满足Ratio = 2^{(Cent/1200)}。

注2—关于 FrameSize (拼帧处理) 的说明：
Phase Vocoder 算法对频域分辨率要求较高，通常需要较长的帧长（建议 384 以上）才能获得良好的音质。

- FrameSize = 0 (默认模式)：模块直接使用系统的全局帧长（Global Frame Length）。如果系统帧长较小（如 64 samples），变调处理可能会出现明显的相位失真（如金属音、模糊感）。
- FrameSize > 0 (拼帧模式)：模块启用内部缓存机制。无论系统输入帧长是多少，模块都会积攒数据，直到凑满 FrameSize 设定的长度（例如 512）才进行一次变调运算。
  - 优势：能在系统低延迟设置下（如帧长64），强行使用大帧长处理，显著提升音质，减少失真。
  - 代价：会引入额外的处理延迟（Latency = FrameSize - SystemFrameSize）。

1.4.2 Phase Vocoder (Formant)

功能 :在Phase Vocoder的基础上，增加了共振峰保护功能。

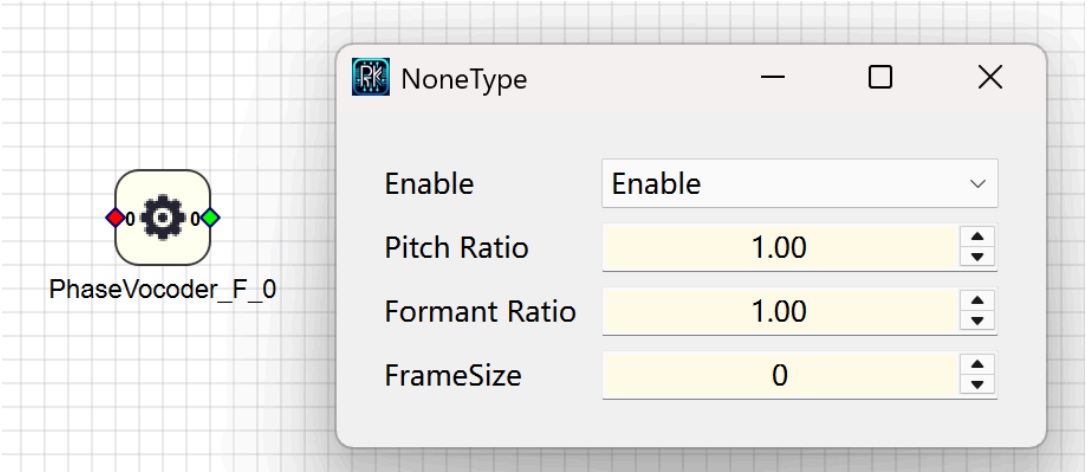


图 1.4.2-1 Phase Vocoder (Formant)控件和设置界面

参数数量：4

参数名	参数位	类型	功能	单位	建议取值	默认取值
Enable	0	float	使能	无	0/1	0
Pitch Ratio	1	float	变调系数，小于1，音调变低，大于1，音调变高	无	0.5 ~ 2	1
Formant Ratio	2	float	共振峰包络变化系数	无	0.5 ~ 2	1
FrameSize	3	int	内部处理帧长：配置为0时跟随系统；配置大于0时开启拼帧处理	Samples	512 / 1024	0

注：
在纯 Phase Vocoder 变调中，谱包络（即共振峰/声道滤波的形状）会随 Pitch Ratio 一起被等比例平移，导致音色也被“抬高/压低”，出现典型的“花栗鼠”（升调）或“闷/口腔变大”（降调）听感。Formant Ratio 用来单独控制谱包络的伸缩/位移程度，从而解耦“音高”与“音色（共振峰位置）”：

Formant Ratio = 1.0（推荐默认）：尽量保持原始共振峰位置不变，即保护音色，只改变基频/谐波——这是“自然变调”的常用设置。

Formant Ratio ≈ Pitch Ratio：让共振峰随音高同步移动，等价于不做共振峰保护，听感与传统 Phase Vocoder 相同。

Formant Ratio > 1：上移共振峰（模拟更小的声道/更尖亮的音色）。

Formant Ratio < 1：下移共振峰（模拟更大的声道/更厚暗的音色）。

1.4.3 WSOLA

功能 :基于WSOLA实现变调，该模块可用于人声变调，对帧长要求稍低，帧长最好在192以上。

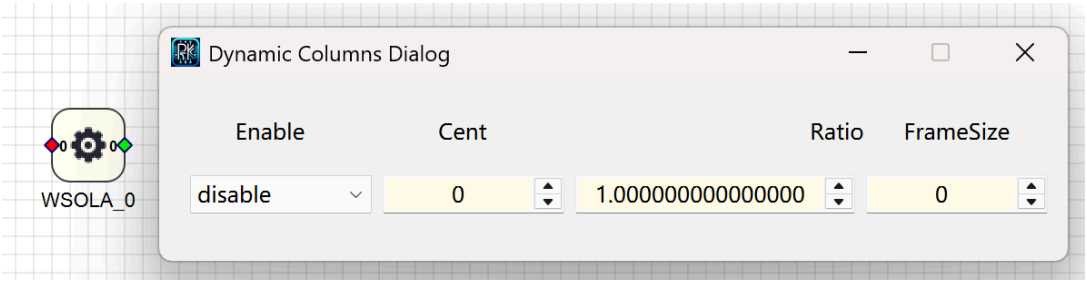


图 1.4.3-1 WSOLA控件和设置界面

参数数量：4

参数名	参数位	类型	功能	单位	建议取值	默认取值
Enable	0	float	使能	无	0/1	0
Cent	1	float	变调大小（单位：音分）	Cent	-1200 ~ 1200	0

参数名	参数位	类型	功能	单位	建议取值	默认取值
Ratio	2	float	变调系数，小于1，音调变低，大于1，音调变高		0.5 ~ 2	1
FrameSize	3	int	内部处理帧长：配置为0时跟随系统；配置大于0时开启拼帧处理	Samples	512 / 1024	0

注1: Cent和Ratio实际上是同一个参数在不同单位下的表示，两者满足Ratio = 2^(Cent/1200)。

注2—关于 FrameSize (拼帧处理) 的说明：

Phase Vocoder 算法对频域分辨率要求较高，通常需要较长的帧长（建议 384 以上）才能获得良好的音质。

- FrameSize = 0 (默认模式)：模块直接使用系统的全局帧长（Global Frame Length）。如果系统帧长较小（如 64 samples），变调处理可能会出现明显的相位失真（如金属音、模糊感）。
- FrameSize > 0 (拼帧模式)：模块启用内部缓存机制。无论系统输入帧长是多少，模块都会积攒数据，直到凑满 FrameSize 设定的长度（例如 512）才进行一次变调运算。
  - 优势：能在系统低延迟设置下（如帧长64），强行使用大帧长处理，显著提升音质，减少失真。
  - 代价：会引入额外的处理延迟（Latency = FrameSize - SystemFrameSize）。

1.4.4 WSOLA (Formant)

功能 :在WSOLA的基础上，增加了共振峰保护功能。

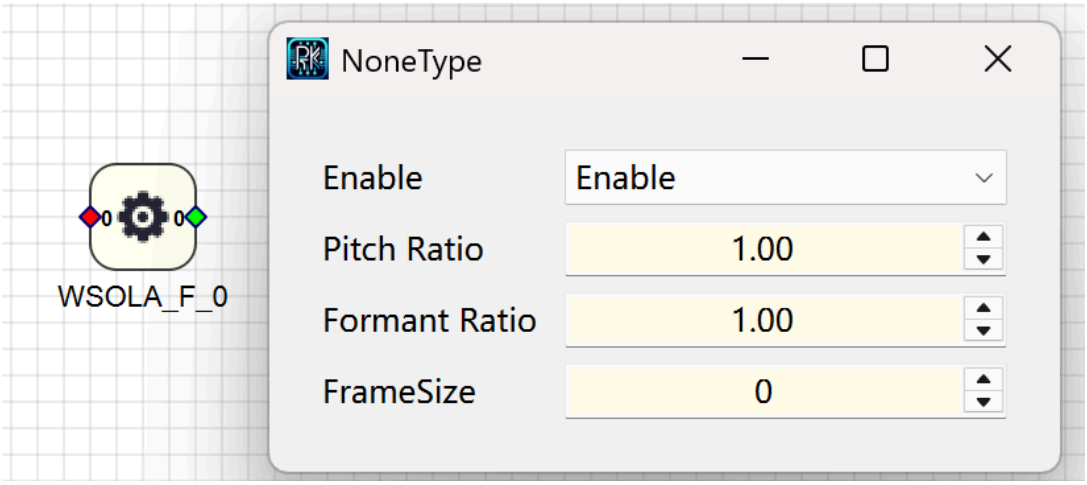


图 1.4-4 WSOLA (Formant)控件和设置界面

参数数量：4

参数名	参数位	类型	功能	单位	建议取值	默认取值
Enable	0	float	使能	无	0/1	0
Pitch Ratio	1	float	变调系数，小于1，音调变低，大于1，音调变高	无	0.5 ~ 2	1
Formant Ratio	2	float	共振峰包络变化系数	无	0.5 ~ 2	1
FrameSize	3	int	内部处理帧长：配置为0时跟随系统；配置大于0时开启拼帧处理	Samples	512 / 1024	0

1.5 Phat Stereo

功能说明:增强立体声效果，使音频听起来更加宽广、饱满和有空间感

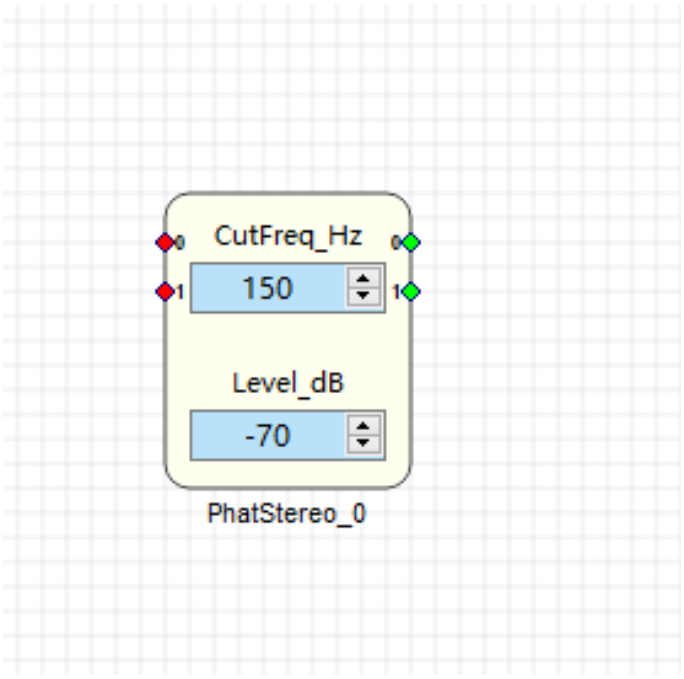


图 1.5-1 Phat Stereo控件

参数数量：2

参数名	参数位	类型	功能	单位	建议取值	默认取值
CutFreq_Hz	0	float	低通滤波器的截止频率	HZ	20 ~ 20000	150
Level_dB	1	float	控制音频的输出强度	dB	-96 ~ 6	-70

1.6 DTS

1.6.1 DTS Virtual:X

DTS Virtual:X 主要用于dts后处理。

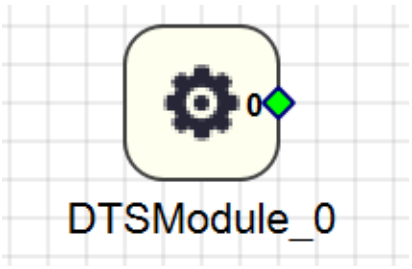


图 1.6.1-1 DTS Virtual:X 控件

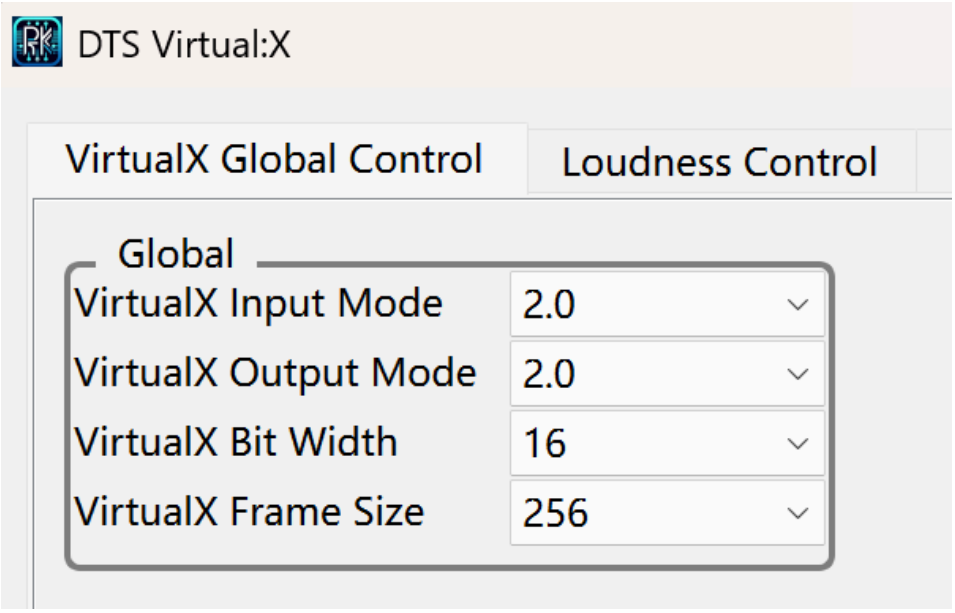


图 1.6.1-2 DTS Virtual:X 参数界面

本控件与版权有关，请与业务/开发人员联系。

1.6.2 DTS:X

DTS：X主要用于dts认证。

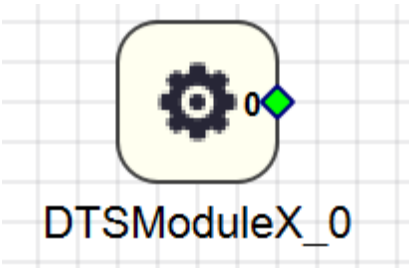


图 1.6.2-1 DTS:X 控件

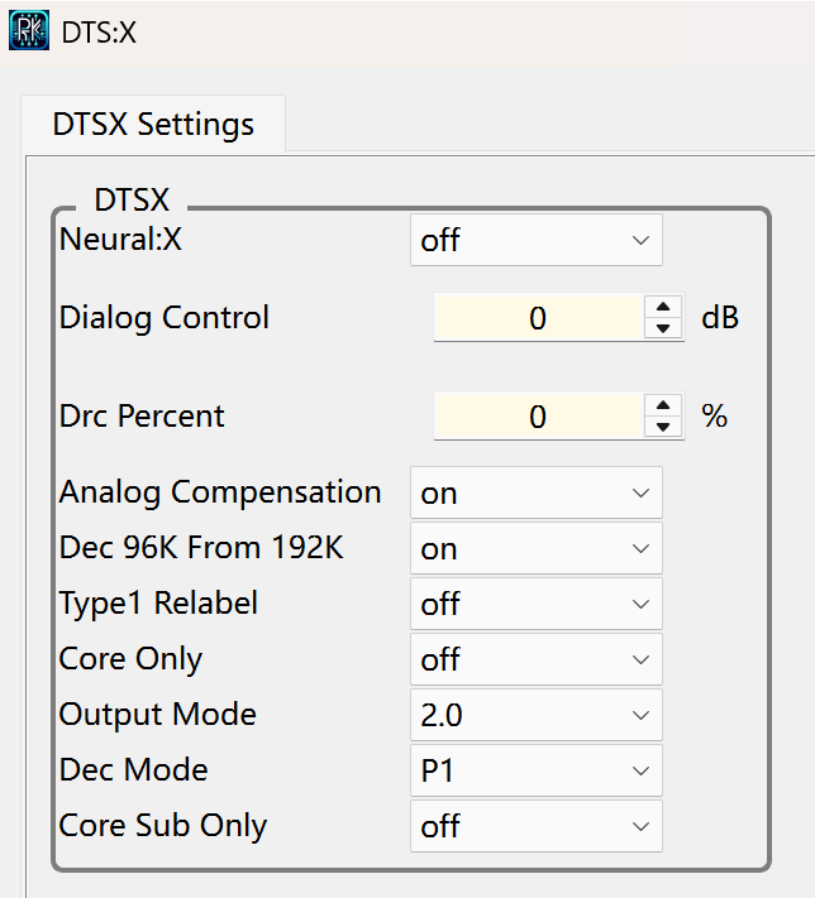


图 1.6.2-2 DTS:X 参数界面

本控件与版权有关，请与业务/开发人员联系。

1.7 Dolby

Dolby插件主要用于dolby认证。

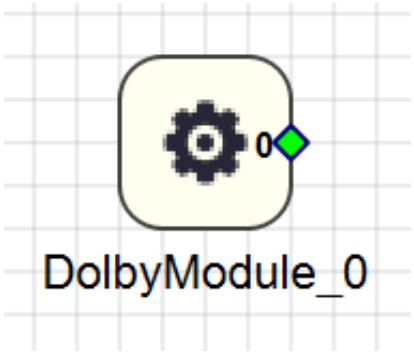


图 1.7-1 Dolby 控件

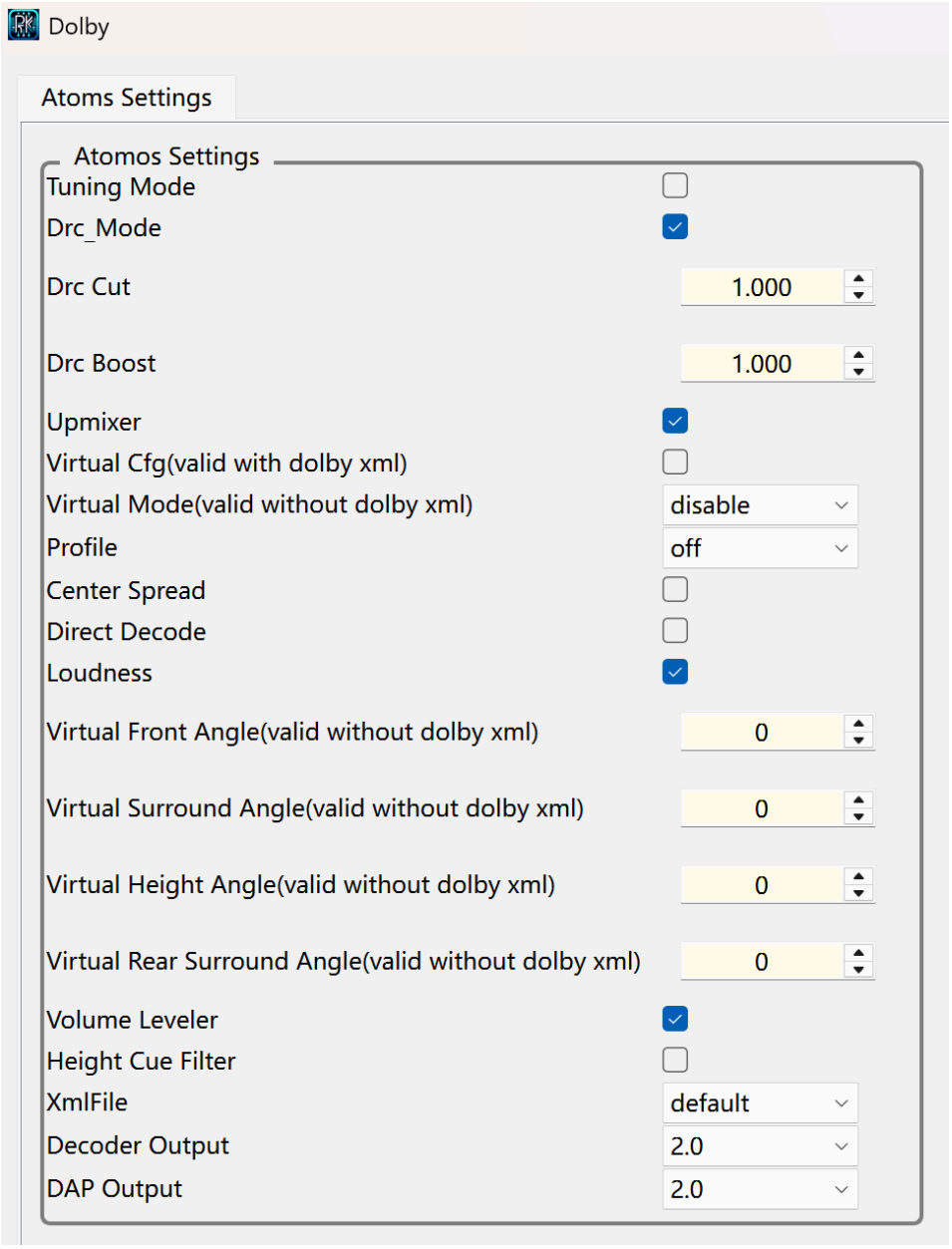


图 1.7-2 Dolby 参数界面

本控件与版权有关，请与业务/开发人员联系。

1.8 BF_ECNR

BF_ECNR用于语音降噪和回声消除。需要手动配置通道数、采样率和帧长。

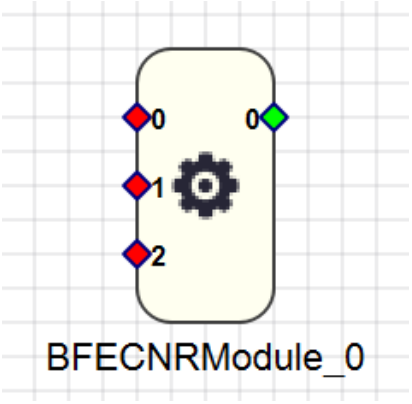


图 1.8-1 BF_ECNR 控件

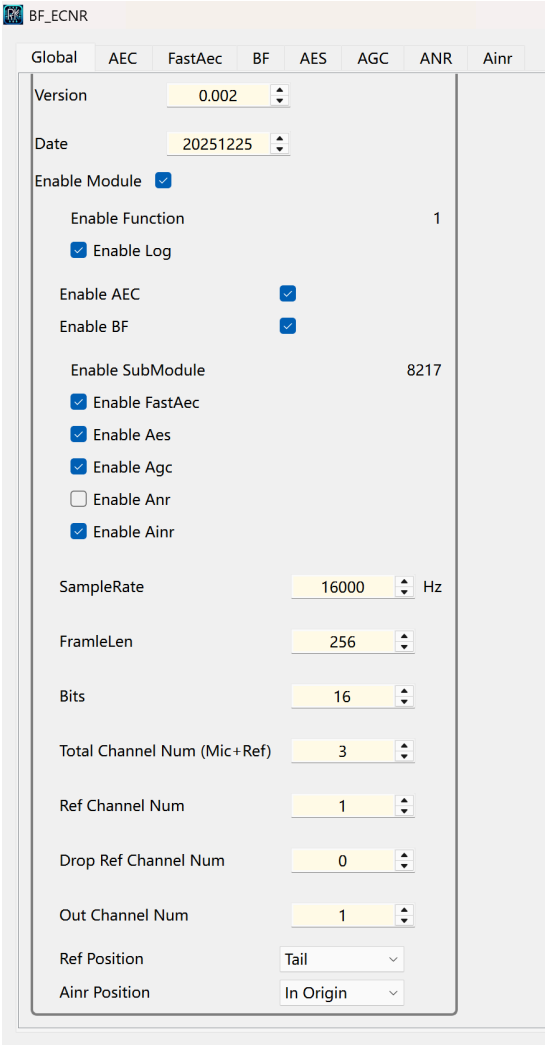


图 1.8-2 BF_ECNR 参数界面

> **总输入通道数 = 麦克风数量 + 参考信号数量**
> **Total Channel Num = Mic Num + Ref Num**

(1) Total Channel Num (Mic+Ref) = 3

- 含义：表示这个算法模块总共需要接入3路输入信号。
- 这 3 路信号是由“麦克风录音数据”和“参考回声数据”组成的。具体各占多少，取决于下面的参数。

(2) Ref Channel Num = 1

- 含义：在上面的 3 路总输入中，有1路是参考信号。既然总数是 3，参考是 1。那么麦克风数量 (Mic Num) = 3 - 1 = 2。
- 结论：这是一个双麦克风（2 Mic）+ 单参考（1 Ref）的配置。

(3) Out Channel Num = 1

- 含义：经过算法处理（降噪、回声消除、波束成形）后，最终输出1路干净的音频。

本控件将在另一篇文档 《Rockchip_Developer_Guide_Audio_Algorithm_VQE_Introduction_CN.pdf》 详细介绍

1.9 AVAS

1.9.1 AVAS

功能：汽车低速行驶警告声，信号源使用wav分区的文件作为输入。

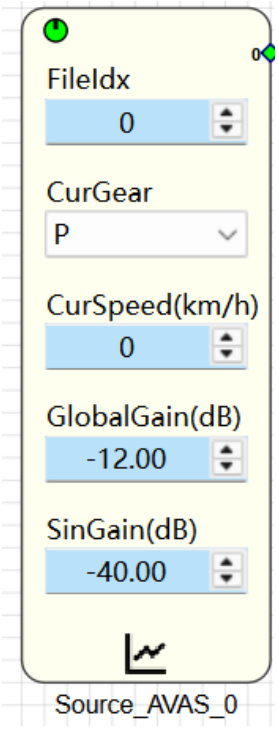
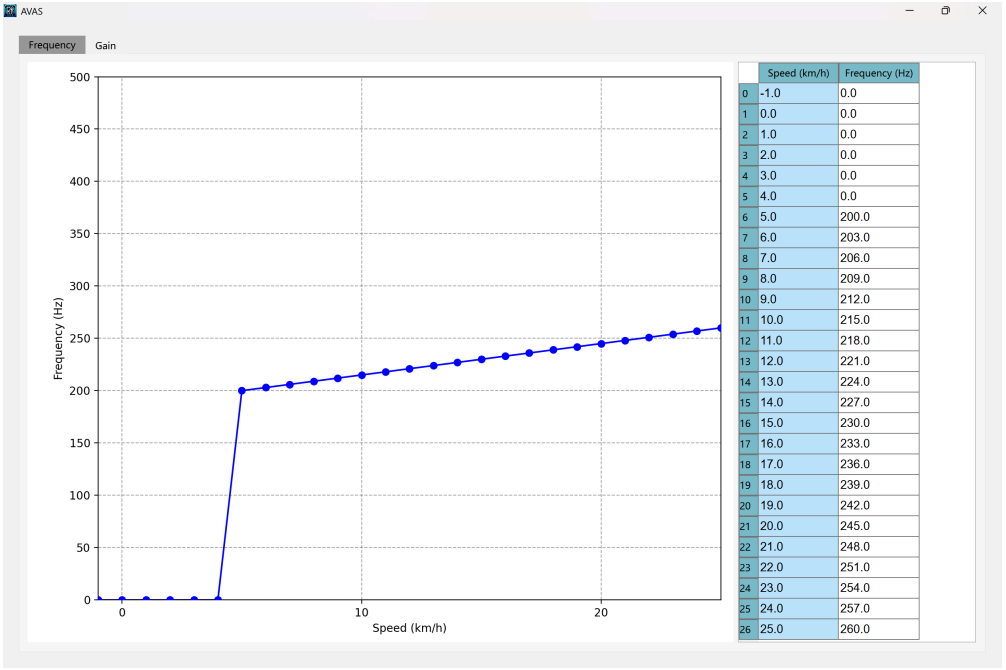
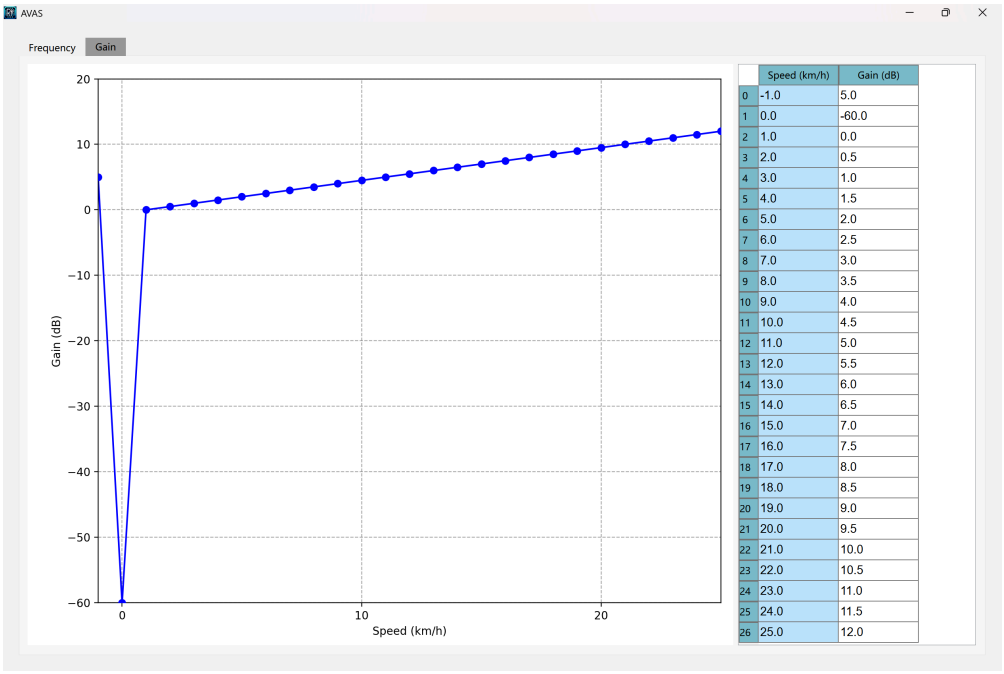


图 1.9.1-1 AVAS控件



	Speed (km/h)	Frequency (Hz)
0	-1.0	0.0
1	0.0	0.0
2	1.0	0.0
3	2.0	0.0
4	3.0	0.0
5	4.0	0.0
6	5.0	200.0
7	6.0	203.0
8	7.0	206.0
9	8.0	209.0
10	9.0	212.0
11	10.0	215.0
12	11.0	218.0
13	12.0	221.0
14	13.0	224.0
15	14.0	227.0
16	15.0	230.0
17	16.0	233.0
18	17.0	236.0
19	18.0	239.0
20	19.0	242.0
21	20.0	245.0
22	21.0	248.0
23	22.0	251.0
24	23.0	254.0
25	24.0	257.0
26	25.0	260.0

图 1.9.1-2 AVAS频率设置界面



	Speed (km/h)	Gain (dB)
0	-1.0	5.0
1	0.0	-60.0
2	1.0	0.0
3	2.0	0.5
4	3.0	1.0
5	4.0	1.5
6	5.0	2.0
7	6.0	2.5
8	7.0	3.0
9	8.0	3.5
10	9.0	4.0
11	10.0	4.5
12	11.0	5.0
13	12.0	5.5
14	13.0	6.0
15	14.0	6.5
16	15.0	7.0
17	16.0	7.5
18	17.0	8.0
19	18.0	8.5
20	19.0	9.0
21	20.0	9.5
22	21.0	10.0
23	22.0	10.5
24	23.0	11.0
25	24.0	11.5
26	25.0	12.0

图 1.9.1-3 AVAS增益设置界面

参数数量：5

参数名	参数位	类型	功能	单位	建议取值	默认取值
FileIdx	0	float	音频文件id	无	0 ~ 127	0
CurGear	1	float	当前档位选择	无	0 ~ 3	0
CurSpeed	2	float	当前车速	km/h	0 ~ 200	0
GlobalGain	3	float	全局增益	dB	-100 ~ 100	-12
SinGain	4	float	信号增益	dB	-100 ~ 100	-40

本模块参数配置旨在满足 **GB/T 37153-2018《电动汽车低速提示音》** 强制性标准，具体的配置逻辑如下：

1. 频率响应 (Frequency Table)
   需配置频率随车速（0-20 km/h）线性增加，以满足标准定义的“频移”要求（频移率 ≥0.8% /(km/h)），从来满足法规要求。
2. 增益控制 (Gain Table)
   需确保在低速段（如 10 km/h, 20 km/h）的输出声压级达到国标最低限值（通常 ≥50 dB），并在超过截止速度后进行衰减。
3. 档位逻辑 (CurGear)
   当系统识别到R档（倒车）时，无论车速如何，均需持续输出倒车警示音，也就是对应speed=-1的参数配置。

1.9.2 AVAS External Source

功能：汽车低速行驶警告声, 功能和AVAS*一样，只是输入源使用输入信号。

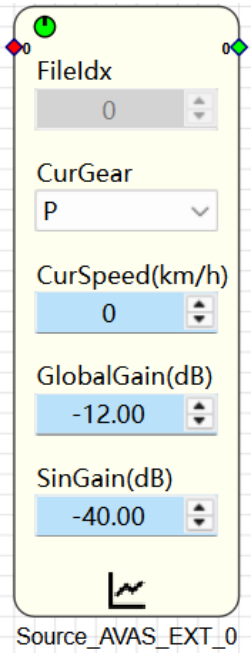


图 1.9.2-1 AVAS External Source控件

1.10 Voice Separators

用传统相干性的方法实现人声和背景音的分离。

1.10.1 Voice Separator (MS)

功能：基于频域互相关的语音分离模块，将立体声信号分离为中间声道M和侧边声道S。

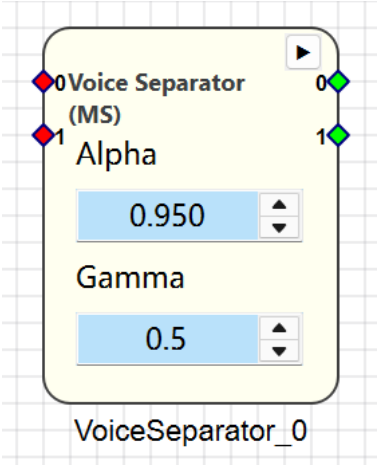


图 1.10.1-1 Voice Separator (MS)控件和设置界面

参数名	参数位	类型	功能	单位	建议取值	默认取值
Alpha	0	float	平滑系数	无	0.5-0.999	0.95
Gamma	1	float	曲线映射系数	无	0.5 ~ 2	0.5

注1：Alpha影响了内部计算出的频域自相关系数在不同帧之间的平滑程度，数值越大，平滑效果越明显，但对突发声音的响应会变慢。该参数主要根据帧长进行修改，帧长越长，建议Alpha值越小。

注2：Gamma影响了频域自相关系数与M通道保留程度的非线性映射曲线，该值越小，意味着很小的自相关系数也会得到较大的保留，从而使得M通道的声音更加完整，该值越小则M通道的声音会更加干净，但也会损失部分声音信息。该参数主要根据实际听感进行调整。

1.10.2 Voice Separator (LRM)

功能：基于频域互相关的语音分离模块，将立体声信号分离为中间声道M、左声道L和右声道R。

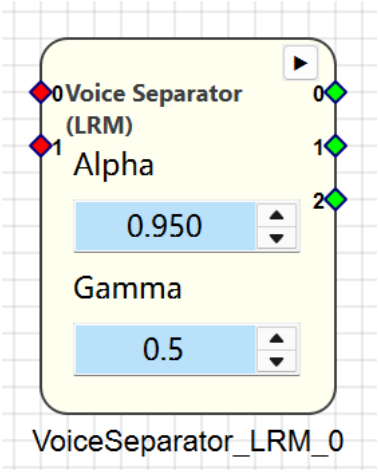


图 1.10.2-1 Voice Separator (LRM)控件和设置界面

参数名	参数位	类型	功能	单位	建议取值	默认取值
Alpha	0	float	平滑系数	无	0.5-0.999	0.95
Gamma	1	float	曲线映射系数	无	0.5 ~ 2	0.5

注1：Alpha影响了内部计算出的频域自相关系数在不同帧之间的平滑程度，数值越大，平滑效果越明显，但对突发声音的响应会变慢。该参数主要根据帧长进行修改，帧长越长，建议Alpha值越小。

注2：Gamma影响了频域自相关系数与M通道保留程度的非线性映射曲线，该值越小，意味着很小的自相关系数也会得到较大的保留，从而使得M通道的声音更加完整，该值越小则M通道的声音会更加干净，但也会损失部分声音信息。该参数主要根据实际听感进行调整。

1.11 CrossTalk Suppressor

功能说明：用于消除双声道音频对两耳的干扰，从而进行声场扩展。

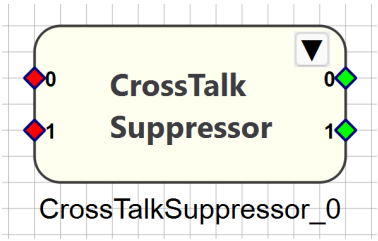
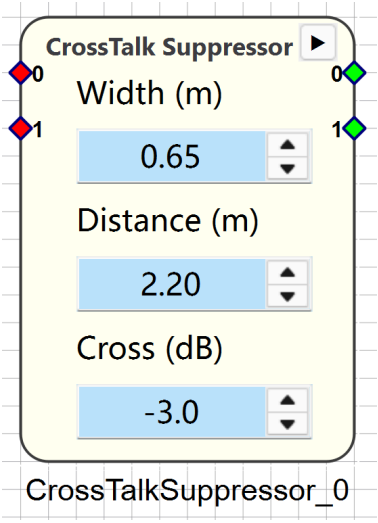


图 1.11-1 CrossTalk Suppressor控件



参数数量：3

参数名	参数位	类型	功能	单位	建议取值	默认取值
Width(m)	0	float	左右声道喇叭间距	m	0.1-2.0	0.65
Distance(m)	1	float	听音位到左右喇叭中心点的距离	m	0.5 ~ 10	2.2
Cross(dB)	2	float	去串扰增益，值越大强度越大	dB	-20 ~ -1	-3

2 Basic DSP

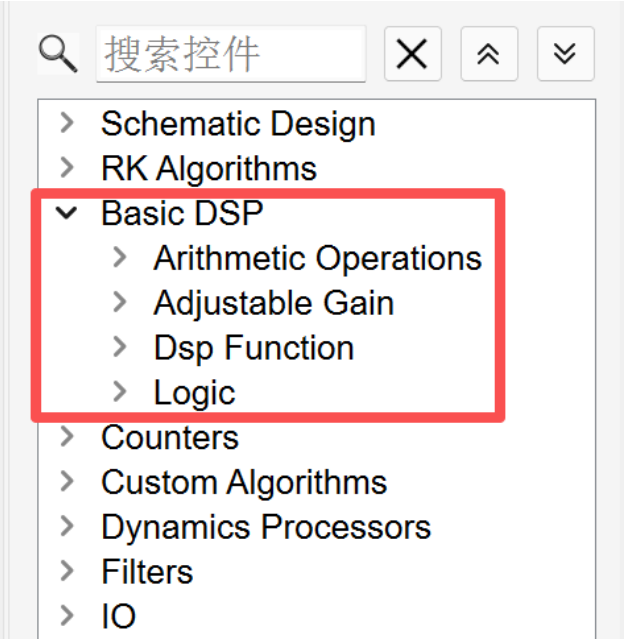


图 2-1 Basic DSP位于算法列表的位置

2.1 Arithmetic Operations

2.1.1 Absolute Value

功能说明：计算数据的绝对值：$output = abs(input)$。

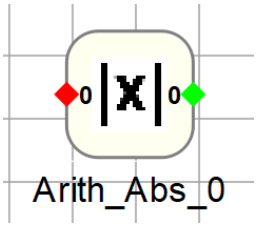


图 2.1.1-1 Absolute Value 控件

参数：无

2.1.2 Inverse Square Root

功能说明：计算数据的开方的倒数：$output = 1/\sqrt{input}$。

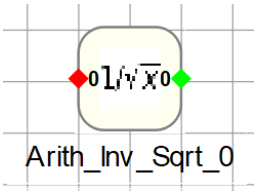


图 2.1.2-1 Inverse Square Root 控件

参数：无

2.1.3 Log2

功能说明：计算数据的log2值：$output = log_2(input)$。

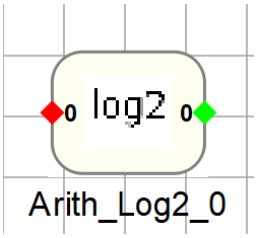


图 2.1.3-1 Log2 控件

参数：无

2.1.4 Log10

功能说明：计算数据的log10值：$output = log_{10}(input)$。

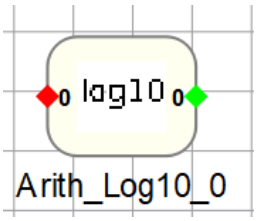


图 2.1.4-1 Log10 控件

参数：无

2.1.5 Exp2

功能说明：计算数据的2的n次方值：$output = 2^{input}$。

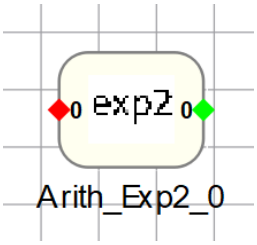


图 2.1.5-1 Exp2 控件

参数：无

2.1.6 Square Root

功能说明：计算数据的开方：$output = \sqrt{input}$。

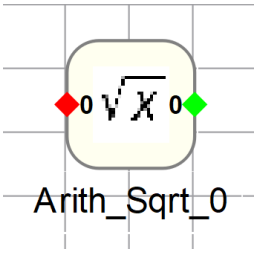


图 2.1.6-1 Square Root 控件

参数：无

2.1.7 Signal Add

功能说明：所有数据累加：$output = input0 + input1 + ...$。

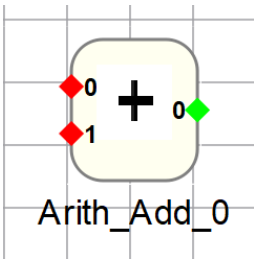


图 2.1.7-1 Signal Add 控件

2.1.8 Signal Subtract

功能说明：所有数据相减：$output = input0 - input1 - ...$。

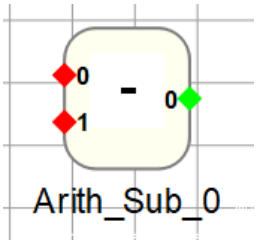


图 2.1.8-1 Signal Subtract 控件

参数：无

2.1.9 Divide

功能说明：两两通道数据相除,第一通道除以第二通道的数据，输出到第一通道，第三通道除以第四通道的数据，输出到第二通道，依次类推，公式如下：
$output0 = input0/input1$
$output1 = input2/input3$

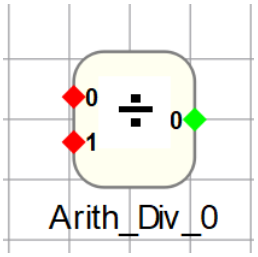


图 2.1.9-1 Divide 控件

参数：无

实际效果展示：
典型工况：输入为DC 0.5和DC 1.0。

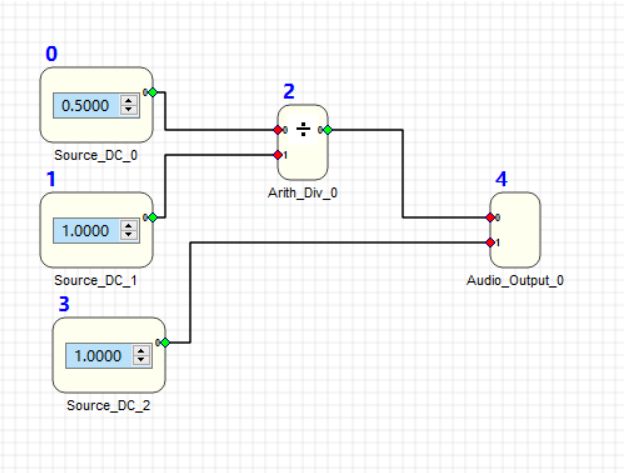


图 2.1.9-2 divide测试样例

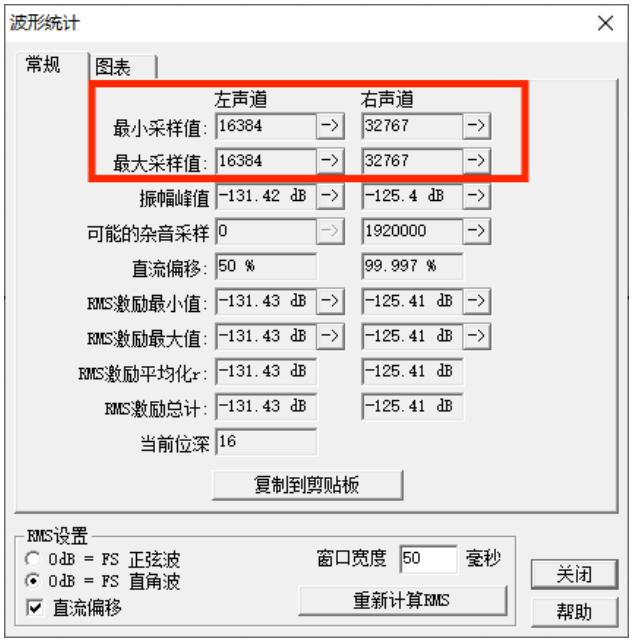


图 2.1.9-3 输出数值图，左声道对应图纸的audio_output的0通道，右声道对应图纸的audio_output的1通道

补充说明，在数字音频中，dc值为1的对应的采样编码值为满刻度值。此次测试的音频采样位数为16bit，范围为-32768 ~ +32767，右声道的输入DC 1.0的采样值为32767

可以从结果看出，divide的输出为16384，对应的dc值为0.5，符合理论结果。

图 2.1.9-4 输出波形对比图，左声道对应图纸的audio_output的0通道，右声道对应图纸的audio_output的1通道

2.1.10 Multiply

功能说明：两两通道数据相乘,第一通道乘以第二通道的数据，输出到第一通道，第三通道乘以第四通道的数据，输出到第二通道，依次类推，公式如下：
$output0 = input0 * input1$
$output1 = input2 * input3$

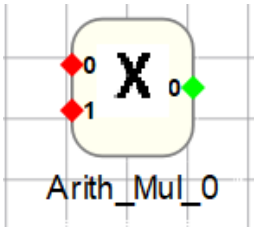


图 2.1.10-1 Multiply 控件

参数：无

2.1.11 Signal Invert

功能说明：信号极性反相，如果选中复选框，则反相，公式为：$output = -input$。

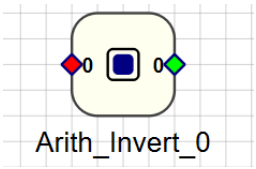


图 2.1.11-1 Signal Invert 控件

参数：无

2.1.12 Multi Signal Invert

功能说明：多信号极性反相,可以每路单路控制是否反相，公式为：
$output0 = -input0$
$output1 = -input1$。

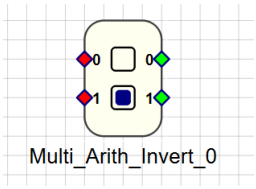


图 2.1.12-1Multi Signal Invert 控件

参数：无

2.2 Adjustable Gain

2.2.1 Gain

功能说明：将输入信号乘以增益后输出，输入框为增益值单位为DB，会将DB转为线性增益*并传给算法使用。

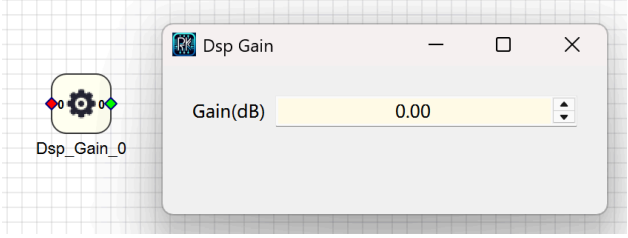


图 2.2.1-1 Gain 模块和设置界面

参数数量：1

参数名	参数位	类型	功能	单位	建议取值	默认取值
Gain	0	float	增益取值	db	-138 ~ 100	0

具体效果参考Gain (Slew)，当前模块只是少了平滑的功能

注：dB转换为线性增益的公式为：

```
def db_to_linear(db, en_rms=False):
    if en_rms:
        return 10**(db/10)
    else:
        return 10**(db/20)
```

2.2 章节的控件均采用else支路计算方法

2.2.2 Gain (Slew)

功能说明：将输入信号乘以增益后输出，输入框为增益值单位为DB，会将DB转为线性增益*并传给算法使用。会根据设置的slew rate，进行增益变化时的平滑操作。

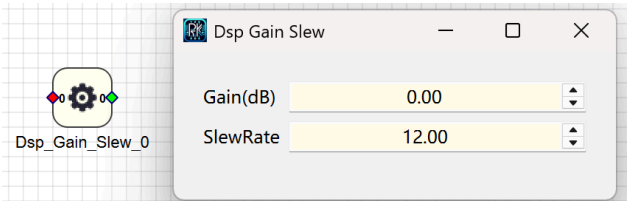


图 2.2.2-1 Gain (Slew) 模块和设置界面

参数数量：2

参数名	参数位	类型	功能	单位	建议取值	默认取值
Gain	0	float	增益取值	db	-138 ~ 100	0
SlewRate	1	float	增益修改平滑速度，slew越大变化越慢	无	1 ~ 23	12

实际效果展示：
输入数据置为1的直流信号，有如下两个典型工况：
工况一：参数按顺序设置为:-12,8， 即gain设置为-12dB，slew rate为8；
工况二：参数按顺序设置为:-6,8， 即gain设置为0.5dB， slew rate为8。

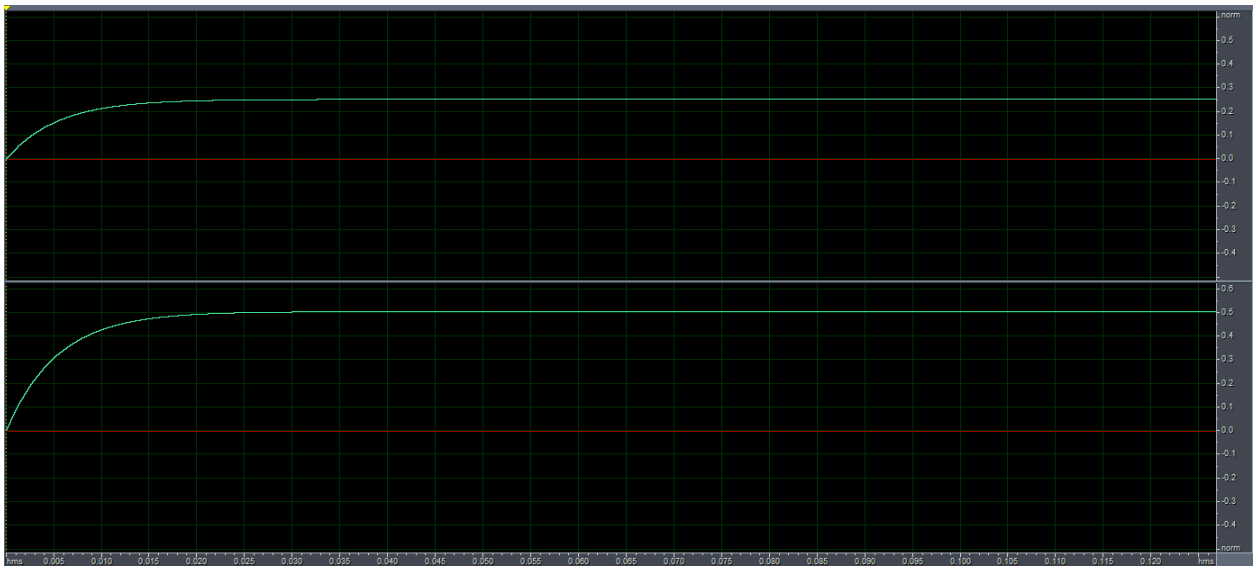


图 2.2.2-2 RKStudio输出时域，工况一为上图，工况二为下图

2.2.3 Gain (Slew Double)

功能说明：将输入信号乘以增益后输出，输入框为增益值单位为DB，会将DB转为线性增益*并传给算法使用。会根据设置的RampUP和RampDown，进行增益变化时的平滑操作。RampUP为增益上升时的平滑系数，RampDown为增益下降时的平滑系数。

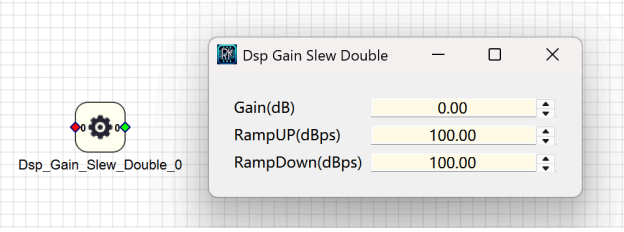


图 2.2.3-1 Gain (Slew Double) 模块和设置界面

参数数量：3

参数名	参数位	类型	功能	单位	建议取值	默认取值
Gain(dB)	0	float	增益取值	db	-138 ~ 100	0
RampUP(dBps)	1	float	增益上升速度	dB/s	1 ~ 1000	200
RampDown(dBps)	2	float	增益下降速度	dB/s	1 ~ 1000	200

实际效果展示：
输入数据置为0.5的直流信号，有如下两个典型工况,可以看到平滑系数的作用：

工况一：参数按顺序设置为:6,200,200， 即gain设置为6，单位为db, RampUP为200dB/s , RampDown为200dB/s ；

工况二：参数按顺序设置为:6,400,400， 即gain设置为6，单位为db, RampUP为400dB/s , RampDown为400dB/s 。

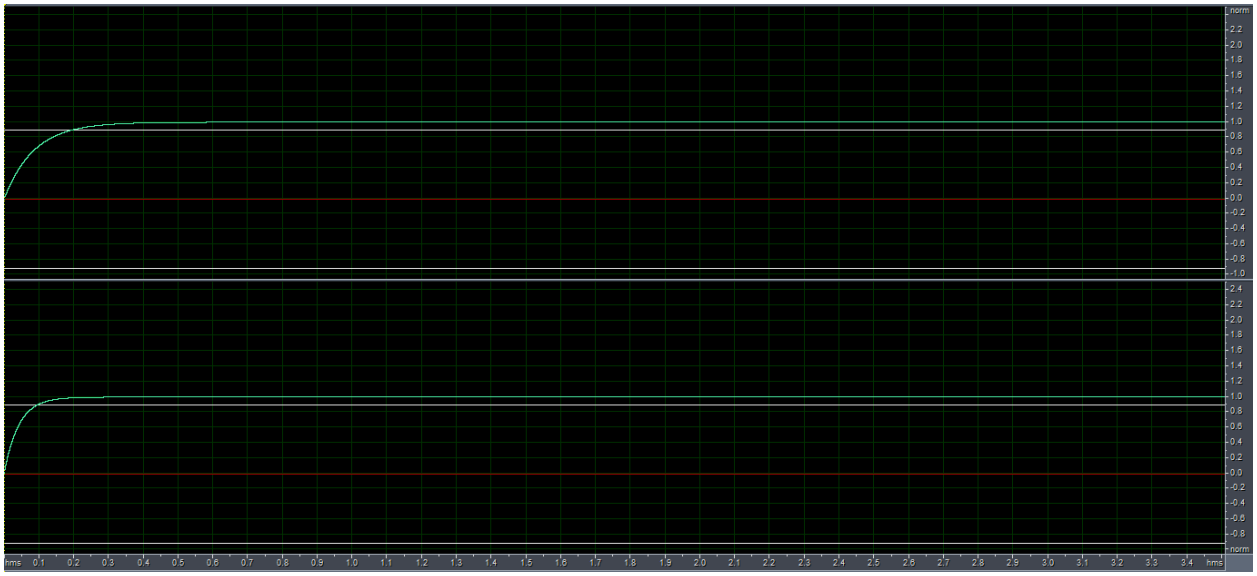


图 2.2.3-2 RKStudio输出时域，工况一为上图，工况二为下图

2.2.4 Gain External Mute (Slew Double)

功能说明：参数设置和功能性和与2.2.3 Gain (Slew Double)一致，dB增益也会转换成线性增益*传入算法计算，这里不作更多说明。主要区别是第一通道是一个外部控制信号，用来控制输出的通道是否mute。如果信号控制信号不为0，那么就会mute输出，否则输出正常的gain计算。

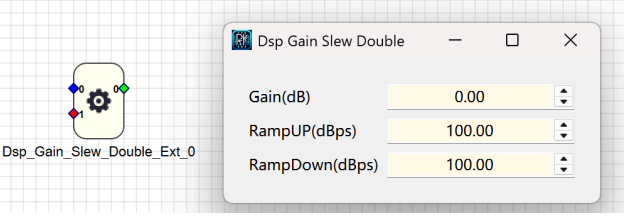


图 2.2.4-1 Gain_Slew_EXT 模块和设置界面

2.3 Dsp Function

2.3.1 Delay

功能说明：通过设置的delay值，输入信号延时delay个点或者delay_ms时间后输出。

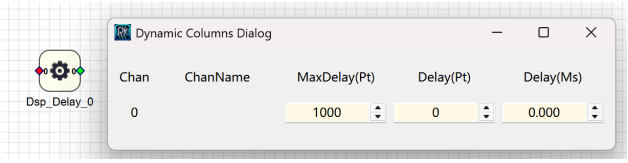


图 2.3.1-1 Delay 模块和设置界面

参数数量：3

参数名	参数位	类型	功能	单位	建议取值	默认取值
MaxDelay	0	float	最大延时	sample（点数）	1 ~ 20000	1000
Delay	1	float	延时取值	sample（点数）	0 ~ MaxDelay	0
Delay_ms	2	float	延时取值	ms	0 ~ MaxDelay/Fs	0

注：第二个参数Delay（单位点数） 和 第三个参数delay_ms （单位ms）是关联的，设置Delay（ 点数）时，会自动计算成延时的时间Delay_ms；反之，设置delay_ms时也会自动计算延时的点数Delay

实际效果展示：
输入数据置为1的直流信号, 有如下两个典型工况：
工况一：参数按顺序设置为:1500,500，--；即最大延时设置为1500，当前延时设置为500，延时单位为samples；
工况二：参数按顺序设置为:1500,--,30； 即最大延时设置为1500，当前延时设置为30，延时单位为ms。

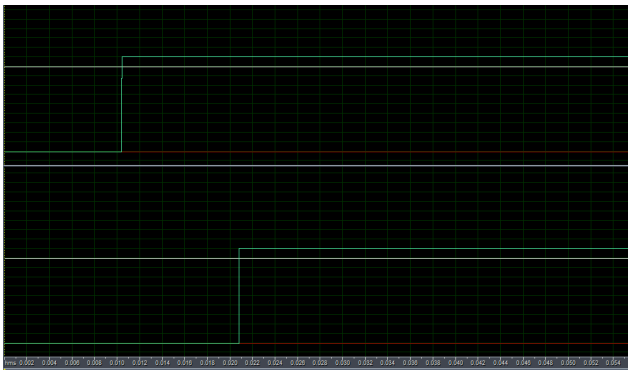


图 2.3.1-2 RKStudio输出时域，工况一为上图，工况二为下图
可以看到，在48k采样率下，延时为10ms和30ms，符合预期。

2.3.2 Multi-Delay

功能说明：通过设置的delay值，输入信号延时delay个点后输出，每路的延时可以单独控制。

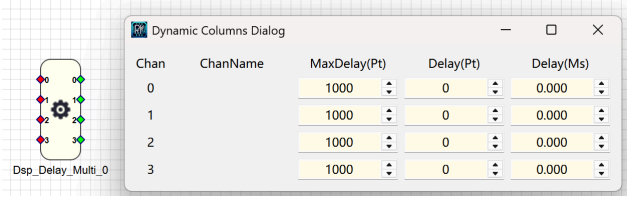


图 2.3.2-1 Multi-Delay 模块和设置界面

参数数量：输入通道数*2

参数名	参数位	类型	功能	单位	建议取值	默认取值
MaxDelay	0	float	最大延时	sample（点数）	1 ~ 20000	1000
Delay	1	float	延时取值	sample（点数）	0 ~ MaxDelay	0
Delay_ms	2	float	延时取值	ms	0 ~ MaxDelay/Fs	0
...						

右键控件，add point 可添加通道数

注：第二个参数Delay（单位点数） 和 第三个参数delay_ms （单位ms）是关联的，设置Delay（ 点数）时，会自动计算成延时的时间Delay_ms；反之，设置delay_ms时也会自动计算延时的点数Delay

运行效果参考Delay模块。

2.3.3 Multi Delay (Index Selectable)

功能说明：索引式多通道延时。与普通 Multi Delay 不同，该模块允许预先配置多组延时参数，通过改变 Index 参数的值，从预设列表中选择当前使用的延时配置。

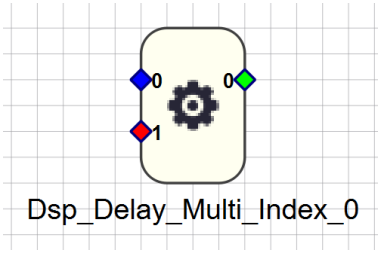


图 2.3.3-1 Multi Delay (Index Selectable) 控件

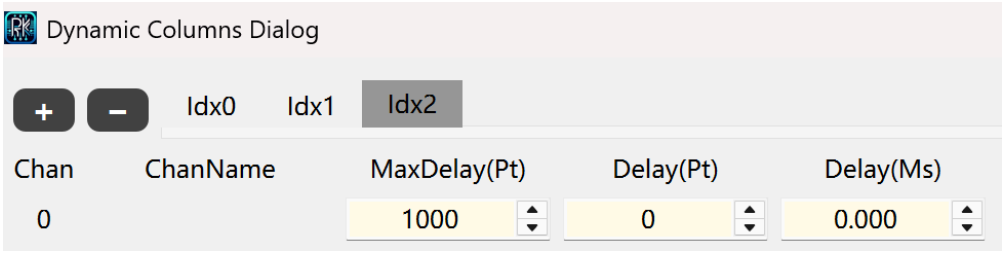


图 2.3.3-2 Multi Delay (Index Selectable) 参数界面

2.3.4 Multi Delay Slew (Index Selectable)

功能说明：带平滑过渡的索引式多通道延时。功能与 Multi Delay (Index Selectable) 类似，但增加了 Slew 功能。当 Index 发生变化时，会有个平滑作用，使得切换过程不会太突兀。

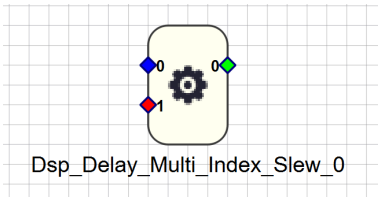


图 2.3.4-1 Multi Delay Slew(Index Selectable) 控件

参数设置界面同Multi Delay (Index Selectable)

2.3.5 DSP Feedback

功能说明：回环通路，将输出信号反馈回去，作为插件的输入信号，可以是当前插件的输出信号，也可以是其他插件的输出信号。

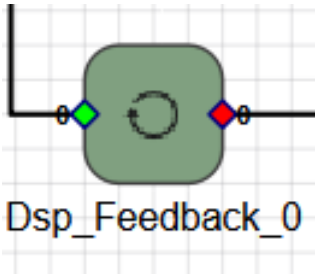


图 2.3.5-1 Feedback 控件

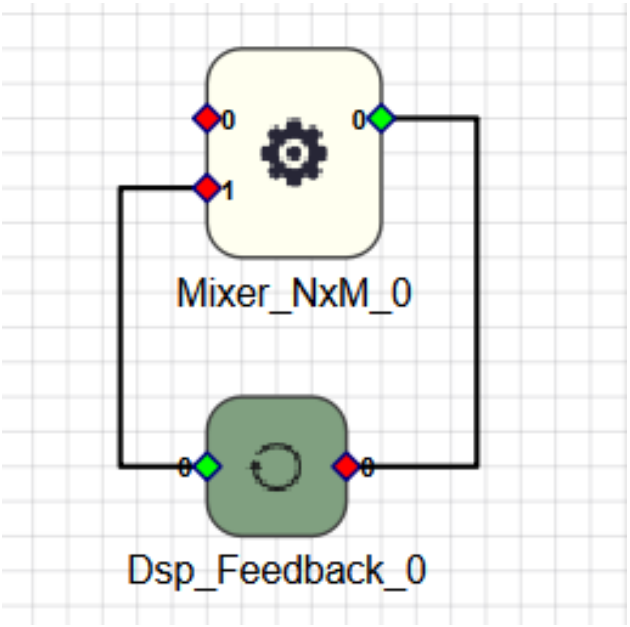


图 2.3.5-2 Feedback 连接示意图

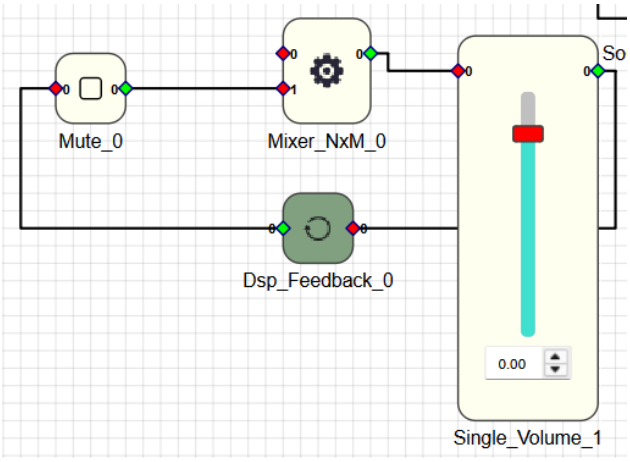


图 2.3.5-3 Feedback 连接示意图

2.4 Logic

2.4.1 AbsMax

功能说明：计算所有输入通道的绝对值的最大值。

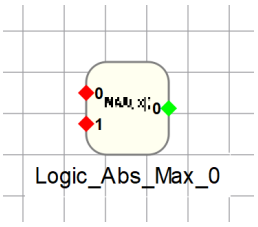


图 2.4.1-1 AbsMax 控件

参数：无

2.4.2 Max

功能说明：计算所有输入通道的最大值。

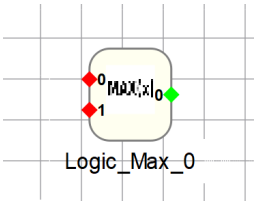


图 2.4.2-1 Max 控件

参数：无

2.4.3 Min

功能说明：计算所有输入通道的最小值。

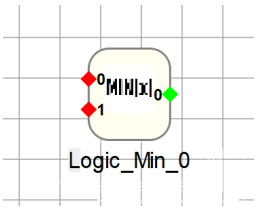


图 2.4.3-1 Min 控件

参数：无

2.4.4 Logic AB

功能说明：通过比较0/1通道的值，并输出对应的0/1通道值，有如下5种情况：

```
0 > 1  ?  0 : 1
0 < 1  ?  0 : 1
0 ≥ 1  ?  0 : 1
0 ≤ 1  ?  0 : 1
0 == 1  ?  0 : 1
```

以第一个case为例，如果第0通道的数据大于第一通道的数据，那么输出第0通道的数据，否则输出第1通道的数据。

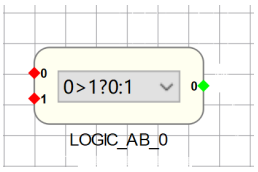


图 2.4.4-1 Logic AB 控件

参数：无

2.4.5 Logic ABCD

功能说明：通过比较0/1通道的值，并输出对应的3/4通道值，有如下5种情况：

```
0 > 1  ?  2 : 3
0 < 1  ?  2 : 3
0 ≥ 1  ?  2 : 3
0 ≤ 1  ?  2 : 3
0 == 1  ?  2 : 3
```

以第一个case为例，如果第0通道的数据大于第1通道的数据，那么输出第2通道的数据，否则输出第3通道的数据。

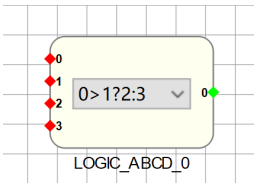


图 2.4.5-1 Logic ABCD 控件

参数：无

2.4.6 Zero Comparator

功能说明：输入值如果等于0，则输出设置的参数值，否则输出0。

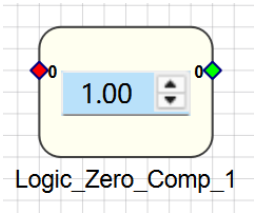


图 2.4.6-1 Logic Zero Compartor 控件

参数：1

2.4.7 Value Hold

功能说明：数值保持器。通常用于“锁存”或“采样保持”当前的输入值。它可以将瞬间的输入信号保持住，以便后续逻辑处理或观察。

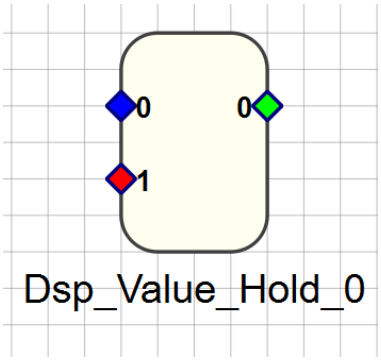


图 2.4.7-1 Value Hold 控件

参数：无

2.4.8 AbsMax Value Hold

功能说明：计算并保持所有输入通道的历史绝对值最大值。

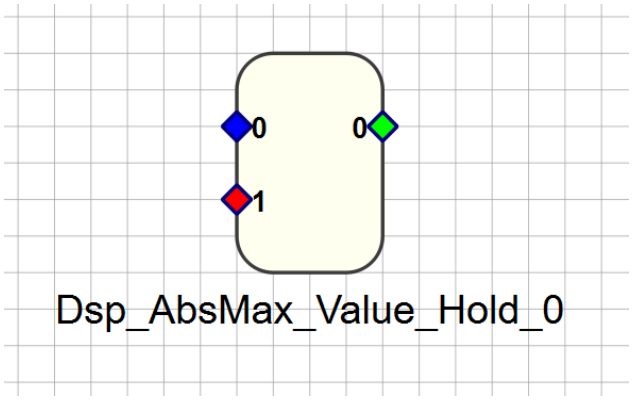


图 2.4.8-1 AbsMax Value Hold 控件

参数：无

2.4.9 Max Value Hold

功能说明：计算并保持所有输入通道的历史最大值。

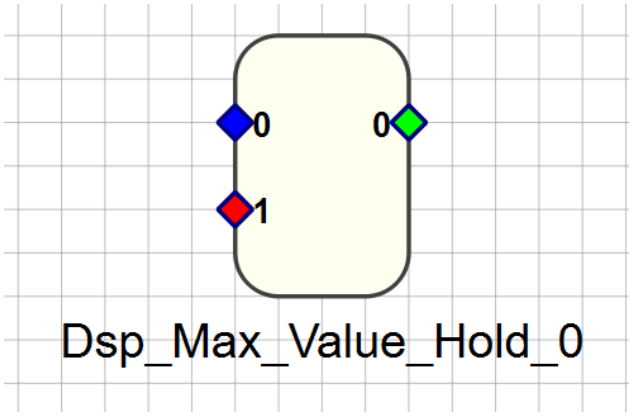


图 2.4.9-1 Max Value Hold 控件

参数：无

2.4.10 Min Value Hold

功能说明：计算并保持所有输入通道的历史最小值。

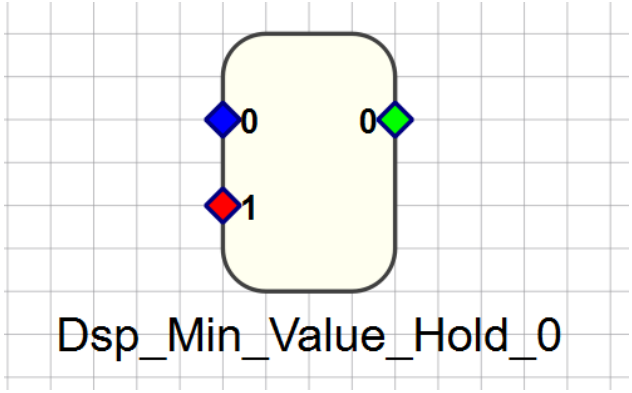


图 2.4.10-1 Min Value Hold 控件

参数：无

3 Dynamics Processors

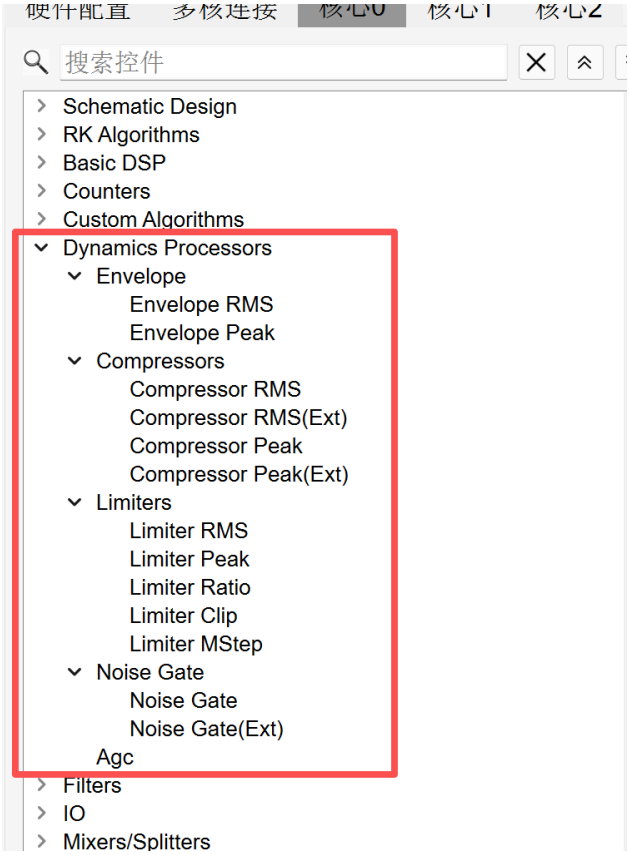


图 3-1 Dynamics Processors位于工具的位置

3.1 Envelope

3.1.1 Envelope RMS

功能说明：平滑跟踪数据的RMS值。

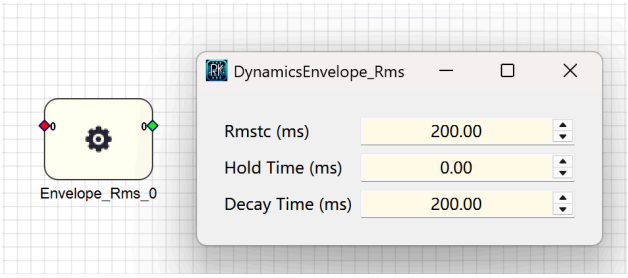


图 3.1.1-1 Envelope RMS 控件与设置界面

参数数量：3

参数名	参数位	类型	功能	单位	建议取值	默认取值
Rmstc（ms）*	0	float	信号变化的响应速度	ms	0 ~ 2000	200
Hold Time（ms）*	1	float	包络下降前保持当前峰值的持续时间	ms	0 ~ 2000	0
Decay Time（ms）*	2	float	增益下降速度	ms	0 ~ 2000	200

实际效果展示：
输入数据白噪声, 有如下两个典型工况：
工况一：Rmstc=20、Hold Time=10、Decay Time=10；
工况二：Rmstc=50、Hold Time=20、Decay Time=20。

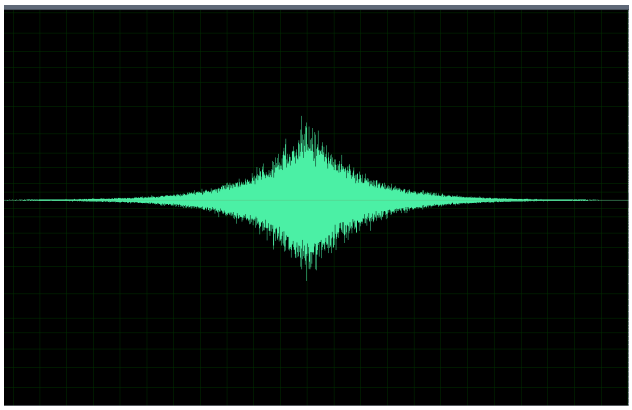


图 3.1.1-2 输入音源图

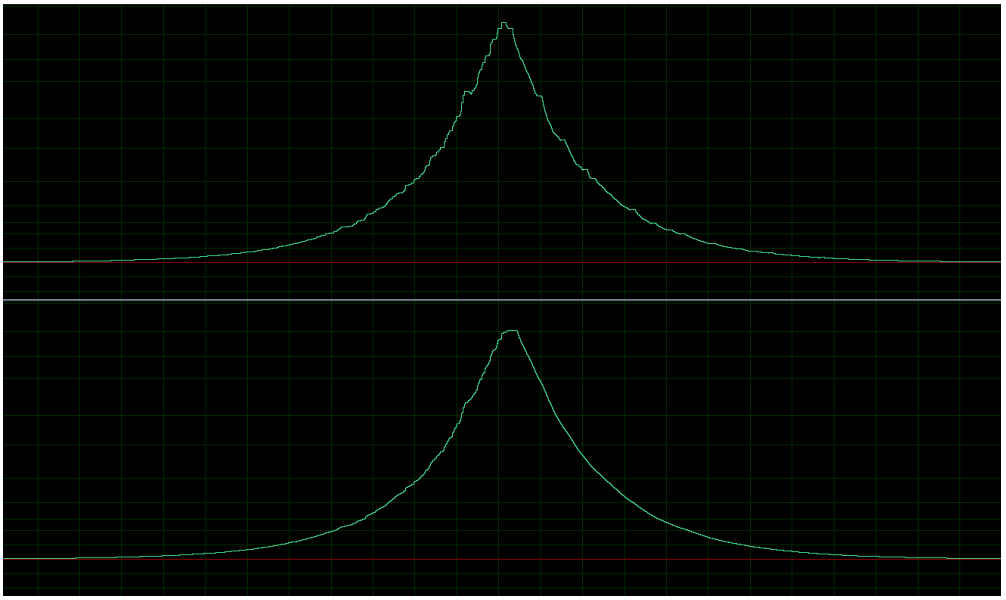


图 3.1.1-3 工况一输出时域(上半图)和工况二输出时域(下半图)

3.1.2 Envelope Peak

功能说明：平滑跟踪数据的Peak值。

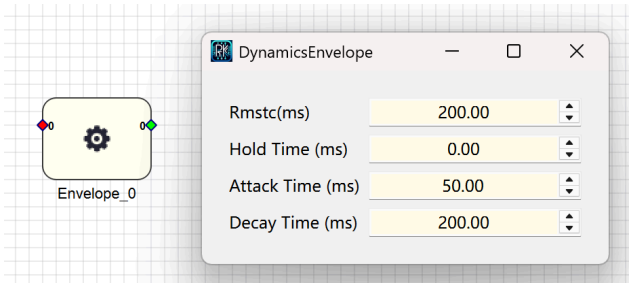


图 3.1.2-1 Envelope Peak 控件与设置界面

参数数量：4

参数名	参数位	类型	功能	单位	建议取值	默认取值
Rmstc（ms）*	0	float	信号变化的响应速度	ms	0 ~ 2000	200
Hold Time（ms）*	1	float	包络下降前保持当前峰值的持续时间	ms	0 ~ 2000	0
Attack Time（ms）*	2	float	包络上升速度	ms	0 ~ 2000	50
Decay Time（ms）*	3	float	包络下降速度	ms	0 ~ 2000	200

实际效果展示：
输入数据白噪声, 有如下两个典型工况：
工况一：Hold Time=10、Decay Time=10；
工况二：Hold Time=50、Decay Time=20；

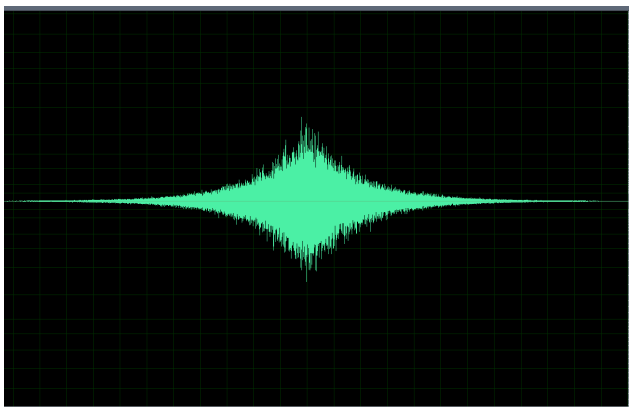


图 3.1.2-2 输入音源图

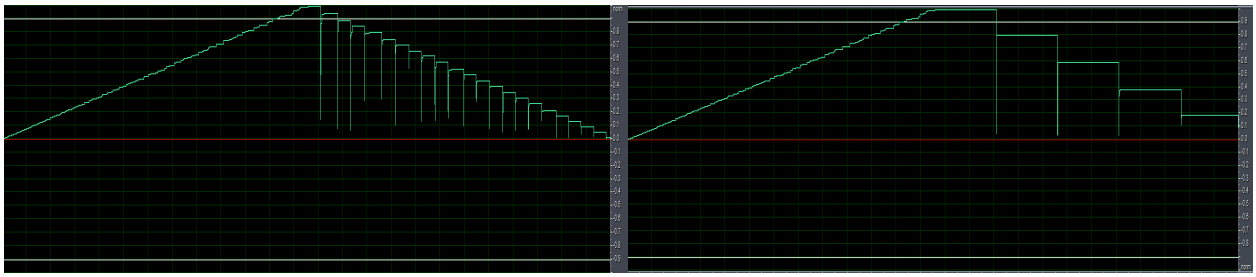


图 3.1.2-3 工况一输出时域(左图)和工况二输出时域(右图)

3.2 Compressors

3.2.1 Compressor Rms

功能说明：压缩器，根据设定的映射曲线调整音频输出范围，Compressor Rms模块根据能量进行曲线映射。双击空白处可进行平滑参数设置，单击按钮可进行映射曲线设置。

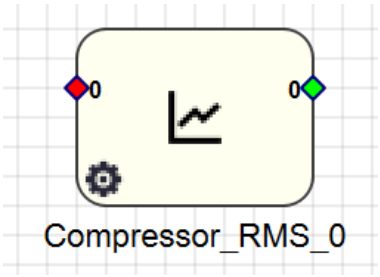


图 3.2.1-1 Compressor Rms 控件

参数数量：6

参数名	参数位	类型	功能	单位	建议取值	默认取值
Rmstc（ms）*	0	float	平滑跟踪信号变化的速度，该值越大变化越慢	ms	0 ~ 2000	20
Hold Time（ms）*	1	float	包络下降前保持当前峰值的持续时间	ms	0 ~ 2000	0
Attack Time（ms）*	2	float	增益上升速度	ms	0 ~ 2000	200
Release Time（ms）*	3	float	增益下降速度	ms	0 ~ 2000	200
Soft Knee Width(dB)	4	float	软拐点宽度，设为0表示硬拐点	dB	0 ~ 100	0
Delay Time（ms）*	5	float	延时时间	ms	0 ~ 100	0

实际效果展示：
输入数据为音乐声和冲击声，相关case配置如下：

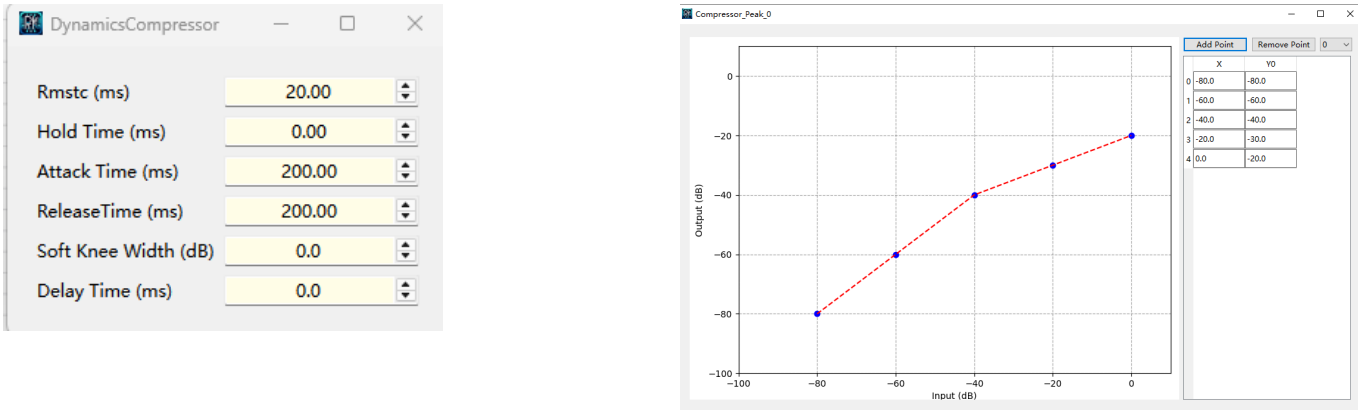


图 3.2.1-2 Compressor Rms 参数(左图)和Compressor Rms 曲线(右图)

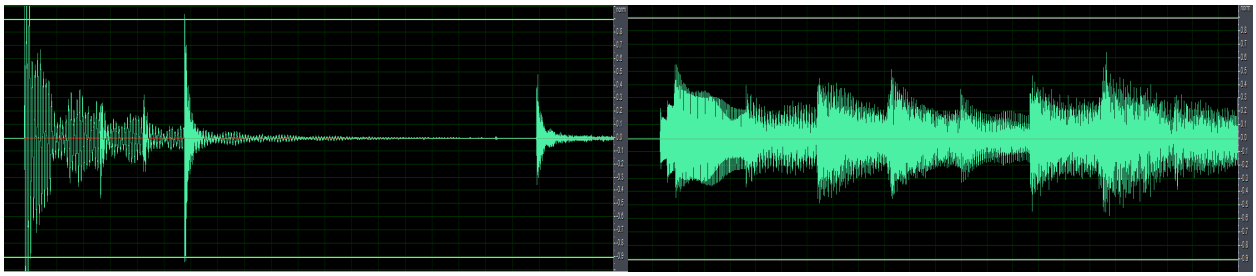


图 3.2.1-3 算法输入：冲击声时域(左图)和Music时域(右图)

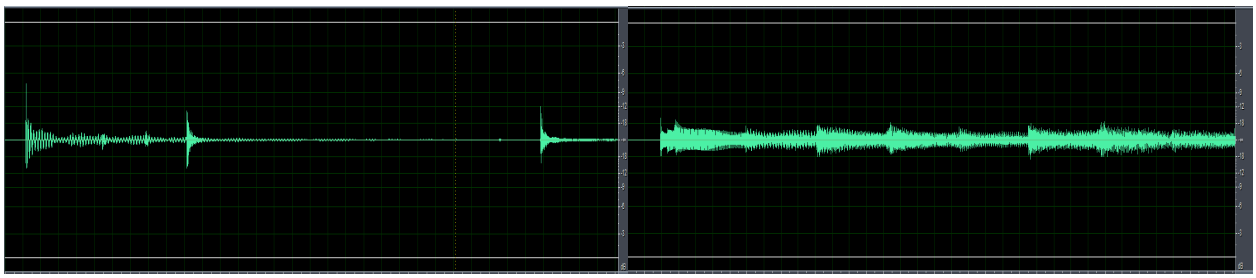


图 3.2.1-4 算法输出：冲击声输出时域(左图)和Music输出时域(右图)

可以从图 3.2.1-2右图看出，我们设置的曲线是
对于（-80dB ~ -40dB）的信号强度输入，保持原样输出;
对于（-40dB ~ 0dB）的信号强度的映射到（-40dB ~-20dB）的强度输出。
从时域的结果上看是低幅度的波形保持原样输出，高幅度的波形被衰减。

3.2.2 Compressor Rms External Input

功能说明：压缩器，根据设定的映射曲线调整音频输出范围， Compressor Rms Ext模块根据ext输入的峰值进行曲线映射。双击控件空白处可进行平滑参数设置，单击按钮可进行映射曲线设置。蓝色的为输入控制信号，红色的为实际受控信号。

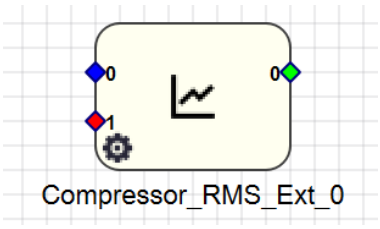


图 3.2.2-1 Compressor Ext Input Rms 控件

具体参数和效果可以参考3.2.1 Compressor Rms*。

3.2.3 Compressor Peak

功能说明：压缩器，根据设定的映射曲线调整音频输出范围，peak模块根据峰值进行曲线映射。双击空白处可进行平滑参数设置，单击按钮可进行映射曲线设置。

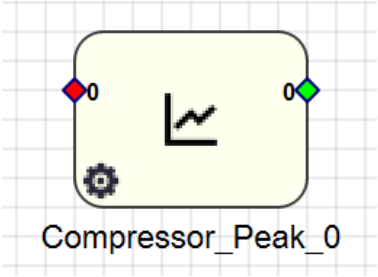


图 3.2.3-1 Compressor Peak 控件

参数数量：5

参数名	参数位	类型	功能	单位	建议取值	默认取值
Hold Time（ms）*	0	float	包络下降前保持当前峰值的持续时间	ms	0 ~ 2000	0
Attack Time *	1	float	增益上升速度	ms	0 ~ 2000	200
Release Time (ms)*	2	float	增益下降速度	ms	0 ~ 2000	200
Soft Knee Width(dB)	3	float	软拐点宽度，设为0表示硬拐点	dB	0 ~ 100	0
Delay Time（ms）*	4	float	延时时间	ms	0 ~ 100	0

实际效果展示：
输入数据为音乐声和冲击声, 相关case配置如下：

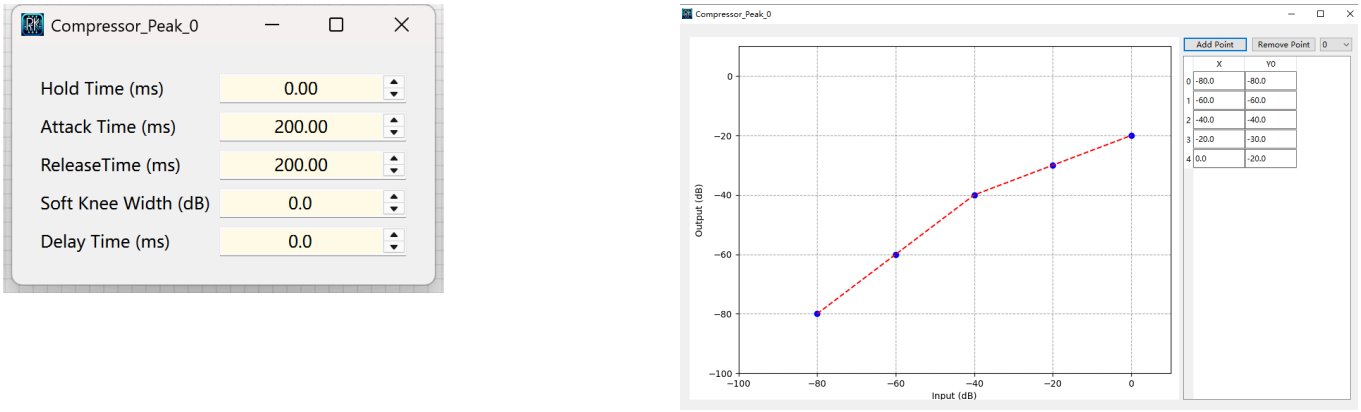


图 3.2.3-2 Compressor Peak 参数(左图)和Compressor Peak 曲线(右图)

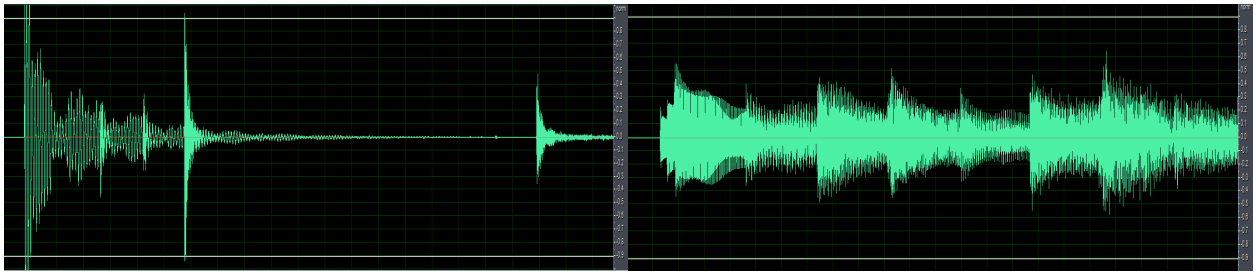
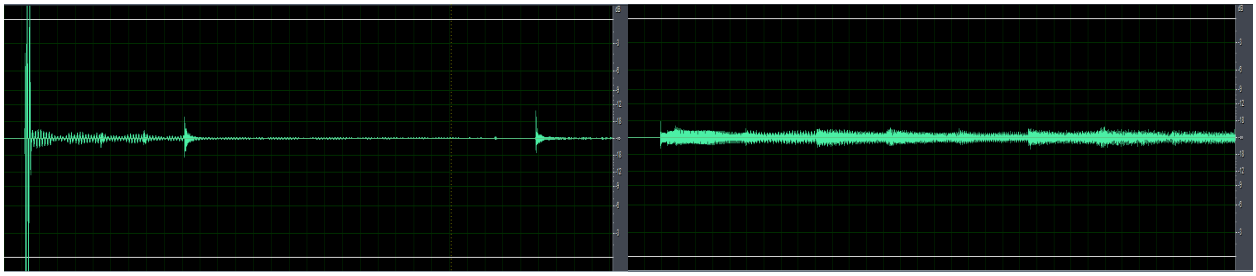


图 3.2.3-3 算法输入：冲击声时域(左图)和Music时域(右图)

图 3.2.3-4 算法输出：冲击声输出时域(左图)和Music输出时域(右图)

可以从图3.2.3-2 右图看出，我们设置的曲线是

对于（-80dB ~ -40dB）的信号峰值输入，保持原样输出;



对于（-40dB ~ 0dB）的信号峰值的映射到（-40dB ~-20dB）的峰值输出。

从时域的结果上看是低幅度的波形保持原样输出，高幅度的波形被衰减。

3.2.4 Compressor Peak External Input

功能说明：压缩器，根据设定的映射曲线调整音频输出范围， Compressor Peak Ext模块根据ext输入的峰值进行曲线映射。双击控件空白处可进行平滑参数设置，单击按钮可进行映射曲线设置。蓝色的为输入控制信号，红色的为实际受控信号。

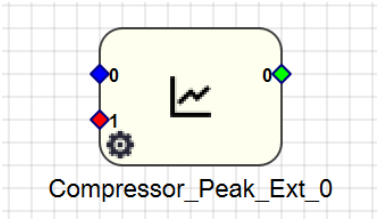


图 3.2.4-1 Compressor Peak EXT 控件

具体效果可以参考3.2.3 Compressor Peak * 。

3.3 Limiter

3.3.1 Limiter RMS

功能说明：限幅器，控制调整音频输出的最大值在固定阈值之下。

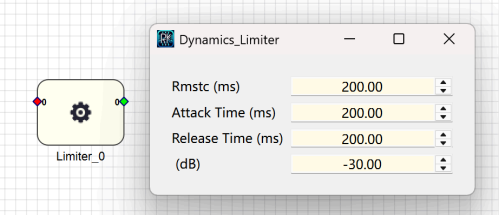


图 3.3.1-1 Limiter RMS 控件

参数数量：4

参数名	参数位	类型	功能	单位	建议取值	默认取值
Rmstc（ms）*	0	float	平滑跟踪信号变化的速度，该值越大变化越慢	ms	0 ~ 2000	200
Attack Time（ms）*	1	float	增益上升速度	ms	0 ~ 2000	200
Release Time（ms）*	2	float	增益下降速度	ms	0 ~ 2000	200
Threshold (dB)	3	float	门限，输入高于该值会被限制在该值之下	dB	-80 ~ 0	-30

实际效果展示：
输入数据为音乐声和冲击声, 相关case配置如下：

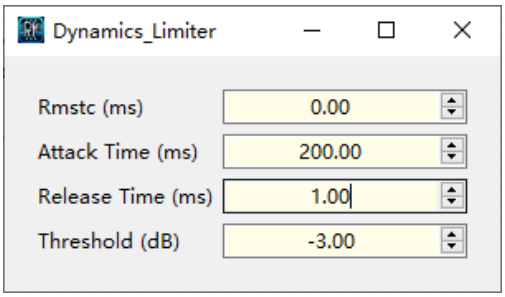


图 3.3.1-2 Limiter 参数设置

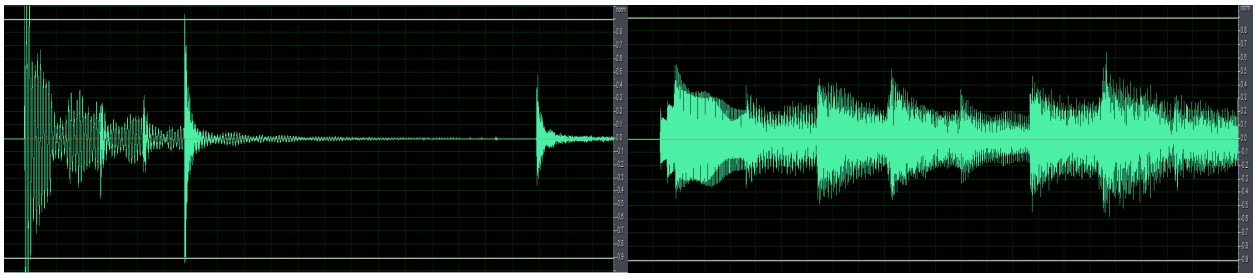


图 3.3.1-3 算法输入：冲击声时域(左图)和Music时域(右图)

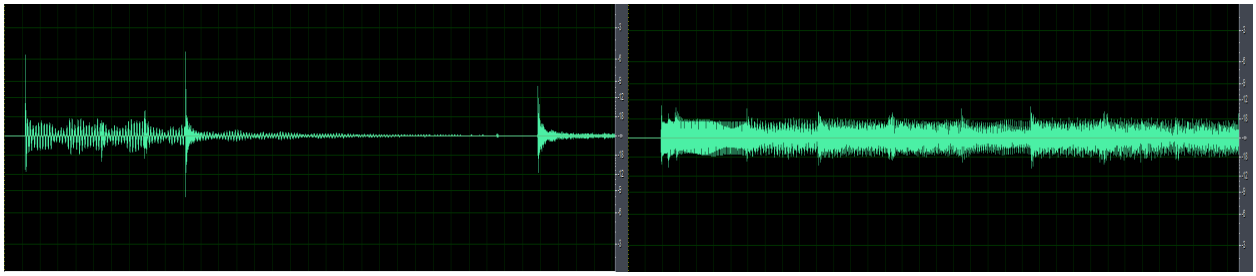


图 3.3.1-4 算法输出：冲击声输出时域(左图)和Music输出时域(右图)

3.3.2 Limiter Peak

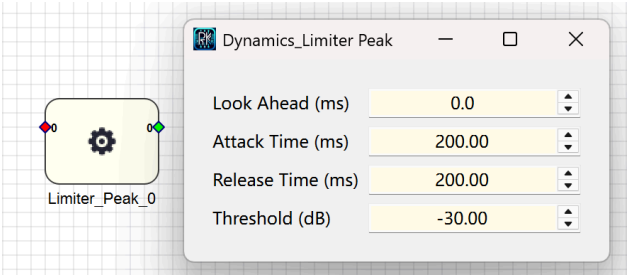


图 3.3.2-1 Limiter Peak控件

参数数量：4

参数名	参数位	类型	功能	单位	建议取值	默认取值
Look Ahead (ms)*	0	float	超前观察时间，指对当前点之后点数进行观测	ms	0 ~ 100	10
Attack Time（ms）*	1	float	增益上升速度	ms	0 ~ 2000	200
Release Time（ms）*	2	float	增益下降速度	ms	0 ~ 2000	200
Threshold (dB)	3	float	门限，输入高于该值会被限制在该值之下	dB	-80 ~ 0	-30

注：为解释参数的意义，以下举例对比：

该控件以下结果的输入信号为：

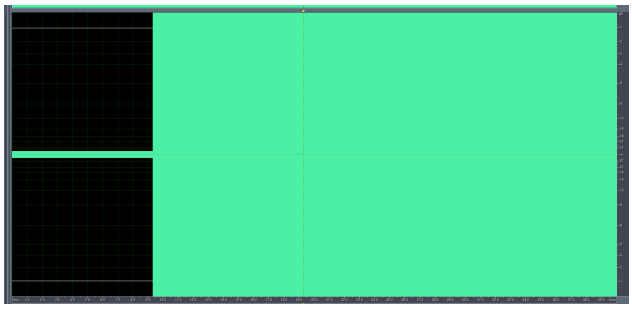


图 3.3.2-2 输入信号为幅值从-32.70db跳变为0db的正弦波

- look ahead 超前观测，本质上为一个延时器，对当前信号的之后的信号观察，值取越大，输出的幅值变化越缓,对冲激信号的抑制越强。**look ahead会增加输出延时**

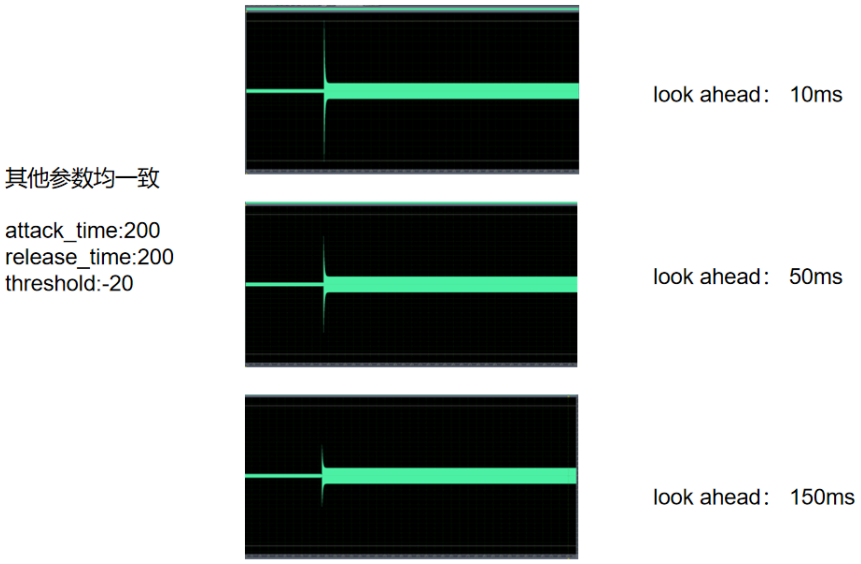


图 3.3.2-3 不同look_ahead对结果的影响

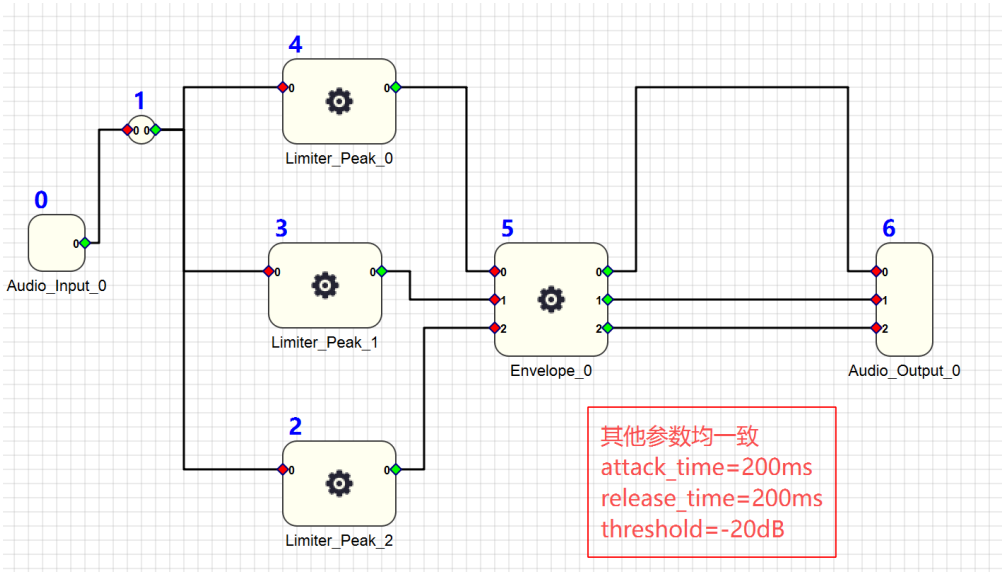


图 3.3.2-4 look_ahead对比测试原理图

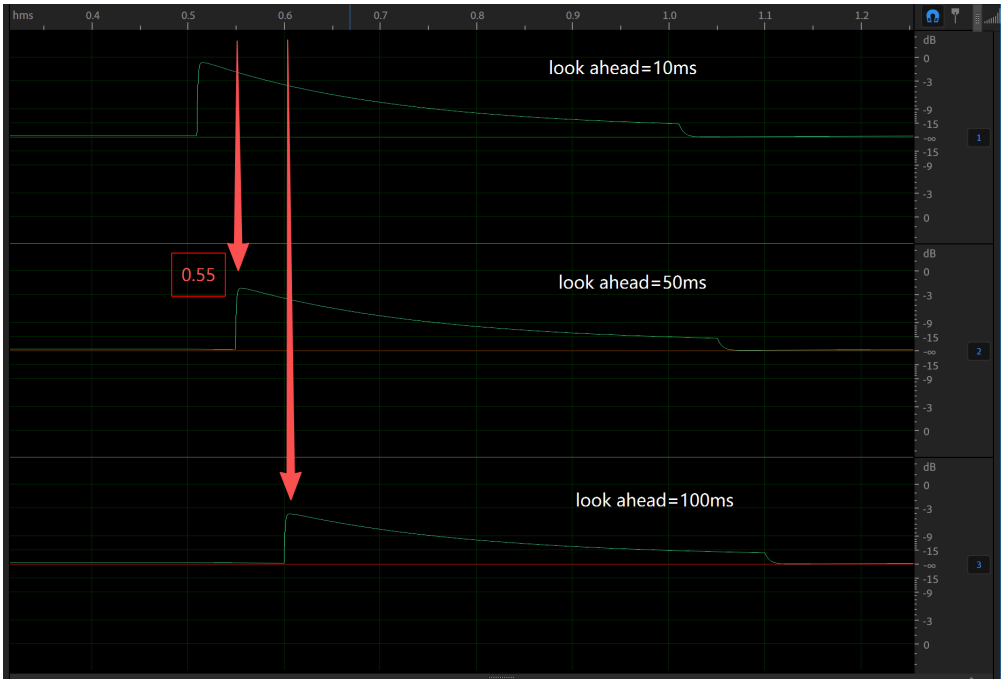


图 3.3.2-5 不同look_ahead对结果的影响

- attack time可理解为从不限制到限制的变化时间，当有一个信号在阈值一下突变高于阈值时（信号值变化），输出会短暂地高于阈值输出，但有很快将幅值限制在门限以下。

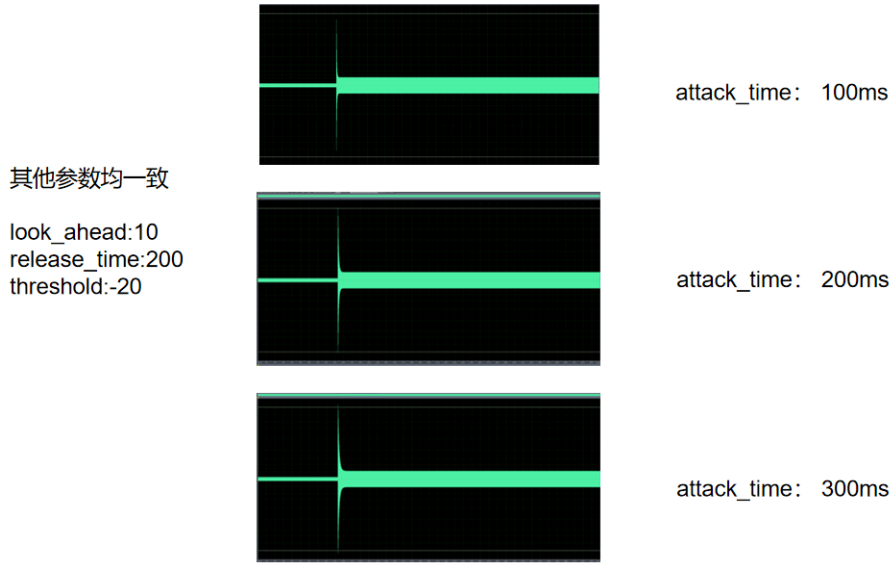


图 3.3.2-6 不同attack_time对结果的影响

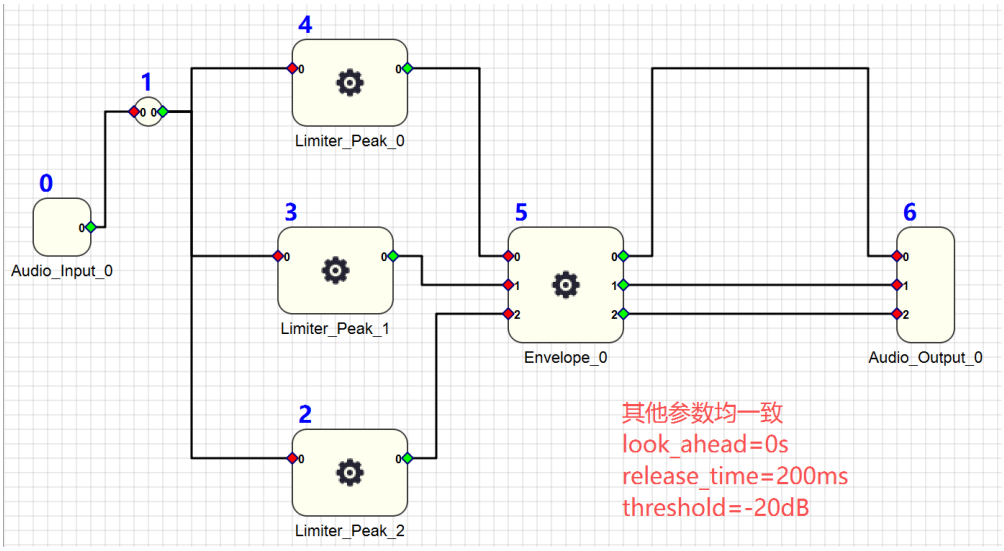


图 3.3.2-7 attack_time对比测试原理图

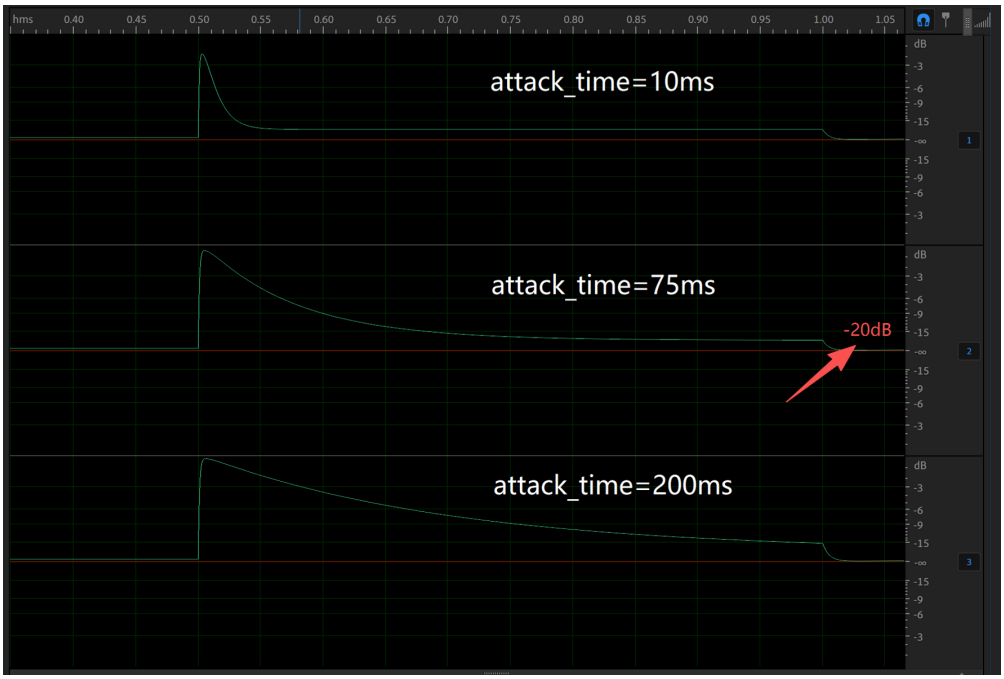


图 3.3.2-8 不同attack_time对结果的影响

- release time 可理解为从限制到不限制的变化时间，有一个信号，而门限值从低于信号值突变为高于信号值（阈值变化），输出会缓慢变大，然后很快与信号值持平。

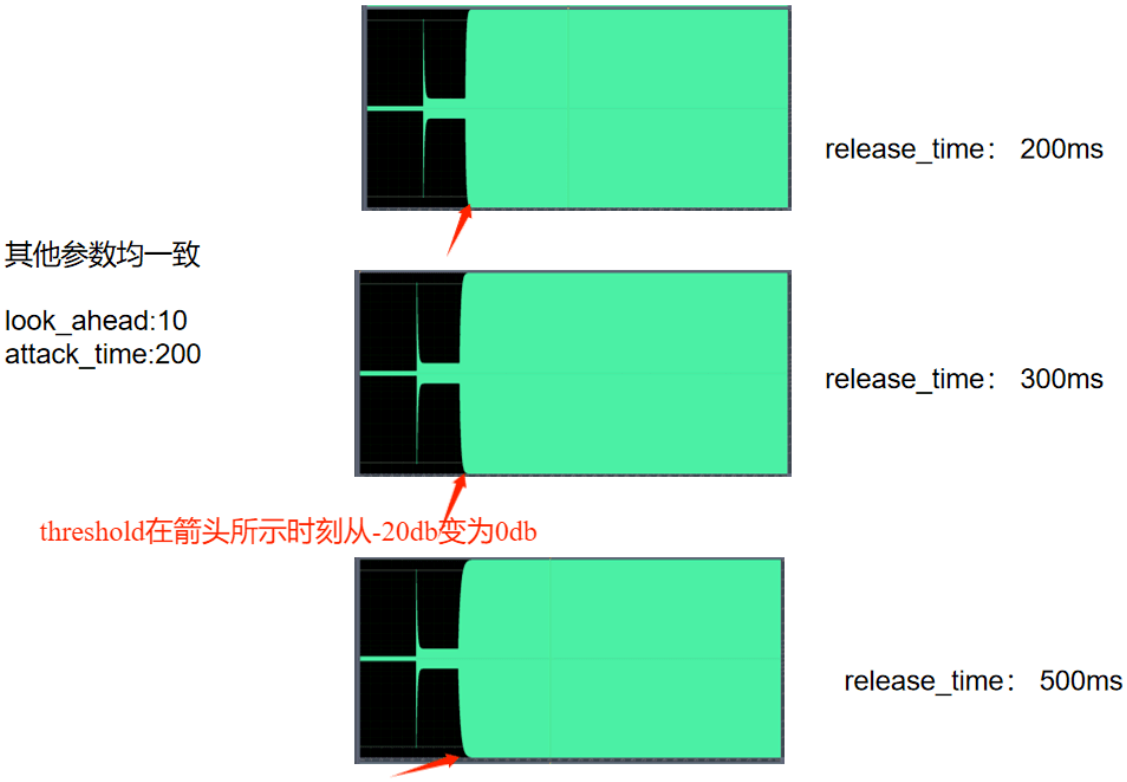


图 3.3.2-9 不同release_time对结果的影响

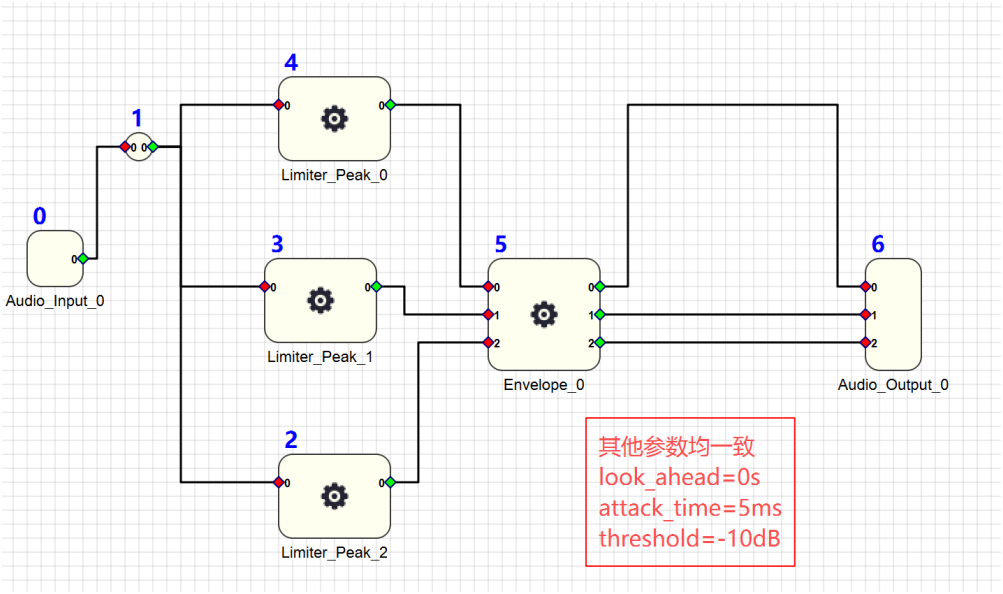


图 3.3.2-10 release_time对比测试原理图

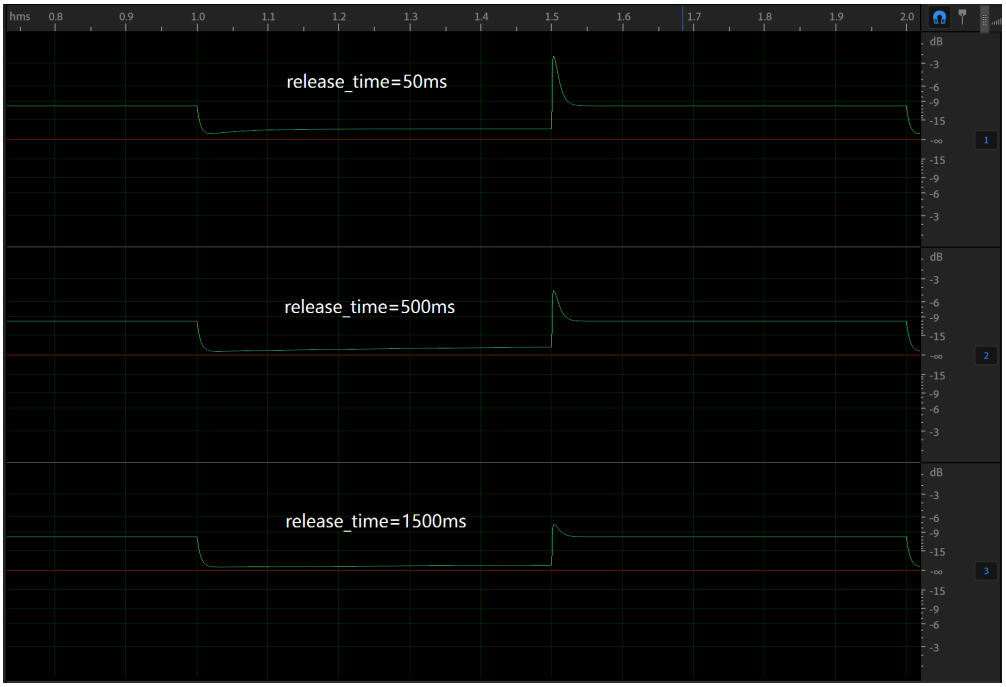


图 3.3.2-11 不同release_time对结果的影响

3.3.3 Limiter Ratio

功能说明：限幅器，控制调整音频输出的最大值, 当超过阈值之后，压缩比例x/y为参数ratio（dB域）。

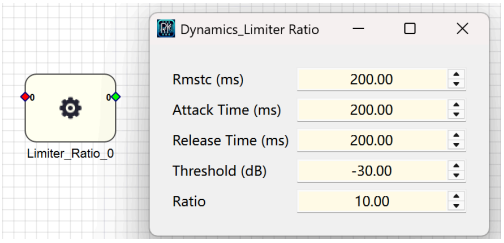


图 3.3.3-1 Limiter Ratio控件

参数数量：5

参数名	参数位	类型	功能	单位	建议取值	默认取值
Rmstc（ms）*	0	float	平滑跟踪信号变化的速度，该值越大变化越慢	ms	0 ~ 2000	200
Attack Time（ms）*	1	float	增益上升速度	ms	0 ~ 2000	200
Release Time（ms）*	2	float	增益下降速度	ms	0 ~ 2000	200
Threshold(dB)	3	float	门限，输入高于该值会被限制在该值之下	dB	-80 ~ 0	-30
Ratio	4	float	压缩比例，x/y的值，ratio越大压缩效果越好	无	1~10000	10

实际效果展示：
输入数据为音乐声和冲击声, 相关case配置如下：

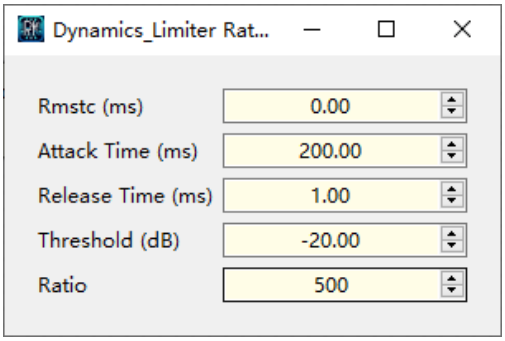


图 3.3.3-2 Limiter Ratio参数设置

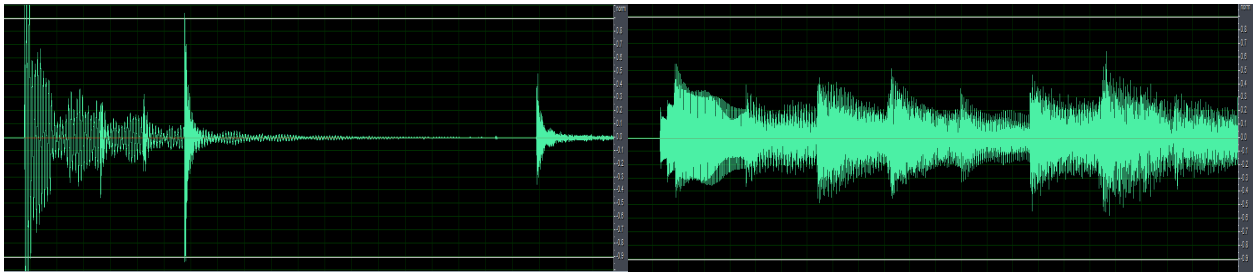


图 3.3.3-3 算法输入：冲击声时域(左图)和Music时域(右图)

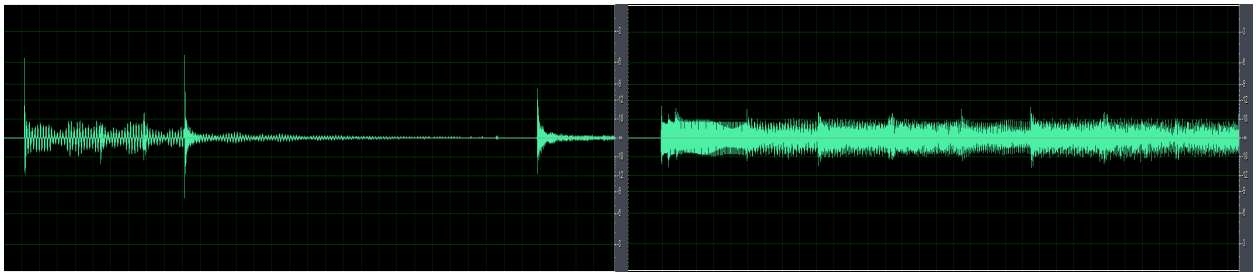


图 3.3.3-4 算法输出：冲击声输出时域(左图)和Music输出时域(右图)

3.3.4 Limiter Clip

功能说明：当输入控制信号为1时，压缩阈值缓慢下降，直到门限值；当控制信号为0时，压缩阈值缓慢上升，直到0。

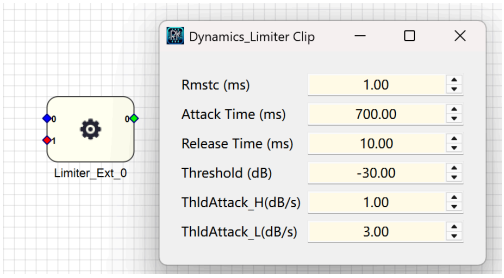


图 3.3.4-1 Limiter Clip控件

参数数量：6

参数名	参数位	类型	功能	单位	建议取值	默认取值
Rmstc（ms）*	0	float	平滑跟踪信号变化的速度，该值越大变化越慢	ms	0 ~ 2000	1
Attack Time (ms) *	1	float	增益上升速度	ms	0 ~ 2000	700
Release Time（ms）*	2	float	增益下降速度	ms	0 ~ 2000	10
Threshold	3	float	初始阈值，也是阈值变化调节过程中的最低阈值	dB	-80 ~ 0	-30
ThldAttack_H(dB/s) *	4	float	阈值上升速度	dB/s	0 ~ 2000	1
ThldAttack_L(dB/s) *	5	float	阈值下降速度	dB/s	0 ~ 2000	3

具体效果可以参考3.3.1 Limiter RMS。

3.4 Noise Gate

3.4.1 Noise Gate

功能说明：噪声门，当信号输入低于门限时，进行压缩；当信号输入高于门限时，输出为输入信号。

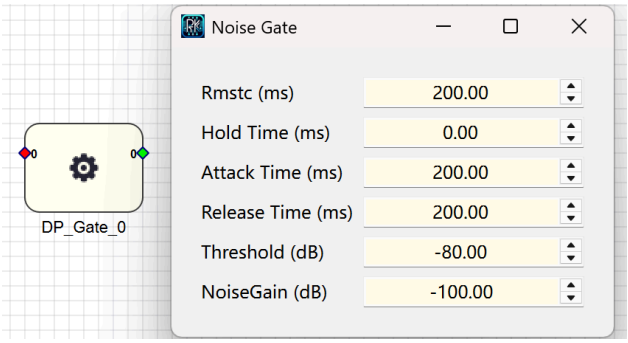


图 3.4.1-1 Noise Gate 控件与设置界面

参数数量：6

参数名	参数位	类型	功能	单位	建议取值	默认取值
Rmstc（ms）*	0	float	平滑跟踪信号变化的速度，该值越大变化越慢	ms	0 ~ 2000	200
Hold Time（ms）*	1	float	包络下降前保持当前峰值的持续时间	ms	0 ~ 2000	0
Attack Time（ms）*	2	float	增益上升速度	ms	0 ~ 2000	200
Release Time（ms）*	3	float	增益下降速度	ms	0 ~ 2000	200
Threahold（dB）*	4	float	噪声门生效阈值	dB	-100 ~ 0	-80
Noise Gain（dB）*	5	float	噪声实际的衰减值	ms	-200 ~ 0	-100

注意与其他的动态处理会输入原始的db参数不同，这里的Threshold和Gain会转换为线性值，Threshold用的是rms的db，Gian用的是peak的db

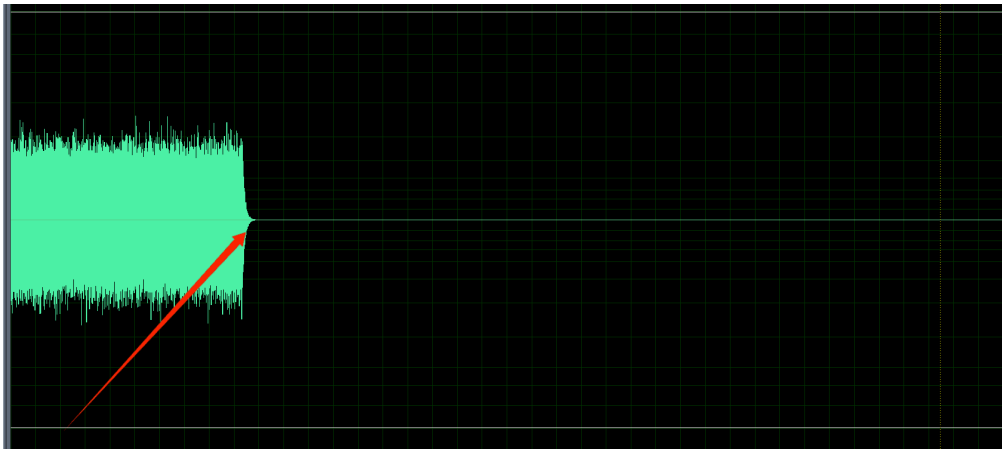


图 3.4.1-2 按默认参数，输入为白噪声，在箭头所指时刻使能控件

3.4.2 Noise Gate (Ext)

功能说明：带有外部侧链输入的噪声门。

- Input 0：外部控制信号（蓝色引脚）。
- Input 1：主音频信号（红色引脚）。

工作原理：模块通过检测 Input 0的电平大小来决定噪声门的开闭。

- 当 Input 0的电平低于设定的阈值时，门关闭，Input 1的信号被衰减。
- 当Input 0的电平高于阈值时，门打开，Input 1的信号正常通过。

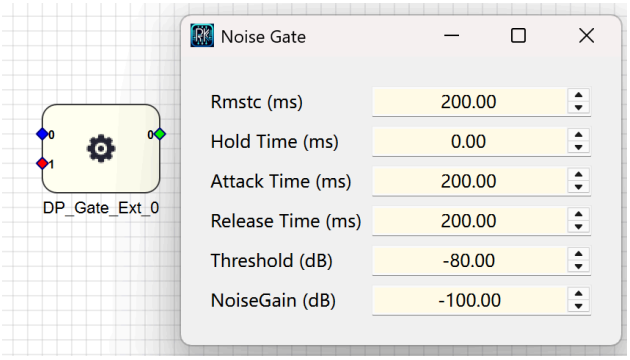


图 3.4.2-1 Noise Gate 控件与设置界面

参数数量：6
参数定义与标准 Noise Gate 一致，请参考之前的表格

3.5 Agc

功能说明：自动增益控制，将输入信号幅度，控制到目标值附近。
实际的增益 = （输入信号幅度 - 目标值）/ ratio

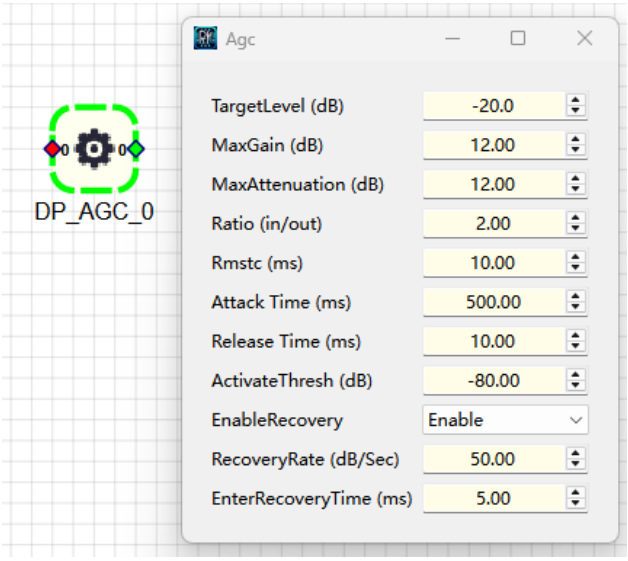


图 3.5-1 Agc 控件与设置界面

参数数量：11

参数名	参数位	类型	功能	单位	建议取值	默认取
TargetLevel （dB）	0	float	控制目标值	dB	-100 ~ 50	-20
MaxGain （dB）	1	float	最大增益值	dB	0 ~ 100	12
MaxAttenuation （dB）	2	float	最大衰减值	dB	0 ~ 100	12
Ratio （in/out）	3	float	增益变化比例	无	1 ~ 100	2
Rmstc（ms）*	4	float	平滑跟踪信号变化的速度，该值越大变化越慢	ms	0 ~ 2000	10
Attack Time（ms）*	5	float	增益上升速度	ms	0 ~ 2000	500
Release Time （ms）*	6	float	增益下降速度	ms	0 ~ 2000	10

参数名	参数位	类型	功能	单位	建议取值	默认取
ActivateThresh（dB）	7	float	工作阈值，高于该值时，开始工作	dB	-200 ~ 0	-80
EnableRecovery	8	float	是否启用增益恢复，启用改功能， 当输入信号幅度低于ActivateThresh时，增益会恢复到0db， 否则增益维持最大的增益值	无	0/1	1
RecoveryRate (dB/Sec)*	9	float	启用recovery时，信号低于阈值的增益衰减速度	db/Sec	1 ~ 200	50
EnterRecoveryTime*	10	float	启用recovery时， 如果输入信号幅度低于ActivateThresh的时间超过EnterRecoveryTime， 强制进入到recovery状态，可以避免当信号掉落过快， 增益会变大导致声音异常	ms	0 ~ 200	5

4 Filters

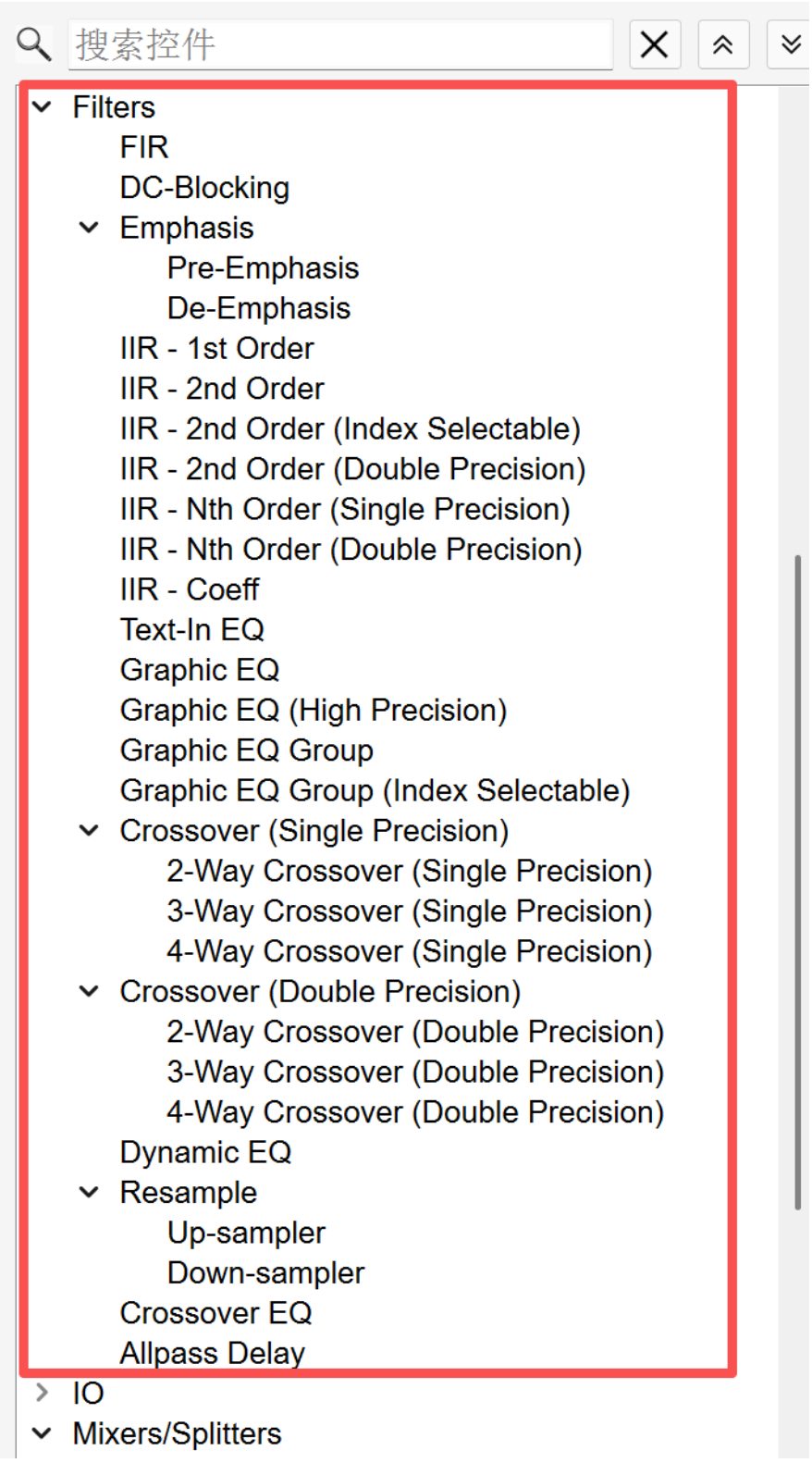


图 4-1 Filter位于工具的位置

4.1 FIR

功能说明：FIR滤波器,控件框中的12决定了滤波器的系数个数为12，此时滤波器的阶数为系数个数-1=11，双击后进入系数配置框，可以导入导出滤波器系数，点击表头后可以进行复制粘贴操作。

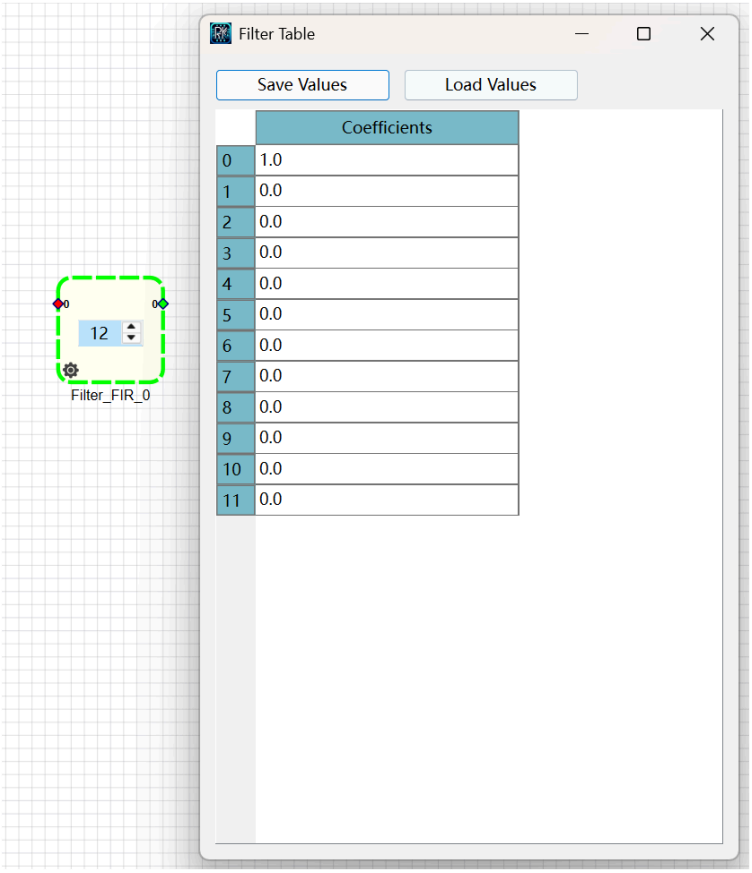


图 4.1-1 模块图标中是滤波器的系数个数

- 工具参数:
  参数数量：滤波器系数个数 + 1

参数名	参数位	类型	功能	单位	建议取值	默认取值
滤波器阶数	0	float	Fir滤波器的系数个数			12
C0	1	float	滤波器第一个系数			1
C1	2	float	滤波器第二个系数			0
Cn...	...	float	滤波器第...个系数			0

- 下发参数 *：与工具参数相同

注：滤波器的系数可以通过多种方式计算得到，以下指出两种常见方法

- 市面上的 FilterDesigner工具或者其他设计工具计算得到,具体方法用户可自行查阅，这里因为版权原因不列出;
- 也可以通过python的scipy库函数scipy.signal.firwin()* 计算。
  注意：由于dsp中计算机制的限制，fir的阶数要能被4整除，如果设计好的滤波器阶数不能刚好满足，那么就需要在尾部补0进行填充。

4.2 DC-Blocking

功能说明：通过使用一阶高通滤波器，去除直流信号。

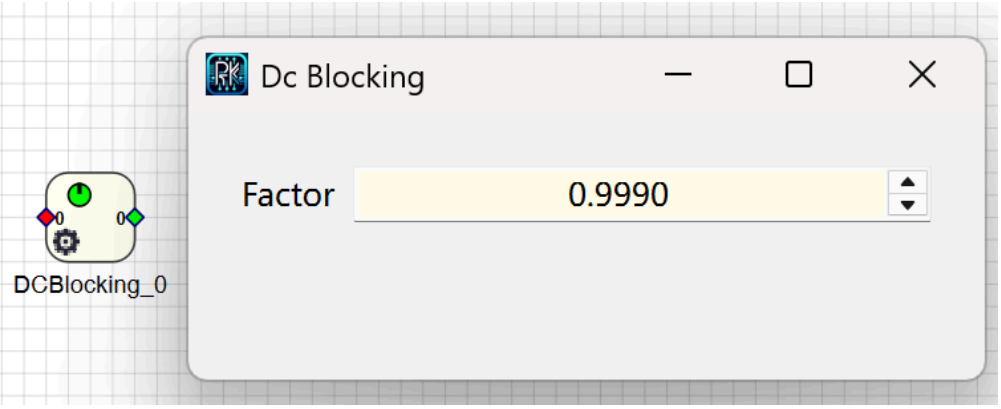


图 4.2-1 DC-Blocking控件

- 工具参数：
  参数数量：2

参数名	参数位	类型	功能	单位	建议取值	默认取值
Enable	0	float	算法使能开关		1	1
Factor	1	float	滤波器系数，越小滤波效果越好，但会滤除部分低频		0 ~ 1	0.9999

- 下发参数 *：与工具参数相同

注：控件左上角的小按钮为使能开关，绿色为1使能，红色为0禁能。下同。

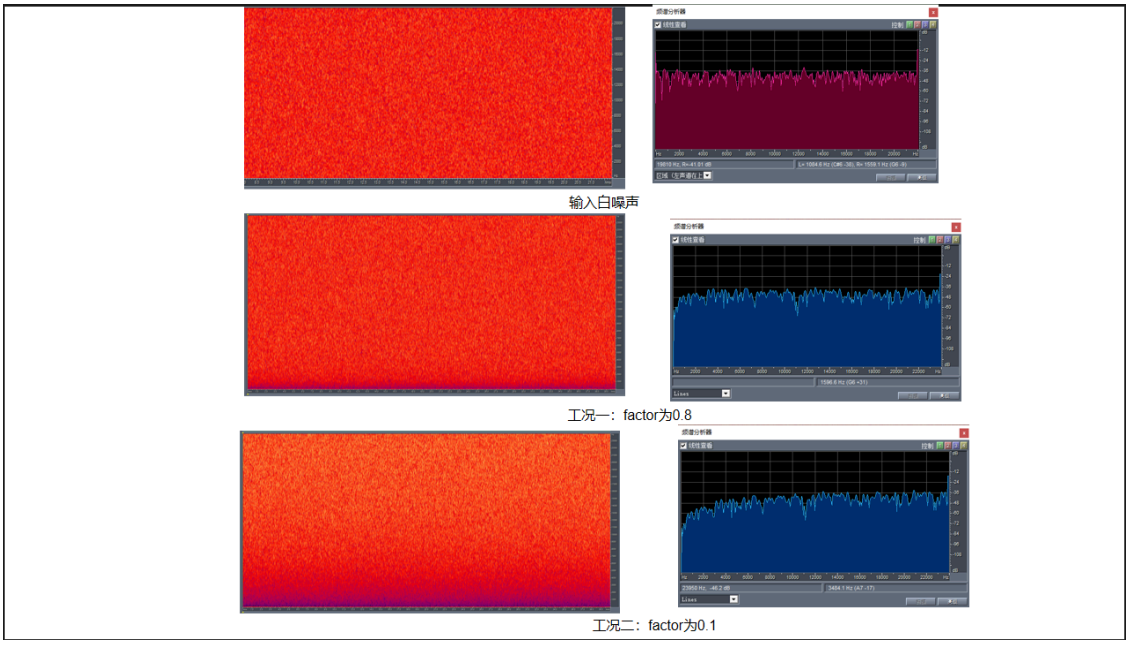


图 4.2-2 左侧的图片为时域上分割为小块时间做fft的频谱，右侧为整个时间段的频谱图，根据参数效果对比，可以看到factor越小，对低频的衰减越大

4.3 Emphasis

4.3.1 Pre-Emphasis

功能说明：预加重，使高频信号增强， 低频信号减弱。

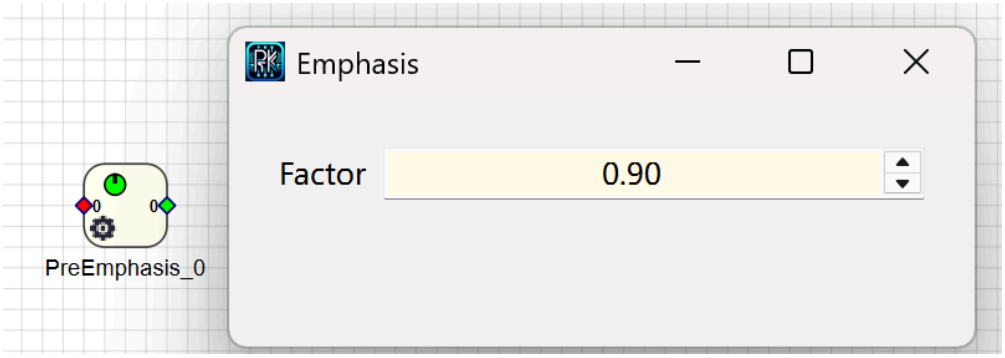


图 4.3.1-1 Pre-Emphasis 控件与设置界面

- 工具参数:
  参数数量：2

参数名	参数位	类型	功能	单位	建议取值	默认取值
Enable	0	float	算法使能开关		1	1
Factor	1	float	预加重系数，越大效果越好		0 ~ 1	0.9

- 下发参数 *：与工具参数相同

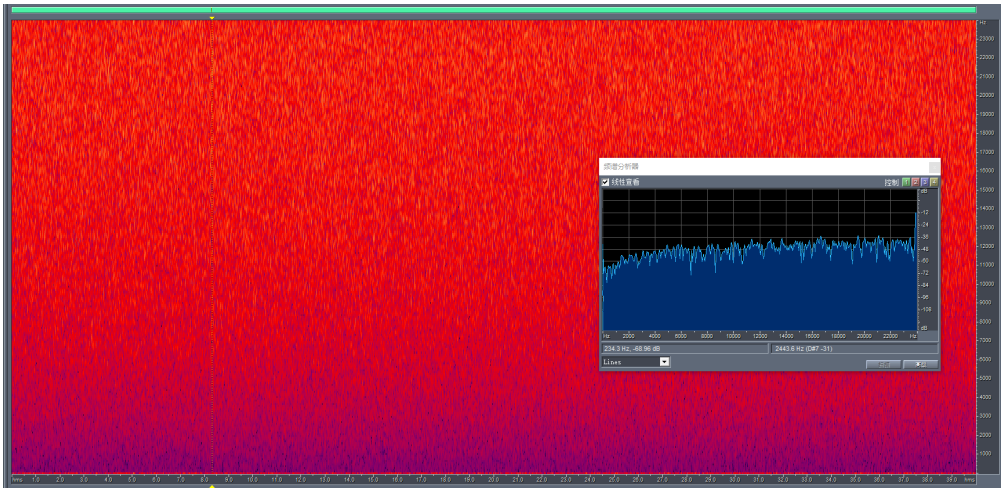


图 4.3.1-2 factor=0.9时的效果，可以看到高频得到增强，低频被抑制

4.3.2 De-Emphasis

功能说明：去加重，使高频信号减弱，低频信号增强 。

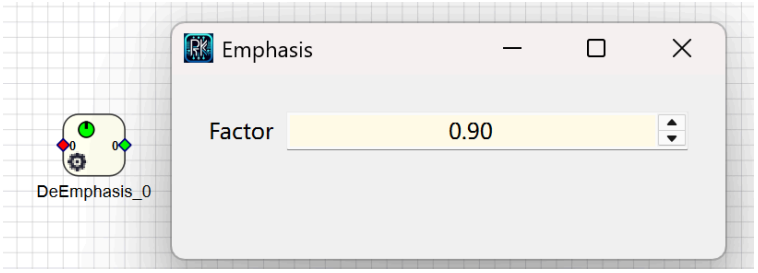


图 4.3.2-1 De-Emphasis 控件与设置界面

- 工具参数:
  参数数量：2

参数名	参数位	类型	功能	单位	建议取值	默认取值
Enable	0	float	算法使能开关		1	1

参数名	参数位	类型	功能	单位	建议取值	默认取值
Factor	1	float	去加重系数，越大效果越好		0 ~ 1	0.9

- 下发参数 *：与工具参数相同

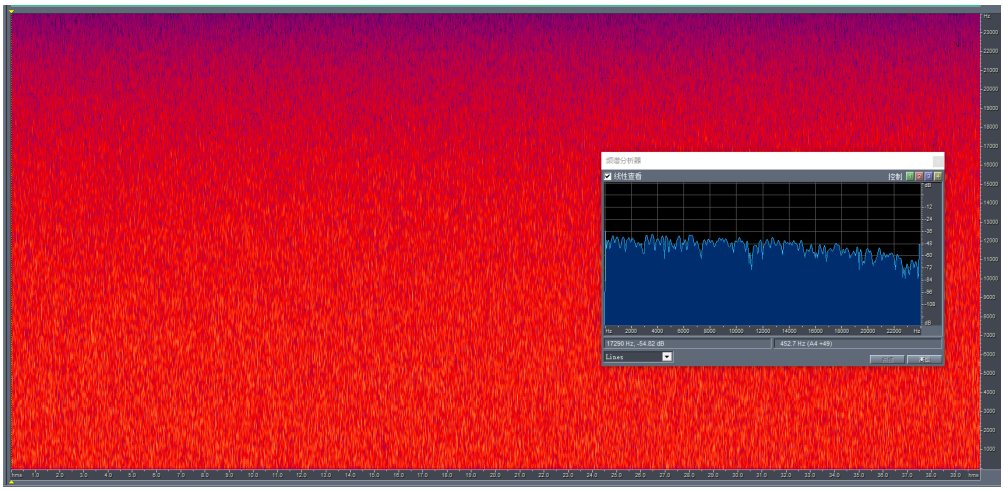


图 4.3.2-2 输入为白噪声，factor=0.9时的效果，可以看到高频被衰减，低频得到增强

4.4 IIR - 1st order

功能说明：通用一阶滤波器，可以选择以下5种滤波器类型：

Type	说明
LP1st	低通滤波器
HP1st	高通滤波器
AP1st	全通滤波器
LS1st	低架滤波器
HS1st	高架滤波器

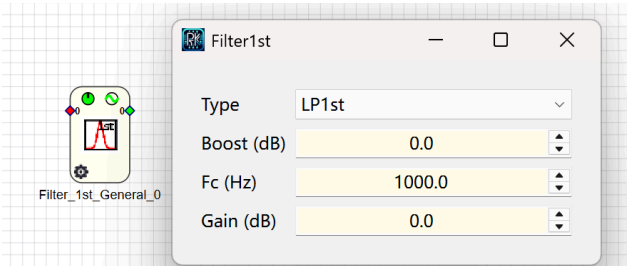


图 4.4-1 IIR-1st Order 控件与设置界面

- 工具参数：
  参数数量：6

工具参数	参数位	类型	功能	单位	建议取值	默认取值
Enable	0	float	使能开关		1	1
Polarity	1	float	反相开关		0	0
Type	2	float	滤波器类型，参考功能说明		0 - 4	0
Boost(dB)*	3	float	滤波器增益	dB	-20-20	0
Fc(Hz)	4	float	滤波器中心频率/截止频率	Hz	20 ~ 24000	1000
Gain(dB)	5	float	输出整体增益	dB	-20 ~ 20	0

注：在本控件中，仅有LS 低架滤波器和HS高架滤波器能设置Boost参数，其他滤波器的Boost参数均无效。

- 下发参数 *： 本控件参数转换逻辑*
  参数数量：3

下发参数	参数位	类型	功能
b0	0	float	滤波器分子系数b0
b1	1	float	滤波器分子系数b1
a1	2	float	滤波器分母系数a1

注：a0固定为1，所以在参数设置没有写出,下同；

一阶滤波器的功能与后面二阶滤波器一致，不过在频响曲线上有区别，我们记下来给出对比：

对于低通滤波器LP,一阶和二阶对比如下

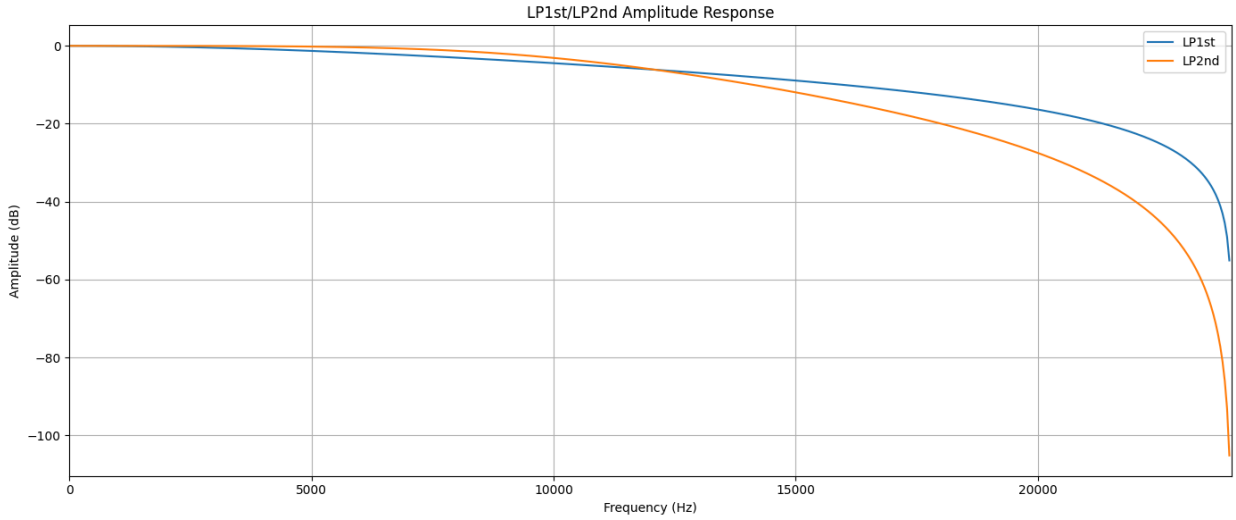


图 4.4-2 LP1st和LP2nd滤波器理论频响曲线

对于高通滤波器HP，一阶和二阶对比如下

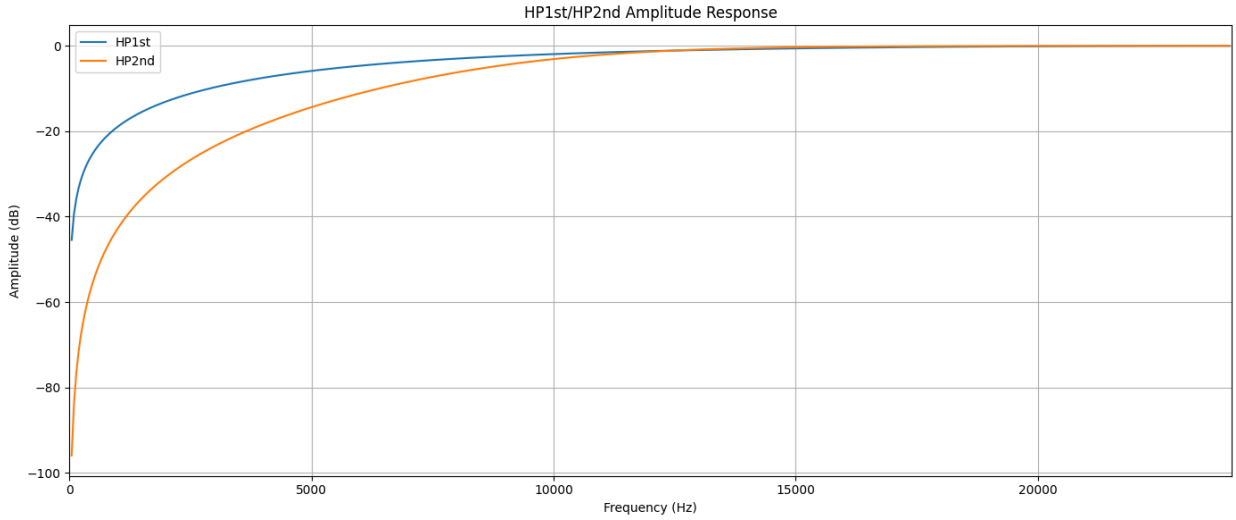


图 4.4-3 HP1st和HP2nd滤波器理论频响曲线

对于全通滤波器AP，一阶和二阶对比如下

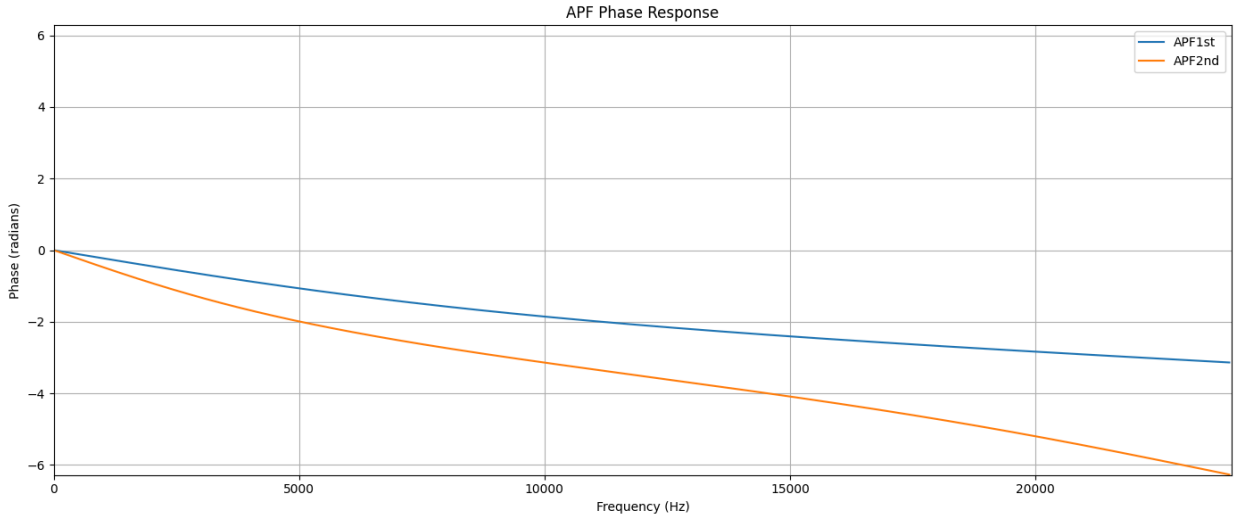


图 4.4-4 AP1st和AP2nd滤波器理论频响曲线

对于低架滤波器LS，一阶和二阶对比如下

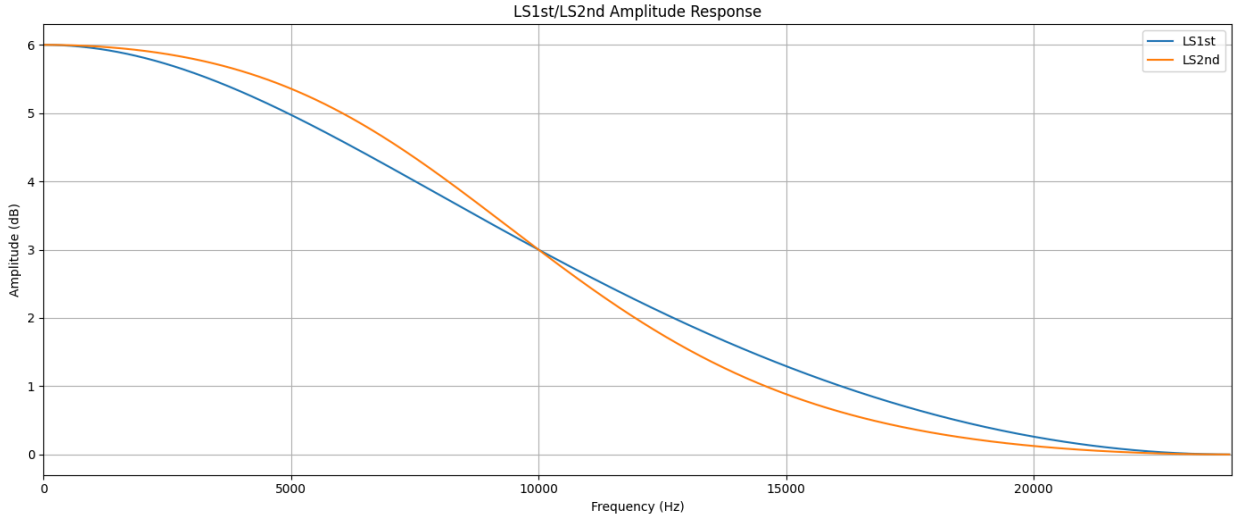


图 4.4-5 LS1st和LS2nd滤波器理论频响曲线

对于高架滤波器HS，一阶和二阶对比如下

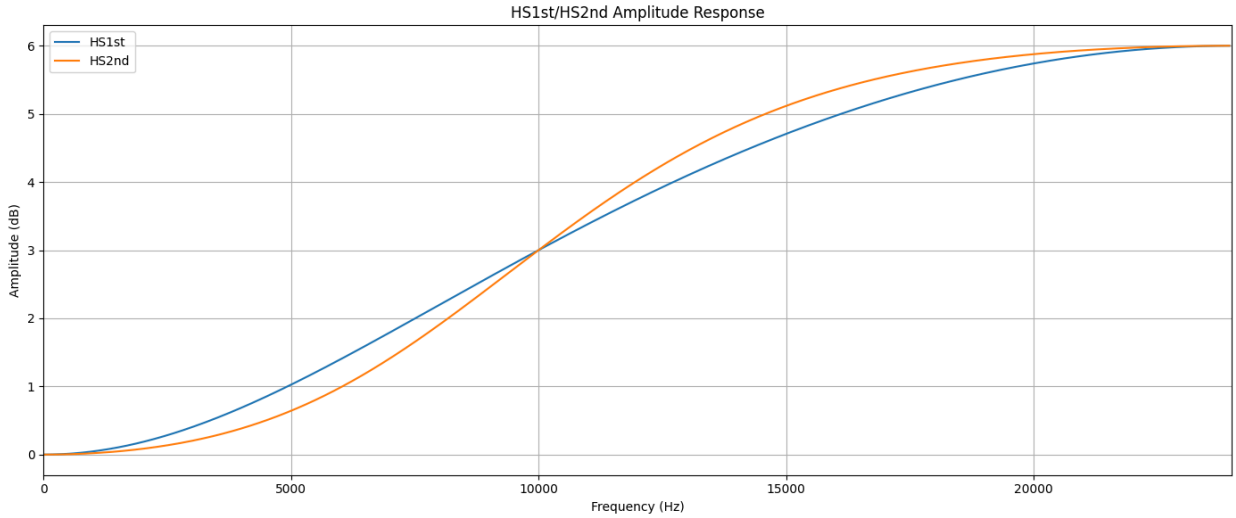


图 4.4-6 HS1st和HS2nd滤波器理论频响曲线

4.5 IIR - 2nd order

4.5.1 IIR - 2nd order

功能说明：通用二阶滤波器，可以选择以下16种滤波器类型：

Type	说明
Parametric	参数滤波器
LowShelf	低架滤波器
HighShelf	高架滤波器
GeneralLP	低通滤波器
GeneralHP	高通滤波器
GeneralBP	带通滤波器
GeneralBS	带阻滤波器
AFP	全通滤波器
Peaking	峰值滤波器
Notch	陷波滤波器
ChebyshevLP	切比雪夫低通滤波器
ChebyshevHP	切比雪夫高通滤波器
BesselLP	贝塞尔低通滤波器
BesselHP	贝塞尔高通滤波器
ButterLP	巴特沃斯低通滤波器
ButterHP	巴特沃斯高通滤波器

- 工具参数：
  参数数量：7(部分滤波器不需要全部参数，此时工具会将对应参数输入框无效化)

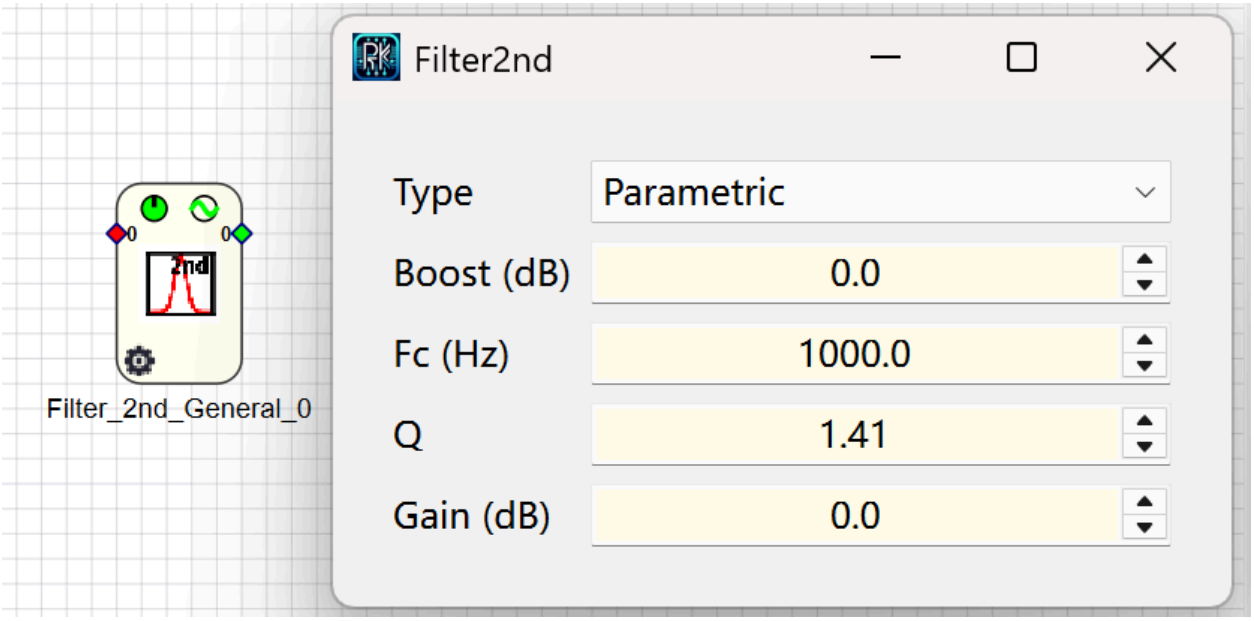


图 4.5.1-1 IIR_2nd控件和参数设置

注意Enable和Polarity的位置。

- Enable对应左上角按钮，为绿时，Enable=1,为红时，Enable=0。

- Polarity对应右上角按钮，为绿时，Polarity=0,为红时，Polarity=1

参数名	参数位	类型	功能	单位	建议取值	默认取值
Enable	0	float	使能开关		1	1
Polarity	1	float	反相开关		0	0
Type	2	float	滤波器类型，参考功能说明		0 - 15	0
Boost(dB)*	3	float	滤波器增益	dB	-20-20	0
Fc(Hz)	4	float	滤波器中心频率	Hz	20 ~ 24000	1000
Q	5	float	滤波器品质因数		0.01 ~ 16	1.41
Gain(dB)	6	float	输出整体增益	dB	-20 ~ 20	0

- 下发参数 *： 本控件参数转换逻辑*
  参数数量： 5

下发参数	参数位	类型	功能
b0	0	float	滤波器分子系数b0
b1	1	float	滤波器分子系数b1
b2	2	float	滤波器分子系数b2
a1	3	float	滤波器分母系数a1
a2	4	float	滤波器分母系数a2

注：a0固定为1，所以在参数设置没有写出，下同；

参数中可以注意到有Boost和Gain *两个控制增益的参数，其中前者是控制滤波器的中心增益的，而后者则是整体增益，用户可以理解为对滤波器的输出额外乘以一个增益，下图直观展示Gain参数的作用

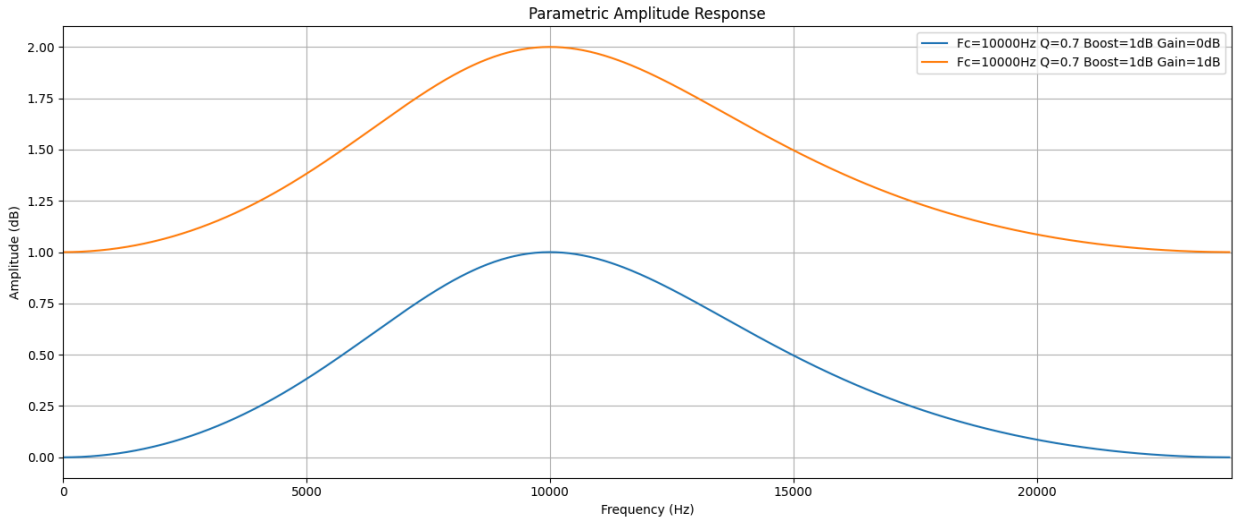


图 4.5.1-2 可以看到，Gain参数对滤波器的频响曲线有整体抬升的作用

理论频响曲线展示：
接下来我们展示每个滤波器的理论频响曲线，以便用户直观理解每个滤波器的作用,并展示Fc参数，Q参数以及Boost参数对幅频响应曲线的影响。

对于Parametric滤波器
展示Fc参数，Q参数以及Boost参数对幅频响应曲线的影响

- Fc: 可以看到Fc参数决定了滤波器影响的中心频率位置，
- Boost: Boost参数滤波器对中心频率的增益，

- Q: 而Q参数影响了滤波器对中心频率附近的影响范围,Q值越大，作用频带越窄：

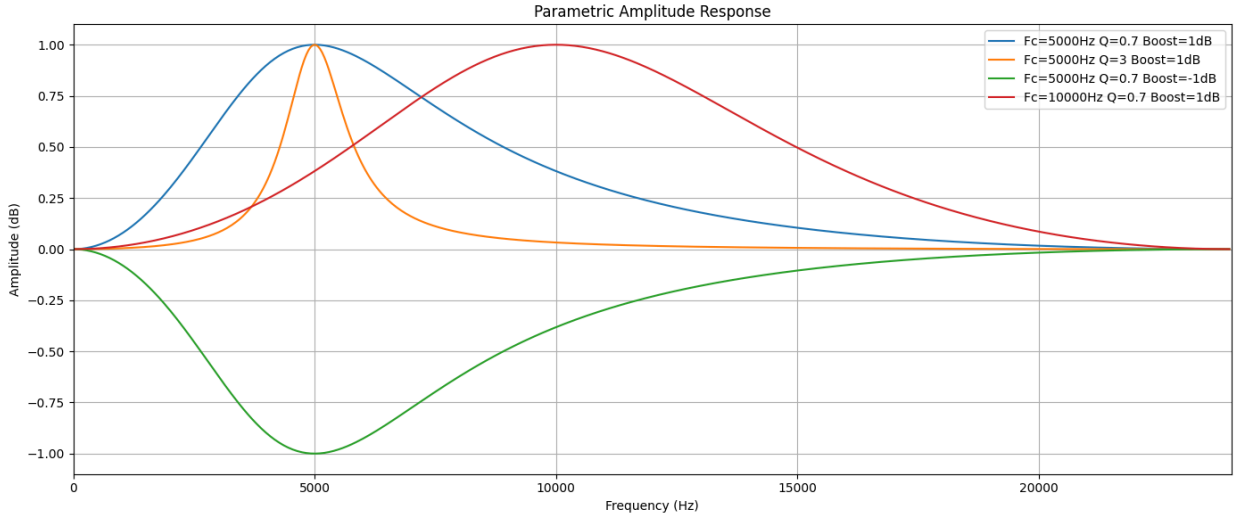


图 4.5.1-3 Parametric滤波器理论频响曲线

对于LowShelf和HighShelf滤波器

可以看到LowShelf影响低频范围，HighShelf影响高频范围，展示Fc参数，Q参数以及Boost参数对幅频响应曲线的影响

- Boost参数滤波器是增益的最大值，
- Fc参数决定了增益下降到一半的位置，
- Q参数影响了Fc附近增益变化的速度：

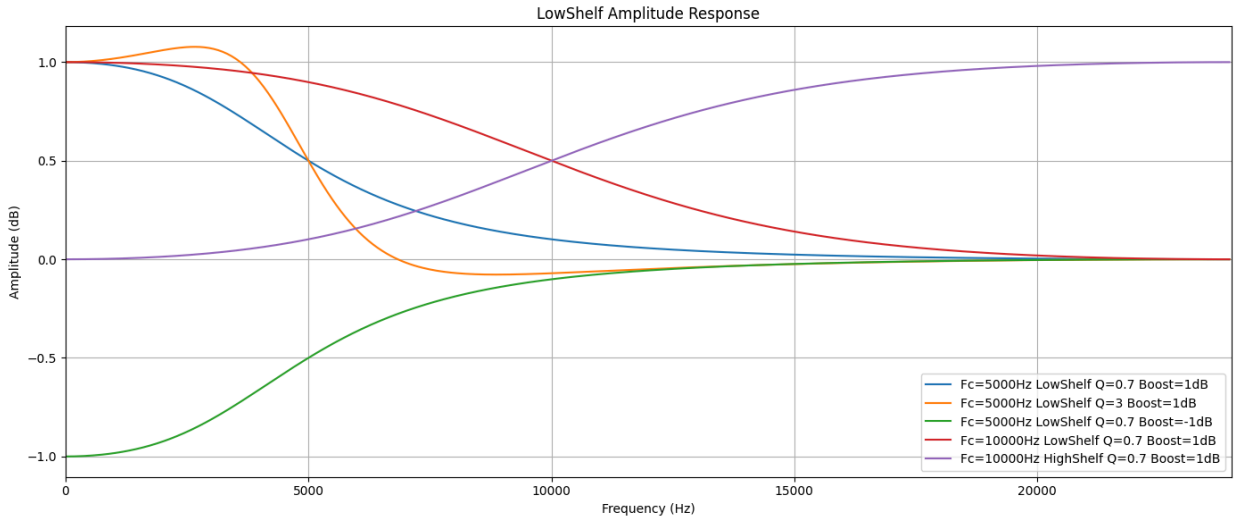


图 4.5.1-4 LowShelf和HighShelf滤波器理论频响曲线

对于GeneralLP、GeneralHP

展示Fc参数，Q参数对幅频响应曲线的影响

- Fc参数为截止频率
- Q参数影响了Fc附近增益变化的速度,当Q> $\sqrt{2}$ 时，会在Fc附近产生峰值 ，等于$\sqrt{2}$时，曲线最为平坦：
- Boost：这两个控件无Boost参数，

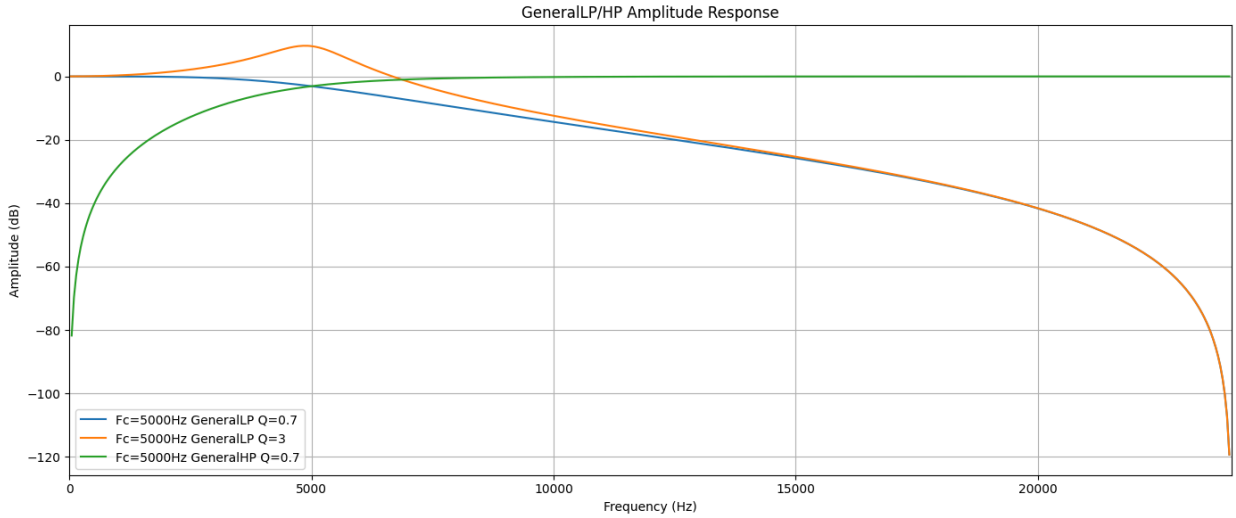


图 4.5.1-5 GeneralLP、GeneralHP滤波器理论频响曲线

对于GeneralBP、GeneralBS

展示Fc参数，Q参数对幅频响应曲线的影响,这些滤波器没Boost参数：

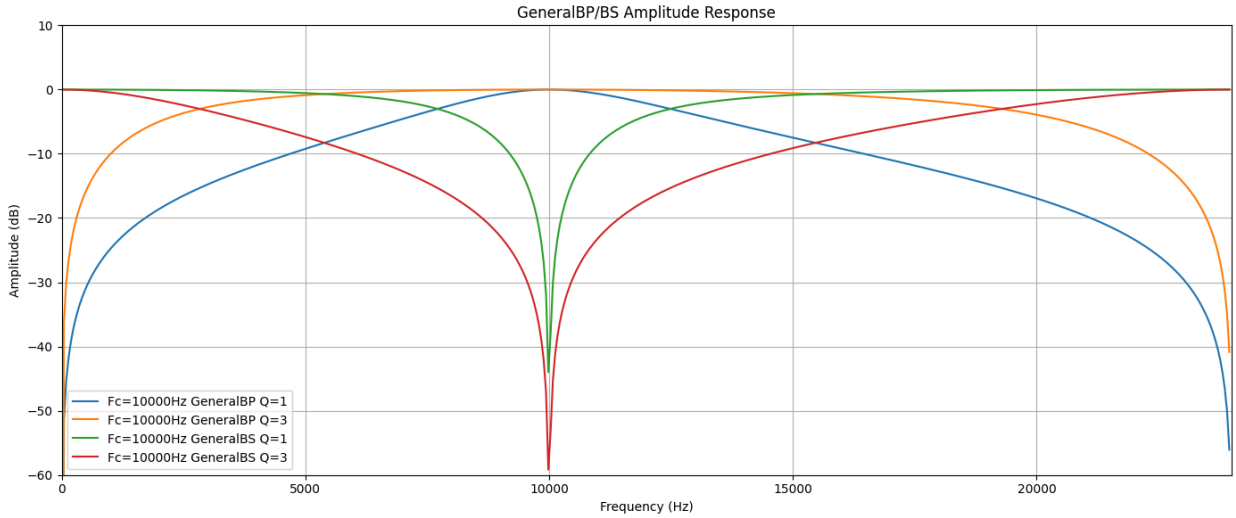


图 4.5.1-6 GeneralBP、GeneralBS滤波器理论频响曲线

对于APF(全通滤波器)

APF的对信号的幅度没有影响，其影响的是相位，展示Fc参数，Q参数对相频响应曲线的影响,这些滤波器没Boost参数。

- 其中Fc参数决定了滤波器造成-pi相位移动的频率，
- Q参数决定了滤波器的中心频率附近相位变化的速度：

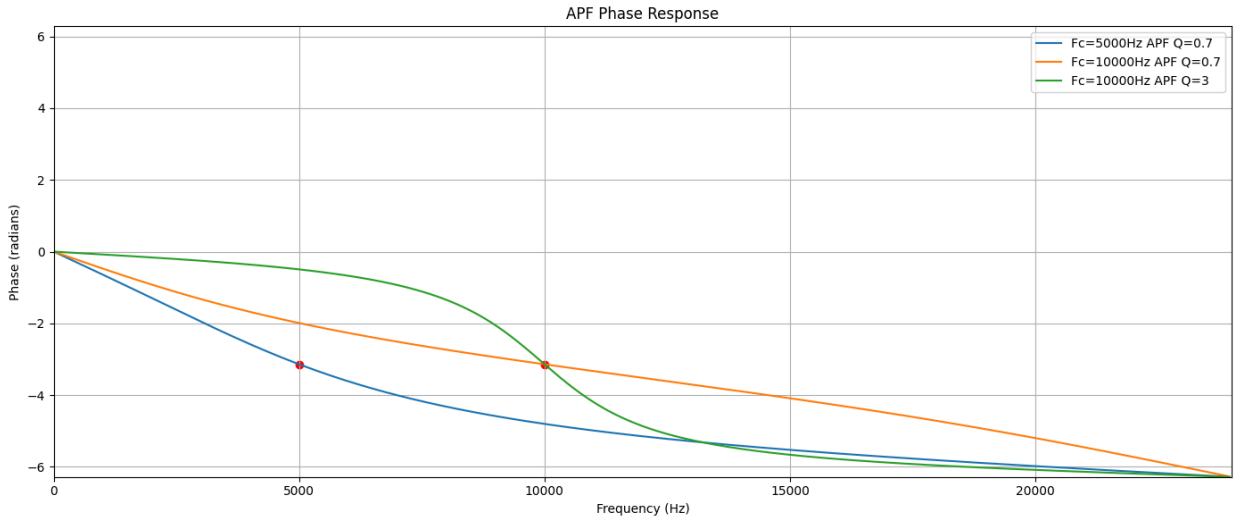


图 4.5.1-7 APF滤波器理论频响曲线

对于Peaking和Notch滤波器

Peaking滤波器类似于GeneralBP，而Notch滤波器类似于GeneralBS，不过相同Q值下，前者影响的范围更集中：

展示Fc参数，Q参数对相频响应曲线的影响,这些滤波器没Boost参数。

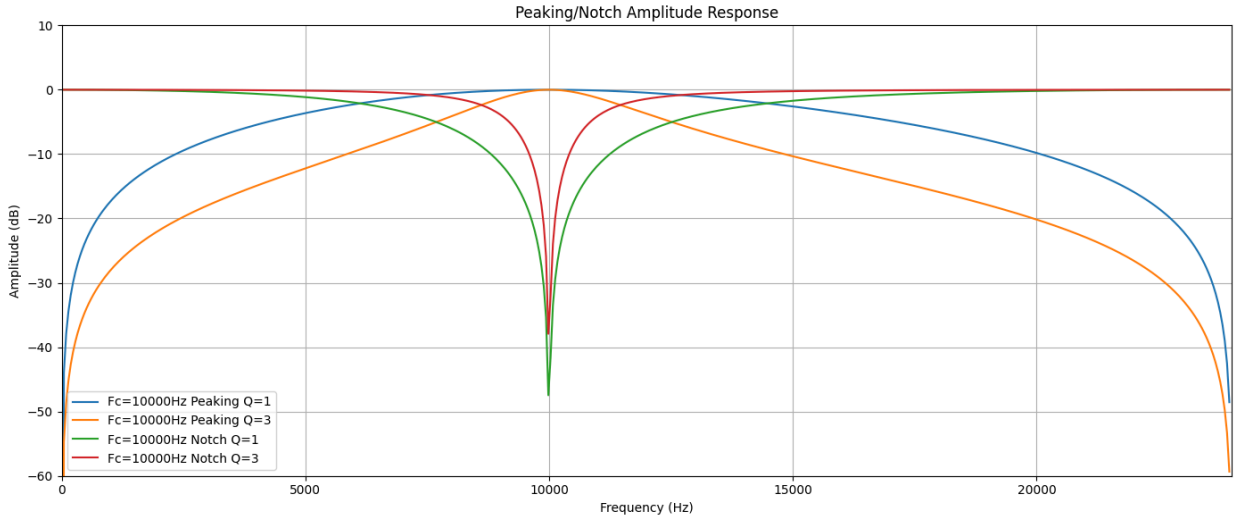


图 4.5.1-8 Peaking和Notch滤波器理论频响曲线

对于Butter和Bessel滤波器

这些滤波器没Q和Boost参数：

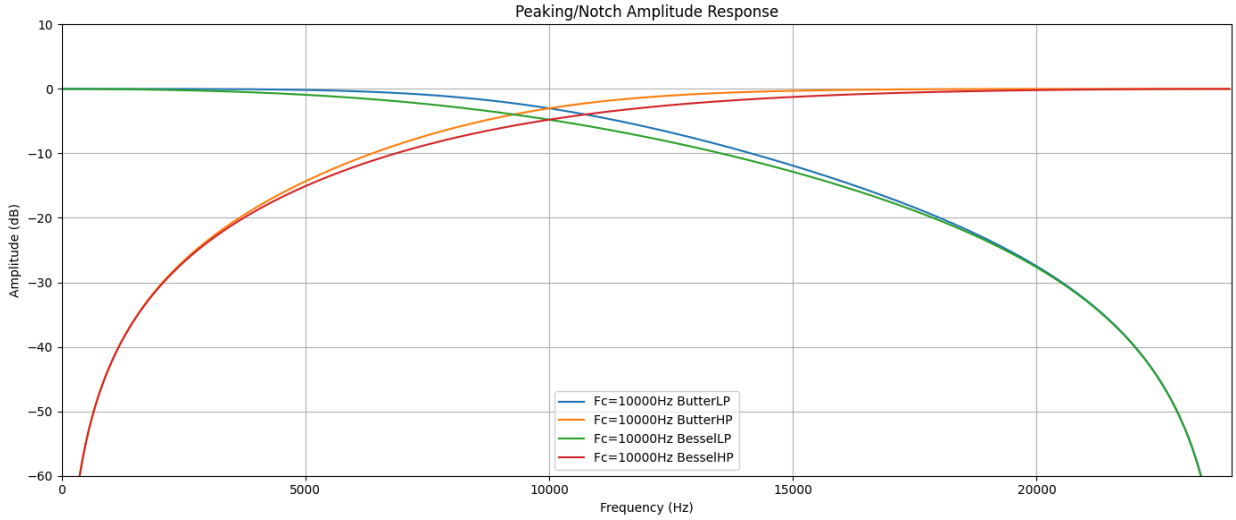


图 4.5.1-9 Butter和Bessel滤波器理论频响曲线

实际效果展示： 输入两通道白噪声源，每路白噪分别经过一个滤波器，获取输出，根据参数不同有以下典型工况：

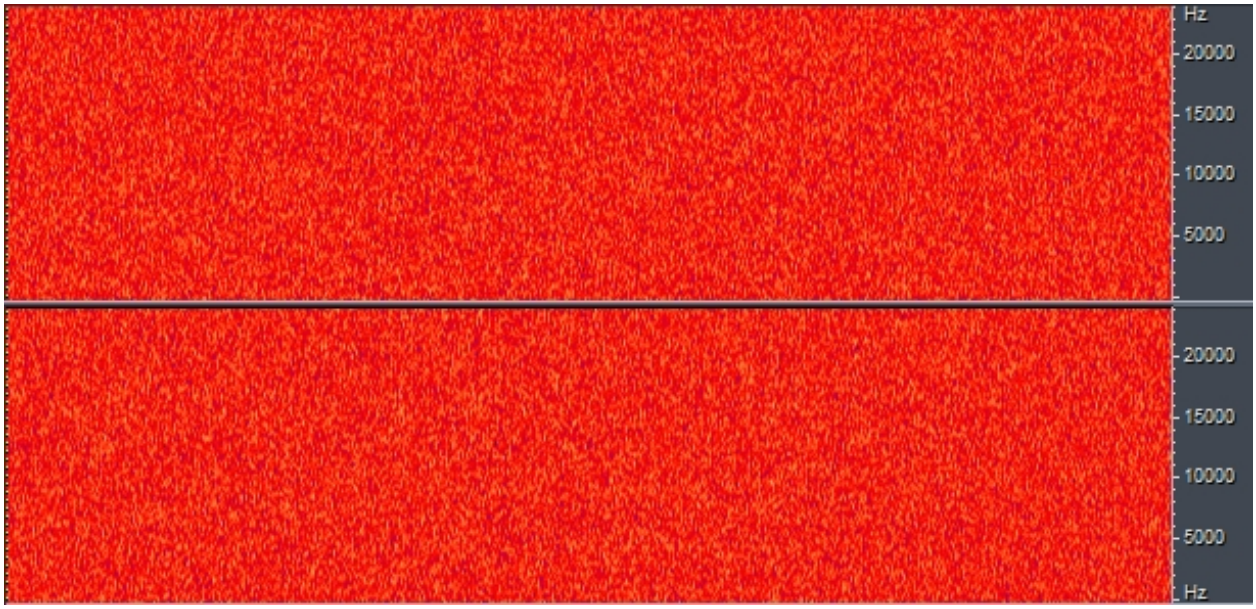


图 4.5.1-10 输入白噪频谱图

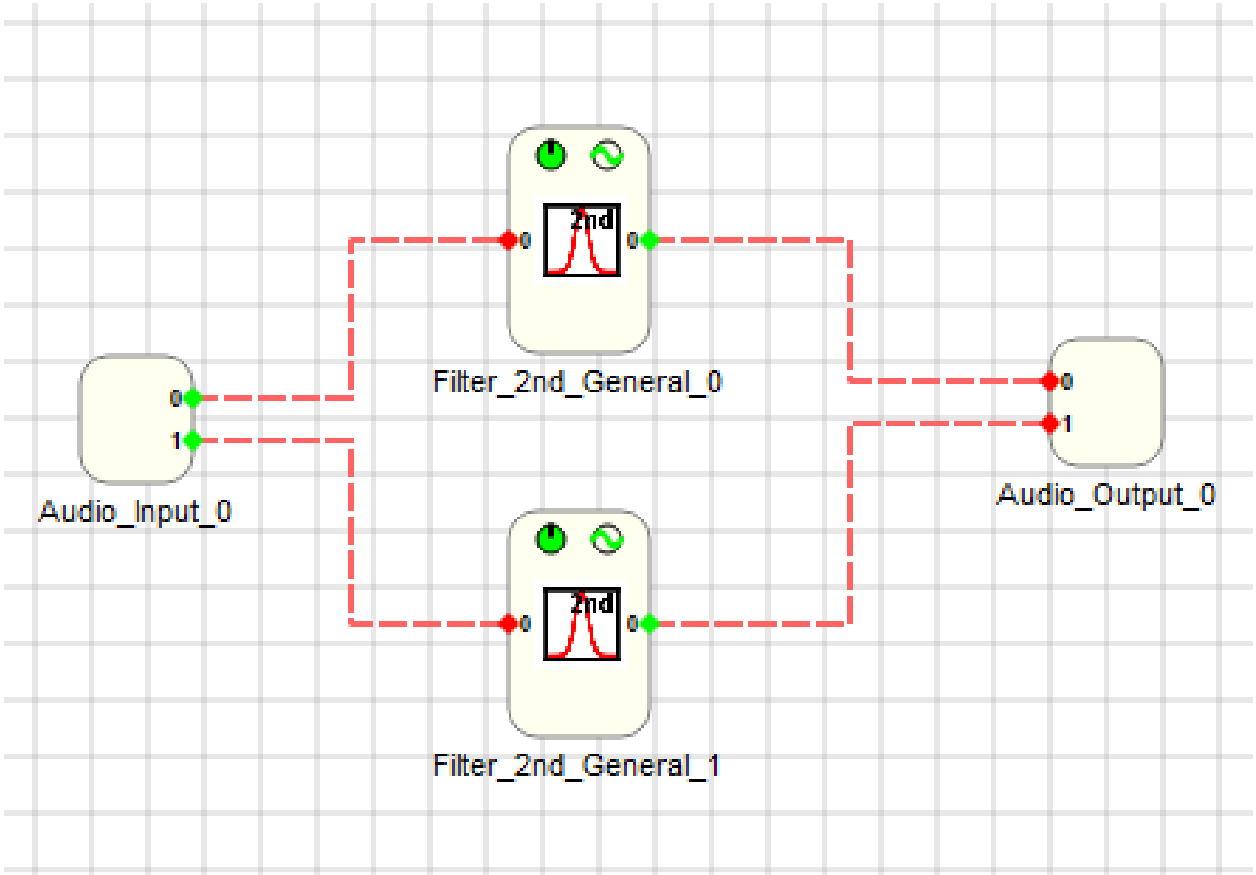


图 4.5.1-11 工况通用音频数据流程图

工况一：（展示Q值的影响）
通道0参数按顺序配置为：[1，0，0，-10，5000，0.7，0]（Q设为0.7）
通道1参数按顺序配置为：[1，0，0，-10，5000，15，0]（Q设为15）
输出如下：

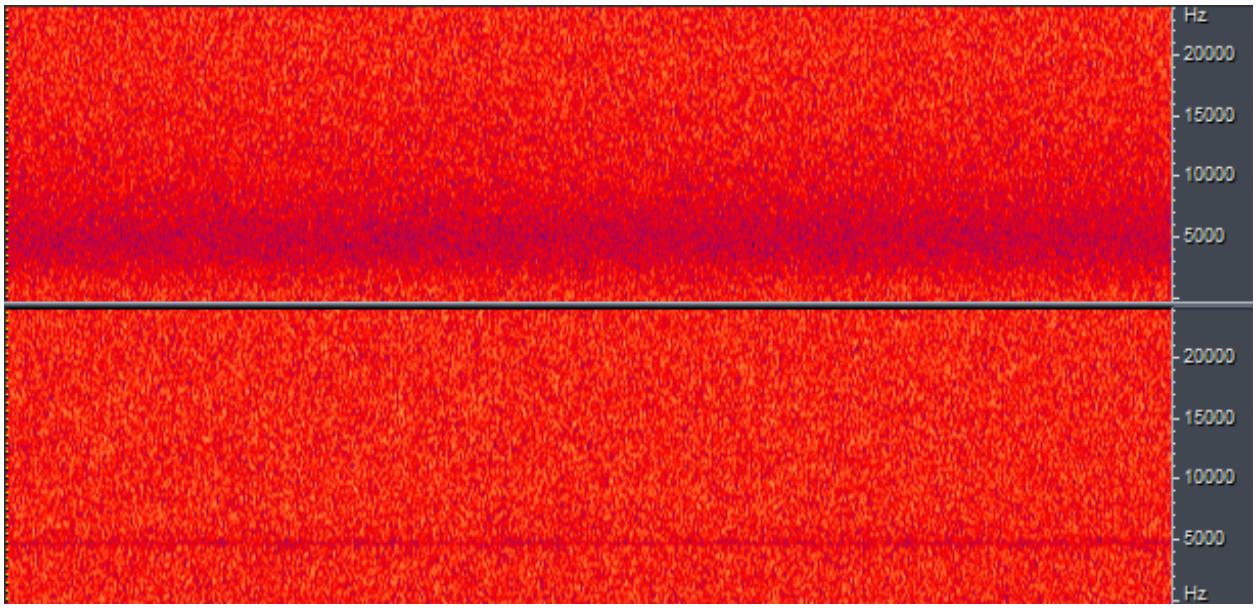


图 4.5.1-12 RKStudio输出频域图，Q值越大，Parametric滤波器作用越集中，作用频带范围越小

工况二：（LS低架滤波器和高架滤波器）
通道0参数按顺序配置为：[1，0，1，10，5000，1.41，0]（第三个参数type为1是低架滤波器）
通道1参数按顺序配置为：[1，0，2，10，5000，1.41，0]（第三个参数type为2是高架滤波器）
输出如下：

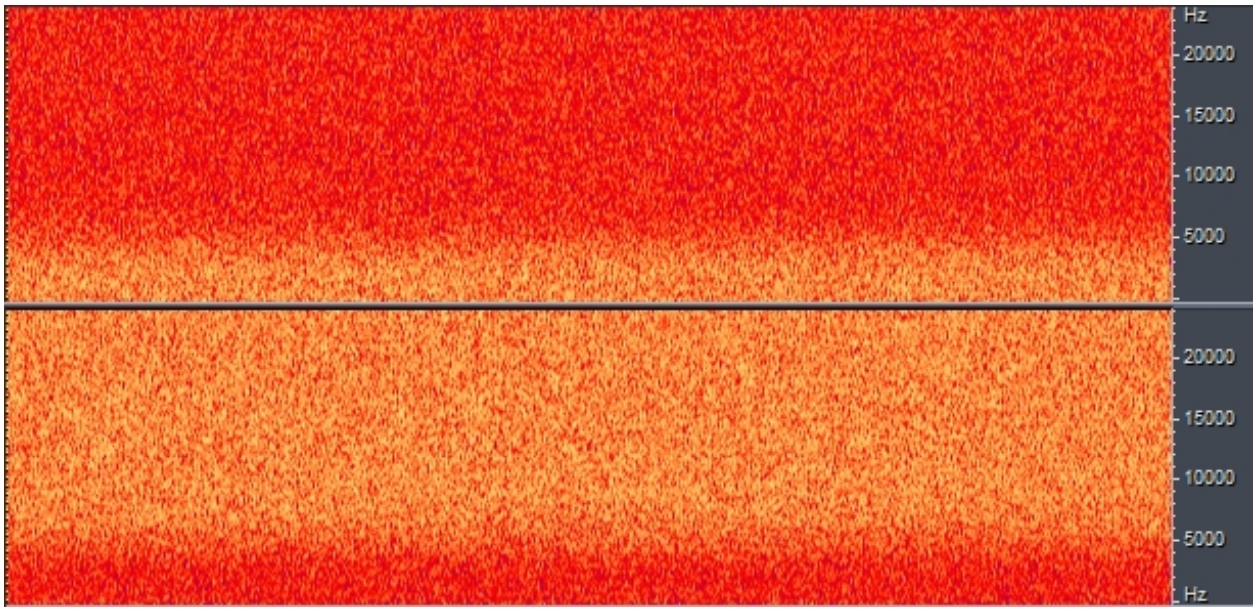


图 4.5.1-13 RKStudio输出频域图，LowShelf对中心频率以下的部分产生影响，HighShelf对中心频率以上部分产生影响

工况三：（BP带通和BS带阻）
通道0参数按顺序配置为：[1，0，5，0，5000，1.41，0]（第三个参数type为5是带通滤波器）
通道1参数按顺序配置为：[1，0，6，0，5000，1.41，0]（第三个参数type为6是带阻滤波器）
输出如下：

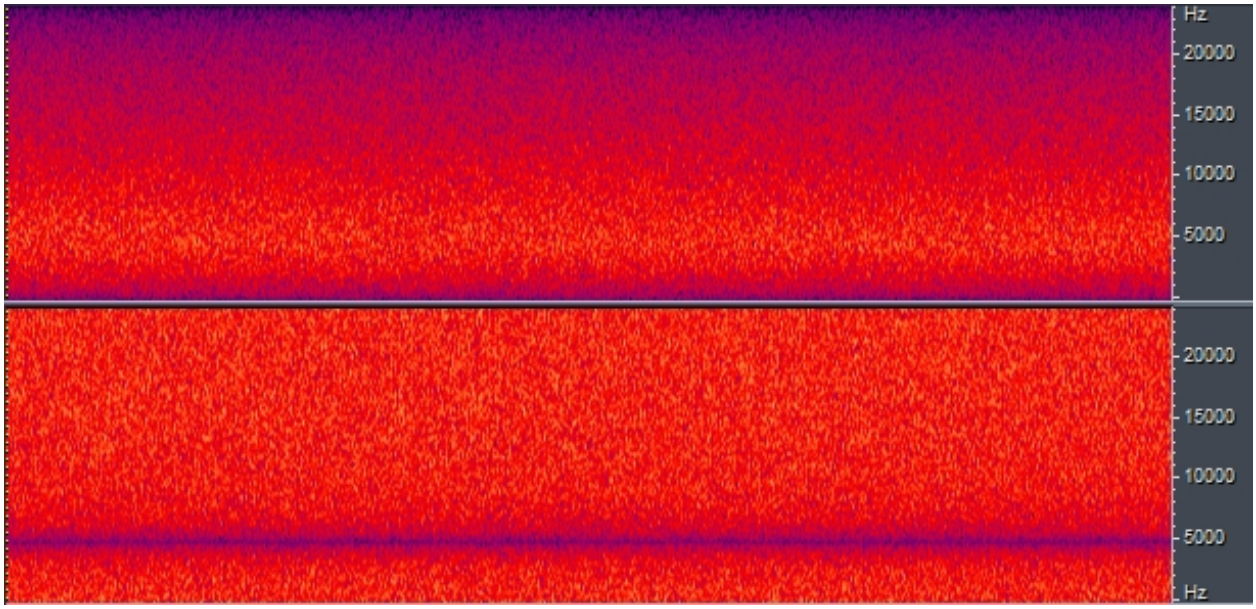


图 4.5.1-14 RKStudio输出频域图，GeneralBP保留中心频率附近的部分，GeneralBS阻止中心频率附近的部分，调整Q值会影响这两个滤波器的影响范围

工况四：（peaking峰值滤波器和Notch陷波滤波器）
通道0参数按顺序配置为：[1，0，8，0，5000，1.41,0]（第三个参数type为8是peaking峰值滤波器）
通道1参数按顺序配置为：[1，0，9，0，5000，1.41,0]（第三个参数type为9是Notch陷波滤波器）
输出如下：

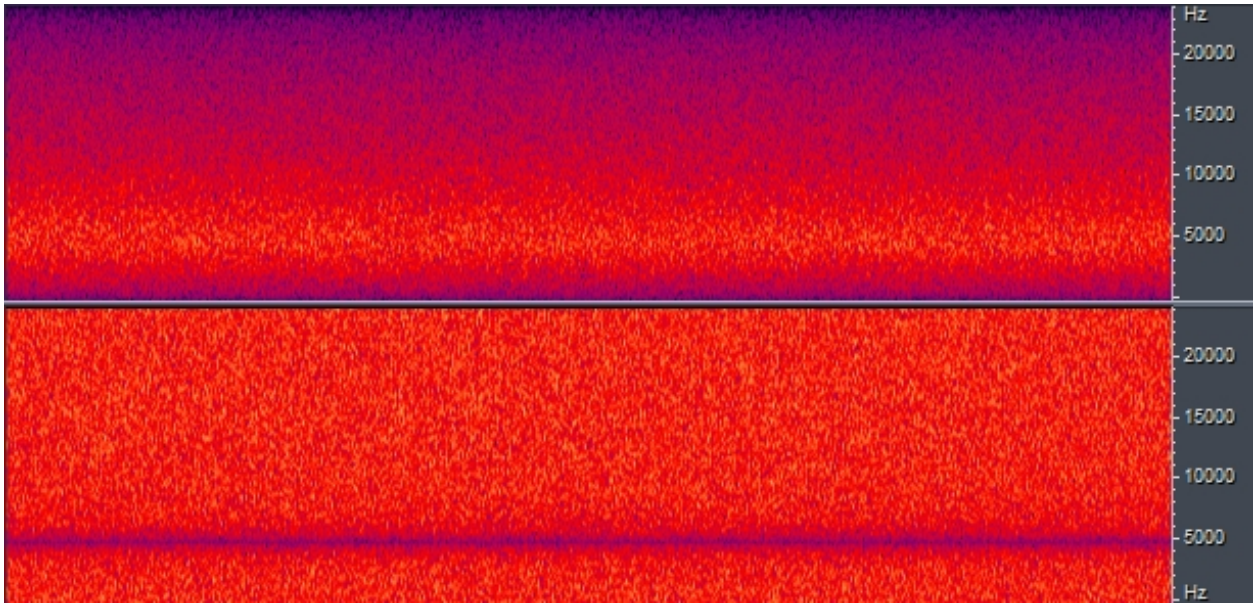


图 4.5.1-15 RKStudio输出频域图，Peaking滤波器类似于GeneralBP，而Notch滤波器类似于GeneralBS，不过相同Q值下，前者影响的范围更集中

4.5.2 IIR - 2nd order (Index Selectable)

功能说明：索引式二阶 IIR 滤波器。功能与标准的 IIR - 2nd Order 完全一致，但增加了一个 Index 参数。允许用户预先配置多组滤波器参数（例如针对不同的音效模式或场景），通过改变Index值来快速切换整套滤波器参数，一般用于多档位eq配置，高低音增强等场景。

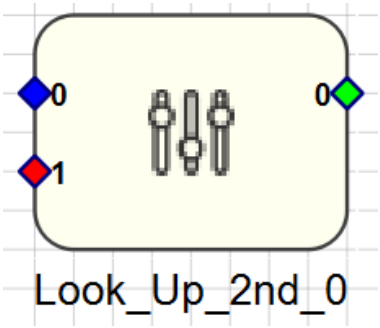


图 4.5.2-1 IIR - 2nd order (Index Selectable)控件

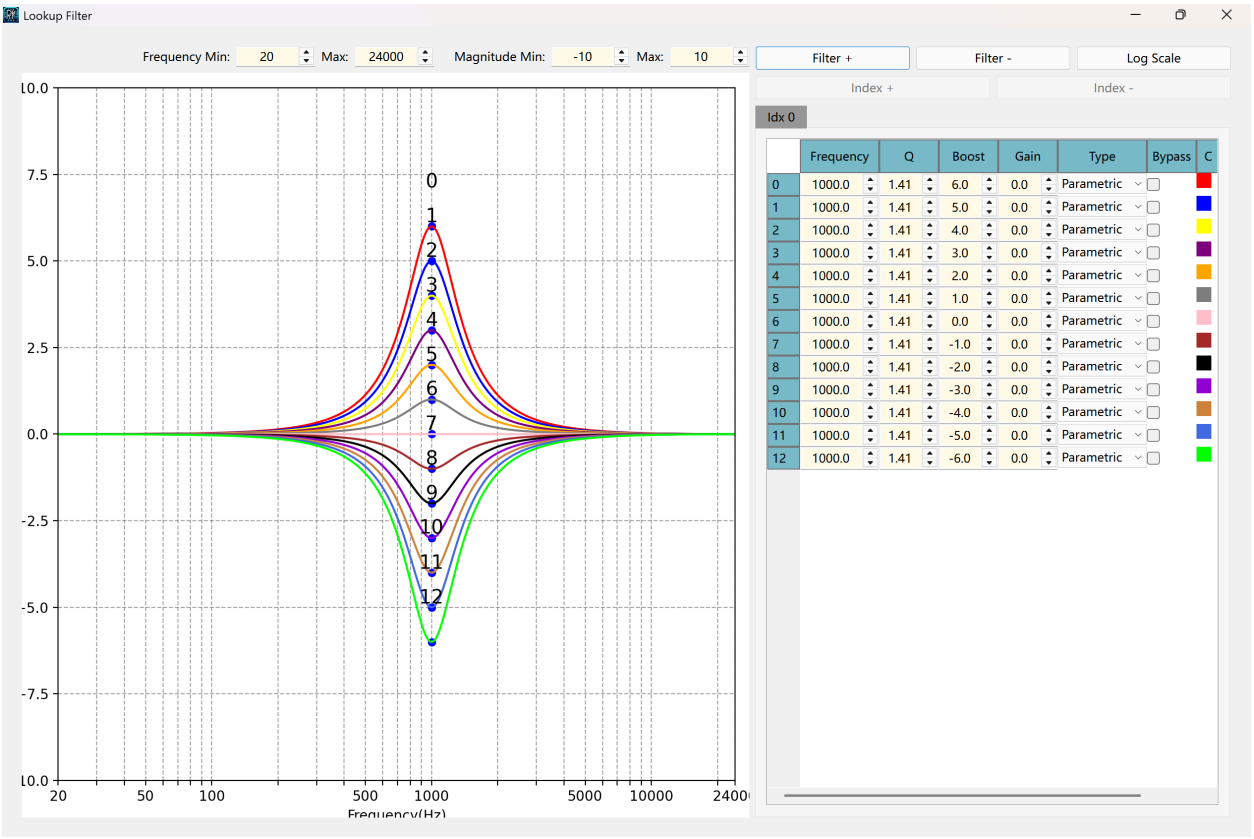


图 4.5.2-2 IIR - 2nd order (Index Selectable)参数设置界面

4.5.3 IIR - 2nd order (Double Precision)

功能说明：双精度二阶 IIR 滤波器。算法逻辑与控件参数与标准的 IIR - 2nd Order 完全一致，但计算精度更高，算力消耗也更大。

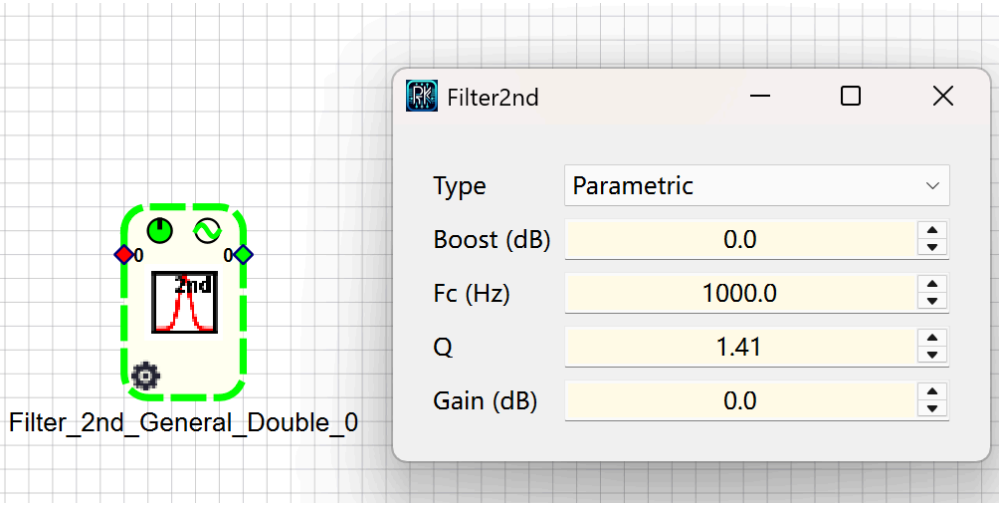


图 4.5.3-1 IIR - 2nd order (Double Precision)控件和设置界面

4.6 IIR Nth Order(Single Precision)

功能说明：高阶滤波器，使用单精度浮点进行运算，实际上是使用二阶IIR滤波器biquads*级联实现高阶滤波器的功能，可以选择以下8种滤波器类型：

Type	说明
ButterLP	巴特沃斯低通滤波器
ButterHP	巴特沃斯高通滤波器
BesselLP	贝塞尔低通滤波器
BesselHP	贝塞尔高通滤波器
Cheby1LP	第一类切比雪夫低通滤波器
Cheby1HP	第一类切比雪夫高通滤波器
Cheby2LP	第二类切比雪夫低通滤波器
Cheby2HP	第二类切比雪夫高通滤波器

- 工具参数：
  参数数量：6(部分滤波器不需要全部参数，此时工具会将对应参数输入框无效化)

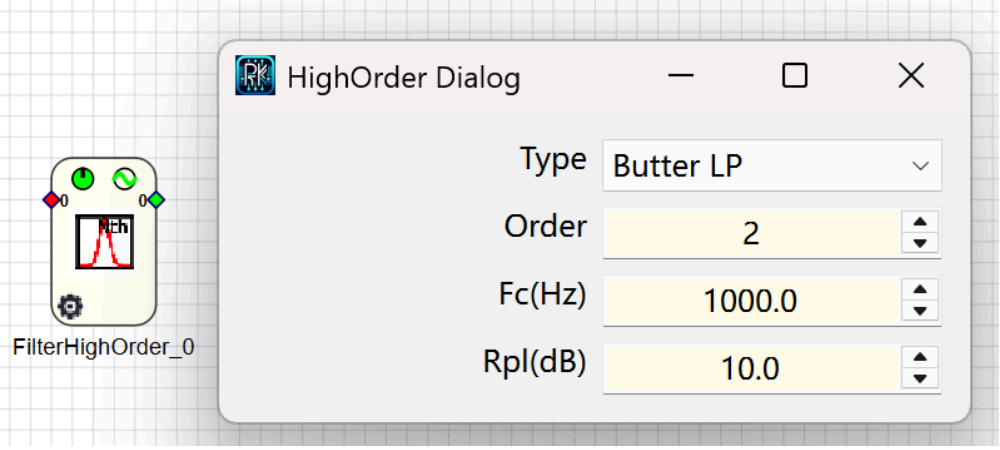


图 4.6-1 参数在模块中的的位置

注：注意Enable和Polarity的位置。左上角按钮为绿时，Enable=1,为红时，Enable=0。右上角按钮为绿时，Polarity=0,为红时，Polarity=1。

参数名	参数位	类型	功能	单位	建议取值	默认取值
Enable	0	float	使能开关		1	1
Polarity	1	float	反相开关		0	0
Type	2	float	滤波器类型，参考功能说明		0 - 9	0
Order	3	float	滤波器阶数，只能是偶数（因为使用二阶级联）		2 - 10	2
Fc(Hz)	4	float	滤波器中心频率	Hz	20 ~ 24000	1000
Rpl(dB)	5	float	通道内最大增益	dB	0 ~ 100	10

- 下发参数 *： 本控件参数转换逻辑*
  参数数量： (滤波器阶数/2)*5 +1

下发参数	参数位	类型	功能
NumSos	0	float	biquads（iir二阶滤波器） 级联数

下发参数	参数位	类型	功能
_0_b0	1	float	第一级滤波器分子系数b0
_0_b1	2	float	第一级滤波器分子系数b1
_0_b2	3	float	第一级滤波器分子系数b2
_0_a1	4	float	第一级滤波器分母系数a1
_0_a2	5	float	第一级滤波器分母系数a2
_1_b0	6	float	第二级滤波器分子系数b0
_x_yn	...	float	第x级滤波器分子/分母系数n

注：a0固定为1，所以在参数设置没有写出，下同；

4.7 IIR Nth Order(Double Precision)

功能说明：高阶滤波器，功能与4.6 IIR Nth Order(Single Precision)相同，但使用双精度浮点进行运算，适用于精度要求更高的iir计算，但是耗时也更长，一般使用单精度浮点运算即可。

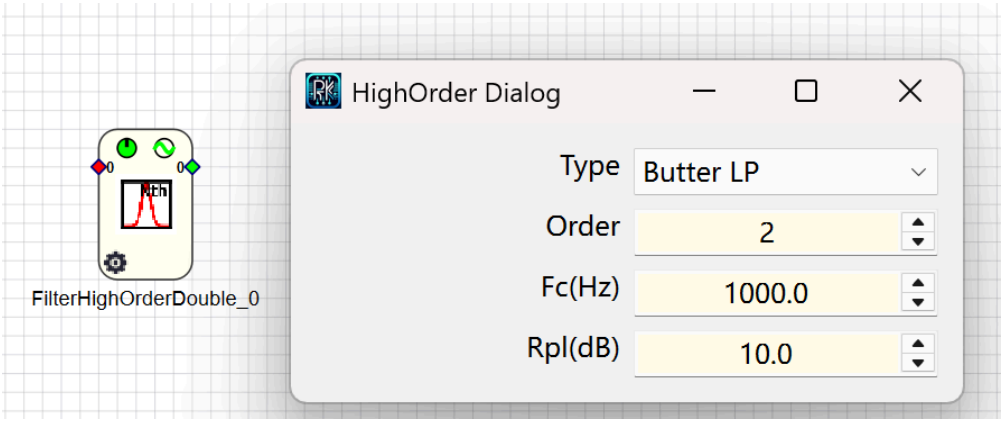


图 4.7-1 双精度浮点的高阶IIR滤波器工具参数设置

4.8 IIR - Coeff

功能说明：IIR滤波器, 可以将参数直接导入，适用于已经设计好滤波器参数的情况。控件框中的2决定了滤波器的阶数为2个biquads* 级联，每个iir二阶滤波器参数为5个（b0,b1,b2,a1,a2）所以总的系数为10个。双击后进入系数配置框，可以导入导出滤波器系数，点击表头后可以进行复制粘贴操作。

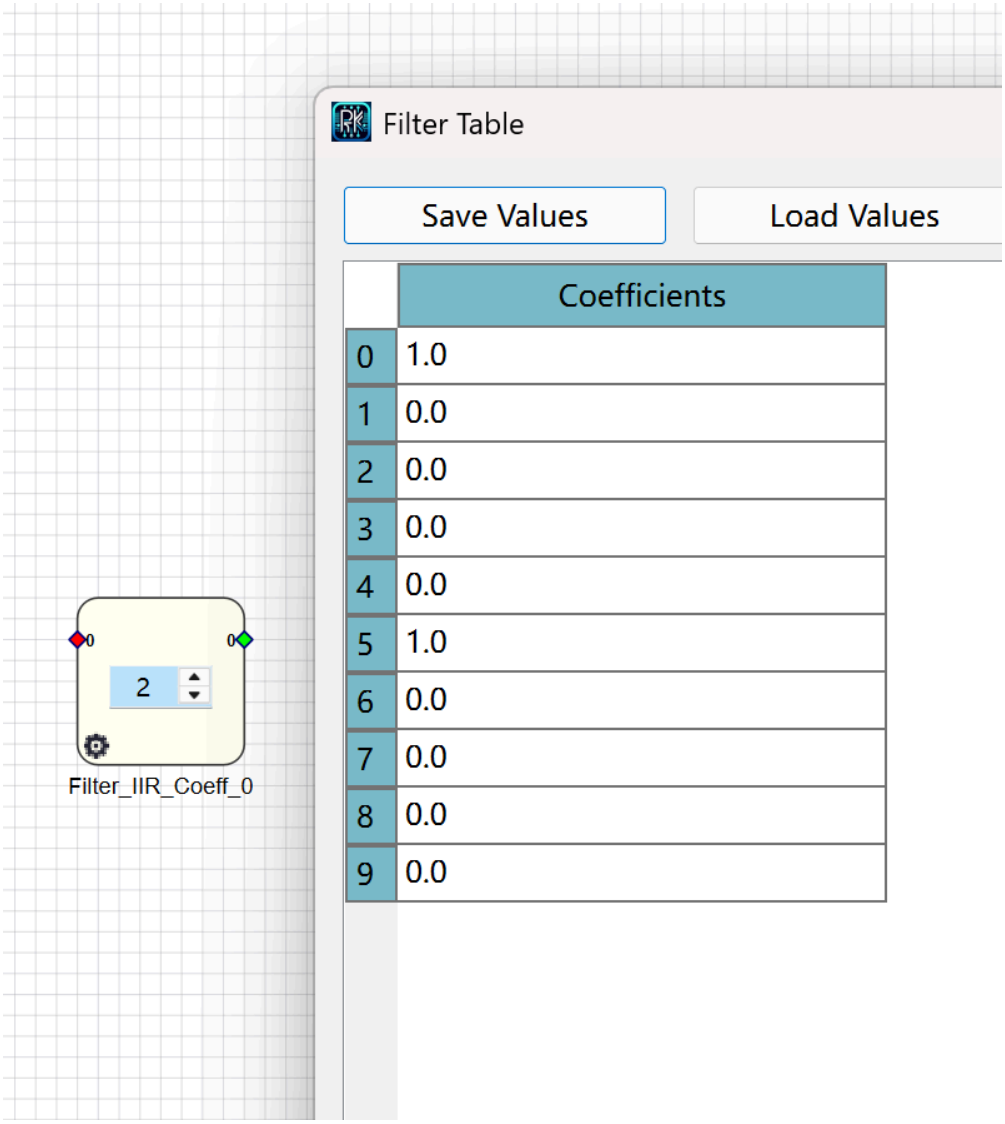


图 4.8-1 模块图标中是滤波器级联个数

- 工具参数：
  参数数量：滤波器级数*5 + 1

参数名	参数位	类型	功能	单位	建议取值	默认取值
numCoeffs	0	float	biquads滤波器串联数（方框内的数值）		1	10
B0_0	1	float	第一级滤波器分子第一个系数		-1000 ~ 1000	1
B1_0	2	float	第一级滤波器分子第二个系数		-1000 ~ 1000	0
B2_0	3	float	第一级滤波器分子第三个系数		-1000 ~ 1000	0
A1_0	4	float	第一级滤波器分母第一个系数		-1000 ~ 1000	0
A2_0	5	float	第一级滤波器分母第二个系数		-1000 ~ 1000	0
B0_1	6	float	第二级滤波器分子第一个系数		-1000 ~ 1000	1
B/A_x_y	...	float	第y级滤波器分子/分母第x个系数		-1000 ~ 1000	

- 下发参数 *：与工具参数相同

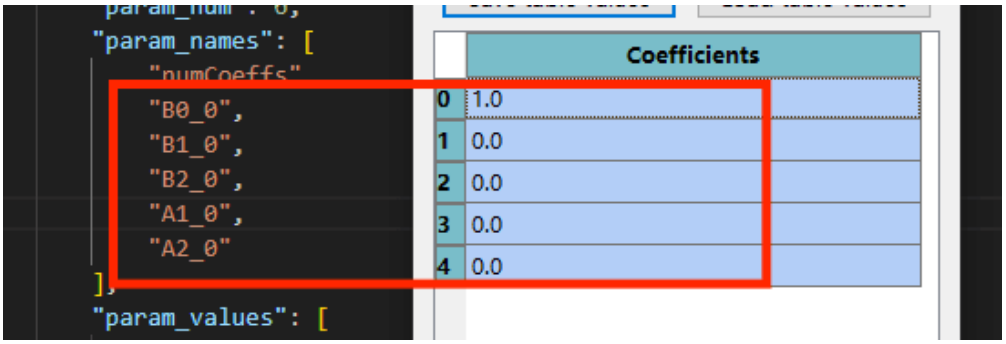


图 4.8-2 图纸json文件与工具设置参数对应，以1级为例

注：

- 图纸的json文件一般位于工程目录的 工程名_param.json

4.9 Graphical EQ

4.9.1 Graphical EQ

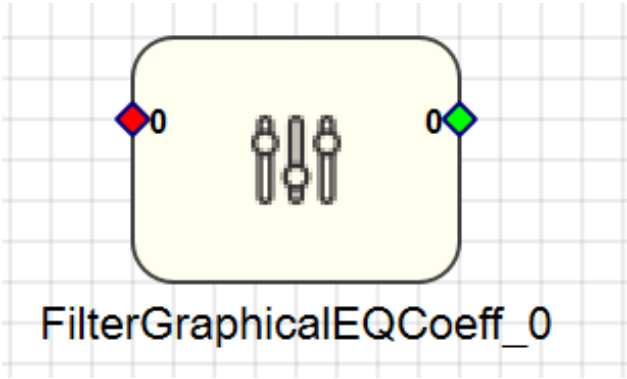


图 4.9.1-1 Graphic EQ 控件

功能说明：图形化EQ滤波器（均衡器）,主要功能是利用多个滤波器对多个频段的信号进行衰减或者增强处理，本质上为多个滤波器串联工作，最后达到增强期望频段、抑制不期望频段的均衡的效果。可以选择以下滤波器类型：

Type	说明
Parametric	参数滤波器
LowShelf	低架滤波器
HighShelf	高架滤波器
GeneralLP	低通滤波器
GeneralHP	高通滤波器
ButterLP	巴特沃斯低通滤波器
ButterHP	巴特沃斯高通滤波器
BesselLP	贝塞尔低通滤波器
BesselHP	贝塞尔高通滤波器
Allpass	全通滤波器

- 工具参数：
  参数数量：n*6(6指的是EQ中每个滤波器的参数数目) +3

参数名	参数位	类型	功能	单位	建议取值	默认取值
SmoothTime（ms）	0	float	平滑时间	ms	20 ~ 22000	1000

参数名	参数位	类型	功能	单位	建议取值	默认取值
Reserved	1		预留			
NumFilter	2	float	滤波器个数（点击add增加，点remove减少）			10
Freq	1*n +2	float	滤波器中心频率	Hz	20 ~ 22000	1000
Boost*	2*n +2	float	滤波器增益	dB	-20-20	0
Gain	3*n +2	float	输出整体增益	dB	-20-20	0
Q	4*n +2	float	滤波器品质因数		0.01 ~ 16	0.71
Type	5*n +2	float	滤波器类型，参考功能说明		0 - 9	0
Bypass	6*n +2	float	失能开关(旁路开关)		0	0

- 下发参数 *： 本控件参数转换逻辑*
  每一个滤波器都用二阶实现，每个二阶滤波器的系数个数为5个。
  参数数量：n*5+3

参数名	参数位	类型	功能
SmoothTime_ms	0	float	平滑时间
Reserved	1		预留
NumFilter	2	float	滤波器个数
_0_b0	3	float	第一个滤波器分子系数b0
_0_b1	4	float	第一个滤波器分子系数b1
_0_b2	5	float	第一个滤波器分子系数b2
_0_a1	6	float	第一个滤波器分母系数a1
_0_a2	7	float	第一个滤波器分母系数a2
_1_b0	8	float	第二级滤波器分子系数b0
_x_yn	.....	float	第x个滤波器分子/分母系数n

点击模块中间的按钮，即可进入EQ设置界面，用户可以在右侧的参数框部分新增或者减少滤波器数目，可以输入滤波器的Fc、Boost以及Q参数，并且可以选择不同的滤波器类型。用户可以bypass掉某些滤波器，被bypass掉的滤波器在左侧的频响曲线上会被标记为红色，且不参与总频响曲线的计算。

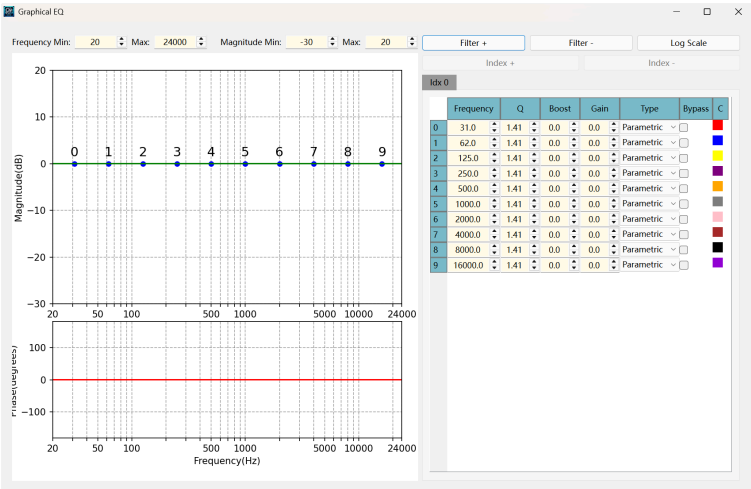


图 4.9.1-2 Graphic EQ 频响曲线图

功能演示：在音频处理中，时常需要增强低频部分来使得声音听起来更加丰满，需要突出人声，和抑制高频的尖锐声，在此处我们为展示效果更加明显，设定增强2000hz附近的频段增益为6dB，品质因数Q为15；增强4000hz的频段的增益为6dB，品质因数Q为7；并抑制7000hz以上的频

段信号；

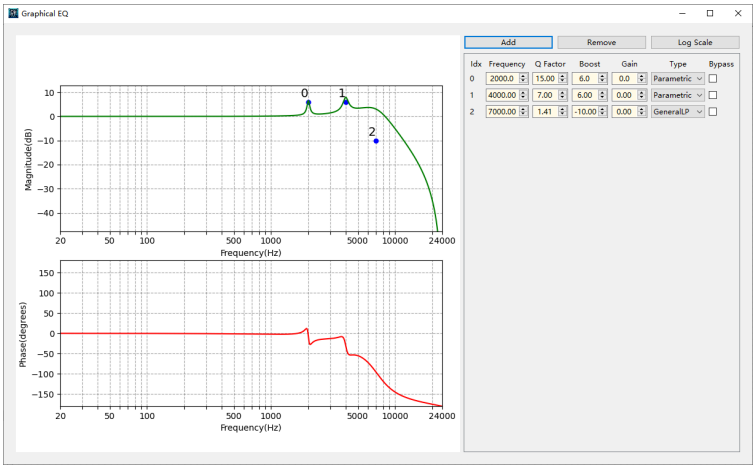


图 4.9.1-3 Graphic EQ 参数设置

输入为白噪声

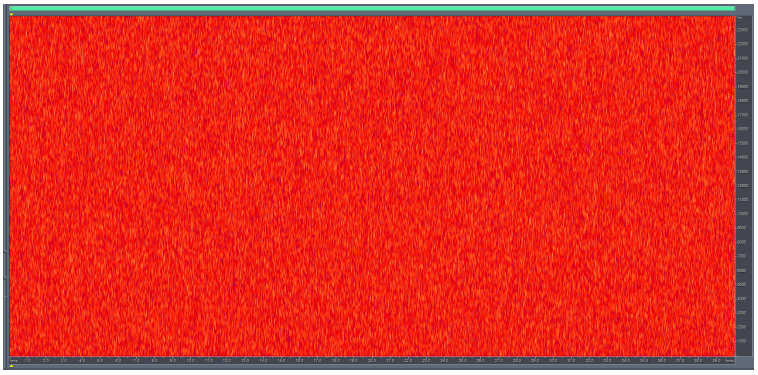


图 4.9.1-4输入白噪声频谱图

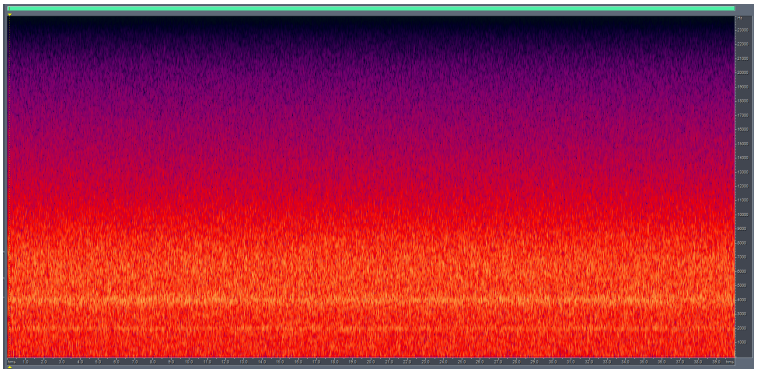


图 4.9.1-5 输出频域图

可以从图中看出，在2000hz频段和4000hz频段有着明显的增强，在7000hz以上频段有着明显的衰减； 同时由图 4.9.1-3 Graphic EQ 参数设置的曲线和输出频域图看出Q值越大，作用越集中，作用频段带宽越小

4.9.2 Graphic EQ (High Precision)

使用方法和“ 4.9.1 Graphical EQ ”一致，使用了特殊运算方式，在提高单浮点运算精度的前提下，算力消耗比单精度计算高一些，但比双精度（Double Precision）更低

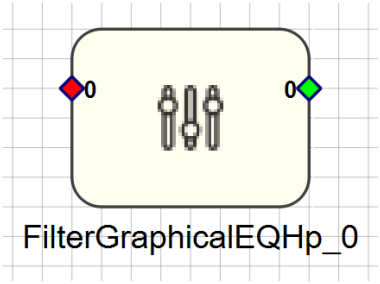


图 4.9.2-1 Graphic EQ (High Precision) 控件

4.10 Graphic EQ Group

4.10.1 Graphic EQ Group

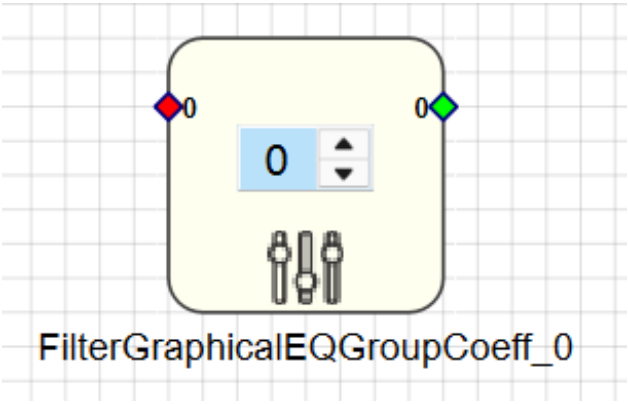


图 4.10.1-1 Graphic EQ Group 控件

功能说明：图形化EQ滤波器组，可以选择以下滤波器类型：

Type	说明
Parametric	参数滤波器
LowShelf	低架滤波器
HighShelf	高架滤波器
GeneralLP	低通滤波器
GeneralHP	高通滤波器
ButterLP	巴特沃斯低通滤波器
ButterHP	巴特沃斯高通滤波器
BesselLP	贝塞尔低通滤波器
BesselHP	贝塞尔高通滤波器
Allpass	全通滤波器

功能说明：图形化EQ滤波器, 基本功能和Graphic EQ一致，但是可以设置多个滤波器组，通过group去切换，工作时候只有选中的那组设置生效，控件上的Group=1表示当前使用的滤波器组group1。

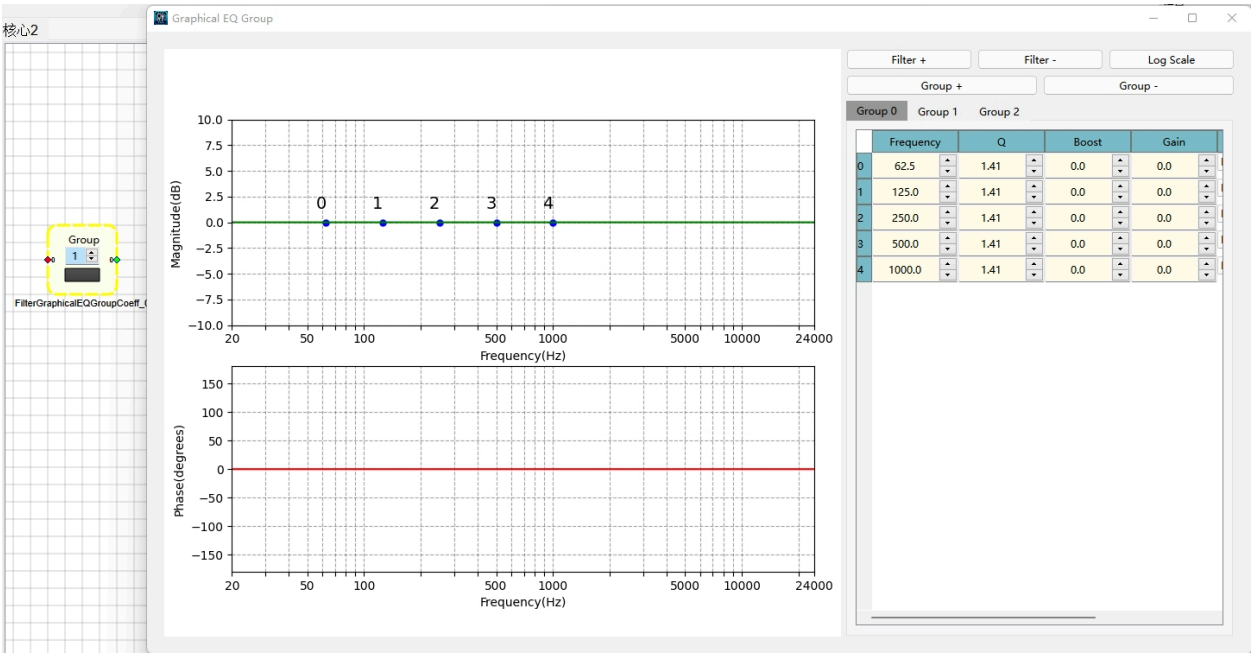


图 4.10.1-2 Graphic EQ 频响曲线图

注：双击组件，可以设置smooth_time，值越大，变化越缓。

4.10.2 Graphic EQ Group (Index Selectable)

通过Ext控制信号控制当前使用的Groupid，和Graphic EQ Group的切换方法一致，只是要通过外接控制信号来进行控制，可以通过DC控制输入控制信号。

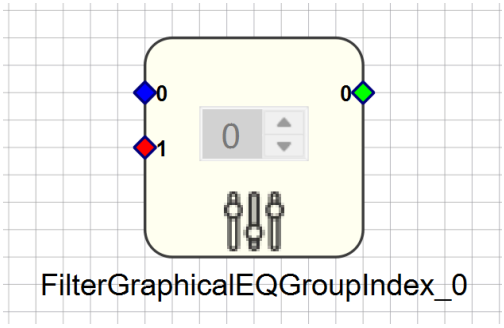


图 4.10.2-1 Graphic EQ Group (Index Selectable)控件

4.11 Crossover (Single Precision)

4.11.1 2-Way Crossover (Single Precision)

功能说明：2路分频器，其内部由两组并行的滤波器组组成，分别负责滤出低频和高频信号。模块会将输入的单路信号分为两路，分别经过这两组滤波器，从而得到两路输出信号。

模块如下图所示，当输入单通道数据时，输出通道0为低频信号，输出通道1为高频信号。而当用户输入多通道数据时，输出通道按照低频-高频-低频-高频的顺序排列。

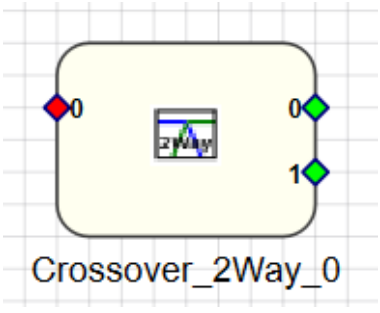


图 4.11.1-1 2-Way Crossover 模块

点击按钮进入参数配置页，可以看到低频部分和高频部分的频响曲线(绿线)以及它们合并后的频响曲线(黑线)。用户可以在下面设置各个滤波器的类型、增益、中心频率参数。其中invert参数用于决定是否将低频部分反相输出。

分频器的滤波器选择以下几种类型

Type	说明
Bypass	旁路，不做处理
Linkwitz-Riley 12	林克维兹-瑞利2阶
Linkwitz-Riley 24	林克维兹-瑞利4阶
Linkwitz-Riley 36	林克维兹-瑞利6阶
Linkwitz-Riley 48	林克维兹-瑞利8阶
Butterworth 12	巴特沃斯2阶
Butterworth 18	巴特沃斯3阶
Butterworth 24	巴特沃斯4阶
Butterworth 36	巴特沃斯6阶
Butterworth 48	巴特沃斯8阶
Bessel 12	贝塞尔2阶
Bessel 18	贝塞尔3阶
Bessel 24	贝塞尔4阶
Bessel 48	贝塞尔8阶

注：滤波器类型后面的数字为滚降速率，单位为dB/倍频，这个数值越大对应的频响曲线在过渡带越陡峭；

- 1阶的是6 dB/倍频
- 2阶的是12 dB/倍频
- 3阶的是18 dB/倍频
- 4阶的是24 dB/倍频
- 5阶的是30 dB/倍频
- 6阶的是36 dB/倍频
- 7阶的是42 dB/倍频
- 8阶的是48 dB/倍频

一下以低通Linkwitz-Riley12（二阶）和低通Linkwitz-Riley24（四阶）举例对比：

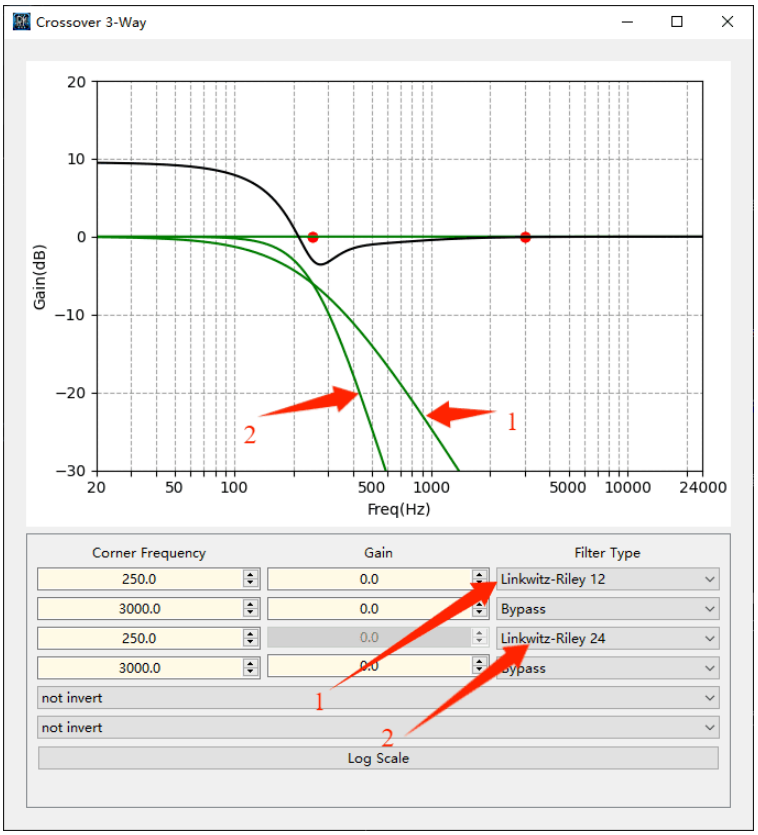


图 4.11.1-2 频响曲线对比（图中界面为3路分频器的设置界面）

可以看到，Linkwitz-Riley24在过渡带的衰减更为陡峭，滚降速率越大的曲线越陡峭，对应的频段划分越清晰。

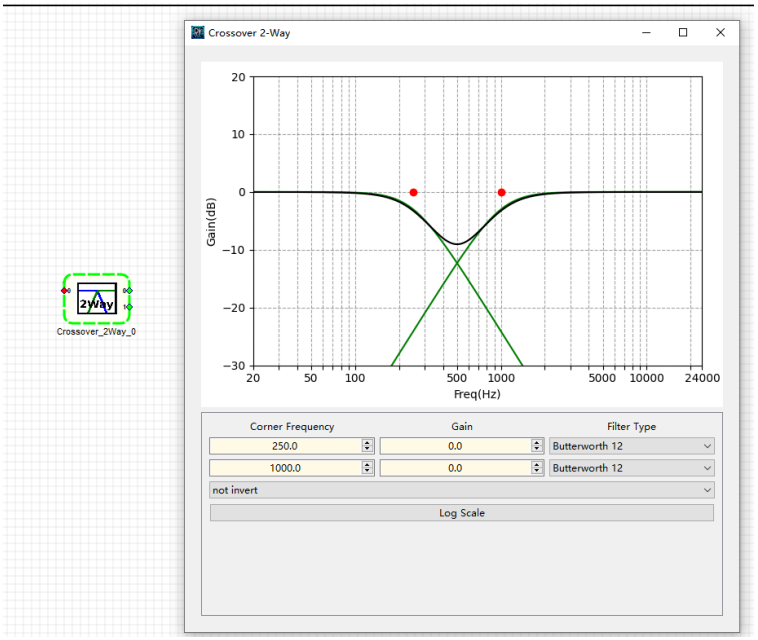


图 4.11.1-3 crossover_2way控件和参数设置界面

注：工具参数与在界面设置的参数位置对应

- 第一行是低通滤波器的LowFc，LowGainDB，LowType
- 第二行是高通滤波器的HighFc，HighGainDB，HighType
  而以下的工具参数的参数为则按 图纸工程文件下的xxx_param.json文件中参数序号排列

工具参数:

参数数量：7

参数名	参数位	类型	功能	单位	建议取值	默认取值
LowType	0	float	低频分量低通滤波器类型		0 ~ 13	5（butterworth12）
LowFc	1	float	低通滤波器截止频率	HZ	20 ~ 24000	250
LowGainDB	2	float	低通滤波器增益	dB	-30 ~ 30	0
HighType	3	float	高频分量高通滤波器类型		0 ~ 13	5（butterworth12）
HighFc	4	float	高通滤波器截止频率	HZ	20 ~ 24000	1000
HighGainDB	5	float	高通滤波器增益	dB	-30 ~ 30	0
Polarity	6	float	极性反向开关			0 （不使能）

- 下发参数 *： 本控件参数转换逻辑*
  参数数量： 12

参数名	参数位	类型	功能
SOSLow	0	float	低通滤波器个数
SOSHigh	1	float	高通滤波器个数
Low_B00	2	float	低通滤波器分子参数b0
Low_B10	3	float	低通滤波器分子参数b1
Low_B20	4	float	低通滤波器分子参数b2
Low_A10	5	float	低通滤波器分母参数a1
Low_A20	6	float	低通滤波器分母参数a2
High_B00	7	float	高通滤波器分子参数b0
High_B10	8	float	高通滤波器分子参数b1
High_B20	9	float	高通滤波器分子参数b2
High_A10	10	float	高通滤波器分母参数a1
High_A20	11	float	高通滤波器分母参数a2

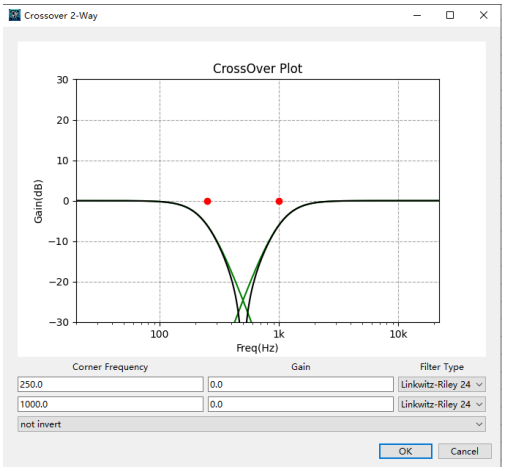


图 4.11.1-4 crossover 2way 配置

2路分频器只有低频和高频两个支路，低频分量对应第一个滤波器（低通滤波器）；高频分量对应第二个滤波器（高通滤波器）。

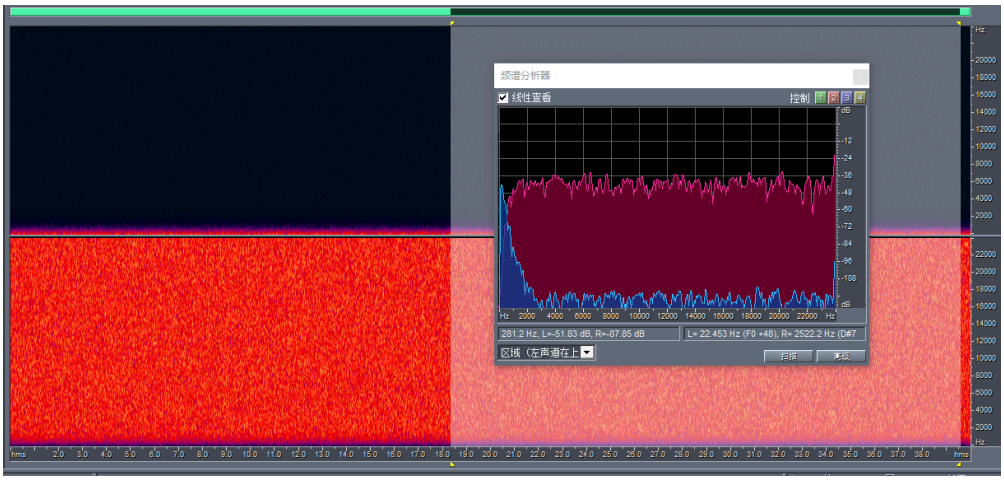


图 4.11.1-5 按上图配置的输出效果，输入为白噪声，图中中间横线以上为低频支路，下方为高频支路

4.11.2 3-Way Crossover(Single Precision)

功能说明：在前面2-Way Crossover的基础上，额外增加了一组滤波器，用于滤出中间频带的信号，注意，该滤波器不是带通滤波器，而是由串联的高通滤波器和低通滤波器组成。 可选滤波器类型* 与4.11.1crossover_2way 相同。

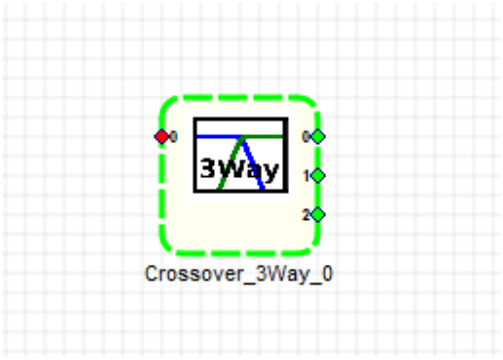


图 4.11.2-1 3-Way Crossover 配置界面

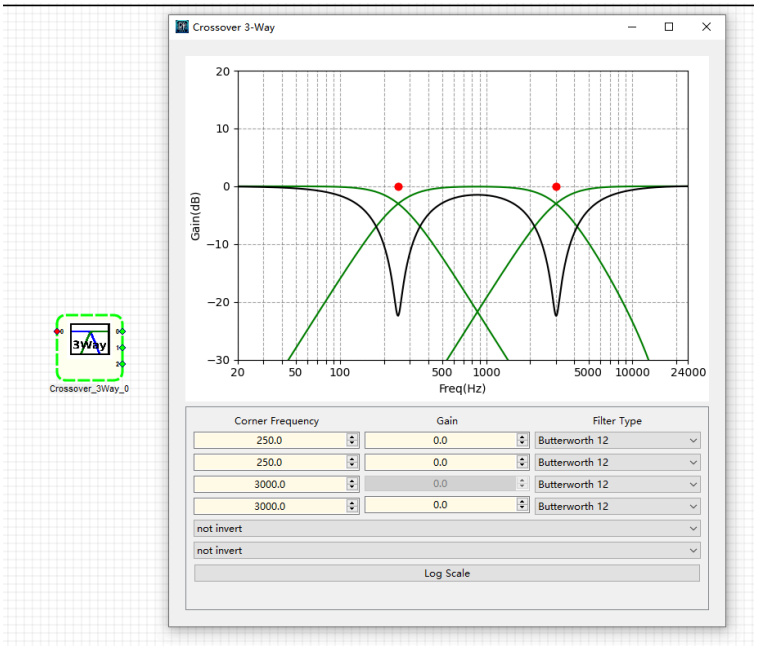


图 4.11.2-2 crossover_3way控件和参数设置界面

注：工具参数与在界面设置的参数位置对应

- 第一行是低频支路的低通滤波器的LowFc，LowGainDB，LowType

- 第二行是中频支路的低通滤波器的MedLPFc，MedLPGainDB，MedLPType
- 第三行是中频支路的高通滤波器的MedHpFc，MedHpGainDB，MedHpType
- 第四行是高频支路的高通滤波器的HighFc，HighGainDB，HighType

而以下的工具参数的参数为则按 图纸工程文件下的xxx_param.json文件中参数序号排列

工具参数：
参数数量：13

参数名	参数位	类型	功能	单位	建议取值	默认取值
LowType	0	float	低频分量低通滤波器类型		0 ~ 13	5（butterworth12）
LowFc	1	float	低通滤波器截止频率	HZ	20 ~ 24000	250
LowGainDB	2	float	低通滤波器增益	dB	-30 ~ 30	0
MedLpType	3	float	中频分量低通滤波器类型		0 ~ 13	5（butterworth12）
MedLPFc	4	float	中频分量低通滤波器截止频率	HZ	20 ~ 24000	250
MedHpType	5	float	中频分量高通滤波器类型		0 ~ 13	5（butterworth12）
MedHPFc	6	float	中频分量高通滤波器截止频率	HZ	20 ~ 24000	3000
MedGainDB	7	float	中频分量低通和高通滤波器增益	dB	-30 ~ 30	0
HighType	8	float	高频分量高通通滤波器类型		0 ~ 13	5（butterworth12）
HighFc	9	float	高通滤波器截止频率	HZ	20 ~ 24000	3000
HighGainDB	10	float	高通滤波器增益	dB	-30 ~ 30	0
Polarity	11	float	低频支路极性反向			0 （不使能）
Polarity	12	float	高频支路极性反向			0 （不使能）

- 下发参数： 本控件参数转换逻辑*
  参数数量： 24

参数名	参数位	类型	功能
SOSLow	0	float	低频分量低通滤波器个数
SOSHigh	1	float	高频分量高通滤波器个数
SOSMedLp	2	float	中频分量低通滤波器个数
SOSMEDHp	3	float	中频分量高通滤波器个数
Low_B00	4	float	低频分量低通滤波器分子参数b0
Low_B10	5	float	低频分量低通滤波器分子参数b1
Low_B20	6	float	低频分量低通滤波器分子参数b2
Low_A10	7	float	低频分量低通滤波器分母参数a1
Low_A20	8	float	低频分量低通滤波器分母参数a2
MedLp_Xn0	9 ~ 13	float	中频分量低通滤波器分子/分母参数
MedHp_Xn0	14 ~ 18	float	中频分量高通滤波器分子/分母参数
High_Xn0	19 ~ 23	float	高频分量高通滤波器分子/分母参数

点击按钮进入参数配置页，操作方法和Crossover 2Way一样，不过多了两组参数需要设置，这两组分别影响中间频带滤波器的高通部分和低通部分。而invert1参数用于决定是否将低频部分反相输出，invert2参数用于决定是否将高频部分反相输出。

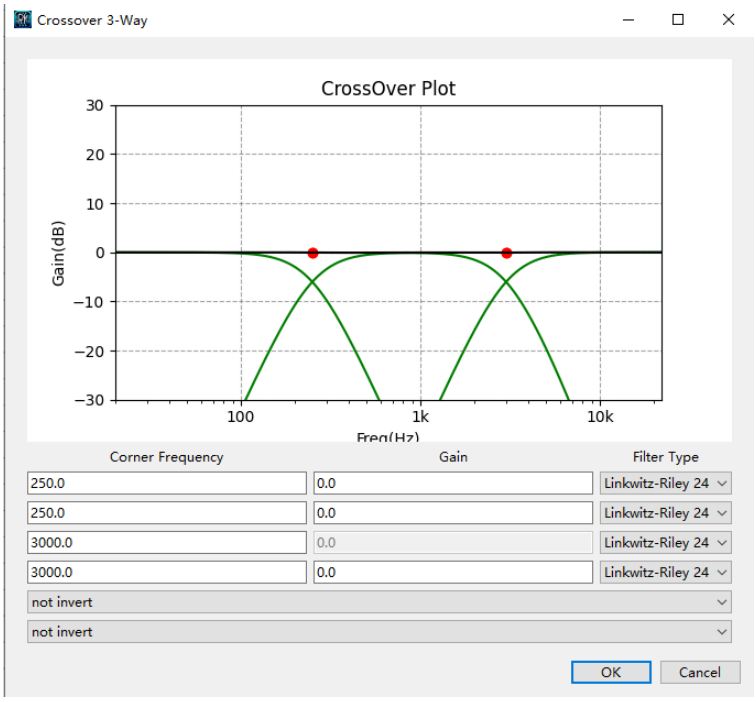
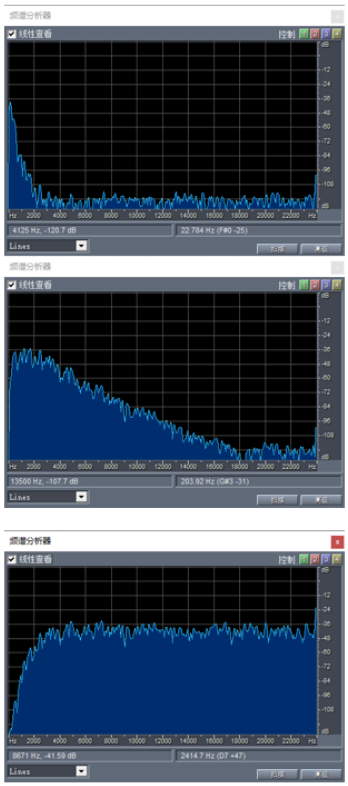


图 4.11.2-3 3-Way Crossover 配置界面

注：图中第一个滤波器(低通)输出低频信号，第二个（高通）和第三个（低通）滤波器串联输出中频信号，第四个（高通）滤波器输出高频信号；



低频分量频谱

中频分量频谱

高频分量频谱

图 4.11.2-4 按上图参数配置的输出结果，输入为白噪声

4.11.3 4-Way Crossover(Single Precision)

功能说明：在 2-Way Crossover 的基础上，额外增加了两组滤波器，用于构建中低频和中高频通道。中间频带的滤波器并非单一的带通滤波器，而是均由串联的高通滤波器和低通滤波器组成。 可选滤波器类型* 与4.11.1crossover_2way 相同。

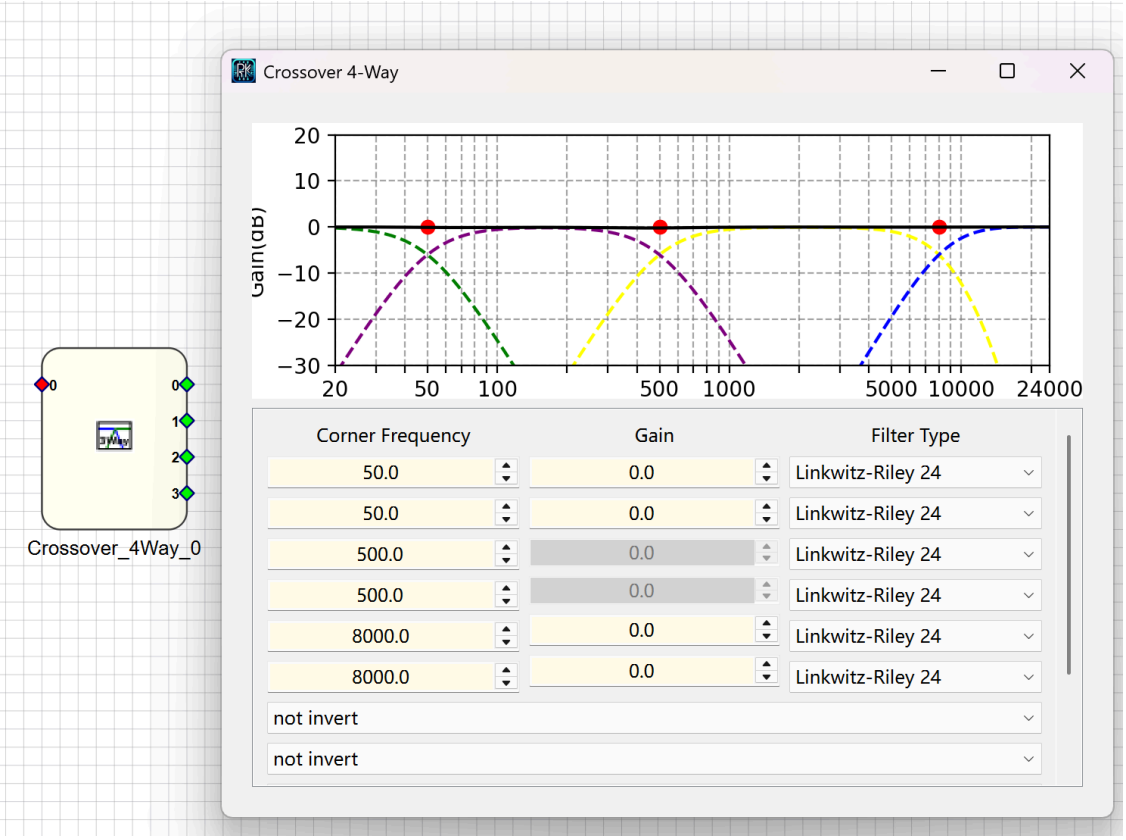


图 4.11.3-1 4-Way Crossover 控件和参数设置界面

4.12 Crossover (Double Precision)

双精度浮点运算的2-Way Crossover、3-Way Crossover和4-Way Crossover，功能与单精度浮点运算的相同，只是运算精度更高，但会耗费更多的计算资源。一般建议使用单精度浮点运算的模块。

4.13 Dynamaic EQ

功能说明：动态EQ均衡器，会根据信号的幅度来调整滤波器，例如，当一个信号的对应频段的强度逐渐增大，达到抑制门限之后，会按设定的比例对其进行增益，并向着设定的目标值靠近，从而达到动态均衡的效果；

实际的boost = （输入信号幅度 - 目标值）/ ratio

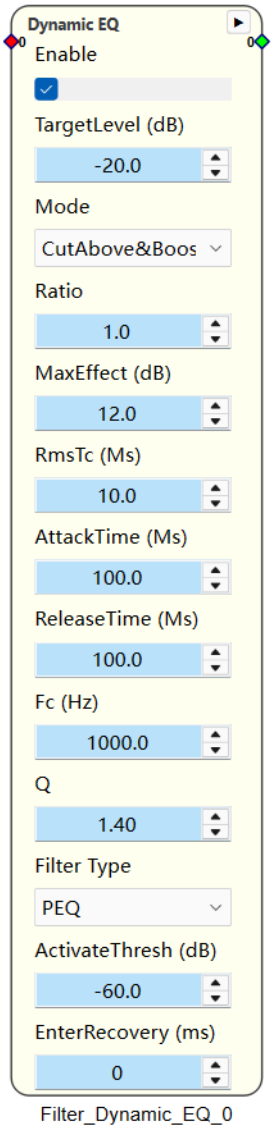


图 4.13-1 Dynamaic EQ 控件与设置界面

参数数量：13

参数名	参数位	类型	功能	单位	建议取值	默认取值
Enable	0	float	模块开关	无	0/1	1
TargetLevel	1	float	控制目标值	dB	-100 ~ 50	-20
Mode	2	float	工作模式选择	无	0~4	4
Ratio (in/out)	3	float	增益变化比例	无	1 ~ 100	1
MaxEffect	4	float	最大增益范围	dB	0 ~ 100	12
Rmstc*	5	float	平滑跟踪信号变化的速度，该值越大变化越慢	ms	0 ~ 2000	10
Attack Time*	6	float	增益上升速度	ms	0 ~ 2000	100
Release Time*	7	float	增益下降速度	ms	0 ~ 2000	100
Fc	8	float	工作中心频点	Hz	20~20000	1000
Q	9	float	滤波器Q值	无	0.1~16	1.4
FilterType	10	float	滤波器类型选择，Bypass/PEQ/LS/HS	无	0 ~ 3	1

参数名	参数位	类型	功能	单位	建议取值	默认取值
ActivateThresh	11	float	低于阈值时，boost会变为0db，如果值等于-120， 那么一直保持工作状态，这个阈值不生效	dB	0 ~ -120	-60
EnterRecovery	12	float	如果当前信号幅度小于ActivateThresh一段时间， 那么会强制进入recovery状态	ms	0 ~ 200	0

工作模式说明：

Boost Above：当信号幅度大于目标值时，增益会增加。

Boost Below：当信号幅度小于目标值时，增益会增加。

Cut Above：当信号幅度大于目标值时，增益会减少。

Cut Below：当信号幅度小于目标值时，增益会减少。

CutAbove & Boost Below：当信号幅度大于目标值时，增益会减少，当信号幅度小于目标值时，增益会增加。

FilterType说明：

Bypass：不使用滤波器。

PEQ：使用PEQ滤波器。

LS：使用低架滤波器。

HS：使用高架滤波器。

4.14 Text-In EQ

功能说明：单波段 EQ 滤波器。

从控件界面来看，该模块的功能与标准的 IIR - 2nd Order 非常相似，提供了一个标准的二阶滤波器单元，包含类型选择、频率、增益和 Q 值设置。该模块可以通过右键->add algo增加并联算法组，讲多个eq整合成一个算法控件，方便在当前页面调参。

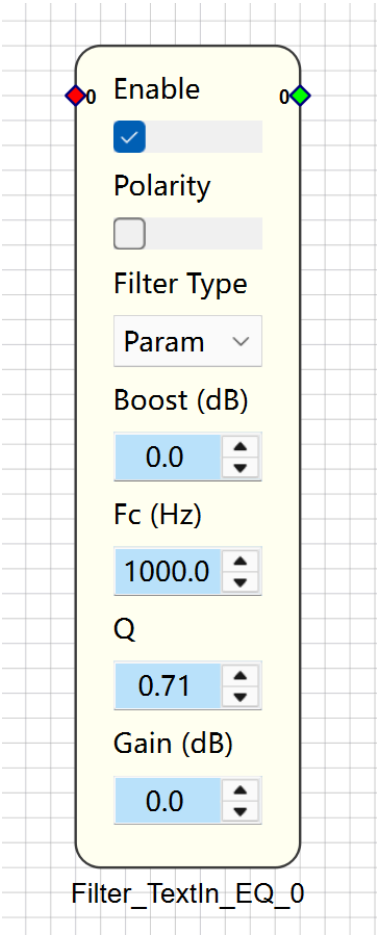


图 4.14-1 Text-In EQ 控件与设置界面

参数：7

4.15 Resample

4.15.1 Up-sampler

功能说明：上采样器（插值）。用于提高音频信号的采样率。
模块会根据设置的倍数（Up Factor）在原始采样点之间插入零值，从而提升采样率。通常建议勾选 Filter 选项以开启插值滤波器（Fir），用于消除混叠。

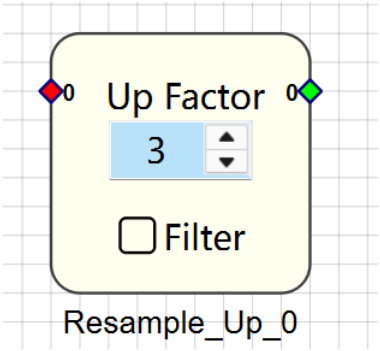


图 4.15.1-1 Up-sampler 控件与设置界面

4.15.2 Down-sampler

功能说明：下采样器（抽取）。用于降低音频信号的采样率。
模块会根据设置的倍数（Dn Factor）每隔 N 个点取一个样，从而降低数据量。建议勾选Filter选项（Fir）以开启抗混叠滤波器。

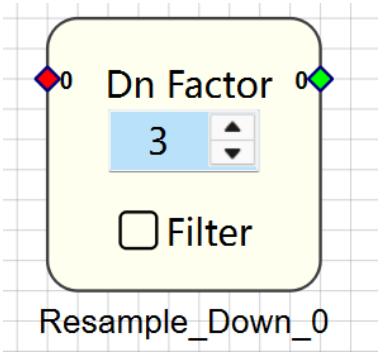


图 4.15.2-1 Down-sampler 控件与设置界面

4.16 Crossover EQ

功能说明：独立分频滤波器。
该模块用于对单路音频信号施加高通或低通滤波处理。通常用于构建自定义的分频网络，或者对扬声器进行单独的频段限制。用户可以自由选择滤波器的类型以及具体的滤波器特性（如巴特沃斯、贝塞尔、林克维兹-瑞利及其对应的斜率）。使用的滤波器和CrossOver插件一致，这个使用起来更加简单。

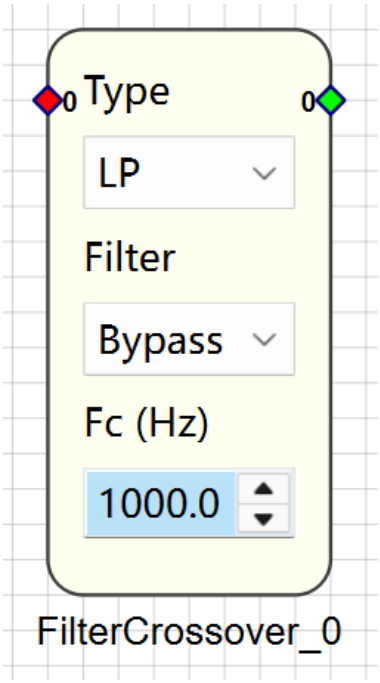


图 4.16.1-1 Crossover EQ 控件与设置界面

4.17 Allpass Delay

功能说明：该模块实现了一个带有延时线的全通滤波器。主要特性：它允许所有频率的信号以相同的幅度通过，但会根据频率引入不同的相位延迟。

典型应用：

1. 人工混响：作为施罗德混响算法的核心构建单元，用于增加回声密度而不染色彩。
2. 相位校正：用于在不改变音色的前提下调整多麦克风系统或分频系统的相位对齐。
3. 去相关：用于将单声道信号转换为伪立体声，或增加声场宽度。

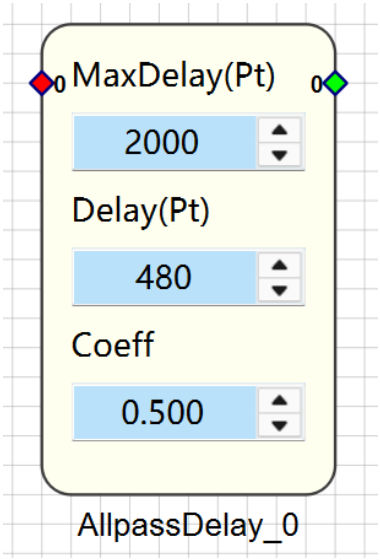


图 4.17.1-1 Allpass Delay 控件与设置界面

参数数量：3

参数名	参数位	类型	功能	单位	建议取值	默认取值
MaxDelay(Pt)	0	float	最大延时	Point	1~20000	2000
Delay(Pt)	1	float	当前延时	Point	1~20000	480

参数名	参数位	类型	功能	单位	建议取值	默认取值
Coeff	2	float	反馈系数	无	-1~1	0.5

5 IO

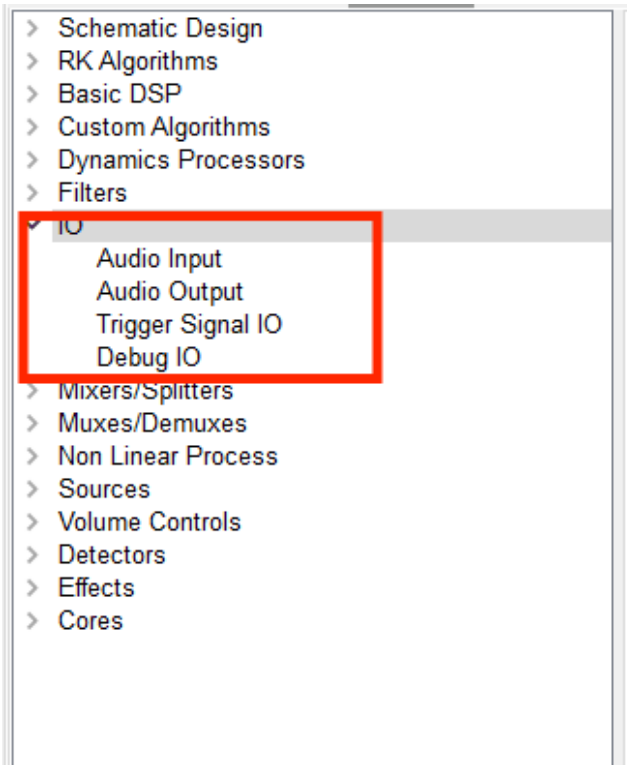


图 5-1 IO位于工具的位置

5.1 Audio Input

功能说明：输入。

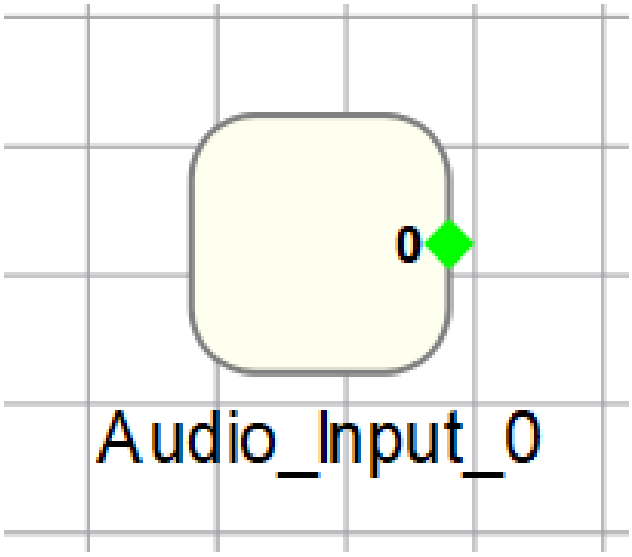


图 5.1-1 Audio Input 控件

参数：无

5.2 Audio Output

功能说明：输出。

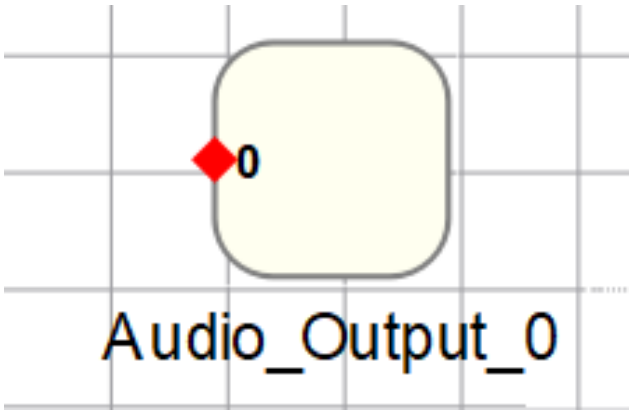


图 5.2-1 Audio Output 控件

参数：无

5.3 Trigger Signal IO

功能说明：GPIO口的信号触发。

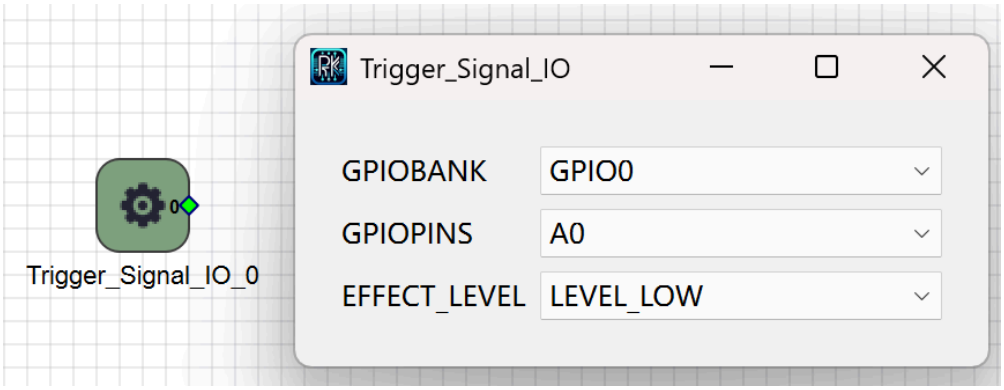


图 5.3-1 Trigger Signal IO 控件和设置界面

参数：3

参数名	参数位	类型	功能	单位	建议取值	默认取值
GPIOBANK	0	float	GPIO引脚所在的BANK		0~4	0
GPIOPINS	1	float	GPIO引脚号		0~31	0
EFFECT_LEVEL	2	float	触发方式， LEVEL开头的是电平触发，INT开头的是中断触发		0~6	0

5.4 Debug IO

功能说明：
debug io连接到output上，配置板端命令：rkstudio_module_debugio 0 1 10 10 1，便可以将核心0，插件id为1的输出通道10，送到output的通道10上。

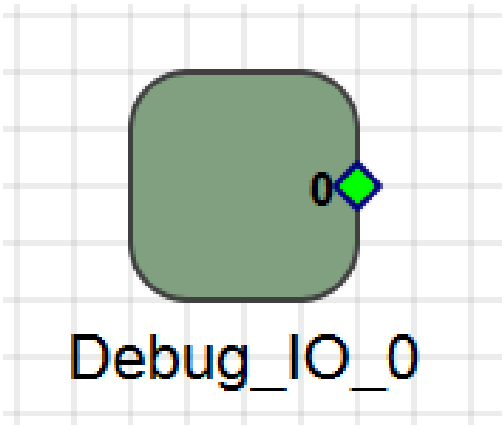


图 5.4-1 Debug IO 控件

参数：无

6 Mixers/Splitters

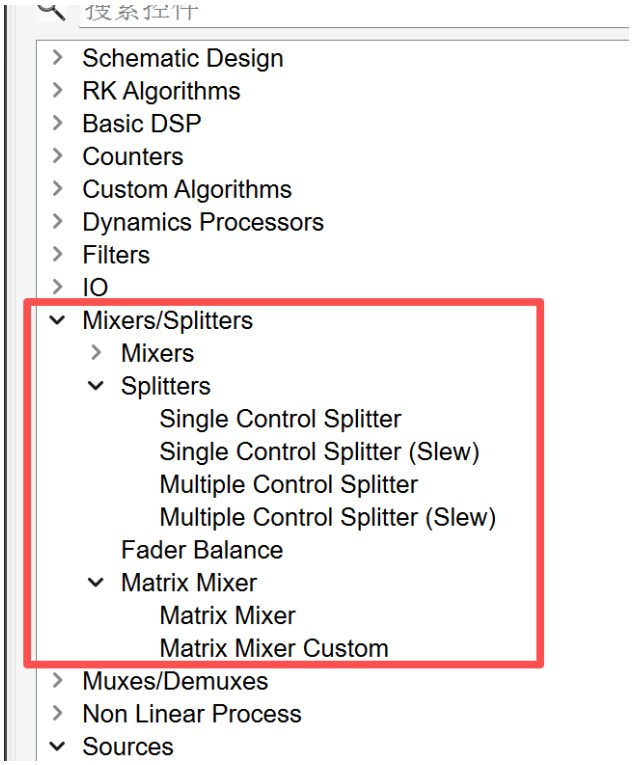


图 6-1 Mixers/Splitters位于工具的位置

6.1 Mixers

6.1.1 Signal Merge

功能说明：多路混成一路，除以通道数后输出。
计算公式：$output = (input0 + input1...)/channel$

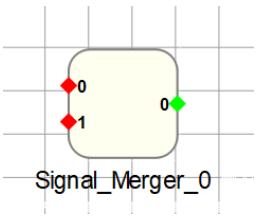


图 6.1.1-1 Signal Merge 控件和设置界面

参数：无

6.1.2 Single Control Mixer

功能说明：多路混成一路，所有输入都对应同一个增益。增益单位为dB（计算时，会将db转为线性增益*）。
计算公式：$output = input0 * gain + input1 * gain...$

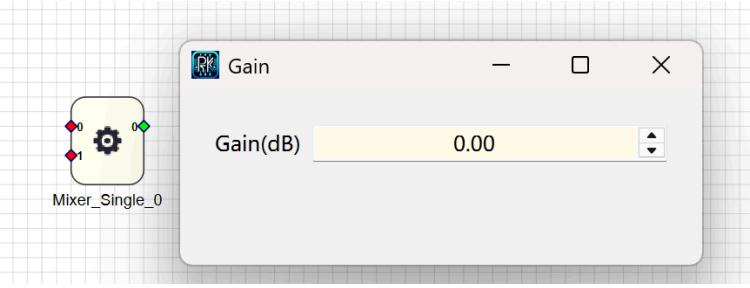


图 6.1.2-1 Single Control Mixer 控件和设置界面

参数数量：1

参数名	参数位	类型	功能	单位	建议取值	默认取值
Gain （dB）	0	float	增益	db	-138~24	0

实际效果展示：
参考6.1.3 Single Control Mixer*模块，本模块少了平滑功能。

注：dB转换为线性增益的公式为：

```
def db_to_linear(db, en_rms=False):
    if en_rms:
        return 10**(db/10)
    else:
        return 10**(db/20)
```

在本控件中，线性增益采用上方程序的else支路计算方式

```
    "custom_id": -1,
    "module_name": "Mixer_Single_0",
    "module_type": "Mixer_Single",
    "block_type": "EN_MIXERS_SINGLE",
    "group_idx": 0,
    "param_type": "float",
    "param_format": "f",
    "param_num": 1,
    "param_names": [    //  下发参数
        "Gain"        //线性增益
    ],
    "param_values": [
        3.1622776601683795  // 10^(10/20)
    ],
    "state_num": 1,
    "state_names": [  // 工具设置参数
        "Gain_dB"     // dB增益
    ],
    "state_values": [
        10.0
    ],
    "state_names_list": [],
    "state_values_list": []
}
```


图 6.1.2-2 工程图纸的json文件

在软件设置的参数（称为工具参数），有些会经过一定的转换传入算法计算（称为下发参数），工具参数和下发参数可以打开对应图纸json文件查看。

6.1.3 Single Control Mixer (Slew)

功能说明：多路混成一路，所有输入都对应同一个增益。增益单位为dB（计算时，会将db转为线性增益*），gain变化时，会根据设置的slew进行平滑。

计算公式：$output = input0 * gain + input1 * gain...$

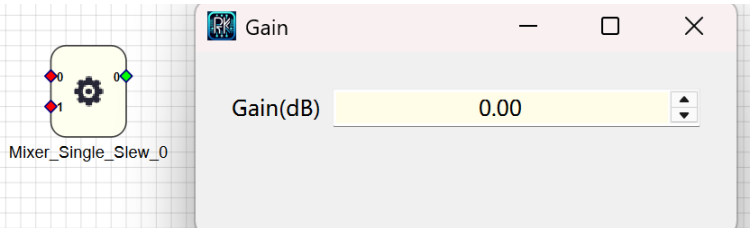


图 6.1.3-1 Single Control Mixer (Slew) 控件和设置界面

参数数量：1

参数名	参数位	类型	功能	单位	建议取值	默认取值
Gain （dB）	0	float	增益	db	无	0

实际效果展示：

输入数据为两个通道，左声道为白噪，右声道为正弦波，有如下两个典型工况：

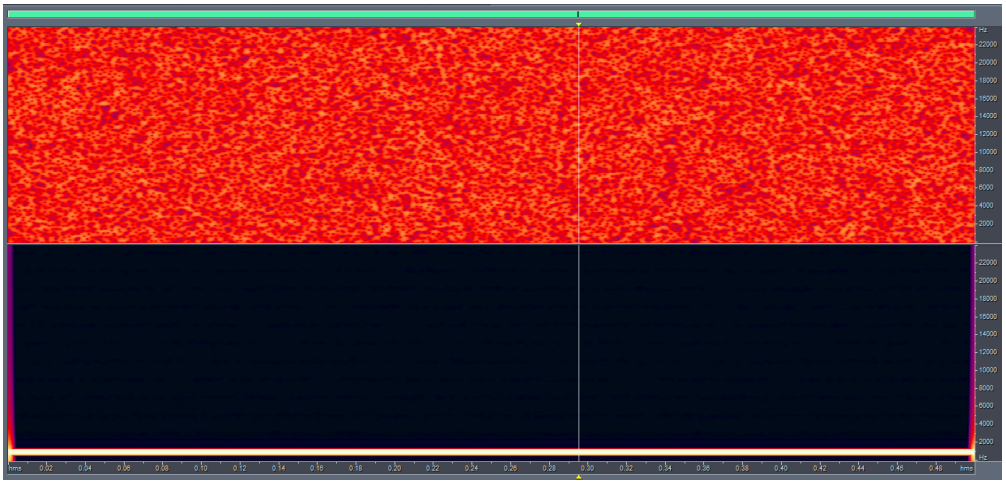


图 6.1.3-2 输入波形频域图

工况一：参数设置为0， 即gain设置为0db；
工况二：参数设置为-12， 即gain设置为-12db。

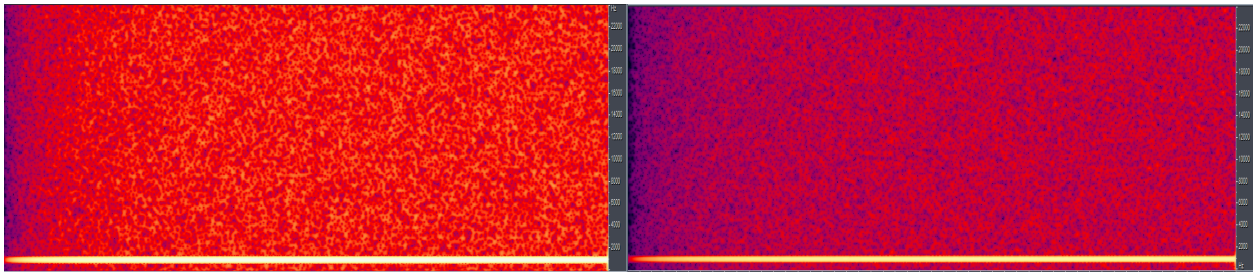


图 6.1.3-3 RKStudio输出频域，左图为工况1，右图为工况2

可以看到初始的时候会有个数据平滑的效果。

6.1.4 Multiple Control Mixer

功能说明：多路混成一路，每个输入分别对应不同增益。增益单位为dB（计算时，会将db转为线性增益*）。
计算公式：$output = input0 * gain0 + input1 * gain1...$

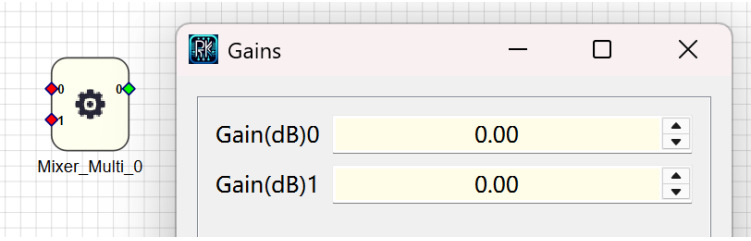


图 6.1.4-1 Multiple Control Mixer 控件和设置界面

参数数量：input_pin

参数名	参数位	类型	功能	单位	建议取值	默认取值
Gain(dB)0	0	float	第0通道增益	db	-138 ~ 24	0
Gain(dB)1	1	float	第1通道增益	db	-138 ~ 24	0

实际效果展示：
效果和Single Control Mixer*相似，本模块可以分别控制每路的增益。

6.1.5 Multiple Control Mixer (Slew)

功能说明：多路混成一路，每个输入分别对应不同增益。增益单位为dB（计算时，会将db转为线性增益*），和上个模块相比，多了slew平滑的功能，gain变化时，会根据设置的slew进行平滑。
计算公式：$output = input0 * gain0 + input1 * gain1...$

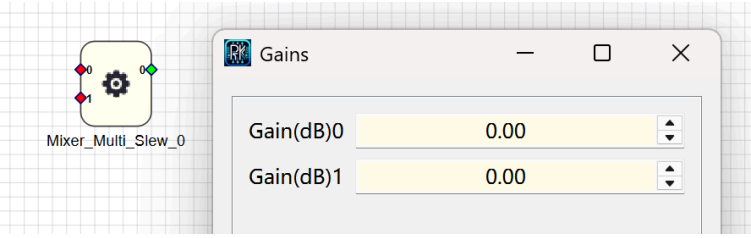


图 6.1.5-1 Multiple Control Mixer (Slew) 控件和设置界面

参数数量：input_pin

参数名	参数位	类型	功能	单位	建议取值	默认取值
Gain(dB)0	0	float	第0通道增益	db	-138 ~ 24	0
Gain(dB)1	1	float	第1通道增益	db	-138 ~ 24	0

实际效果展示：
效果和Single Control Mixer*相似，本模块可以分别控制每路的增益。

6.1.6 Cross Mixer

功能说明：多路混成多路，gain存于参数中。如果是2路输入，3路输出，那么gain的顺序是：

- gain00, gain10
- gain01, gain11
- gain02, gain12

计算公式：$output0 = input0 * gain00 + input1 * gain10$

$$output1 = input0 * gain01 + input1 * gain11$$

$$output2 = input0 * gain02 + input1 * gain12$$

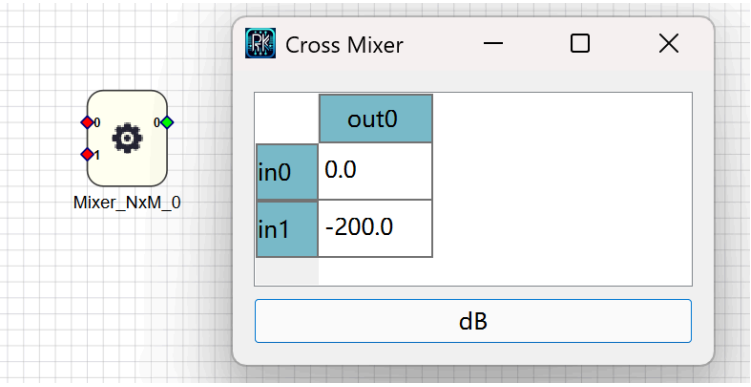


图 6.1.6-1 Cross Mixer 控件和设置界面

增益单位为下方按键显示单位，点击下方的按键“linear”可切换成dB单位。但在传入算法时统一转换为线性增益。

参数数量：input_pin * output_pin

参数名	参数位	类型	功能	单位	建议取值	默认取值
Gain00	0	float	增益	db	-138 ~ 24	0
Gain01	1	float	增益	db	-138 ~ 24	0
Gain02	2	float	增益	db	-138 ~ 24	0
Gain10	3	float	增益	db	-138 ~ 24	0
Gain11	4	float	增益	db	-138 ~ 24	0
Gain12	5	float	增益	db	-138 ~ 24	0

参数名	参数位	类型	功能	单位	建议取值	默认取值
Gain_x_y	x*output_pin+y	float	增益	db	-138 ~ 24	0

实际效果展示：
效果和Single Control Mixer*相似，本模块可以有多个输出，并且可以控制每路的增益。

6.1.7 Cross Mixer (Slew)

功能说明：多路混成多路，gain存于参数中。如果是2路输入，3路输出，那么gain的顺序是：

- gain00, gain01, gain02
- gain10, gain11, gain12

计算公式：$output0 = input0 * gain00 + input1 * gain10$

$$output1 = input0 * gain01 + input1 * gain11$$

$$output2 = input0 * gain02 + input1 * gain12$$

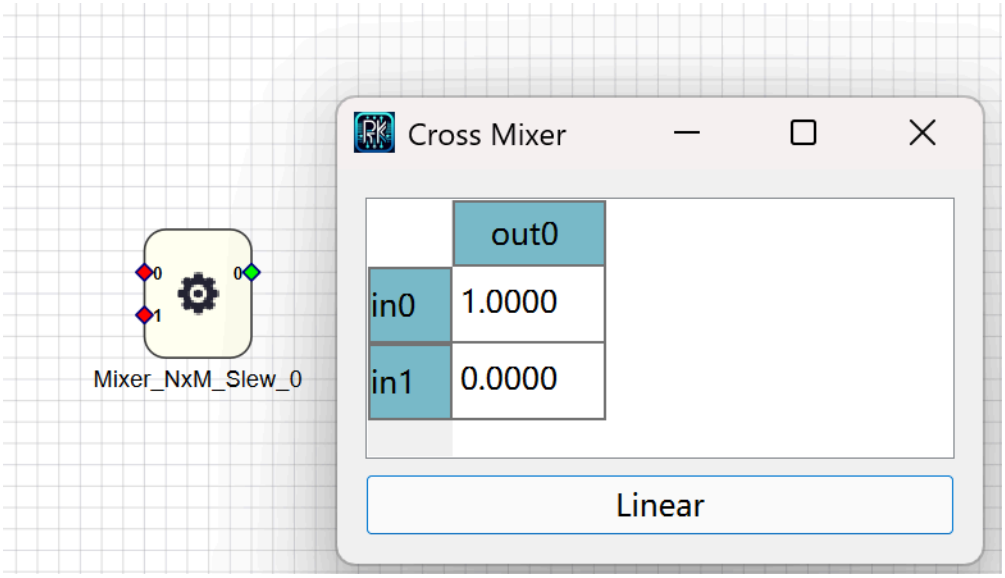


图 6.1.7-1 Cross Mixer (Slew) 控件和设置界面
增益单位为下方按键显示单位，点击下方的按键“linear”可切换成dB单位。但在传入算法时统一转换为线性增益。

参数数量：input_pin * output_pin

参数名	参数位	类型	功能	单位	建议取值	默认取值
Gain00	0	float	增益	db	-138 ~ 24	0
Gain01	1	float	增益	db	-138 ~ 24	0
Gain02	2	float	增益	db	-138 ~ 24	0
Gain10	3	float	增益	db	-138 ~ 24	0
Gain11	4	float	增益	db	-138 ~ 24	0
Gain12	5	float	增益	db	-138 ~ 24	0
Gain_x_y	x*output_pin+y	float	增益	db	-138 ~ 24	0

实际效果展示：
效果和Single Control Mixer*相似，本模块可以有多个输出，并且可以控制每路的增益。

6.1.8 Cross Mixer Fast

功能说明：多路混成多路，基本功能和Cross Mixer*相似。区别在于这个模块计算效率会更高，gain为0的通道不进行计算，但内存占用会更大。

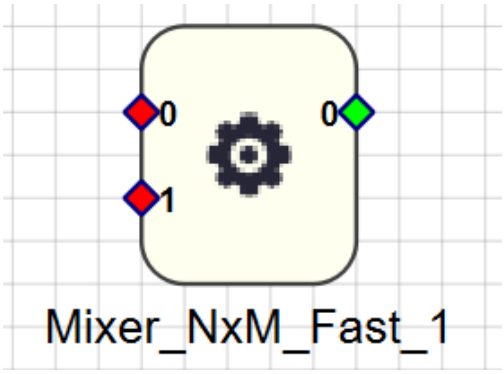


图 6.1.8-1 Cross Mixer Fast 控件和设置界面

6.1.9 Cross Mixer Fast (Slew)

功能说明：多路混成多路，基本功能和Cross Mixer (Slew)*相似。区别在于这个模块计算效率会更高，gain为0的通道不进行计算，但内存占用会更大。并且在参数变化时，对应的输出通道使用淡入淡出的方式进行平滑。

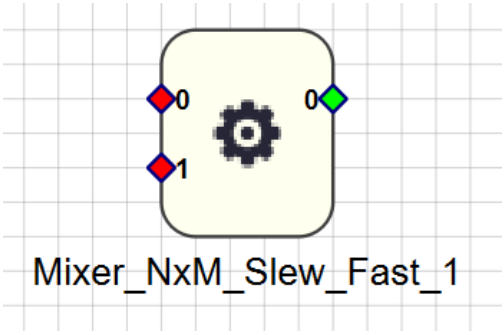


图 6.1.9-1 Cross Mixer Fast (Slew) 控件和设置界面

6.1.10 Cross Mixer Slew (Index)

功能说明：带索引功能的平滑混音矩阵。与普通 Cross Mixer 不同，该模块允许用户预设多组混音增益配置,并且通过控制信号进行Idx的切换。

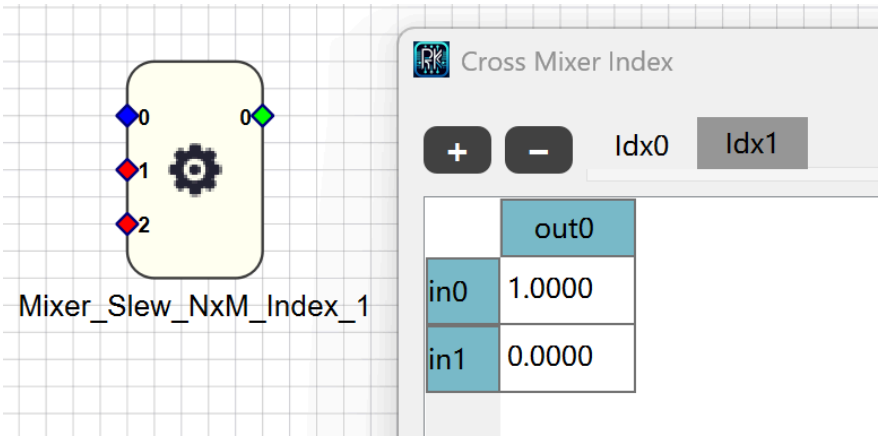


图 6.1.10-1 Cross Mixer Slew (Index) 控件和设置界面

6.2 Splitters

6.2.1 Single Control Splitter

功能说明：一路分为多路，所有输出都对应同一个增益。增益单位为dB（计算时，会将db转为线性增益*）。

计算公式：$output0 = input * gain$

$$output1 = input * gain$$

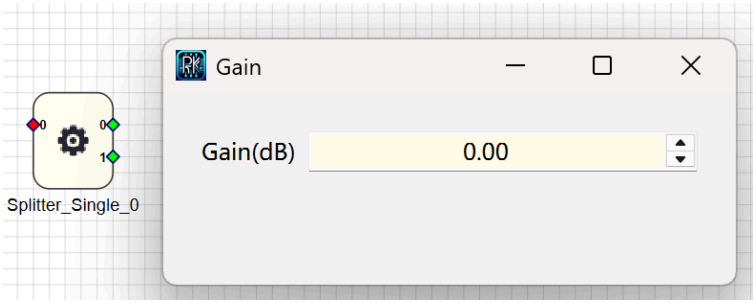


图 6.2.1-1 Single Control Splitter 控件和设置界面

参数数量：1

参数名	参数位	类型	功能	单位	建议取值	默认取值
Gain(dB)	0	float	增益	db	-138~24	0

实际效果展示：
效果和 Multiple Control Splitter (Slew)*相似，本模块一个增益控制多个输出通道。

6.2.2 Single Control Splitter (Slew)

功能说明：一路分为多路，所有输出都对应同一个增益。增益单位为dB（计算时，会将db转为线性增益*）。

计算公式：$output0 = input * gain$

$$output1 = input * gain$$

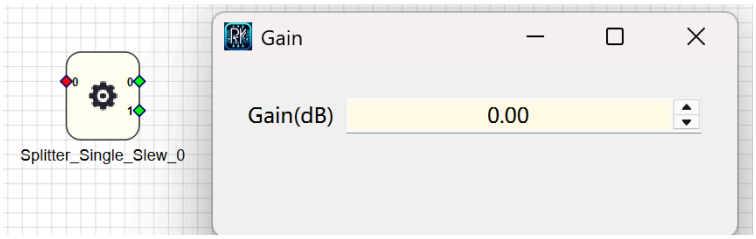


图 6.2.2-1 Single Control Splitter (Slew) 控件和设置界面

参数数量：1

参数名	参数位	类型	功能	单位	建议取值	默认取值
Gain(dB)	0	float	增益	db	-138~24	0

实际效果展示：
效果和 Multiple Control Splitter (Slew)*相似，本模块一个增益控制多个输出通道。

6.2.3 Multiple Control Splitter

功能说明：一路分为多路，每个输出分别对应不同增益。增益单位为dB（计算时，会将db转为线性增益*）。

计算公式：$output0 = input * gain0...$

$$output1 = input * gain1...$$

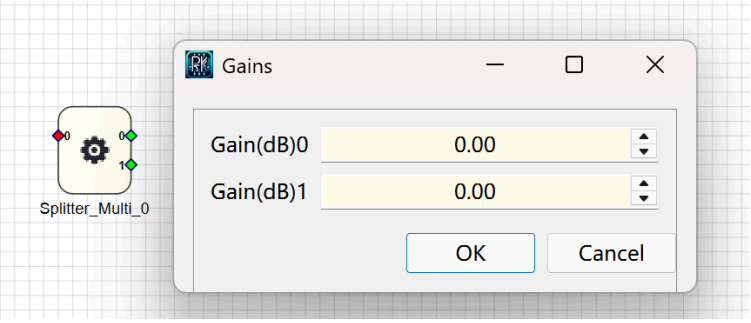


图 6.2.3-1 Multiple Control Splitter 控件和设置界面

参数数量：output_pin

参数名	参数位	类型	功能	单位	建议取值	默认取值
Gain(dB)0	0	float	增益	db	-138~24	0
Gain(dB)1	1	float	增益	db	-138~24	0

实际效果展示：
效果和 Multiple Control Splitter (Slew)* 相似，本模块少了平滑功能。

6.2.4 Multiple Control Splitter (Slew)

功能说明：一路分为多路，每个输出分别对应不同增益。增益单位为dB（计算时，会将db转为线性增益*）。

计算公式：$output0 = input * gain0...$

$output1 = input * gain1...$

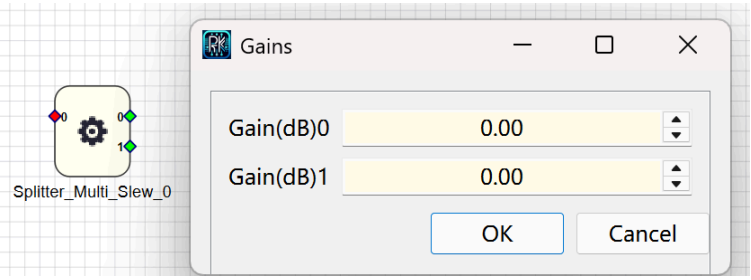


图 6.2.4-1 Multiple Control Splitter 控件和设置界面

参数数量：output_pin

参数名	参数位	类型	功能	单位	建议取值	默认取值
Gain(dB)0	0	float	增益	db	-138~24	0
Gain(dB)1	1	float	增益	db	-138~24	0

实际效果展示：
输入单声道白噪，输出两通道。
参数按顺序设置为-10, -20，即gina0为-10db，gain1为-20db。

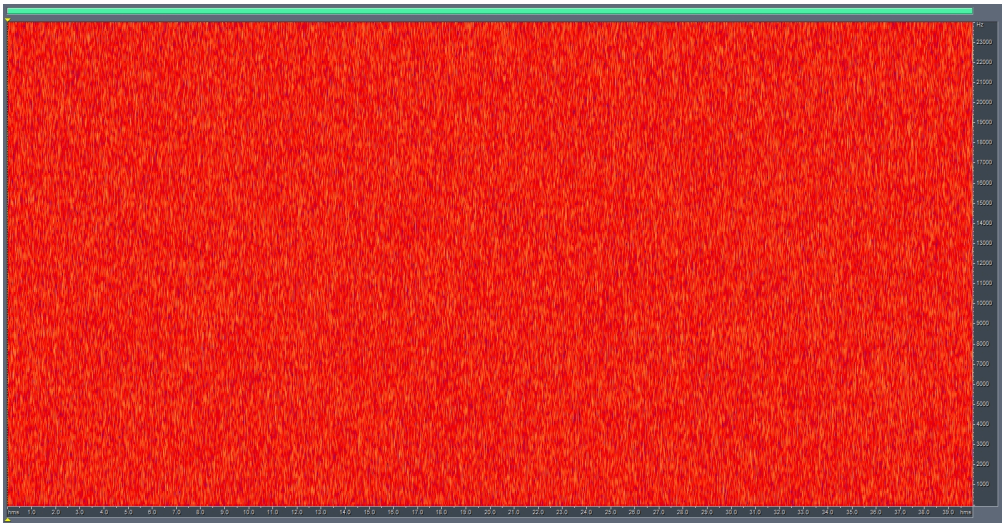


图 6.2.4-2 输入白噪频域图

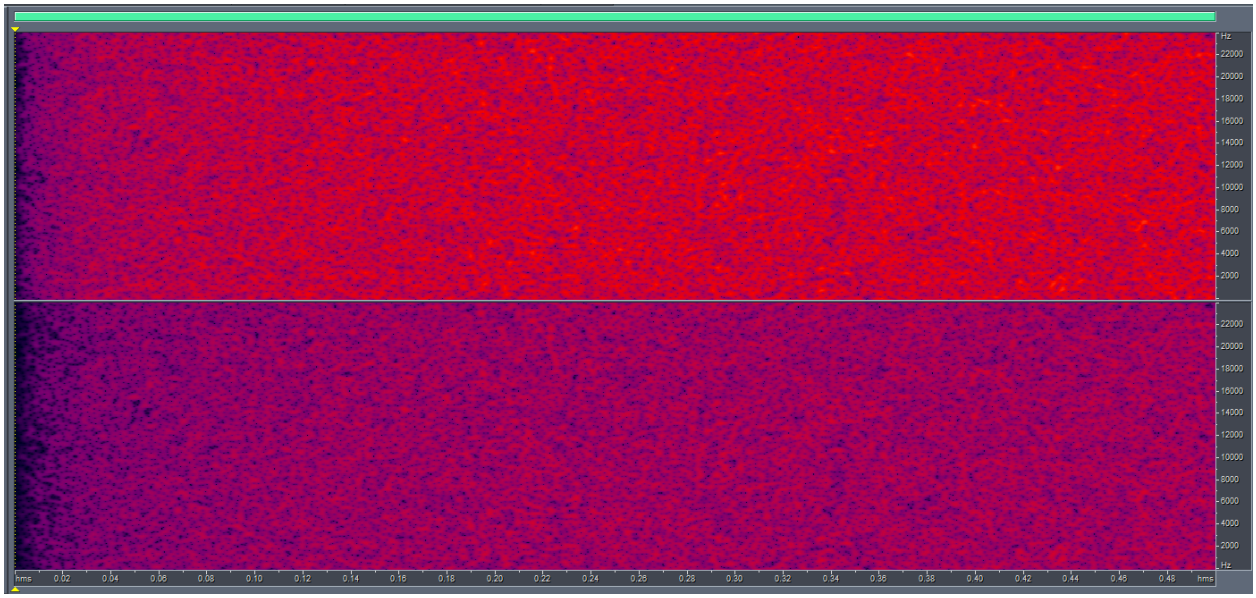


图 6.2.4-3 RKStudio输出频域(图中中间横线上方对应-10dB增益，下方对应-20dB增益)

6.3 Fader Balance

功能说明：实现声场均衡，fader表示声场的前后平衡，balance表示声场的左右平衡。step对应的是增益的调整的步进，step为0时，声场处于中间位置，一般增益不变。控件中的fader表示step为1时，fader的增益对应[0/0/0/-1/-1/0],这时候FL和FR不变，但是RL和RR增益为-1，这时候声场就会往前一些。表格中的增益数据，要根据实际的情况进行调整。

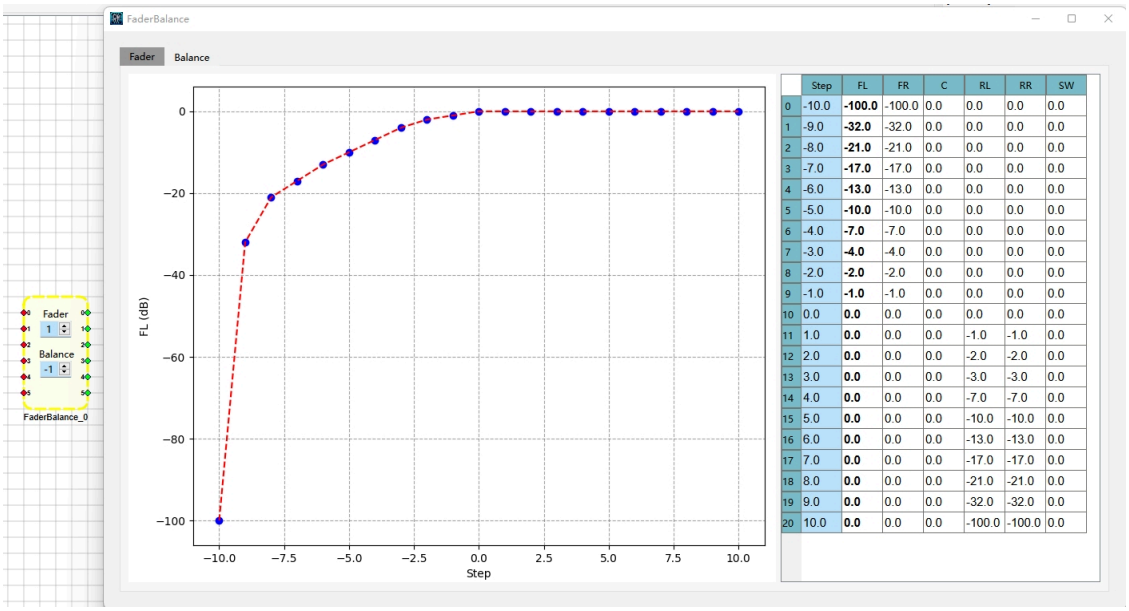


图 6.3-1 Fader Balance 控件和设置界面

由于参数比较多，下面列出控制参数：

参数数量：2

参数名	参数位	类型	功能	单位	建议取值	默认取值
FaderIdx	0	float	fader的步进	无	-10-10	0
BalanceIdx	1	float	balance的步进	无	-10-10	0

6.4 Matrix Mixer

功能说明：数字矩阵混音器，可以实现可变多通道的混音，输入通道数和输出通道数可以自定义，一般用于输入通道数会变化，输出通道数固定的场景。控件输入通道需要设置到可变通道数的最大值，比如输入通道数为6，然后可以在layout中选择使用哪几个通道进行混音，Coefficient则是混音的权重.

目前这个插件只能部署一个实例，只能配合rkplayer进行参数切换。如果要部署多个实例或者进行自行切换参数，需要用Matrix Mixer Custom插件。

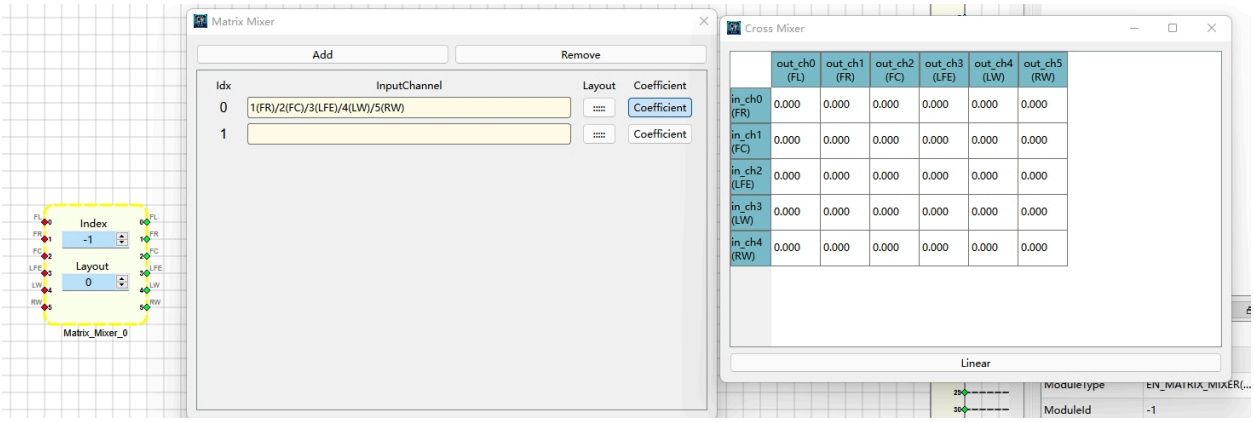


图 6.4-1 Matrix Mixer 控件和设置界面

控件中的Index表示当前选择第几个idx，也就是选择控件的第几组配置。假如index设置成-1，那么就会使用Layout来选择。如下图中的Layout选择了2和4通道，那么对应的layout的值为：1 << 2 | 1 << 4 = 0b00010100=20，（在对应bit位置1表示选中）。

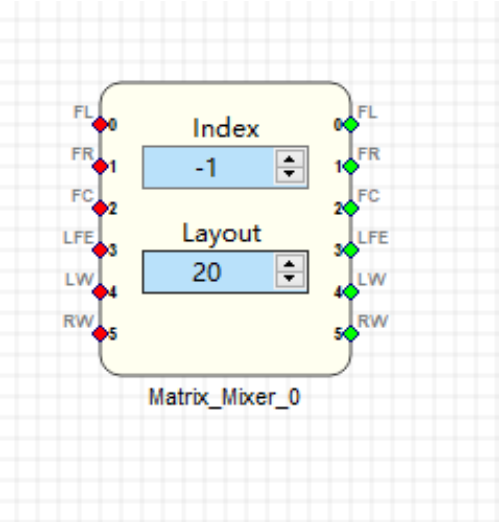


图 6.4-3 若将idx设置为-1时，将按照layout（20=0b00010100）的值进行选中2通道和4通道（对应bit位，从二进制数右边第0位开始数）输入信号混音；若将idx设置为0，则按照图6.4-3的第idx=0的配置

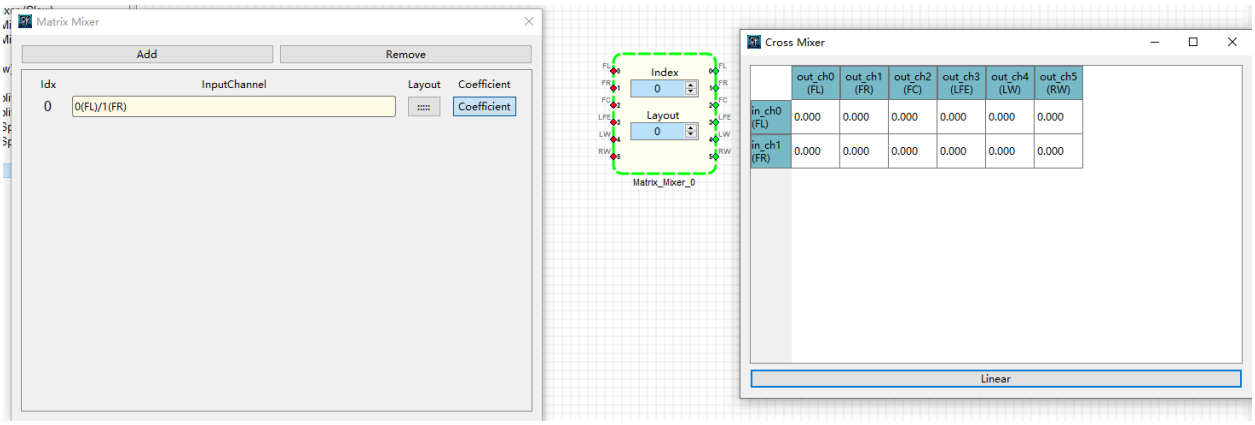


图 6.4-3 点击coefficient设置对应的coeff权重

如上我们选择了idx=0，选中FL和FR两个通道进行混音，输出为6个通道。 则输出为

out_fl=in_fl*coeff_0_0+in_fr*coeff_0_1;

out_fr=in_fl*coeff_1_0+in_fr*coeff_1_1;

out_fc=in_fl*coeff_2_0+in_fr*coeff_2_1;

依次类推.....

由于参数比较多，下面列出控制参数： 参数数量：2

参数名	参数位	类型	功能	单位	建议取值	默认取值
Index	0	float	选择当前使用第几组配置	无	0 ~ 15	-1
Layout	1	uint32_t	选择当前输入通道组成的Hex	无	0 ~ 0xffffffff	0

6.5 Matrix Mixer Custom插件

功能参考6.4 Matrix Mixer，适用于自行设参和部署插件。

7 Muxes/DeMuxes

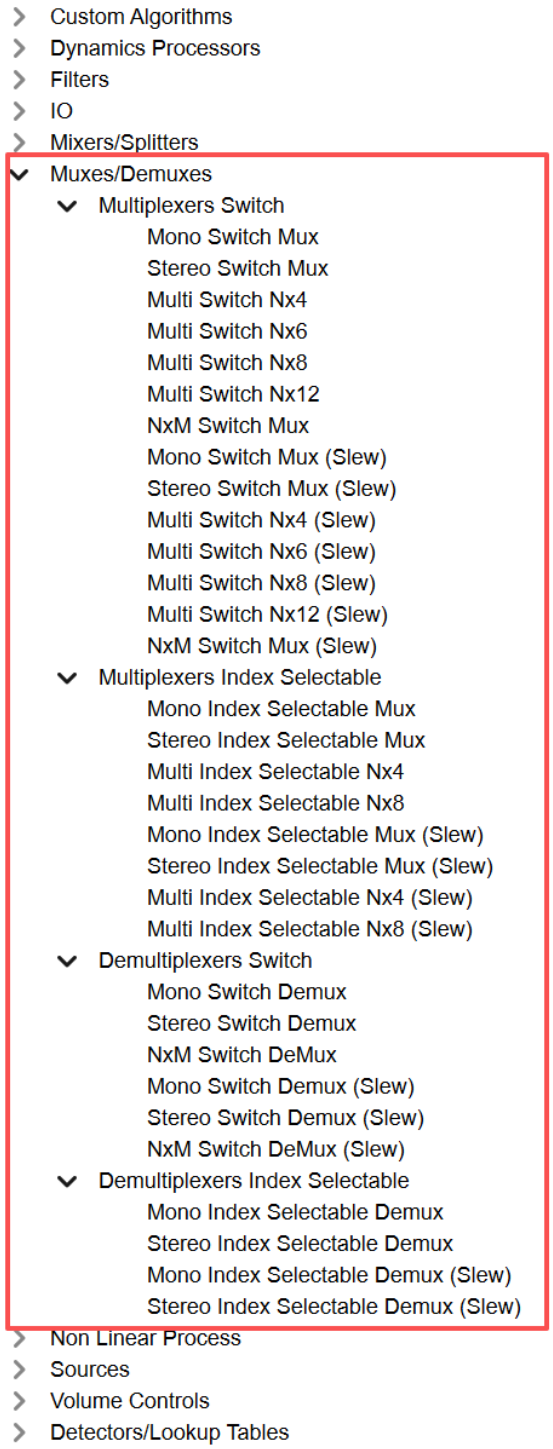


图 7-1 Muxes/DeMuxes位于工具的位置

首先简单介绍Muxes/DeMuxes的概念。

复用器(Multiplexers, Muxes)，根据控制信号，从多组输入(每组输入可以是单声道、双声道或多声道)中选择一组路由到输出。以图7-2为例，该复用器输入输出数据为4声道，根据控制信号的不同，复用器从第一组和第二组中选择一组，路由到输出上。

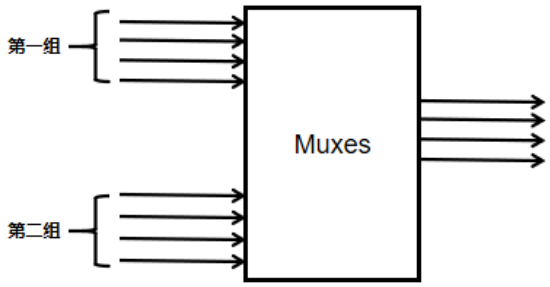


图 7-2 2选1复用器，输入输出信号为4声道

解复用器(DeMultiplexers, DeMuxes)，根据控制信号，将一组输入(输入可以是单声道或双声道)路由到多组输出中的一组，其他组输出则输出空信号。以图7-3为例，解复用器可以将输入路由到第一组输出上，并在第二组输出上输出0。

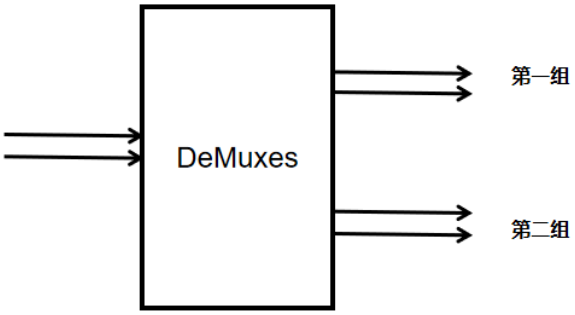


图 7-3 2路解复用器，输入输出信号为2声道

7.1 Multiplexers Switch

7.1.1 Mono Switch Mux

功能说明：根据按钮(Switch)，从多组输入中选择一组输出，每组输入都是单声道。默认是两组选一组，用户可以右键->Add Point来添加更多组输入。

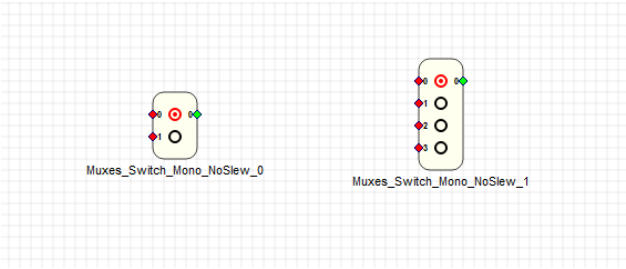


图 7.1.1-1 单通道复用器

实际效果展示：

输入两通道数据，通道1和通道2为不同类型数据，输出单通道，有下图所示两个典型工况：

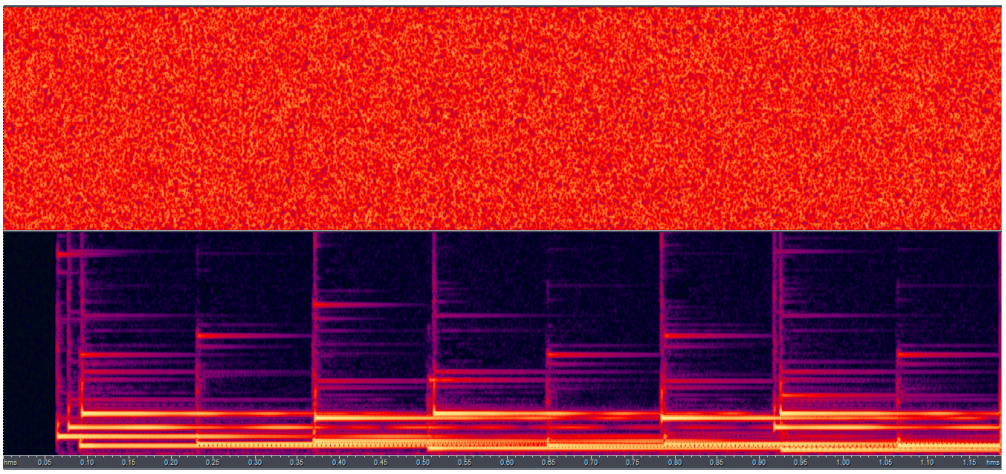


图 7.1.1-2 输入数据，通道1为白噪声，通道2为音乐

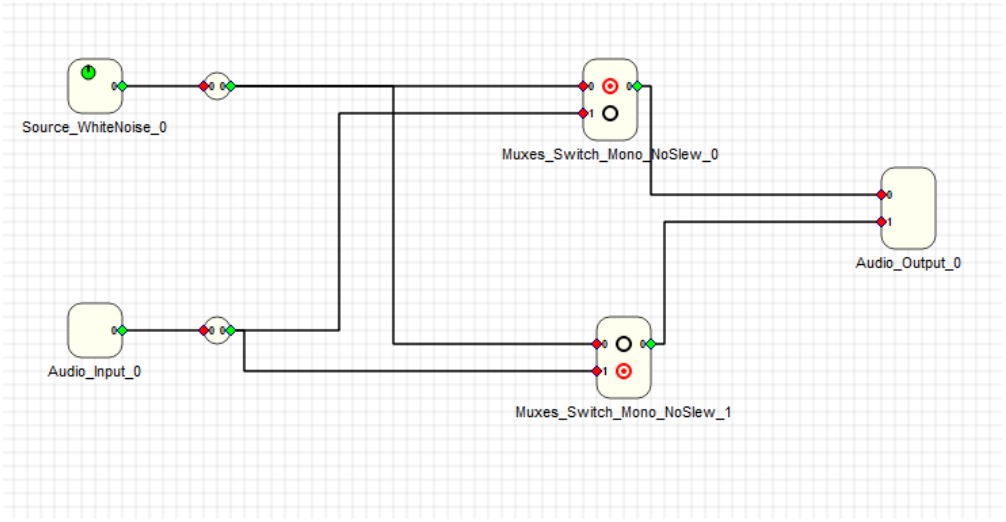


图 7.1.1-3 工况1，选择通道0，工况2，选择通道1

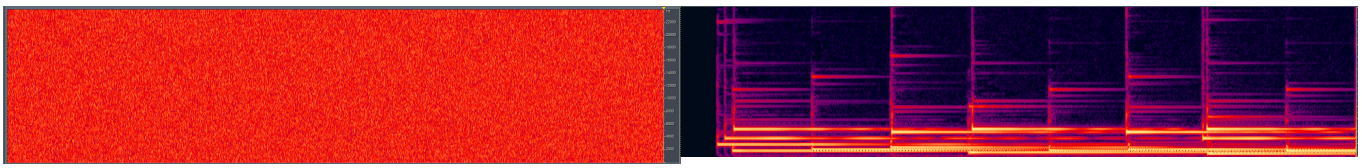


图 7.1.1-4 RKStudio输出频域，左图为工况1，右图为工况2

可以看到，不同的Switch选择对应的输出不一样。

7.1.2 Stereo Switch Mux

功能说明：与7.1 Mono Switch Mux类似，但每组输入是双声道，输出是双声道。

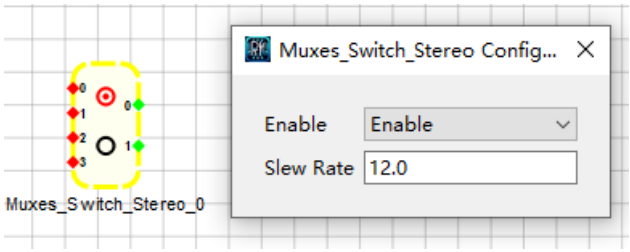


图 7.1.2-1 立体声复用器，与7.1的区别在于，每组输入和输出都是双声道数据，以上图为例，Switch选择第一组，则输出第一组的左声道和右声道，即通道0和1，选择第二组，则输出第二组的左声道和右声道，即通道2和3。

7.1.3 Multi Switch Nx4

功能说明：与7.1.1 Mono Switch Mux类似，但每组输入是4声道，输出是4声道。

7.1.4 Multi Switch Nx6

功能说明：与7.1.1 Mono Switch Mux类似，但每组输入是6声道，输出是6声道。

7.1.5 Multi Switch Nx8

功能说明：与7.1.1 Mono Switch Mux类似，但每组输入是8声道，输出是8声道。

7.1.6 Multi Switch Nx12

功能说明：与7.1.1 Mono Switch Mux类似，但每组输入是12声道，输出是12声道。

7.1.7 Multi Switch NxM

功能说明：与7.1.1 Mono Switch Mux类似，输入输出通道数可自定义，控件个数会更具通道数的比例变化，但是输入通道数一定要是输出通道数的整数倍。

7.1_ Multiplexers Switch (Slew)

以下控件与上方multiplexer Switch相似，但都多了Slew平滑的功能
slewRate平滑速度会转换成平滑系数传入算法，转换代码参考：SlewRate*。

以下以7.1.8 Mono Switch Mux (Slew)组件为例

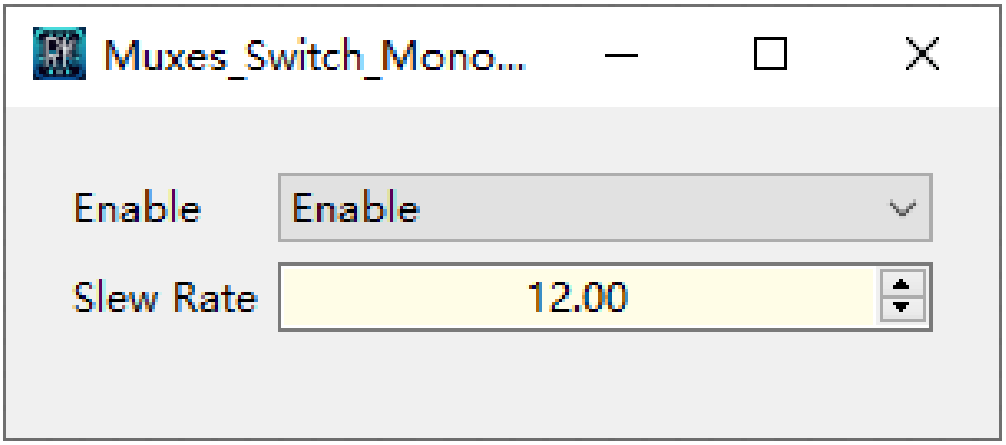


图 7.1_-1 设置Slew平滑功能；参数0为是否使能，参数1为平滑速度，值越大平滑效果越好，平滑曲线越缓

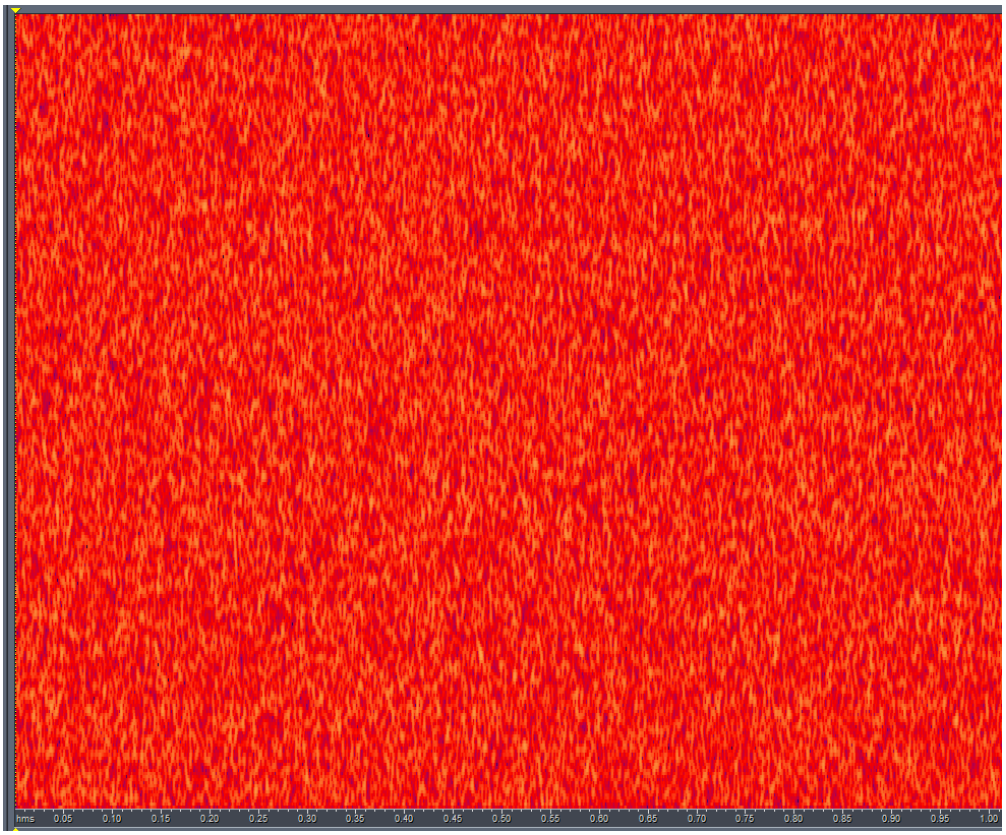


图 7.1_-2输入为白噪声(频域图)

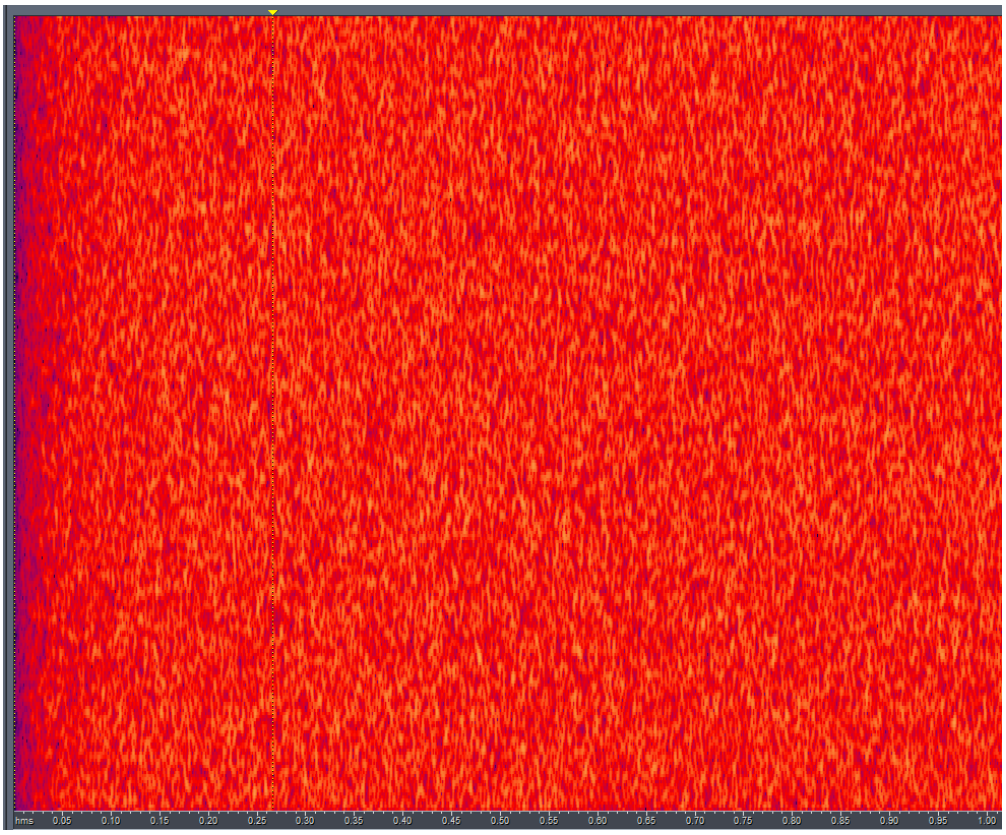


图 7.1_-3 输出频域图，在开始时刻可观察到平滑过程，Slew Rate越大平滑效果越好

7.1.8 Mono Switch Mux (Slew)

功能说明：与7.1.1 Mono Switch Mux类似，但是多了Slew平滑*的功能。

7.1.9 Stereo Switch Mux (Slew)

功能说明：与7.1.1 Mono Switch Mux类似，但是每组输入是4声道；并且多了Slew平滑*的功能。

7.1.10 Multi Switch Nx4 (Slew)

功能说明：与7.1.1 Mono Switch Mux类似，但是每组输入是4声道，输出是4声道；并且多了Slew平滑*的功能。

7.1.11 Multi Switch Nx6 (Slew)

功能说明：与7.1.1 Mono Switch Mux类似，但是每组输入是6声道，输出是6声道；并且多了Slew平滑*的功能。

7.1.12 Multi Switch Nx8 (Slew)

功能说明：与7.1.1 Mono Switch Mux类似，但是每组输入是8声道，输出是8声道；并且多了Slew平滑*的功能。

7.1.13 Multi Switch Nx12 (Slew)

功能说明：与7.1.1 Mono Switch Mux类似，但是每组输入是12声道，输出是12声道；并且多了Slew平滑*的功能。

7.1.14 NxM Switch Mux (Slew)

功能说明：与7.1.1 Mono Switch Mux类似，输入输出通道数可自定义，控件个数会更具通道数的比例变化，但是输入通道数一定要是输出通道数的整数倍；并且多了Slew平滑*的功能。

7.2 Multiplexers Index Selectable

7.2.1 Mono Index Selectable Mux

功能说明：根据控制信号，从多组输入中选择一组输出，每组输入都是单声道。默认是两组选一组，用户可以右键->Add Point来添加更多组输入。

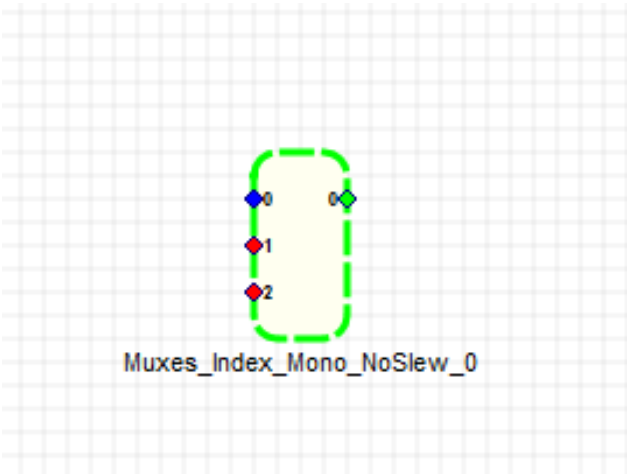


图 7.2.1-1 注意与图7.1.1的区别，该模块通过控制信号而不是按钮来选择输出，控制信号为正整数，为0则选择第一组输入，为1则选择第二组输入，依次类推。

实际效果展示：
输入两通道数据与7.1.1一样的双通道数据，然后使用DC模块来模拟控制信号，有下图所示两个典型工况：

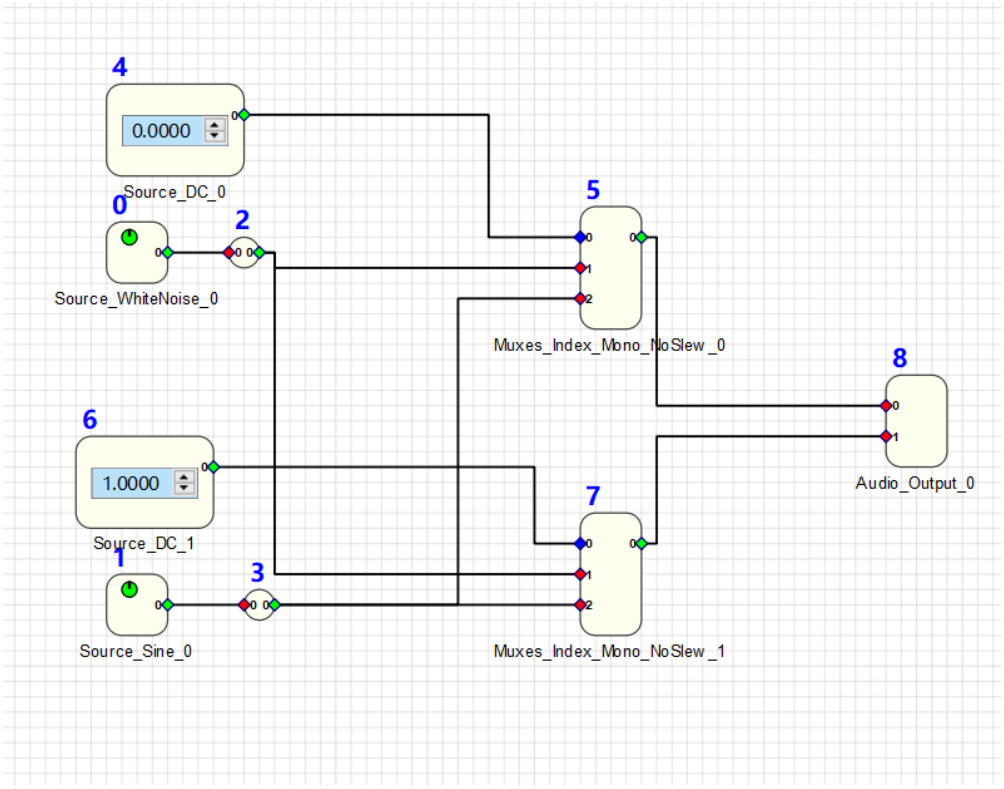


图 7.2.1-2 工况1，选择通道0，工况2，选择通道1

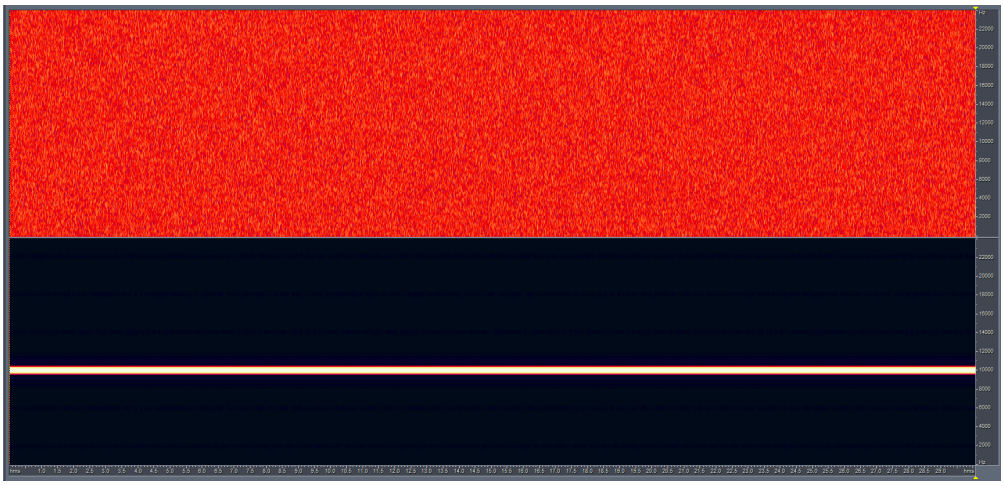


图 7.2.1-3 RKStudio输出频域，上图为工况1，下图为工况2

可以看到，不同的ext控制信号，选择对应的输出不一样。

7.2.2 Stereo Index Selectable Mux

功能说明：与7.2.1 Mono Index Mux类似，但每组输入是2声道，输出是2声道。

7.2.3 Multi Index Selectable Nx4

功能说明：与7.2.1 Mono Index Mux类似，但每组输入是4声道，输出是4声道。

7.2.4 Multi Index Selectable Nx8

功能说明：与7.2.1 Mono Index Mux类似，但每组输入是8声道，输出是8声道。

7.2_ Multiplexers Index Selectable (Slew)

以下控件与上方multiplexer Index Selectable 相似，但都多了Slew平滑的功能

slewRate平滑速度会转换成平滑系数传入算法，转换代码参考：SlewRate*。

以下以7.2.5 Mono Index Selectable Mux (Slew)组件为例

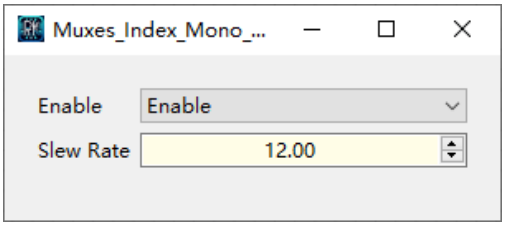


图 7.2_-1 设置Slew平滑功能；参数0为是否使能，参数1为平滑速度，值越大平滑效果越好，平滑曲线越缓

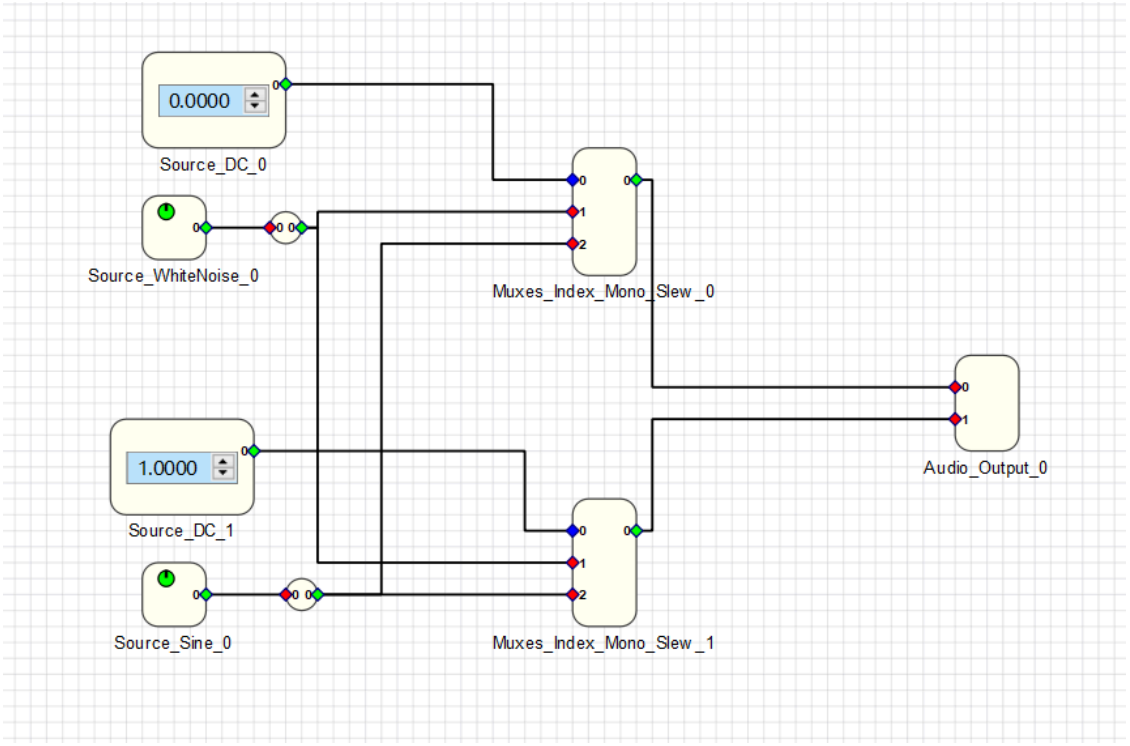


图 7.2_-2 输入为白噪声和10k的正弦波； 工况1，选择通道0，使能slew，slew rate=23；工况2，选择通道1，使能slew，slew rate=23

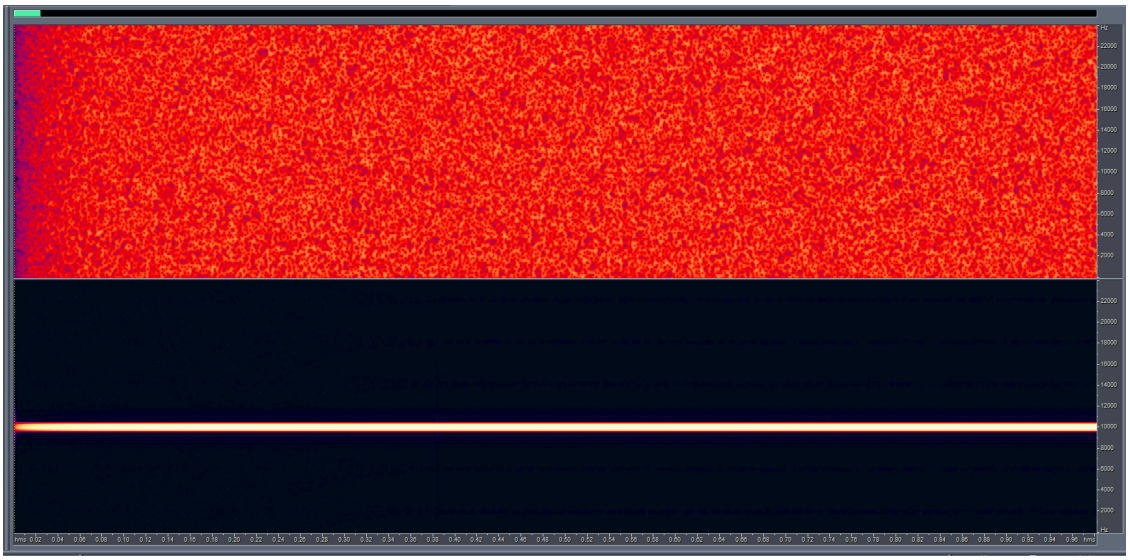


图 7.2_-3 输出频域图，图中上部分为工况一输出，下部分为工况2输出，相比较图7.2.1-3 可以看到在起始时刻有着明显的平滑效果

7.2.5 Mono Index Selectable Mux（Slew）

功能说明：与7.2.1 Mono Index Mux类似，但是多了Slew平滑*的功能。

7.2.6 Stereo Index Selectable Mux Slew）

功能说明：与7.2.1 Mono Index Mux类似，但是每组输入是2声道，输出是2声道；并且多了Slew平滑*的功能。

7.2.7 Multi Index Selectable Nx4 （Slew）

功能说明：与7.2.1 Mono Index Mux类似，但是每组输入是4声道，输出是4声道；并且多了Slew平滑*的功能。

7.2.8 Multi Index Selectable Nx8（Slew）

功能说明：与7.2.1 Mono Index Mux类似，但是每组输入是8声道，输出是8声道；并且多了Slew平滑*的功能。

7.3 DeMultiplexers Switch

7.3.1 Mono Switch Demux

功能说明：根据按钮(Switch)，将一组输入路由到多组输出中的一组，用户可以右键->Add Point来添加更多组输出。

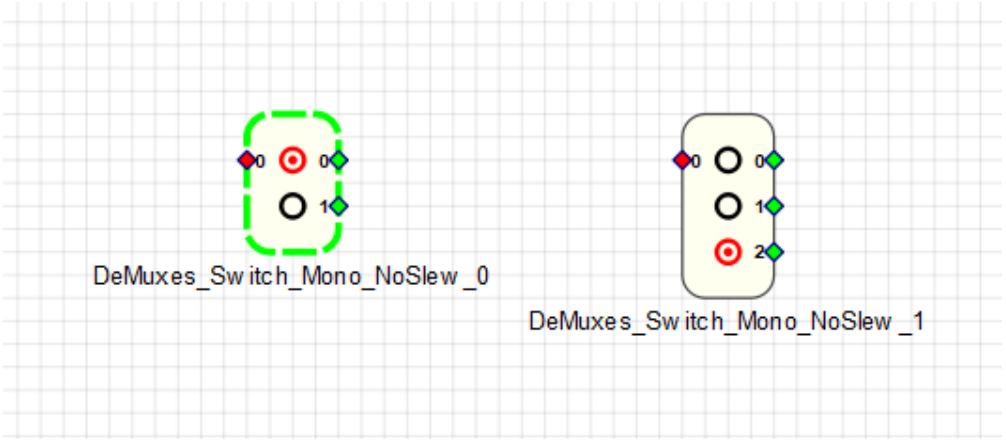


图 7.3.1-1 单通道解复用器

一下举两个常见的工况为例： 输入为白噪声

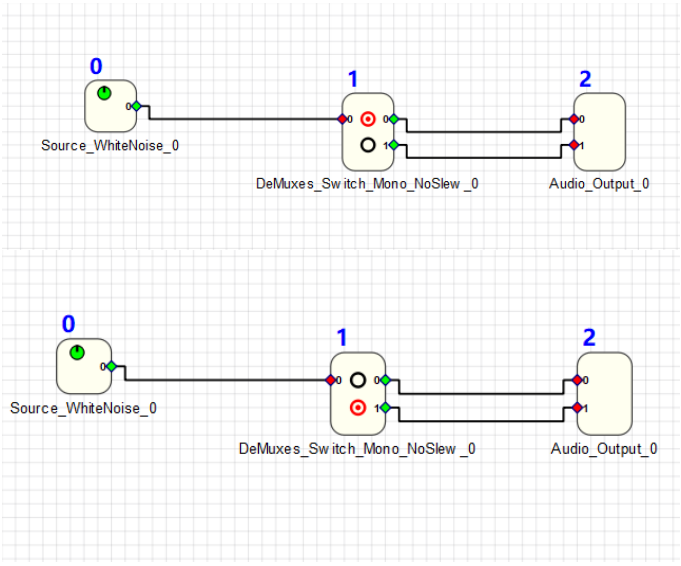


图 7.3.1-2上图工况一，选择0通道输出；下图工况二，选择1通道输出

图 7.3.1-3 上图为工况一输出结果；下图为工况二输出结果

7.3.2 Stereo Switch Demux

功能说明：与7.3.1 Mono Switch Demux类似，但输入是双声道，每组输出是双声道。

7.3.3 NxM Switch Demux

功能说明：7.3.1 Mono Switch Demux类似，输入输出通道数可自定义，控件个数会根据通道数的比例变化，但是输出通道数一定要是输入通道数的整数倍。

7.3.4 Mono Switch Demux（Slew）

功能说明：与7.3.1 Mono Switch Demux类似,但是多了Slew平滑的功能

SlewRate平滑速度会转换成平滑系数传入算法，转换代码参考：SlewRate*。

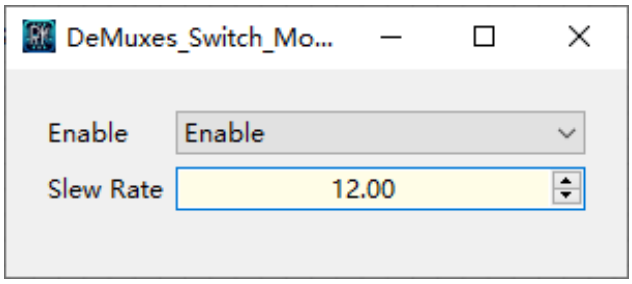


图 7.3.4-1 设置Slew平滑功能；参数0为是否使能，参数1为平滑速度，值越大平滑效果越好，平滑曲线越缓

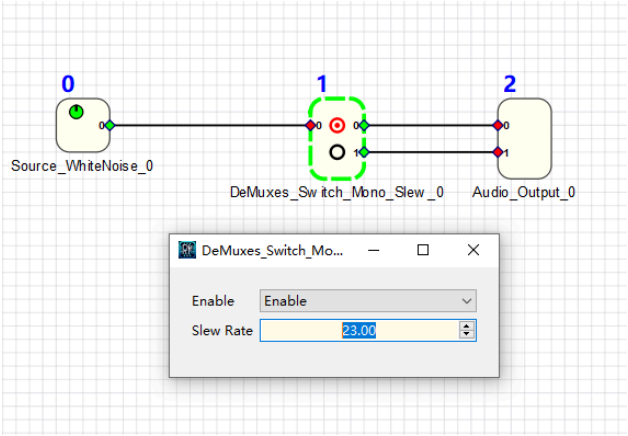


图 7.3.4-2 使能slew，slew rate=23

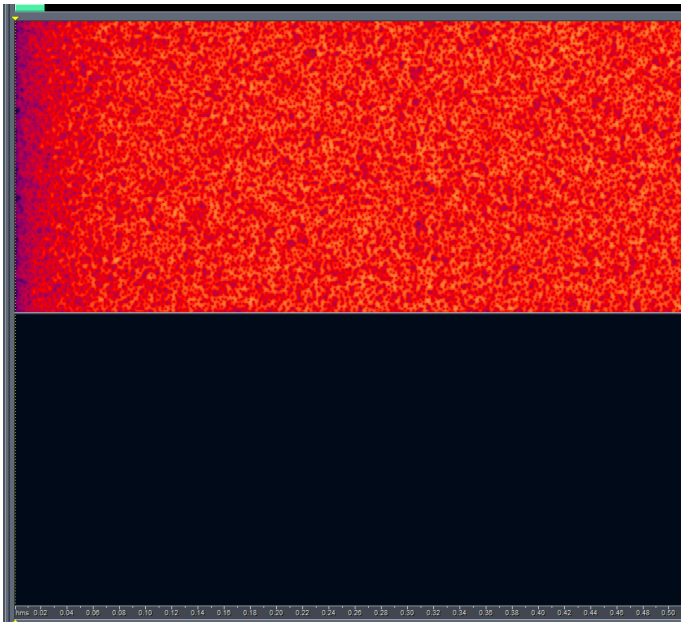


图 7.3.4-3 输出频域图，对比输出白噪声的频谱可以看到开始时刻，有明显的平滑过程

7.3.5 Stereo Switch Demux（Slew）

功能说明：7.3.1 Mono Switch Demux类似，但是输入是双声道，每组输出是双声道；并且多了Slew平滑*的功能（平滑功能可参考7.3.4 Mono Switch Demux（Slew））

7.3.6 NxM Switch Demux（Slew）

功能说明：7.3.1 Mono Switch Demux类似，输入输出通道数可自定义，控件个数会根据通道数的比例变化，但是输出通道数一定要是输入通道数的整数倍；并且多了Slew平滑*的功能（平滑功能可参考7.3.3 Mono Switch Demux（Slew））

7.4 DeMultiplexers Index Selectable

7.4.1 Mono Index Selectable Demux

功能说明：根据控制信号，将一组输入路由到多组输出中的一组，用户可以右键->Add Point来添加更多组输出。

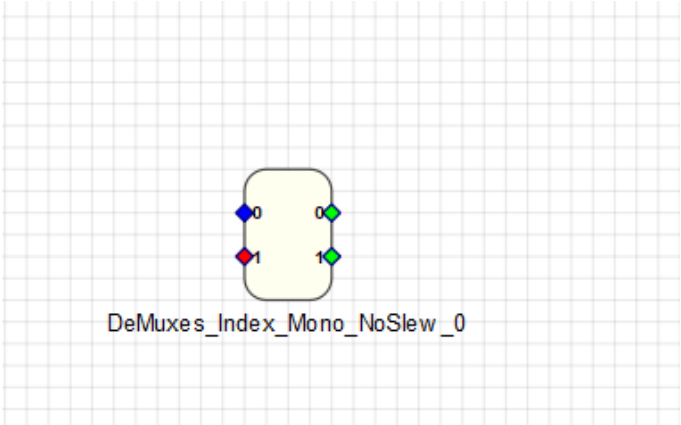


图7.4.1-1 注意与图7.3.1的区别，该模块通过控制信号而不是按钮来选择输出，控制信号为正整数，为0则选择第一组输入，为1则选择第二组输入，依次类推

以下举例两个常见的工况：

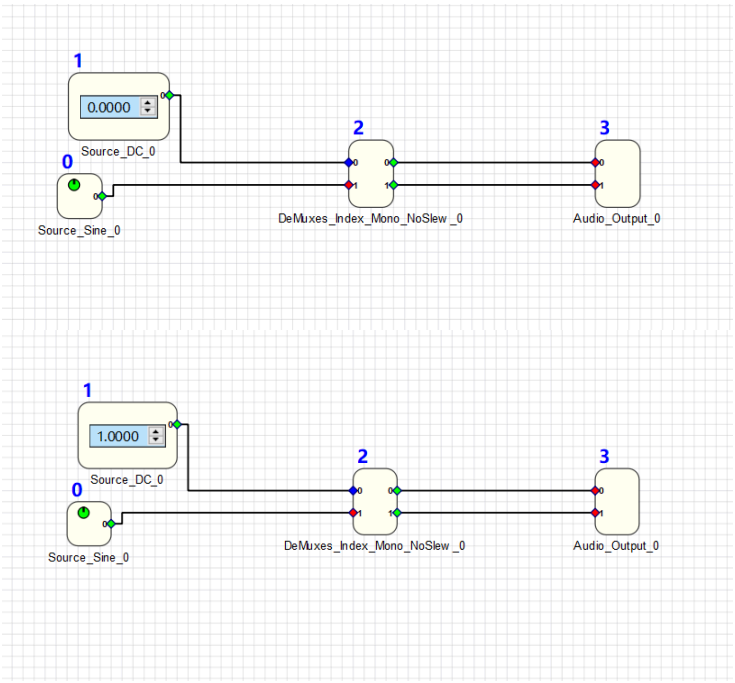
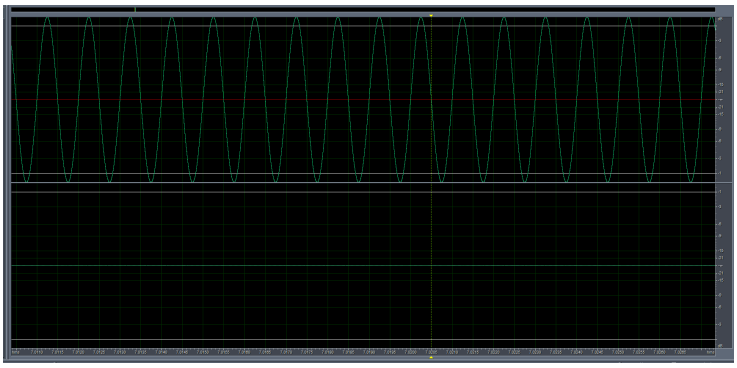


图7.4.1-2 上图工况一，ext输入为0选择0通道输出；下图工况二，ext输入为1选择1通道输出



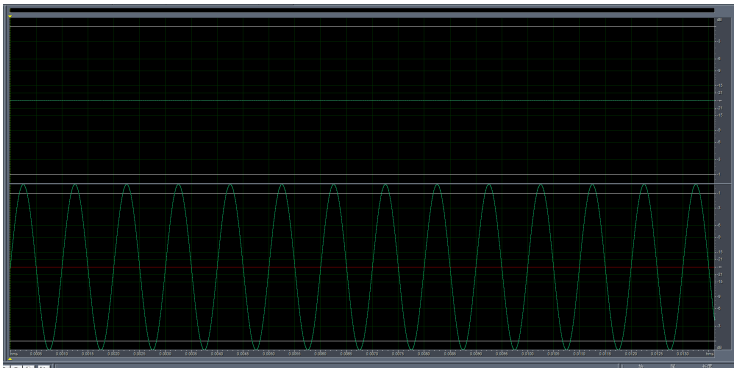


图7.4.1-3 输出时域图 上图为工况一输出结果；下图为工况二输出结果

7.4.2 Stereo Index Selectable Demux

功能说明：与7.4.1 Mono Index Selectable Demux类似，但输入是双声道，每组输出是双声道。

7.4.3 Mono Index Selectable Demux (Slew)

功能说明：与7.4.1 Mono Index Selectable Demux类似，但是多了Slew平滑的功能
SlewRate平滑速度会转换成平滑系数传入算法，转换代码参考：SlewRate*。

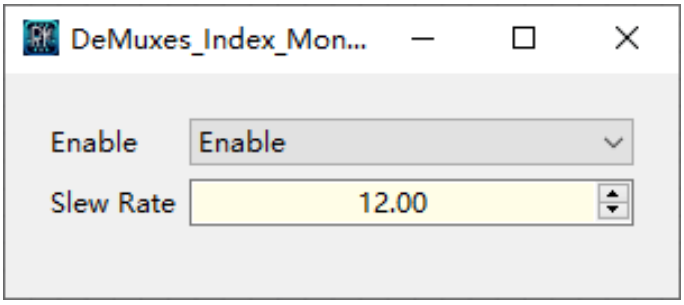


图 7.4.3 -1 设置Slew平滑功能；参数0为是否使能，参数1为平滑速度，值越大平滑效果越好，平滑曲线越缓

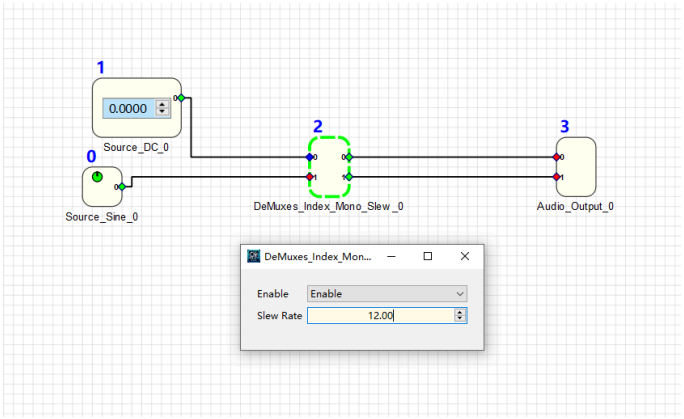


图 7.4.3-2 输入为正弦波，使能slew，slew rate=12

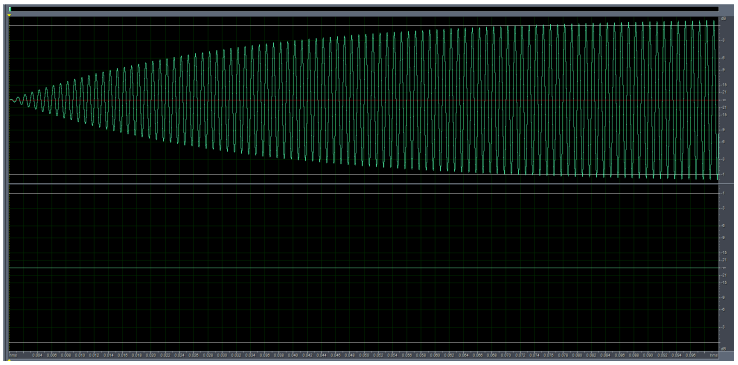


图 7.4.3-3 输出结果时域图，对比输出正弦波可以看到开始时刻，有明显的平滑过程

7.4.4 Stereo Index Selectable Demux (Slew)

功能说明：与7.4.1 Mono Index Selectable Demux类似，但是输入是双声道，每组输出是双声道；并且多了Slew平滑*的功能 （平滑功能可参考7.4.3 Mono Index Selectable Demux (Slew)）

8 Source

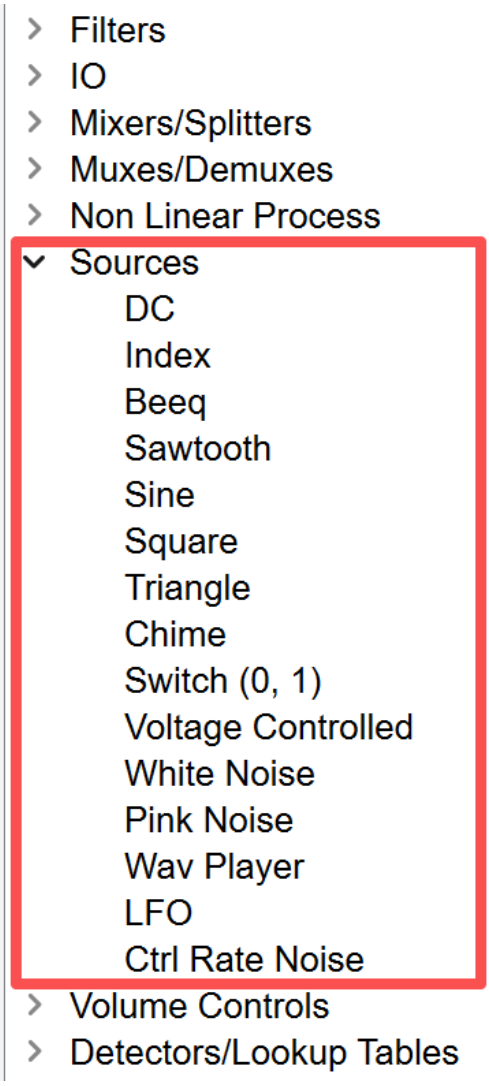


图 8-1 source位于工具的位置

8.1 DC

功能说明：直流控件用于生成直流信号（一个恒定数值），通过修改输入框的值，可以调整直流信号大小

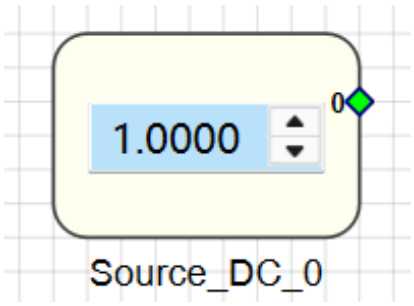


图 8.1-1 DC 控件和设置界面

参数数量：1

参数名	参数位	类型	功能	单位	建议取值	默认取值
Value	0	float	直流信号大小		-1.0 ~ 1.0	1.0

典型工况：参数设为0.01，对应输入结果如下：

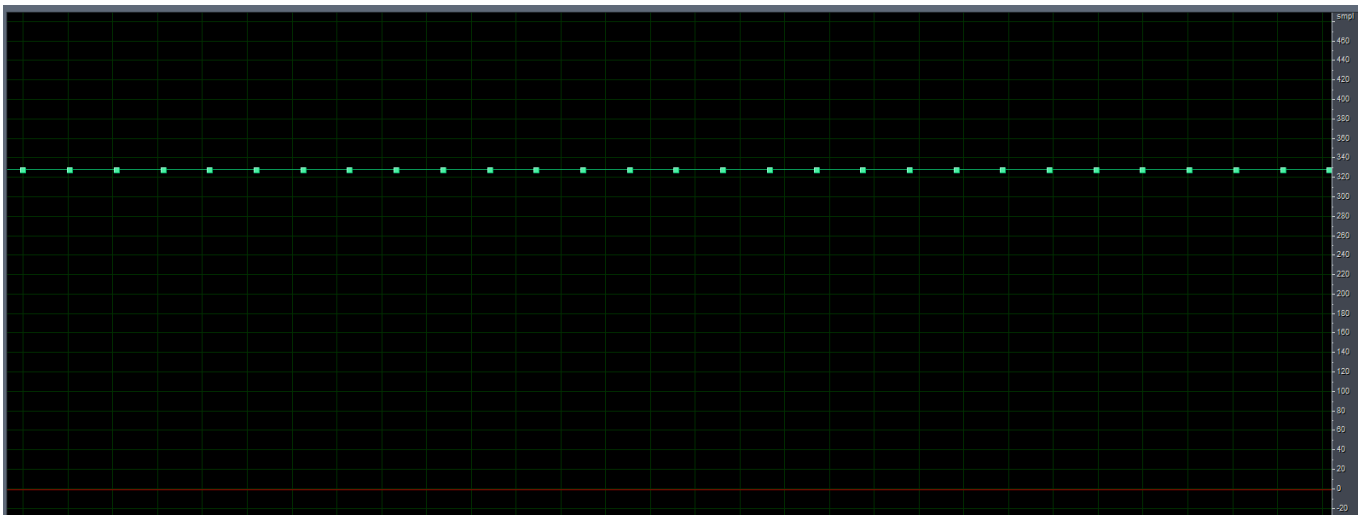


图 8.1-2 RKStudio直流声源输出时域

8.2 Index

功能和DC类似，但是输入框限制输入为整数，方向进行id的控制，一般用作Index的控制信号。

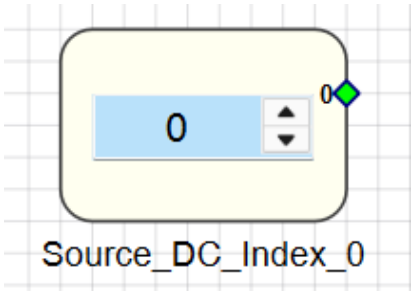


图 8.2-1 Index控件

8.3 Beeq

功能说明：振荡器算法用于生成蜂鸣信号，修改 Freqency 得到所需频率信号。

图 8.3-1 Beeq 控件和设置界面

参数数量：2

参数名	参数位	类型	功能	单位	建议取值	默认取值
Enable	0	float	使能	无	0/1	1
Freqency(HZ)	0	float	频率大小	Hz	小于采样率一半	1000

典型工况：参数设为1200，对应输入结果如下：

图 8.3-2 RKStudio蜂鸣声源输出频域

8.4 Sawtooth

功能说明：振荡器算法用于生成锯齿波信号，修改Freqency得到所需频率信号。

图 8.4-1 Sawtooth 控件和设置界面

参数数量：2

参数名	参数位	类型	功能	单位	建议取值	默认取值
Enable	0	float	使能	无	0/1	1
Freqency(HZ)	0	float	频率大小	Hz	小于采样率一半	1000

典型工况：参数设为1200，对应输入结果如下：

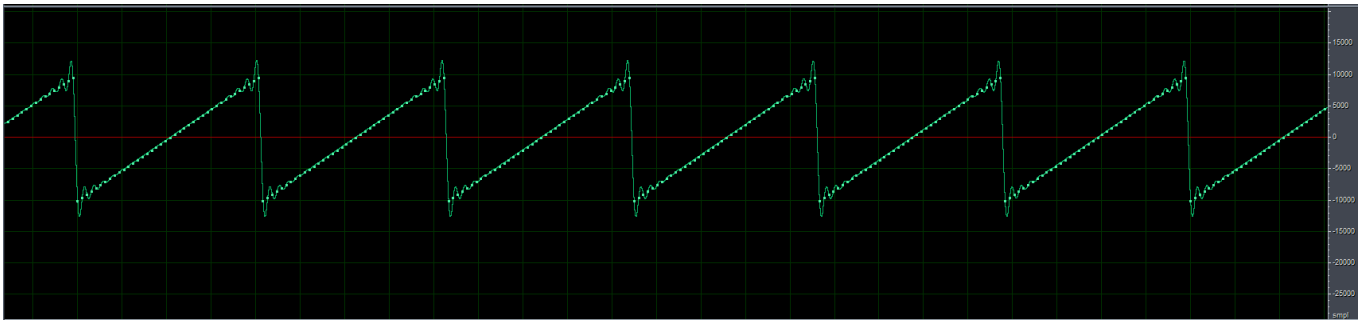


图 8.4-2 RKStudio锯齿波声源输出时域

8.5 Sine

功能说明：振荡器算法用于生成正弦波信号，修改 Freqency 得到所需频率信号。

图 8.5-1 Sine 控件和设置界面

参数数量：2

参数名	参数位	类型	功能	单位	建议取值	默认取值
Enable	0	float	使能	无	0/1	1
Freqency(HZ)	0	float	频率大小	Hz	小于采样率一半	1000

典型工况：参数设为1200，对应输入结果如下：

图 8.5-2 RKStudio正弦波声源输出频域

8.6 Square

功能说明：振荡器算法用于生成方波信号，修改 Freqency 得到所需频率信号

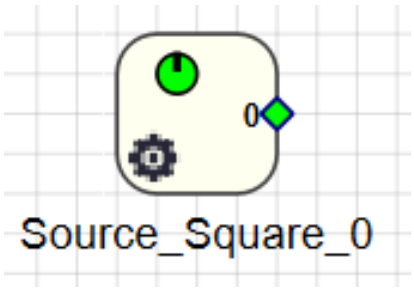


图 8.6-1 Square 控件和设置界面

参数数量：2

参数名	参数位	类型	功能	单位	建议取值	默认取值
Enable	0	float	使能	无	0/1	1
Freqency(HZ)	0	float	频率大小	Hz	小于采样率一半	1000

典型工况：参数设为1200，对应输入结果如下：

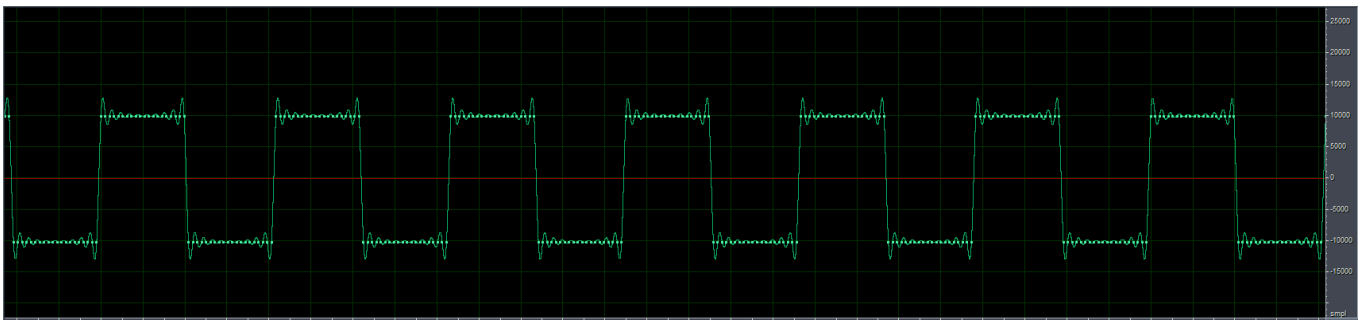


图 8.6-2 RKStudio方波声源输出时域

8.7 Triangle

功能说明：振荡器算法用于生成三角波信号，修改 Freqency 得到所需频率信号

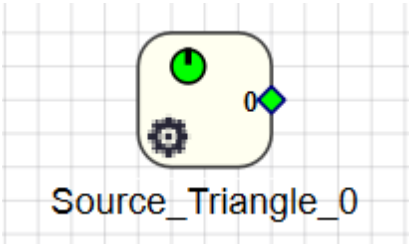


图 8.7-1 DC 控件和设置界面

参数数量：2

参数名	参数位	类型	功能	单位	建议取值	默认取值
Enable	0	float	使能	无	0/1	1
Freqency(HZ)	0	float	频率大小	Hz	小于采样率一半	1000

典型工况：参数设为1200，对应输入结果如下：

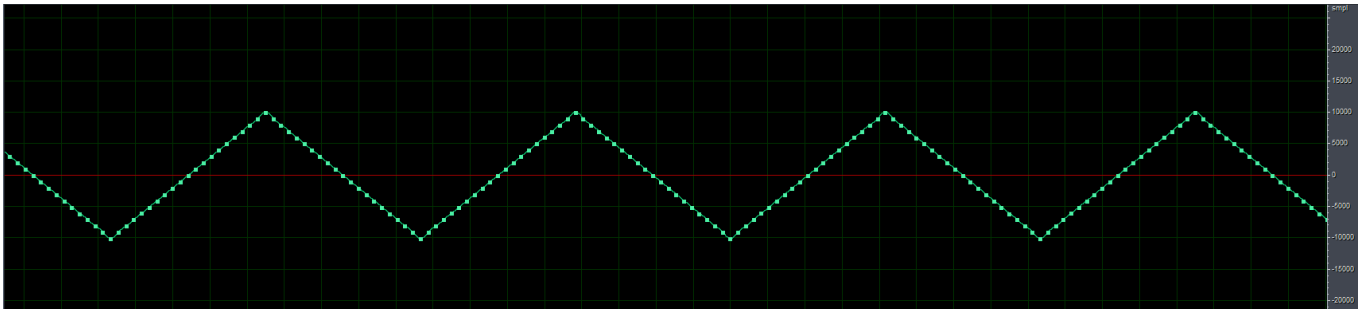


图 8.7-2 RKStudio三角波声源输出时域

8.8 Chime

功能说明：通过设置时间和频率的变化，生成一段可变频率音频，可用于生成扫频信号。控件中绿色的按钮用于启动/停止扫频，圆圈用于设置是否循环播放。

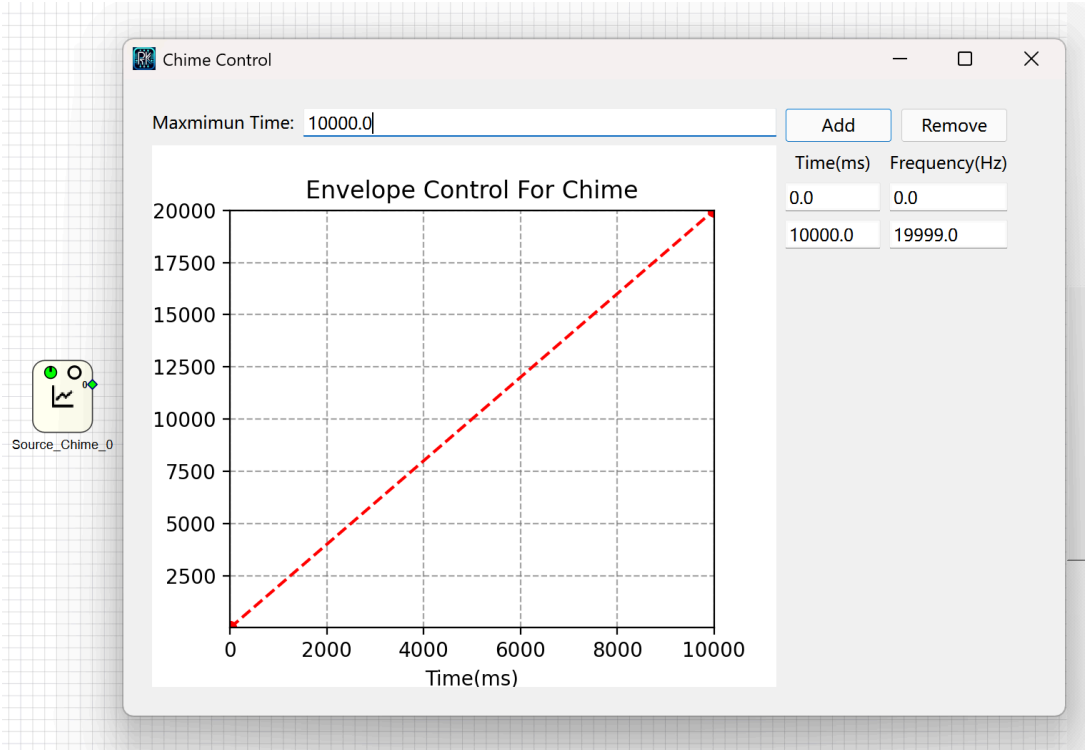


图 8.8-1 Chime 控件和设置界面

典型工况：当配置为8.8-1图中的参数后，输出的频谱如下图所示：

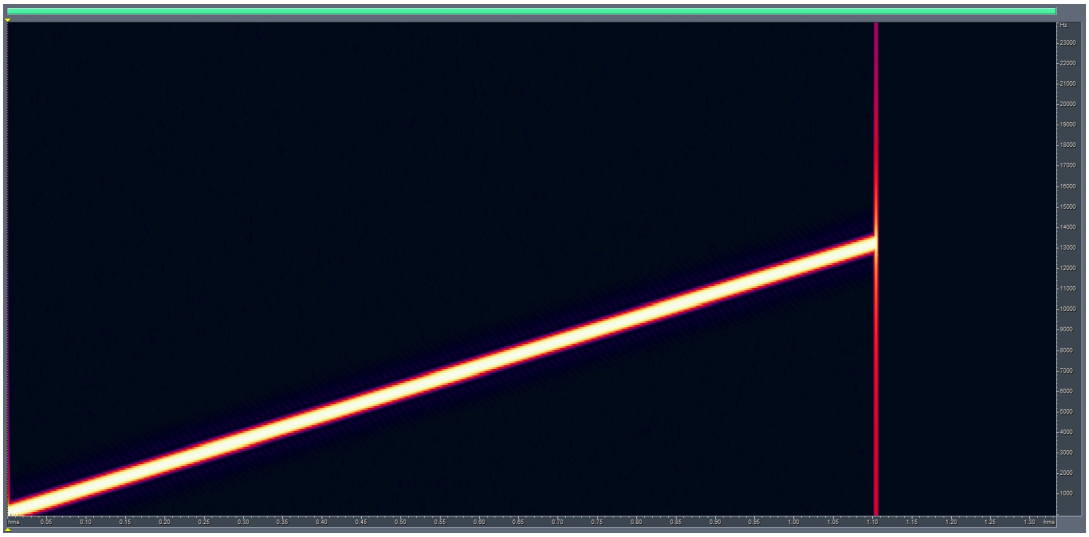


图 8.8-2 CHime输出波形的频谱

8.9 Switch(0,1)

功能说明：通过按钮控制输出信号。绿色是1，红色是0。

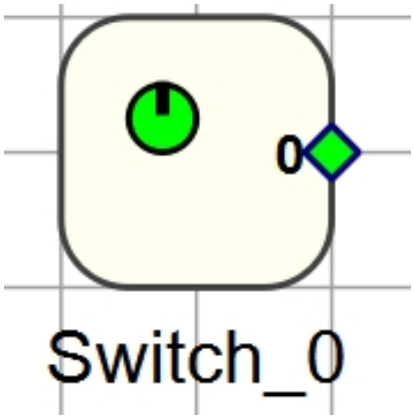


图 8.9-1 Switch 控件和设置界面

参数数量：1

参数名	参数位	类型	功能	单位	建议取值	默认取值
开关状态	0	float	是否开关	无	0/1	1

8.10 Voltage Controlled

功能说明：压控振荡器算法，通过输入大小调节输出频率（input * samplerate / 2）,输入大与等于1则输出频率对应采样率的一般，小于等于0输出频率为0.

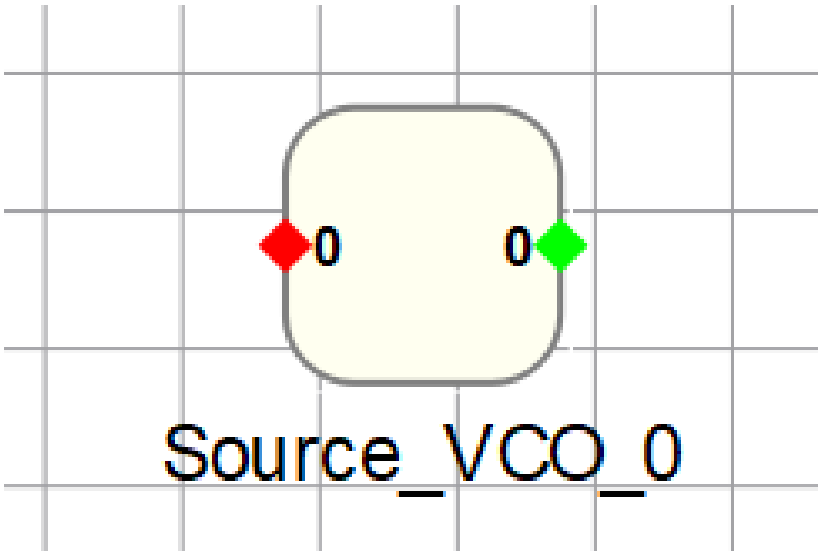


图 8.10-1 VCO 控件和设置界面

参数数量：0

典型场景：前置添加DC直流模块，设置为0.5接到VCO模块

图 8.10-2 RKStudio压控振荡器声源输出频谱

8.11 White Noise

功能说明：高斯白噪声算法，修改均值mean以及方差sigma来调节白噪声大小

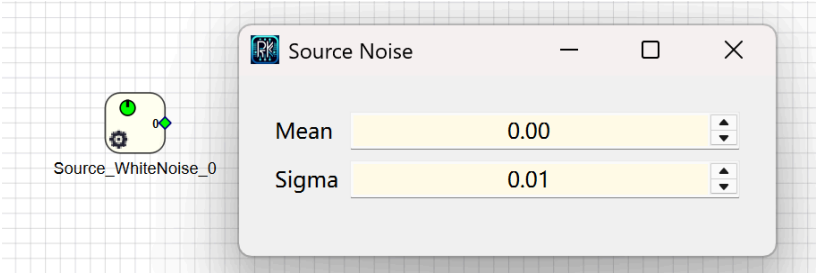


图 8.11-1 White Noise 控件和设置界面

参数数量：2

参数名	参数位	类型	功能	单位	建议取值	默认取值
Mean	0	float	均值，对应直流偏置		0 ~ 1	0
Sigma	1	float	方差，对应输出幅值		0 ~ 1	0.01

典型工况：设置参数为MeanNeed= 0， SigmaNeed = 0.1：

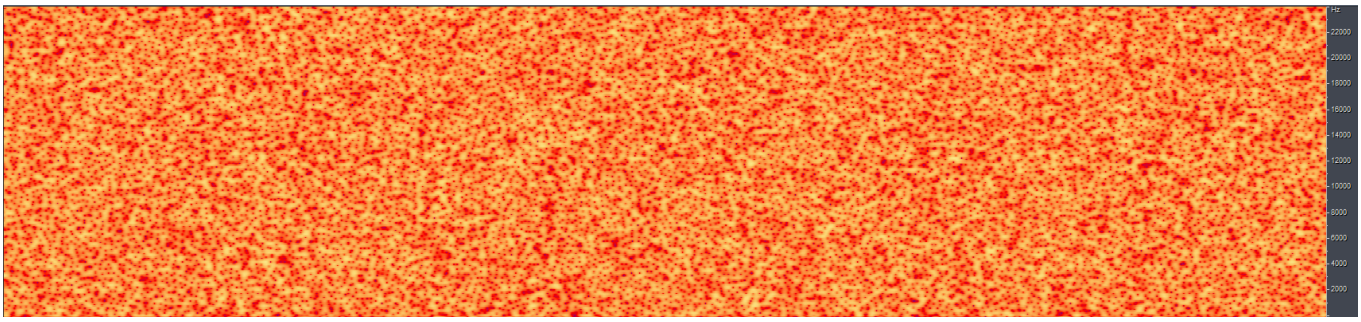


图 8.11-2 RKStudio白噪声声源输出频域

8.12 Pink Noise

功能说明：粉色白噪声算法，修改均值mean以及方差sigma来调节粉色噪声大小

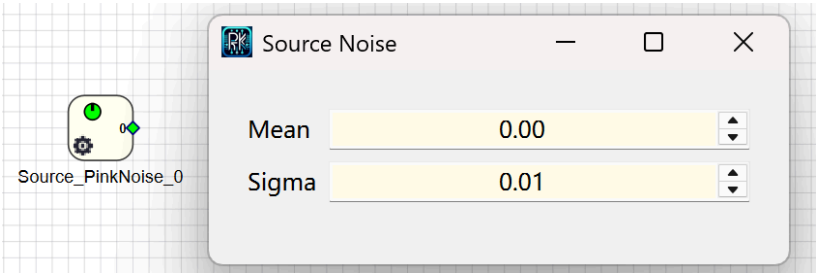


图 8.12-1 Pink Noise 控件和设置界面

参数数量：2

参数名	参数位	类型	功能	单位	建议取值	默认取值
Mean	0	float	均值，对应直流偏置		0 ~ 1	0
Sigma	1	float	方差，对应输出幅值		0 ~ 1	0.01

典型工况：设置参数为MeanNeed= 0， SigmaNeed = 0.1：

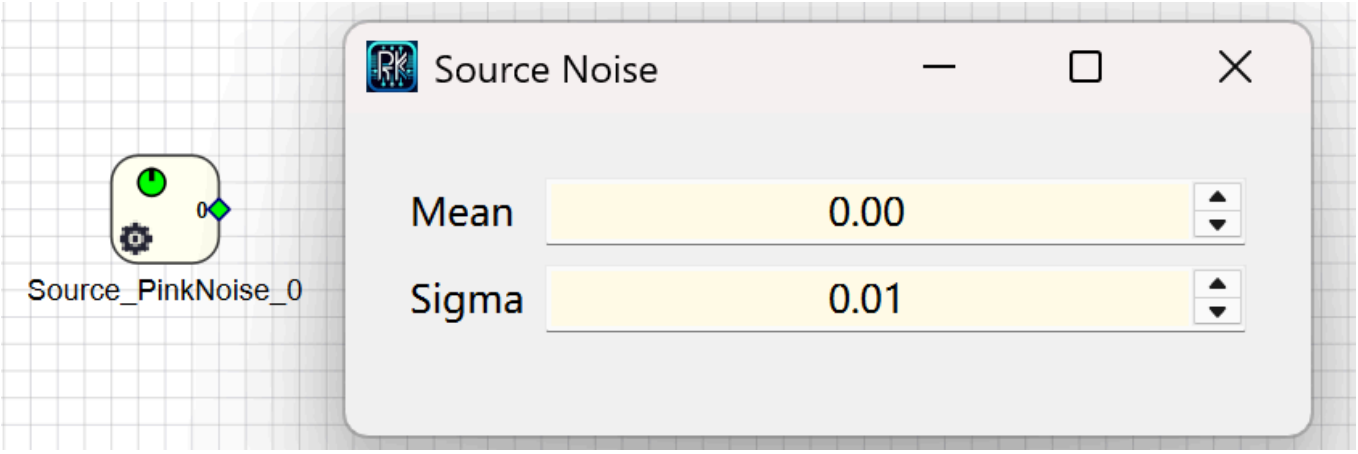


图 8.12-2 RKStudio粉色噪声声源输出频域

8.13 Wav Player

功能说明：播放wav或者pcm音频文件，如果是wav文件，那么输出chan会根据实际的格式自动调整，并且支持重采样。输入框表示当前的文件idx，三角形表示播放按钮，正方型是停止按钮，圆形是循环播放按钮。双击后看到其他的参数配置窗口。

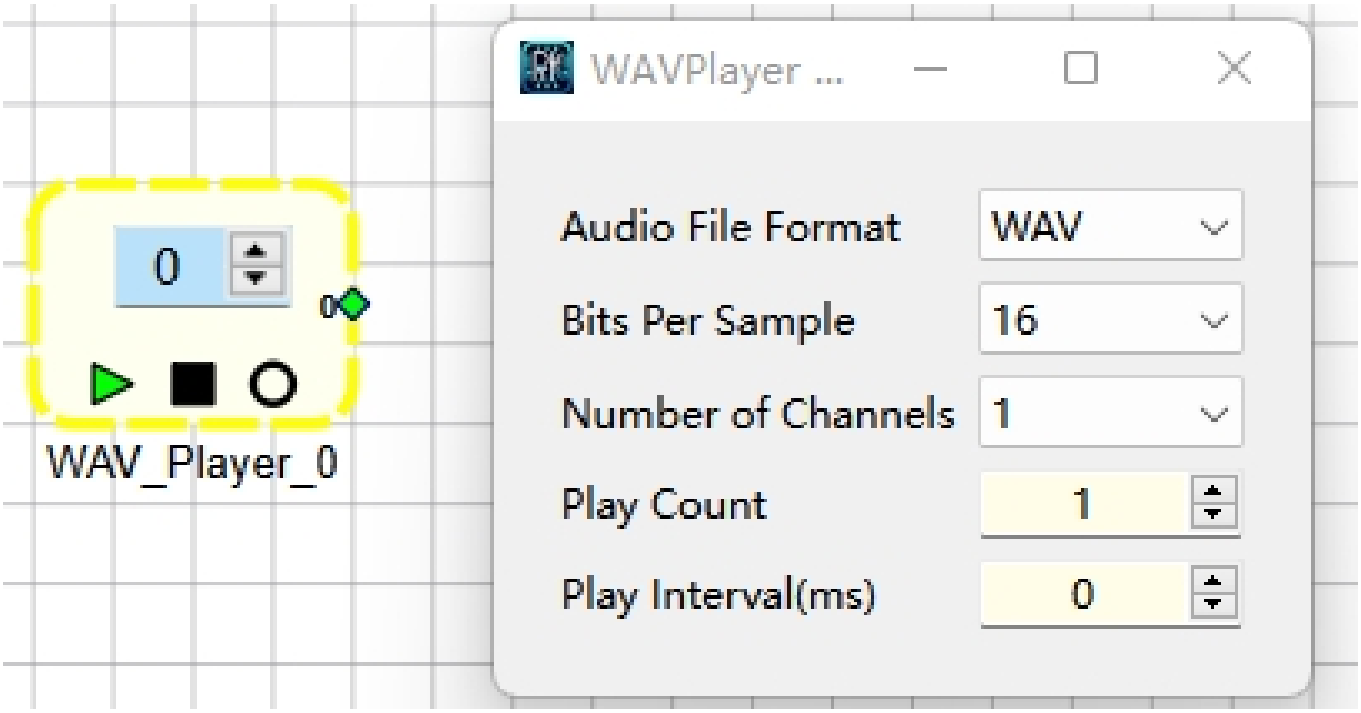


图 8.13-1 wavPlayer控制设置界面

参数数量：8

参数名	参数位	类型	功能	单位	建议取值	默认取值
Index	0	float	当前播放文件的idx	无	0 ~ 127	0
PlayState	1	float	播放状态，0为停止，1为暂停，2为播放， 这个参数即是状态显示，也可以控制播放状态 （图中绿色三角区域）	无	0/1/2	0
Circular Mode	2	float	循环模式，0为单次播放，1为循环 （图中圆圈区域，黑色为0，变绿为1）	无	0/1	0
Format	3	float	当前音频文件的格式，0为pcm，1为wav	无	0/1	0
Bits	4	float	当前音频的采样量化bit数，只用于pcm格式	无	0/1/2/3(0对应8,1对应16,2对应24,3对应32)	1
Chan	5	float	当前音频的chan数，只用于pcm格式	无	0/1(0对应1chan，1对应2chan)	0

参数名	参数位	类型	功能	单位	建议取值	默认取值
Play Count	6	float	播放次数，只用于单次播放模式	无	0 ~ 1000	1
Play Interval	7	float	播放时间间隔	ms	0 ~ 10000	0

wavPlayer中的文件，还依赖于rkwav.img分区中的烧录文件，在分区配置和烧录工具上可以看到这个分区。这个分区文件中包含了wav文件，可以直接通过idx进行播放。配置入口在工具-》文件分区配置中。点击ok后，会自动生成一个rkwav.img文件，需要烧录到对应的分区中。

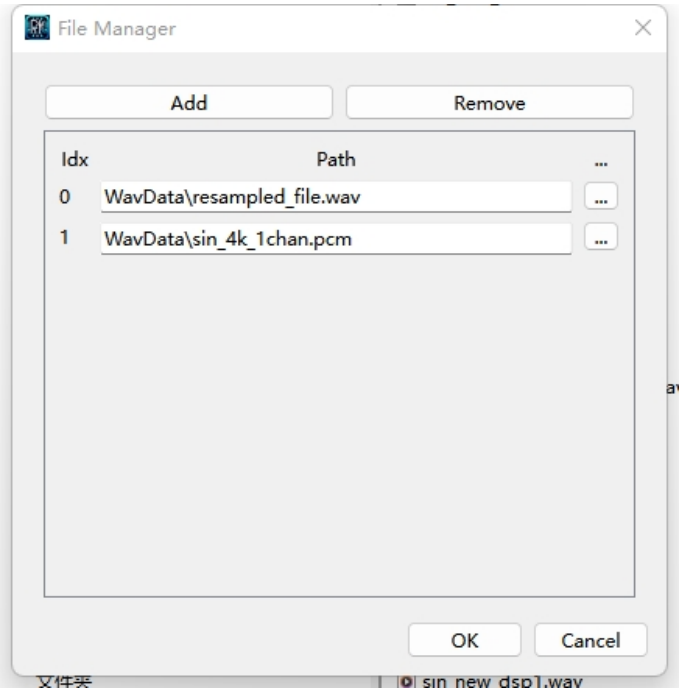


图 8.13-2 wav分区管理设置界面

8.14 LFO

功能说明：低频振荡器（Low Frequency Oscillator）。
该模块用于产生低于音频频率（通常 < 20Hz）的周期性波形信号。LFO 本身通常不直接用于发声，而是作为调制源去控制其他音频处理模块的参数。例如：控制音量可产生颤音，控制频率可产生指颤音，或者用于自动声像移动等效果。

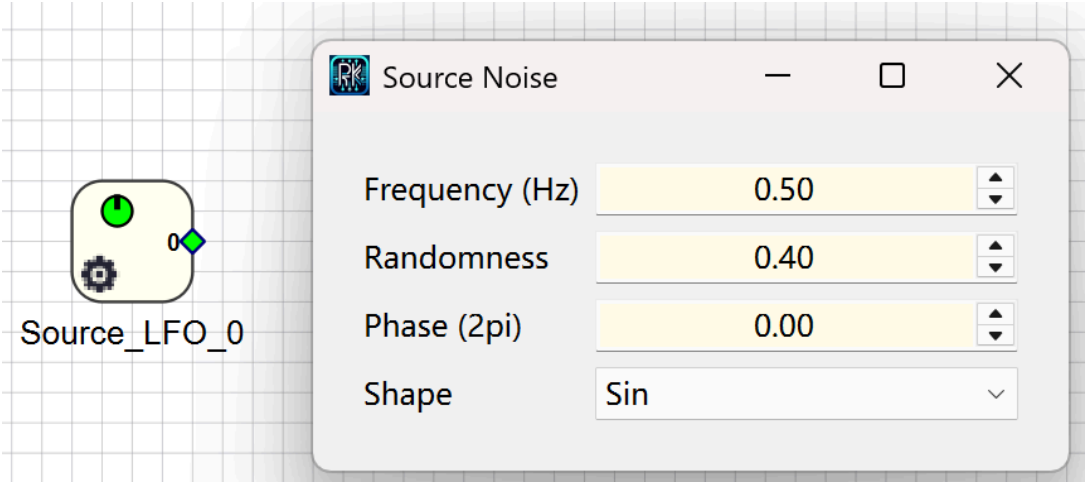


图 8.14-1 LFO控件设置界面

8.15 Ctrl Rate Noise

功能说明：控制率噪声发生器。
该模块产生一个低频的、随机变化的控制信号。不同于音频噪声，它的频率通常较低，且变化平滑。主要作为调制源使用，用于控制其他模块的参数，以产生非周期性的、更有机的听感效果。

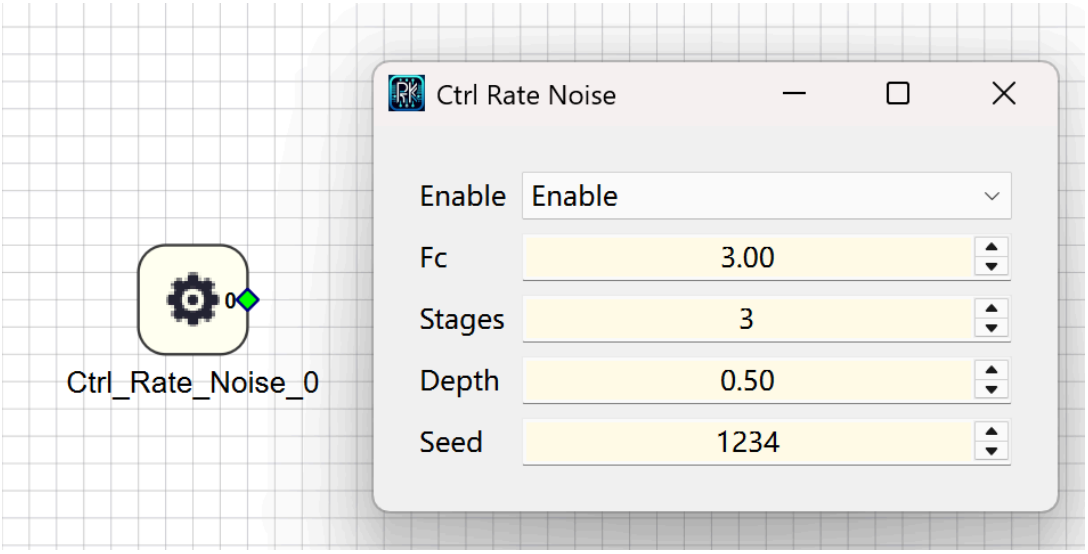


图 8.15-1 Ctrl Rate Noise控件设置界面

9 Volume Controls

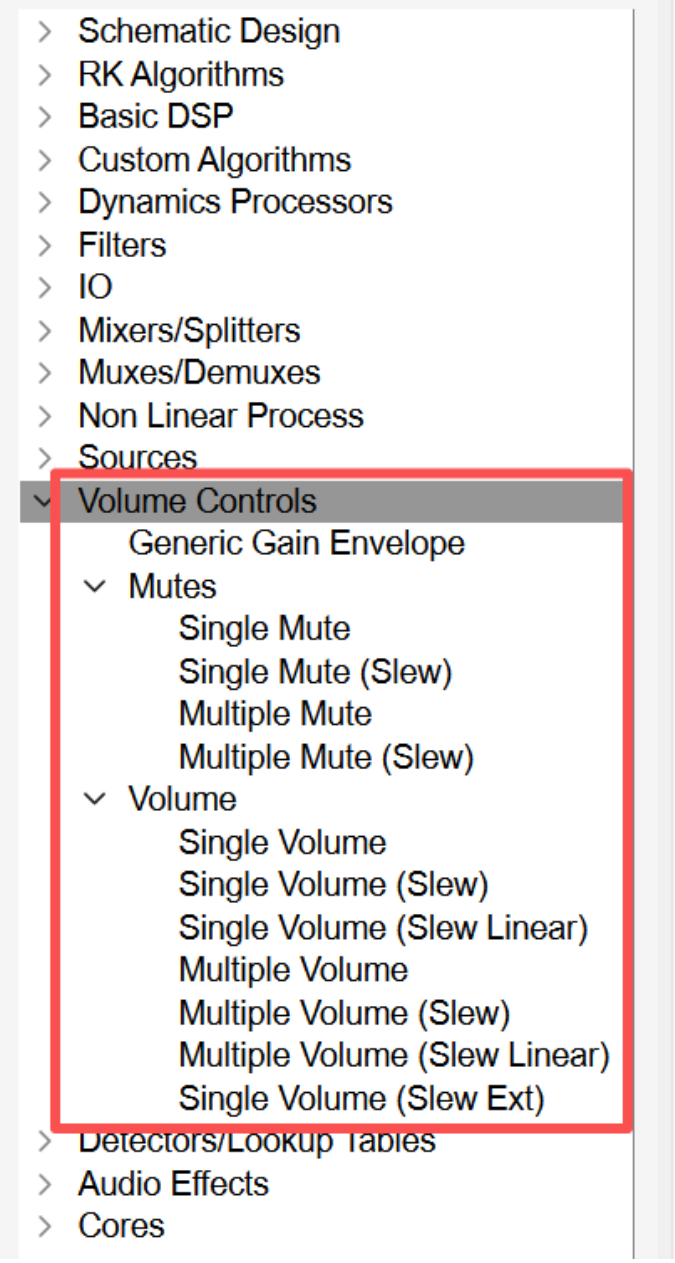


图 9-1 VolumeControl位于算法列表的位置

9.1 Generic Gain Envelope

该模块为增益随时间发生变化的模块，输入2路白噪进行测试，白噪RMS为-20dB
测试工况：模块的控制点横坐标为:{0,2,502,504,1004,1104},纵坐标为:{-100,0,-100,0,-70,0}，具体如下图所示：

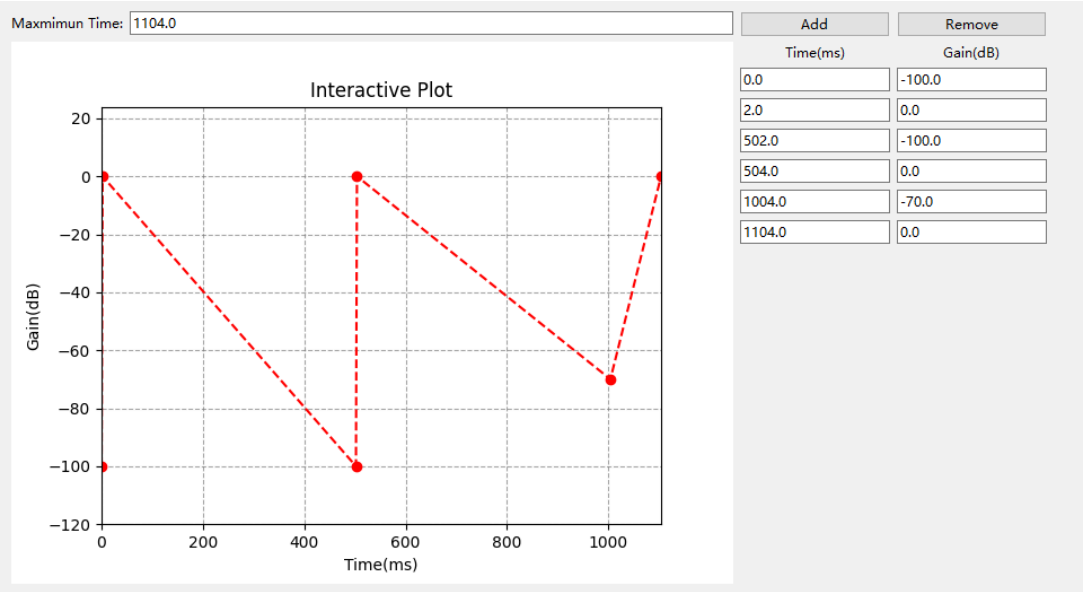


图 9.1-1 Generic Gain Envelope参数设置

RKStudio输出如下：

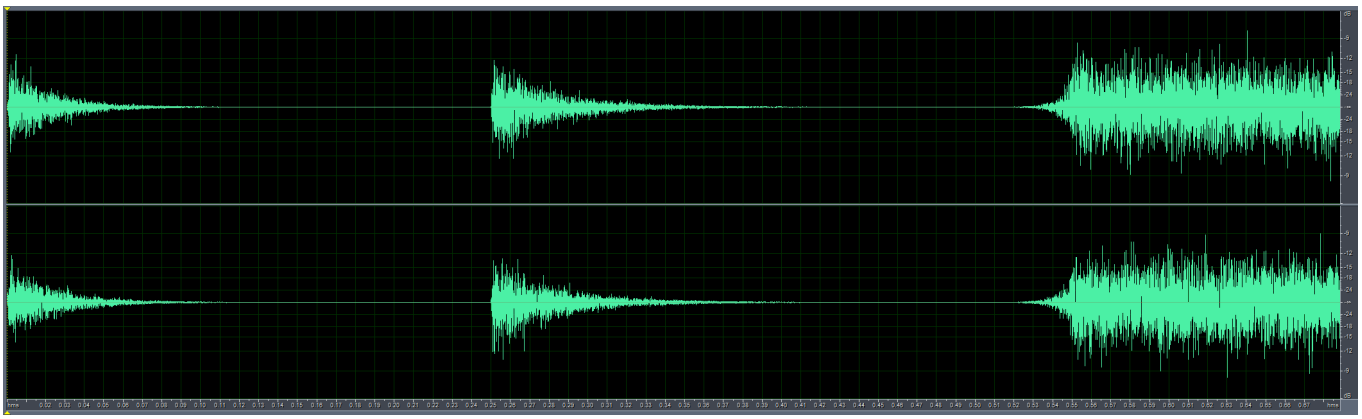


图 9.1-2 Generic Gain Envelope波形输出

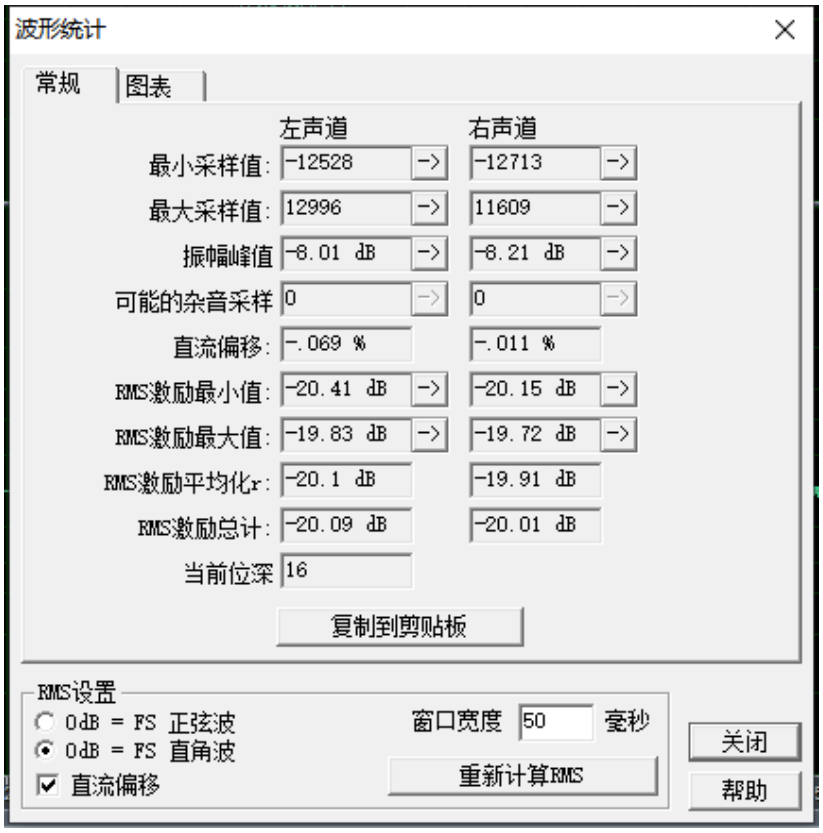


图 9.1-3 Generic Gain Envelope波形统计

9.2 Mute

9.2.1 Single Mute

功能说明：静音。

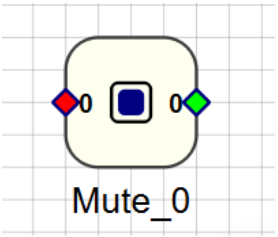


图 9.2-1 Mute 控件

参数说明：enable使能,选中则静音。

9.2.2 Single Mute(Slew)

功能说明：切换带平滑的静音。

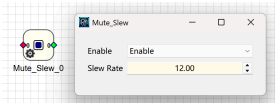


图 9.2-2 Mute(Slew) 控件

参数说明：enable使能,选中则静音。

9.2.3 Multiple Mute

功能说明：静音。

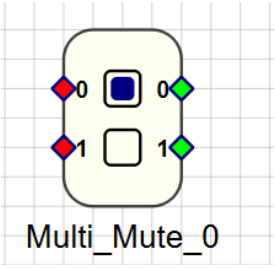


图 9.2-3 Multiple Mute 控件

参数说明：enable使能,选中则静音。

9.2.4 Multiple Mute(Slew)

功能说明：切换带平滑的静音。

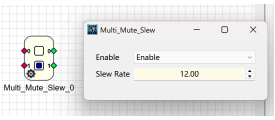


图 9.2-4 Multiple Mute(Slew) 控件

参数说明：enable使能,选中则静音。

9.3 Volume

9.3.1 Single Volume

功能说明：输入 * 固定增益 = 输出。

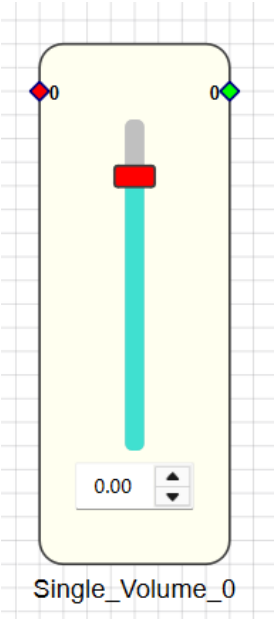


图 9.3.1-1 Single Volume 控件

参数数量：1

参数名	参数位	类型	功能	单位	建议取值	默认取值
Gain	0	float	所有通道的增益	dB	-120 ~ 20	0

实际效果展示：
输入数据白噪声, 增益设置为-10dB：

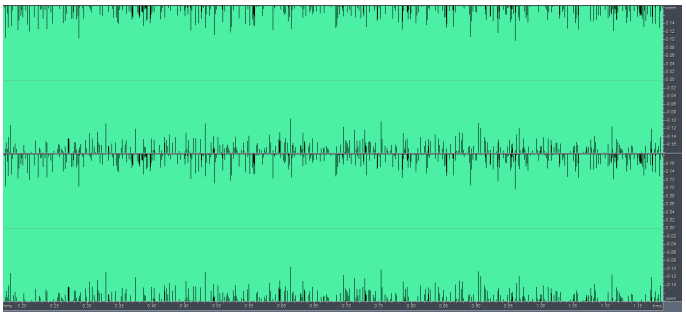


图 9.3.2-2 输入音源图

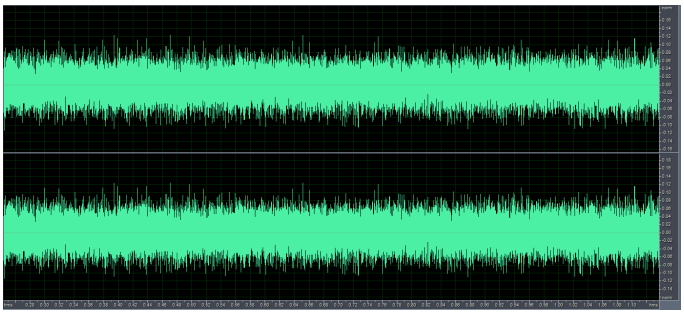


图 9.3.2-3 Single Volume Control输出时域

9.3.2 Single Volume(Slew)

功能说明： 输出 =输入 * 固定增益,并且增益变化会进行帧间的平滑操作。

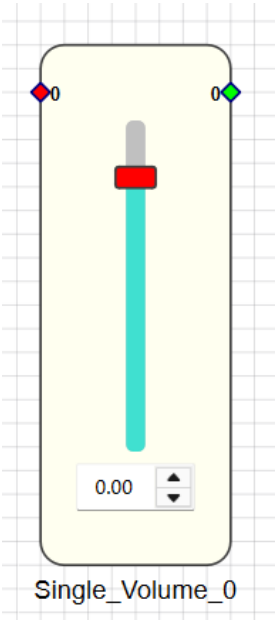


图 9.3.2-1 Single Volume(Slew)控件

参数数量：1

参数名	参数位	类型	功能	单位	建议取值	默认取值
Gain	0	float	所有通道的增益	dB	-120 ~ 20	0

实际效果展示：
输入数据白噪声, 增益设置为-10dB：

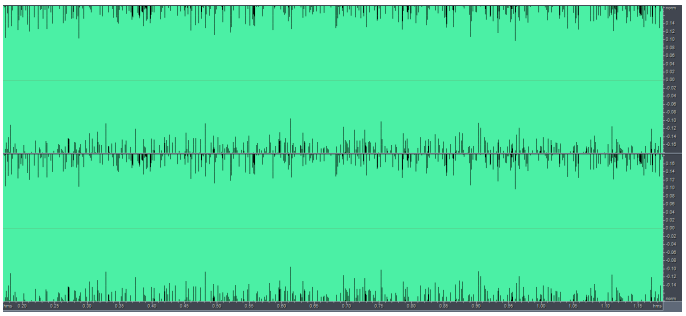


图 9.3.2-2 输入音源图

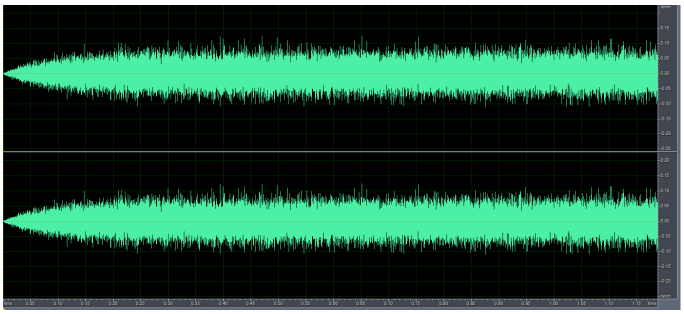


图 9.3.2-3 Single Volume Control(Slew)输出时域

9.3.3 Multiple Volume

功能说明：每个通道输出 = 每个通道输入 * 每个通道固定增益 。

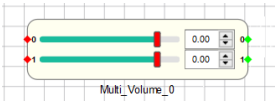


图 9.3.3-1 Multiple Volume控件

参数数量：input_bin

参数名	参数位	类型	功能	单位	建议取值	默认取值
Gain	0 ~ input_bin-1	float	每个通道的增益	dB	-120 ~ 20	0

实际效果展示：
输入数据白噪声, 增益设置为-10dB，参数配置如下：

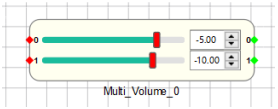


图 9.3.3-2 Multiple Volume 参数

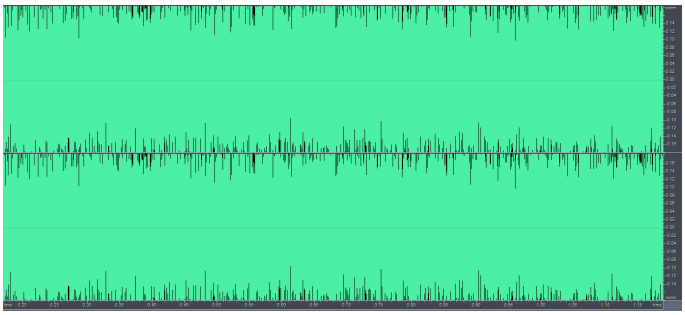


图 9.3.3-3 输入音源图

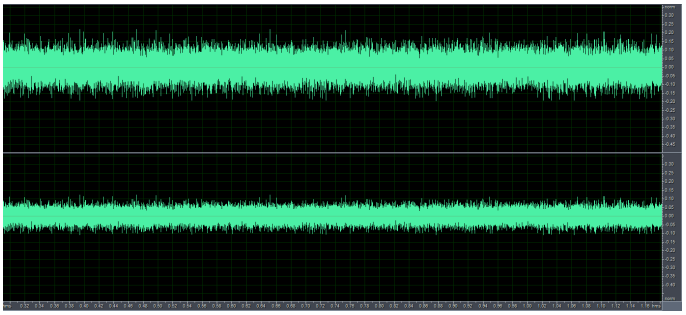


图 9.3.3-4 Single Volume Control输出时域

9.3.4 Multiple Volume(Slew)

功能说明：每个通道输出= 每个通道输入 * 每个通道固定增益 ， 并且增益变化会进行帧间的平滑操作。

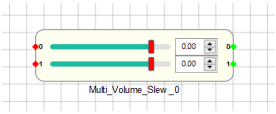


图 9.3.4-1 Multiple Volume(Slew)控件

参数数量：input_bin

参数名	参数位	类型	功能	单位	建议取值	默认取值
Gain	0 ~ input_bin-1	float	每个通道的增益	dB	-120 ~ 20	0

实际效果展示：
输入数据白噪声, 参数配置如下：

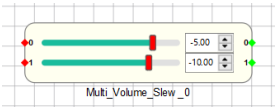


图 9.3.4-2 Multiple Volume(Slew)参数

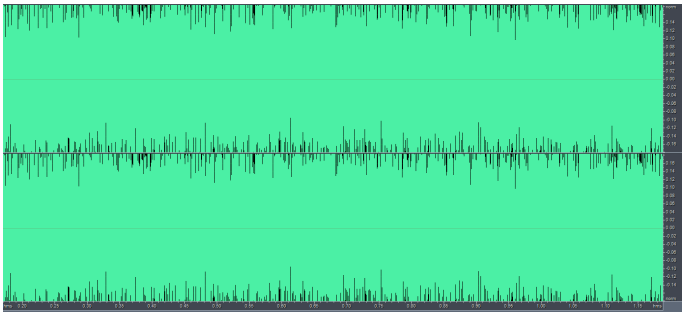


图 9.3.4-3 输入音源图

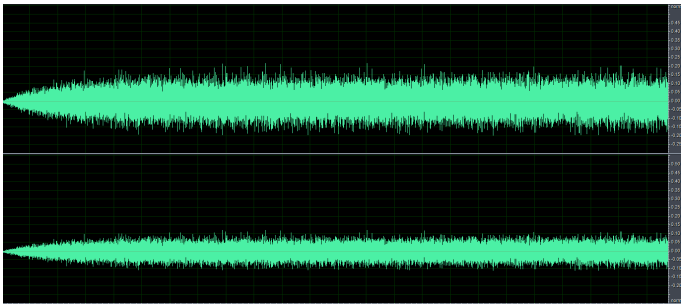


图 9.3.4-4 Multiple Volume 输出时域

9.3.5 Single Volume(Slew Linear)

功能说明：线性平滑音量控制。
在具备音量调节功能的基础上，增加了平滑时间的控制参数。当用户改变增益时，模块会在设定的时间内，以线性的方式从当前增益平滑过渡到目标增益。

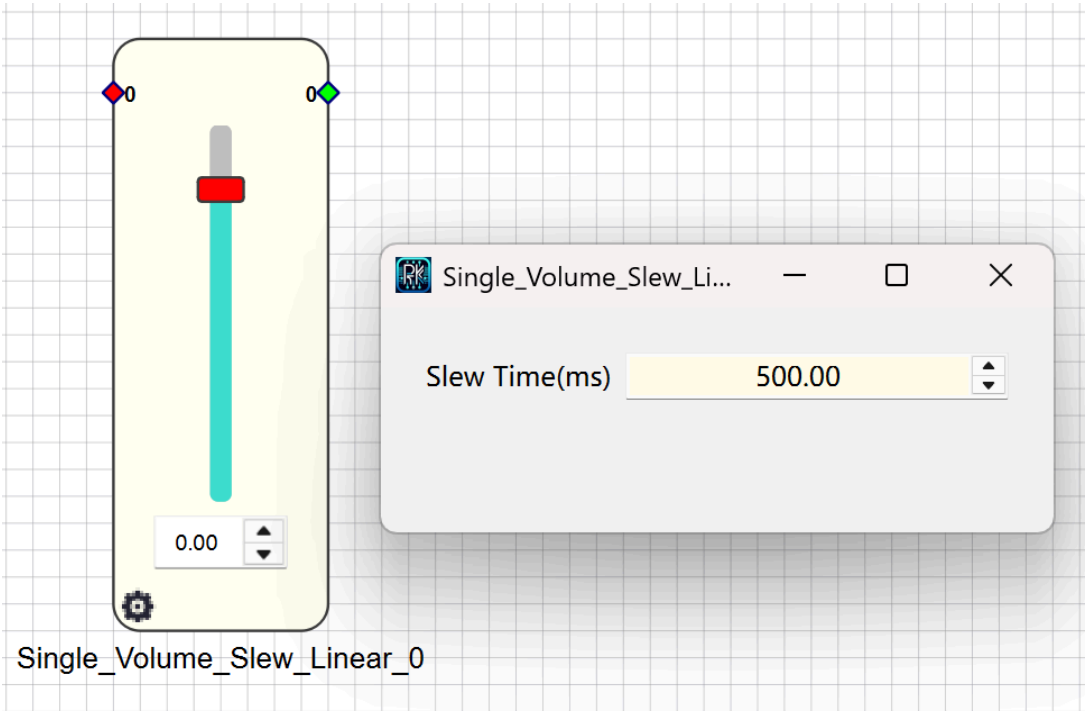


图 9.3.5-1 Single Volume(Slew Linear) 控件和设置界面

9.3.6 Multiple Volume(Slew Linear)

功能说明：多通道线性平滑音量控制。
该模块结合了Multiple Volume的多通道独立控制功能和 Slew Linear的自定义平滑时间功能。

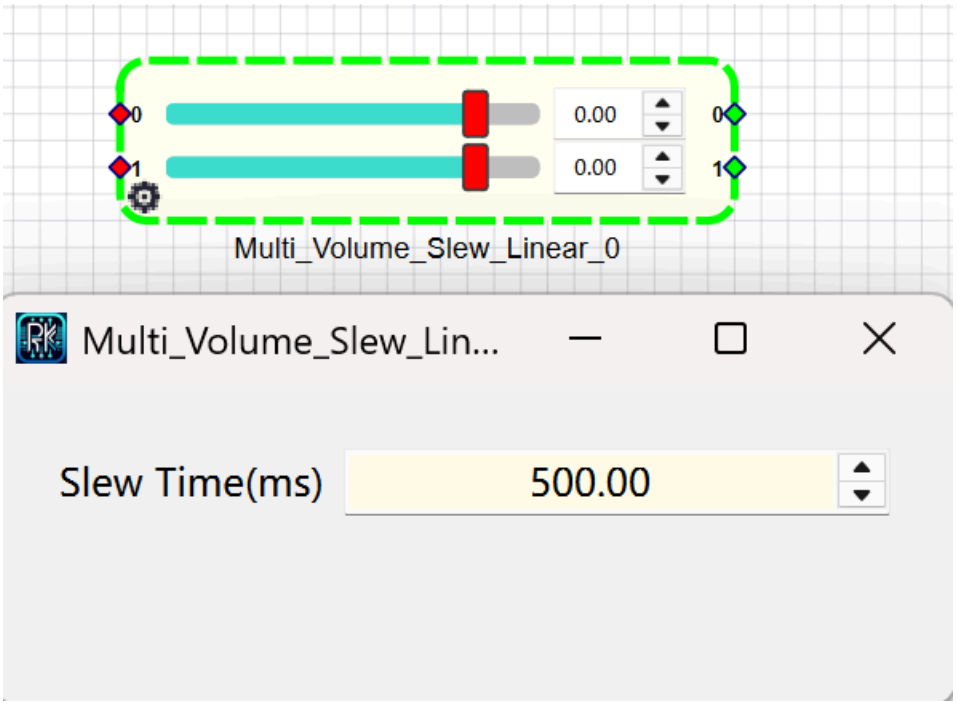


图 9.3.6-1 Multiple Volume(Slew Linear) 控件和设置界面

9.3.7 Single Volume(Slew Ext)

功能说明：外部控制的带平滑单通道音量。

- 信号流向：Input 0（上方蓝色引脚）为控制信号输入，Input 1（下方红色引脚）为音频信号输入。
- 工作原理：模块利用 Input 0 的数值作为增益系数应用到 Input 1上。
- 平滑处理：当Input 0的控制值发生变化时，模块不会立即跳变增益，而是根据 Slew Rate设定的速率进行平滑过渡，常用于实现包络跟随控制音量或闪避效果。

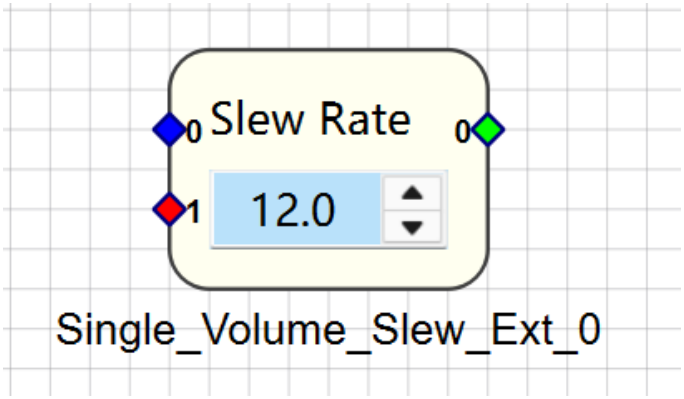


图 9.3.7-1 Single Volume(Slew Ext) 控件和设置界面

10 Detectors/Lookup Tables

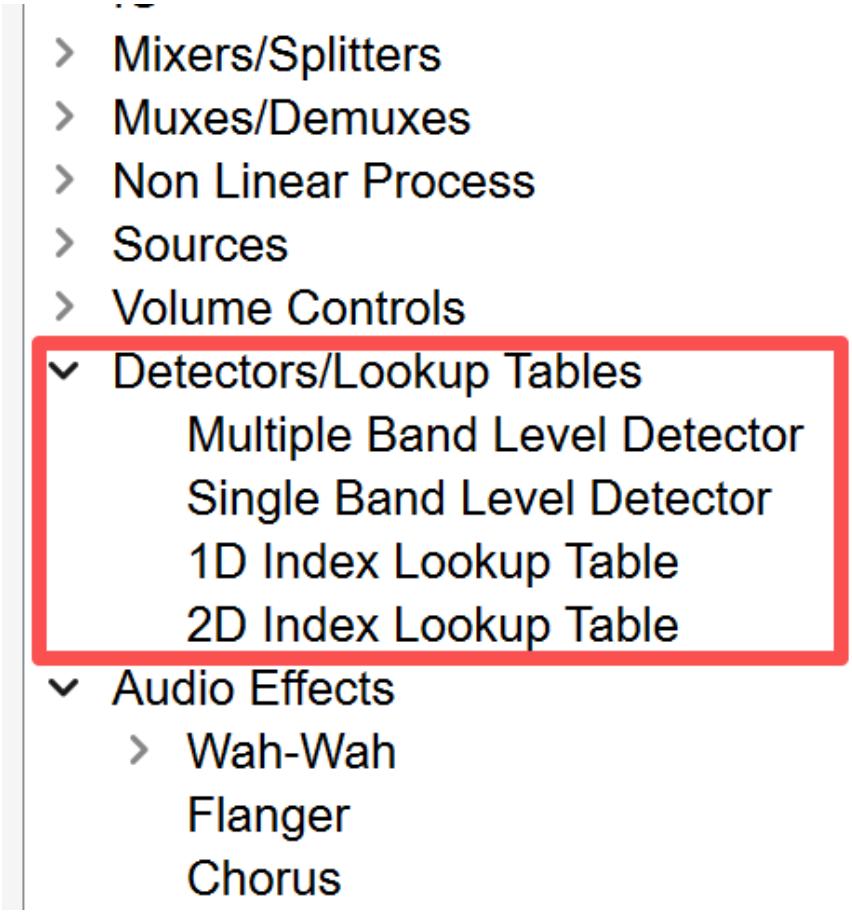


图 10-1 Detectors位于工具的位置

10.1 Multiple Band Level Detector

功能说明：检测多频带的能量，并通过参数读取的方式获取。原理为设置多个并行的滤波器，每路滤波器对应一个频段的检测。

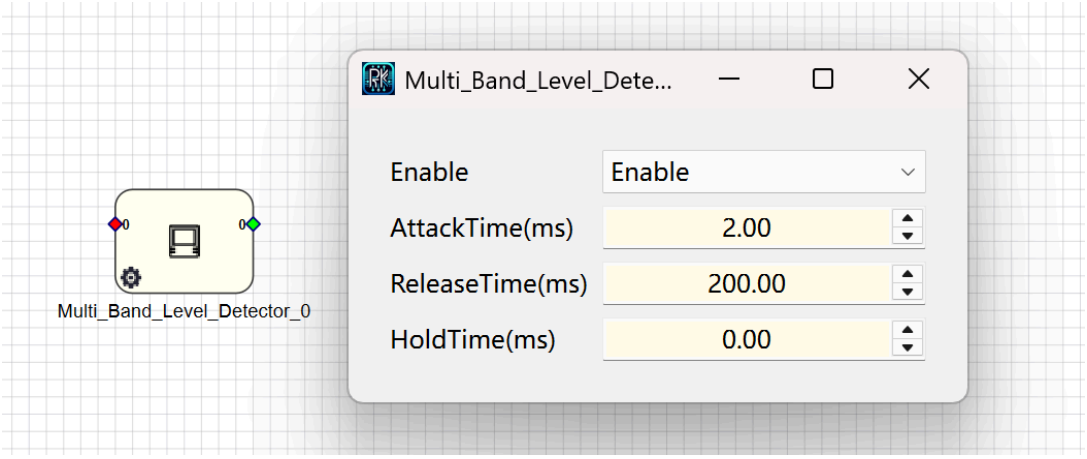


图 10.1-1 控件和通用参数设置界面

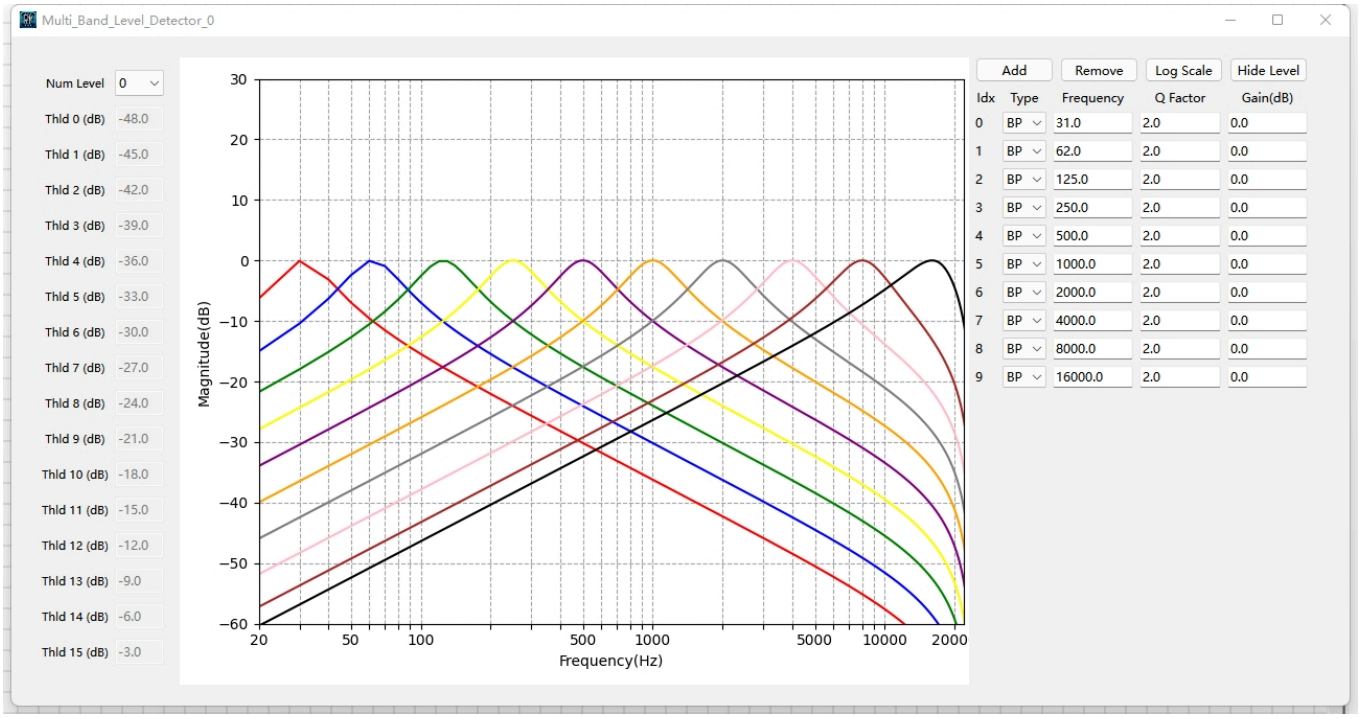


图 10.1-2 频带设置界面

图 10.1-2为多频带检测器参数设置界面(点击中间按钮可以进入)，点击右上角Show Level，会显示频带等级划分功能，一般可以不用。左边的Thld是设置阈值区间，会返回对应的能量等级。

通用参数数量6：

参数名	参数位	类型	功能	单位	建议取值	默认取值
Enable	0	float	是否使能		0/1	1
Type	1	float	检测类型 （Peak/Rms）		0/1	0
Attack Time (ms)	2	float	能量统计的上升时间	ms	0~1000	0
Release Time (ms)	3	float	能量统计的衰减时间	ms	0~1000	0
Detect threhold(dB)	4	float	总检测阈值	dB	-138 ~ 24	0
Band Num	5	float	频带数量		1~32	10
Thld Num	6	float	Level数量		1~16	0

每一路滤波器的参数设置:
参数数量：4

参数名	参数位	类型	功能	单位	建议取值	默认取值
idx	...	float	滤波器序号			
Type	...	float	滤波器类型		0 /1/2	0
Frequency	..	float	检测频带中心频率	HZ	20 ~ 23760	0
Q_Factor	...	float	品质参数Q		0.01 ~ 16	2
Gain(dB)	...	float	增益	dB	0.01 ~ 16	0

如果定义了10个band，16个Thld Level，那么接下来的参数为(从7开始是因为有一个参数位做了预留)：

参数名	参数位	类型	功能	单位	建议取值	默认取值
Coeff	7~(7 + 5 * 10)	float	滤波器系数			
Thld	57~(57 + 16)	float	阈值划分，level等级 ，如果Thld Num为0，则不存储			
Rms	73 ~ (73 + 10)	float	每个频带的能量，需要应用对参数进行读取使用	dB		
Level	83 ~ (83 + 10)	float	每个频带能量对应的level，需要应用对参数进行读取使用			

10.2 Single Band Level Detector

功能说明：检测音频信号能量，并通过参数读取的方式获取

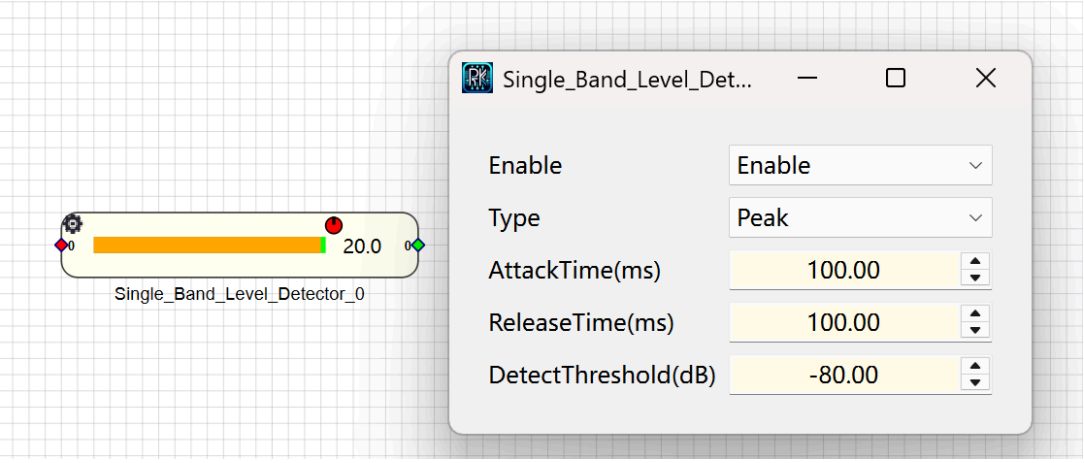


图 10.2-1 控件和参数设置界面

参数数量：5

参数名	参数位	类型	功能	单位	建议取值	默认取值
Enable	0	float	是否使能		0/1	1
Type	1	float	检测类型 （Peak/Rms）		0/1	0
Attack Time (ms)	2	float	能量统计的上升时间	ms	0~2000	100
Release Time (ms)	3	float	能量统计的衰减时间	ms	0~2000	100
Detect Threshold(dB)	4	float	检测阈值	dB	-138 ~ 24	-80

10.3 1D Index Lookup Table

功能说明：一维索引查找表。
该模块将输入信号视为索引值，并在内部预定义的表格中查找对应的数值作为输出。

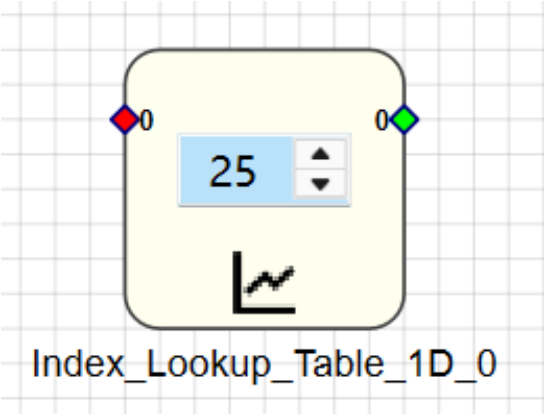


图 10.3-1 1D Index Lookup Table控件

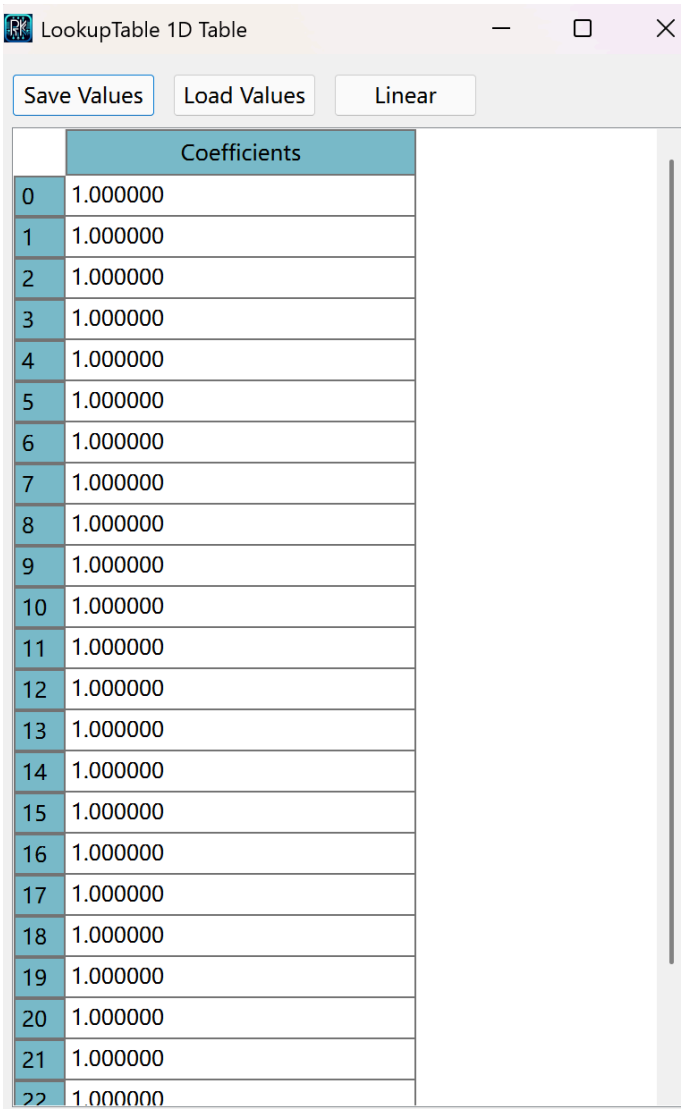


图 10.3-2 1D Index Lookup Table参数设置界面

10.4 2D Index Lookup Table

功能说明：二维索引查找表。
该模块接收两个输入信号（Input 0 对应 X轴，Input 1 对应 Y轴），将它们作为坐标索引，在内部定义的二维矩阵（表格）中查找对应的数值并输出。

- X 索引：决定表格的列（Column）。
- Y 索引：决定表格的行（Row）。

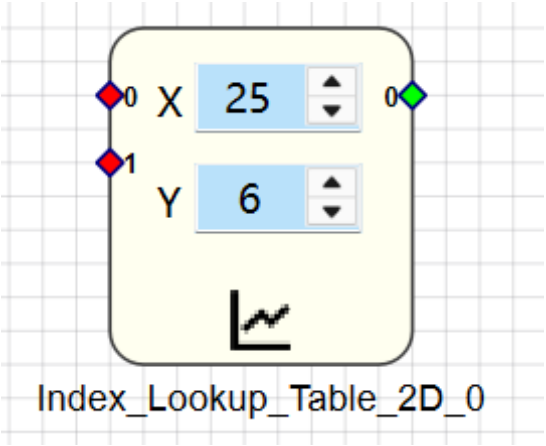
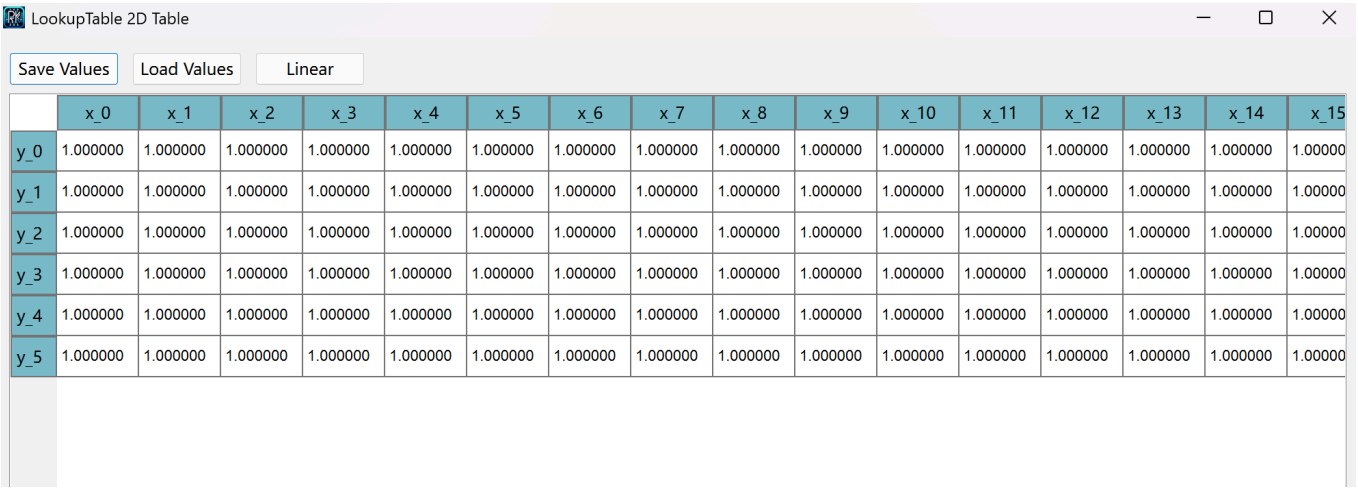


图 10.4-1 2D Index Lookup Table控件



	x_0	x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9	x_10	x_11	x_12	x_13	x_14	x_15
y_0	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.00000
y_1	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.00000
y_2	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.00000
y_3	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.00000
y_4	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.00000
y_5	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.00000

图 10.4-2 2D Index Lookup Table参数设置界面

11 Audio Effects

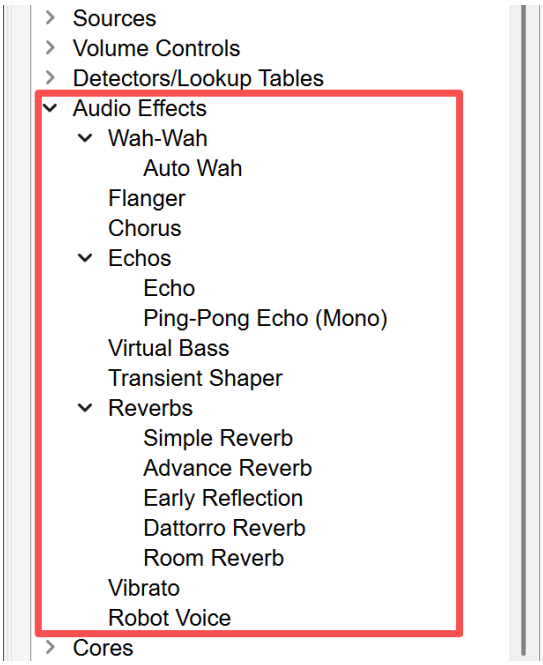


图 11-1 Audio Effects位于工具的位置

11.1 Wah-Wah

11.1.1 Auto Wah

功能说明：一种音频效果控件，用于模拟传统哇音踏板（Wah-Wah Pedal）的效果

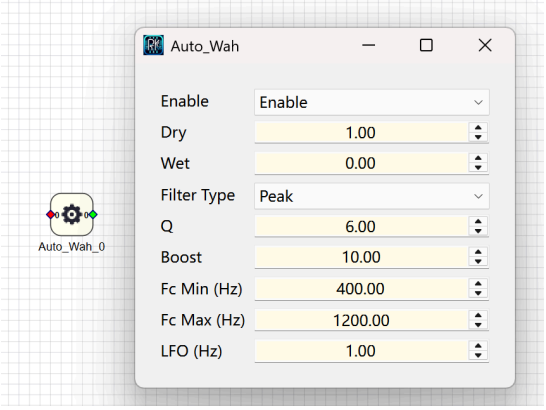


图 11.1.1-1 Auto Wah控件及参数设置

参数数量：9

参数名	参数位	类型	功能	单位	建议取值	默认取值
Enable	0	float	是否使能		0/1	1
Dry	1	float				1
Wet	2	float				0
Filter Type	3	float	滤波器类型		0/1	0
Q	4	float	品质因数			6
Boost	5	float				10
Fc Min (HZ)	6	float	最小截止频率	HZ		400
Fc Max (HZ)	7	float	最大截止频率	HZ		1200
LFO （HZ）	8	float		HZ		1

11.2 Flanger

功能说明：通过LFO配合延迟，实现特殊音效，结构框图如下所示。

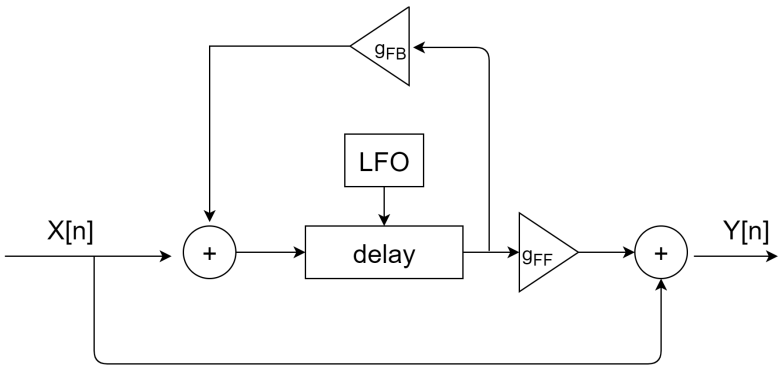


图 11.2-1 Flanger控件框图

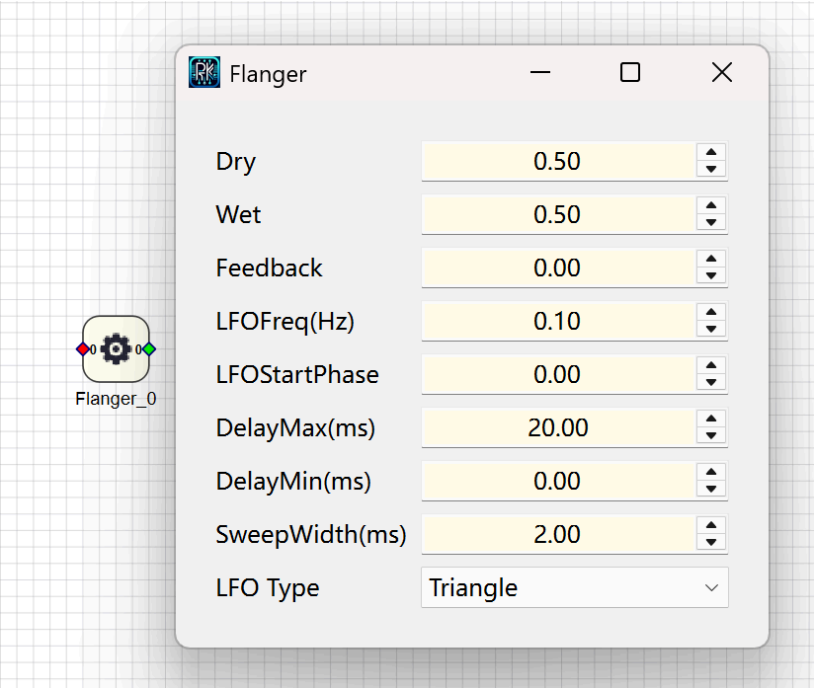


图 11.2-2 Flanger控件及参数设置

参数数量：9

参数名	参数位	类型	功能	单位	建议取值	默认取值
Dry	0	float	干信号系数(原始信号音量)	-	0.0 ~ 1.0	0.5
Wet	1	float	湿信号系数(效果信号音量)	-	0.0 ~ 1.0	0.5
Feedback	2	float	反馈系数，对应图11.3-1的gFB	-	0.0 ~ 0.99	0
LFOFreq(Hz)	3	float	低频振荡器频率	Hz	0.1 ~ 10.0	0.10
LFOStartPhase	4	float	起始相位	degree	0 ~ 360	0.00
DelayMax(ms)	5	float	延迟buffer最大长度	ms	1.0 ~ 50.0	20.00
DelayMin(ms)	6	float	最小延迟时间	ms	0.0 ~ 10.0	0.00
SweepWidth(ms)	7	float	延迟变化宽度	ms	0.1 ~ 10.0	2.00
LFO Type	8	int	低频振荡器波形类型	-	0 ~ 3	Triangle

11.3 Echo

11.3.1 Echo

功能说明：基于多抽头的delayline，实现回声效果，结构框图如下所示，该模块会从延迟buffer里取出多个数据，并乘以不同系数得到输出结果，同时输出结果经过低通滤波器后会反馈回去，和输入数据一起存入延迟buffer中。

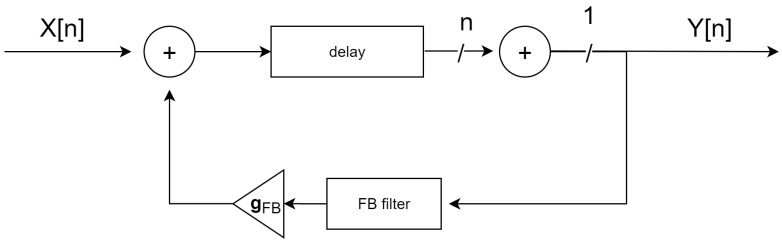


图 11.3.1-1 Echo控件框图

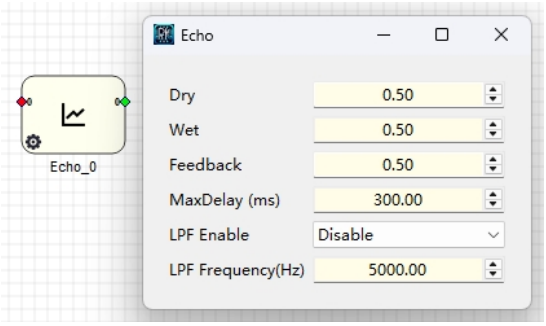


图 11.3.1-2 点击模块左下角按钮，配置Echo控件整体参数

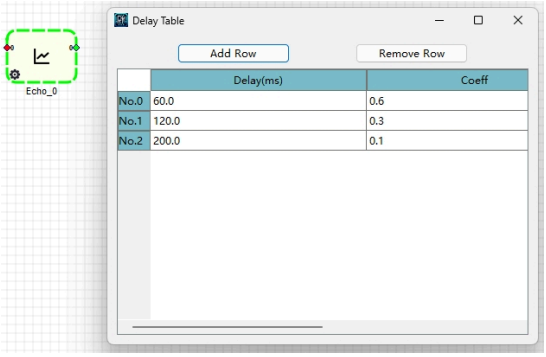


图 11.3.1-3 点击模块中间按钮，配置抽头参数

参数数量：5+5+1+2*n（n为延迟点数目）

整体参数

参数名	参数位	类型	功能	单位	建议取值	默认取值
Dry	0	float	干信号系数			0.5
Wet	1	float	湿信号系数			0.5
Feedback	2	float	反馈系数，对应图11.3-1的gFB			0.5
MaxDelay	3	float	延迟buffer最大长度，(抽头延迟在该范围内时支持实时调参)	ms		300
LPF Enable	4	float	低频滤波器使能开关	Hz		Disable
LPF Frequency	5	float	低频滤波器截止频率	Hz		5000

第i个抽头参数(i=1,2...n)

参数名	参数位	类型	功能	单位	建议取值	默认取值
Delay	11+i-1	float	抽头对应的延时	ms		
Coeff	11+n+i	float	抽头对应的系数			

注：使用一个以上的抽头可能会使回声显得杂乱，初步调试时建议使用1个抽头。

11.3.2 Ping-Pong Echo (Mono)

功能说明：单声道乒乓回声。
该模块接收单声道输入信号，将其分为左右两个通道进行处理。通过设置不同的左右声道延时时间（Delay L 和 Delay R），并在两个声道之间引入交叉反馈，使得回声在左、右声道之间来回跳跃，产生宽广的立体声空间感。

- 输入：单声道。
- 输出：立体声，包含 L 和 R 两个通道。

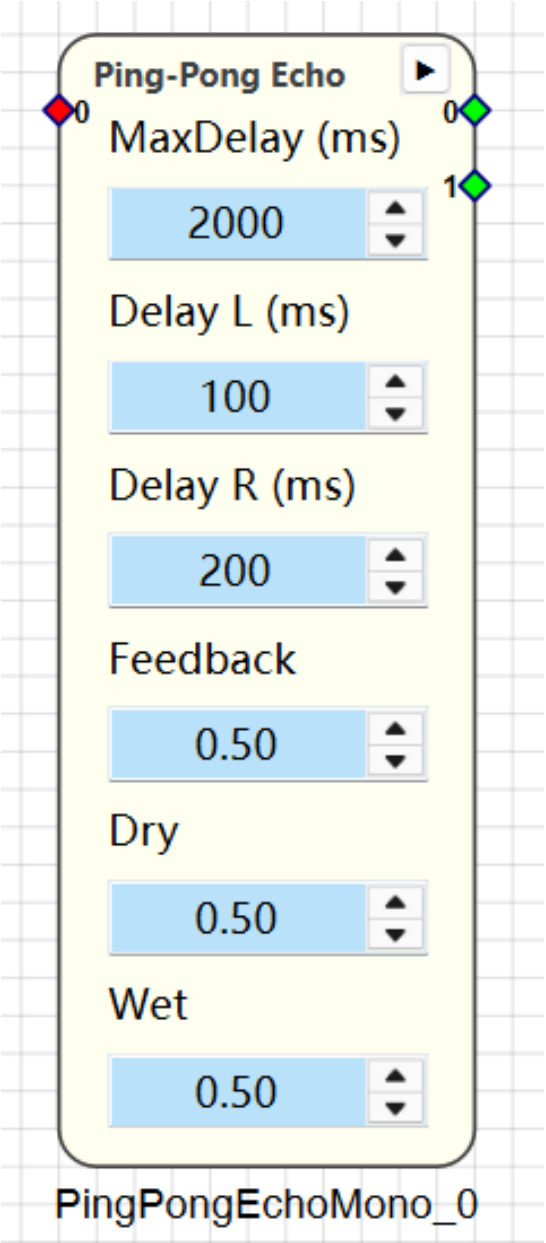


图 11.3.2-1 Ping-Pong Echo (Mono)控件框图

11.4 Virtual Bass

功能说明：利用missing fundamental原理，实现虚拟低音效果，逻辑框图如下，其中HFIL为四阶link-witz高通滤波器，FIL1为4阶link-witz低通滤波器+4阶link-witz高通滤波器，FIL2为4阶link-witz低通滤波器+4阶link-witz高通滤波器。

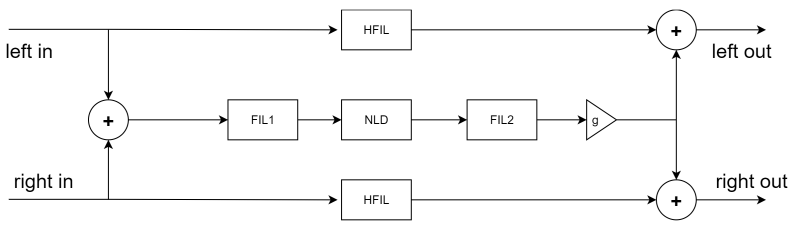


图 11.4-1 Virtual Bass逻辑框图

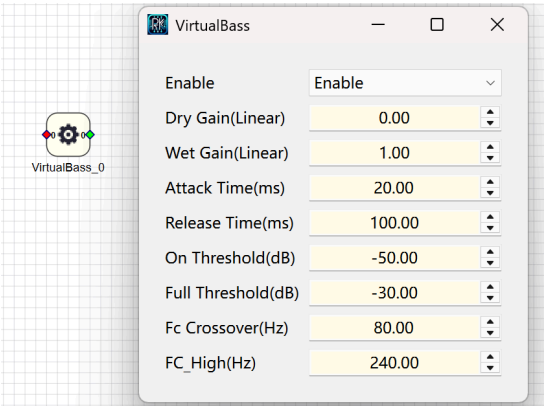


图 11.4-2 Virtual Bass参数设置

参数数量：9

参数名	参数位	类型	功能	单位	建议取值	默认取值
Enable	0	float	模块使能开关	-	0 / 1	1.0
Dry Gain(Linear)	1	float	干信号增益（线性值）	-	0.0 ~ 2.0	0.00
Wet Gain(Linear)	2	float	湿信号增益（增强信号线性值）	-	0.0 ~ 2.0	1.00
Attack Time(ms)	3	float	检测器建立时间	ms	1 ~ 100	20.00
Release Time(ms)	4	float	检测器释放时间	ms	10 ~ 1000	100.00
On Threshold(dB)	5	float	开启阈值，高于此电平开始增强	dB	-90 ~ 0	-50.00
Full Threshold(dB)	6	float	满幅阈值，高于此电平达最大增强	dB	-90 ~ 0	-30.00
Fc Crossover(Hz)	7	float	分频频率，低于此频率进行低音处理	Hz	40 ~ 200	80.00
FC_High(Hz)	8	float	产生谐波的高频截止频率	Hz	100 ~ 500	240.00

除了使用该模块实现Virtual Bass效果外，工具也可以通过滤波器+NLP配合的方式实现同样效果，示意图如下：

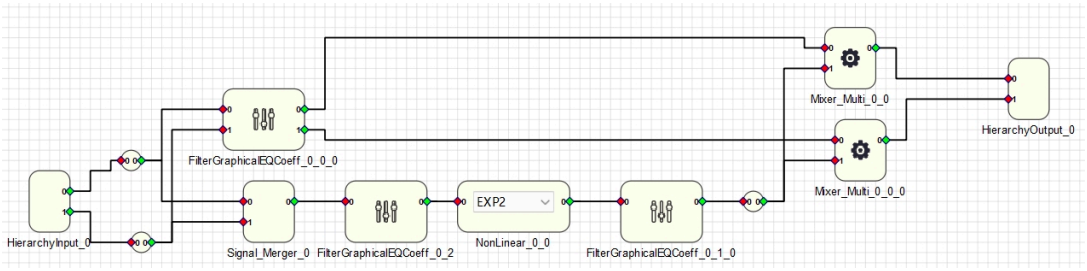


图 11.4-3 利用滤波器+NLP实现Virtual Bass

11.5 Reverbs

11.5.1 Simple Reverb

功能说明：实现简单的混响效果，支持单声道和立体声。

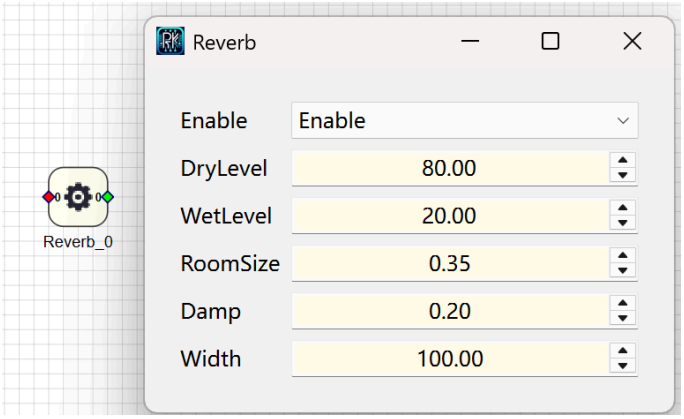


图 11.5.1-1 Simple Reverb控件和设置界面

参数数量：6

参数名	参数位	类型	功能	单位	建议取值	默认取值
Enable	0	float	使能	无	0/1（0禁能/1使能）	1
DryLevel*	1	float	干声(原始声音)比例	%	0 ~ 100	80
WetLevel*	2	float	湿声(混响系统处理输出声音)比例	%	0 ~ 100	20
RoomSize	3	float	混响强度系数,值越大效果越越明显		0 ~ 1	0.35
Damp	4	float	低通滤波器，系数越大，滤波效果越明显		0 ~ 1	0.2
Width	5	float	立体声宽度	%	0 ~ 100	100

注1：RoomSize对应内部反馈式梳状滤波器的反馈系数，所以该数值越接近1，混响拖尾效果越明显，但是注意不能为1，否则算法会溢出。并且混响拖尾的长度与该参数不为线性关系。

注2：Damp对应内部一阶低通滤波器系数，因此该数值越大，混响拖尾的高频衰减也就越明显。

注3：Width影响了该模块输出的左右两通道数据的相互混音比例，若设为0，则两通道会完全混音，从而导致两通道的混响拖尾完全一致。设为100，则不进行混音。

实际效果展示，输入立体声音乐，相关case配置如下：

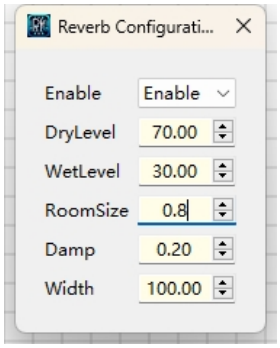


图 11.5.1-2 Simple Reverb参数

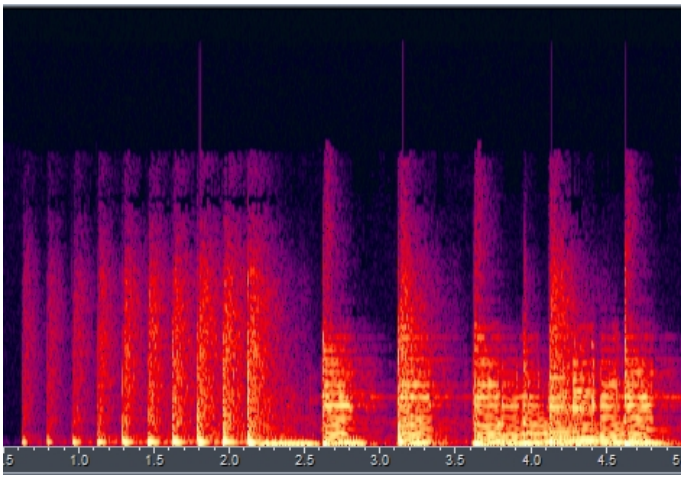


图 11.5.1-3 输入音乐频谱图

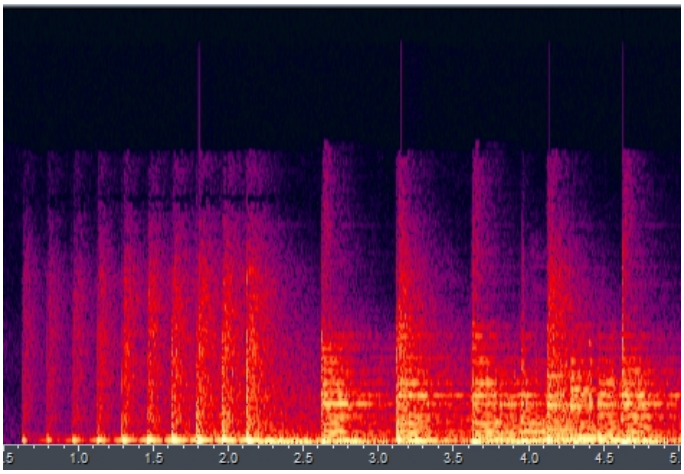


图 11.5.1-4 输出音乐频谱图

下图展示了其他参数相同情况下(DryLevel=0,WetLevel=100,RoomSize=0.8,Width=100)，Damp为0.2和0.5的区别。

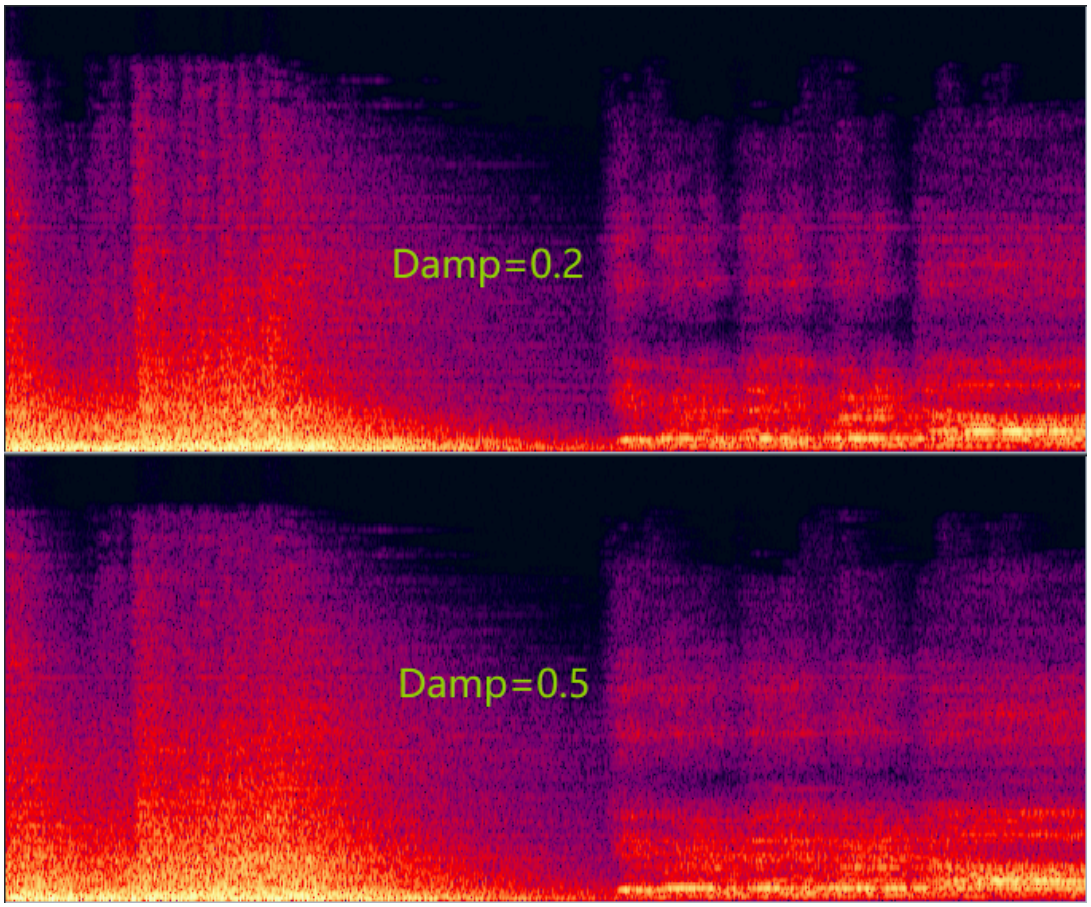


图 11.5.1-5 damp对混响效果的影响

11.5.2 Dattorr Reverb

功能说明：参考Effect Design Part1: Reverberator and Other Filters文献设计的混响模块，该模块的详细结构以及各参数意义请参考上述文献。

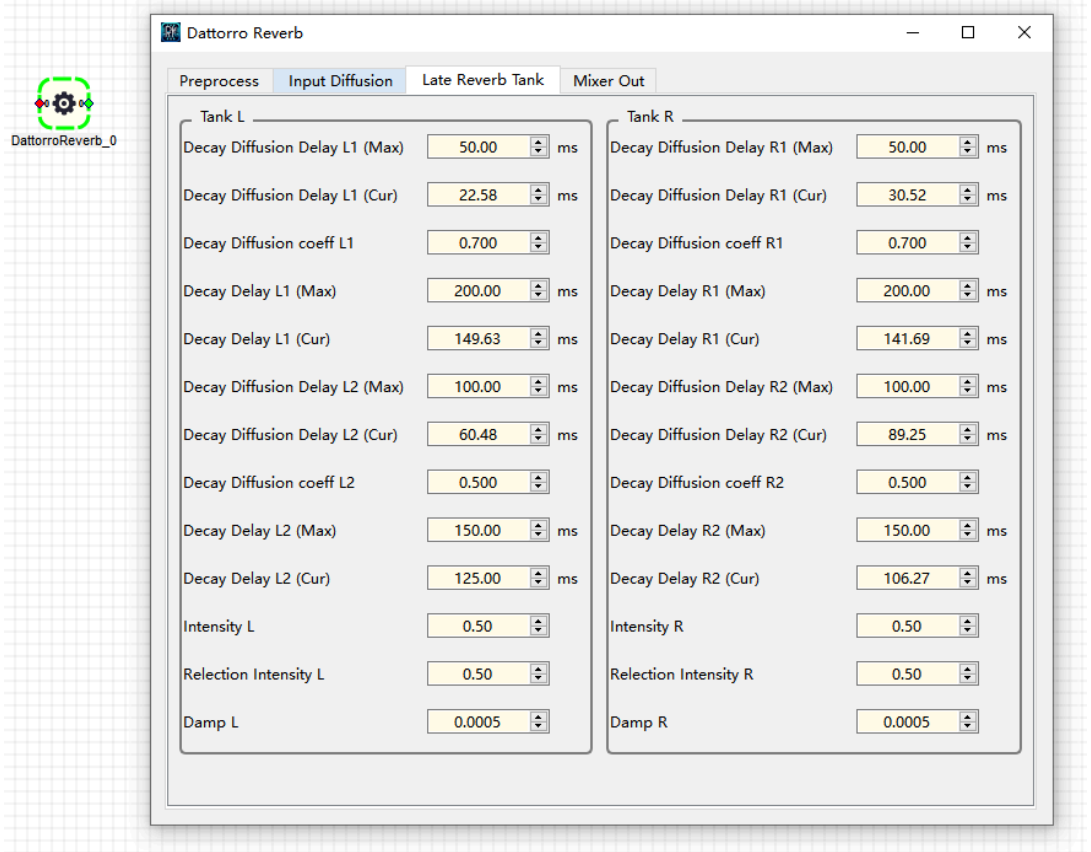


图 11.5.2-1 Dattorr Reverb参数

注1：原文的默认参数基于29761Hz采样率计算，工具中已经换算到48000Hz。

注2：如果想简单调整混响的强度，可以尝试修改Late Reverb Tank部分中的 Intensity L\R 以及Relection Intensity L\R参数。

11.5.3 Advance Reverb

功能说明：高级混响。
相比 Simple Reverb，提供了更多的参数控制，允许用户对混响的扩散度、衰减特性以及空间宽度进行更精细的调节。

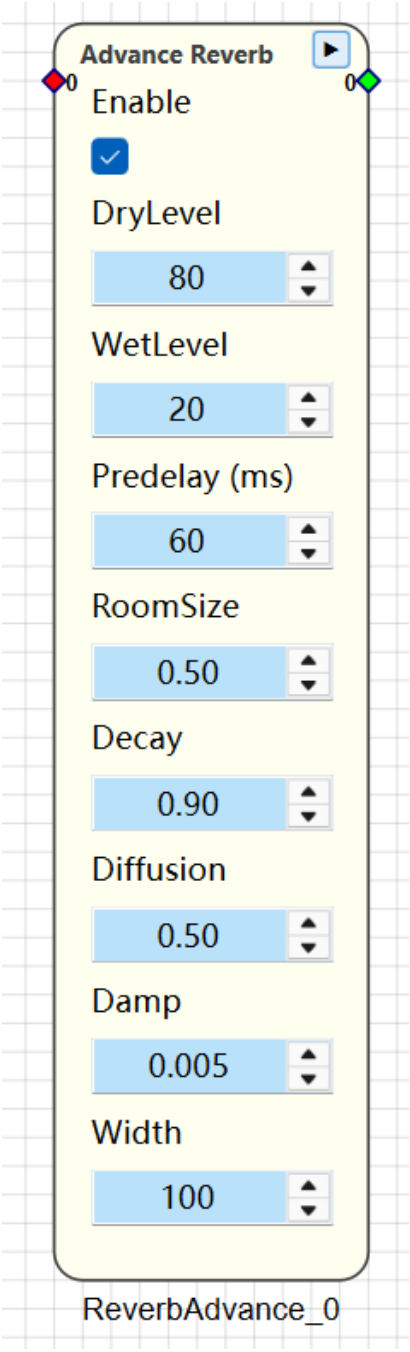


图 11.5.3-1 Advance Reverb参数

参数数量：9

参数名	参数位	类型	功能	单位	建议取值	默认取值
Enable	0	float	模块使能开关	-	0/1	1
DryLevel	1	float	干信号（直达声）音量	-	0 ~ 100	80
WetLevel	2	float	湿信号（混响声）音量	-	0 ~ 100	20

参数名	参数位	类型	功能	单位	建议取值	默认取值
Predelay (ms)	3	float	预延时。直达声到达后，反射声开始出现的时间差。	ms	0 ~ 200	60
RoomSize	4	float	房间大小系数。影响内部延迟线的长度，值越大空间感越大。	-	0 ~ 1.0	0.50
Decay	5	float	衰减时间系数。决定混响尾音持续的长短。	-	0 ~ 1.0	0.90
Diffusion	6	float	扩散度。值越高，反射声越密集（如平滑的墙壁）；值越低，回声颗粒感越强。	-	0 ~ 1.0	0.50
Damp	7	float	高频阻尼。模拟空气或墙壁对高频的吸收，值越大高频衰减越快，声音越“暗”。	-	0 ~ 1.0	0.005
Width	8	float	立体声宽度。	%	0 ~ 100	100

11.5.4 Early Reflection

功能说明：早期反射模拟器。
该模块专门用于模拟房间的反射模型，用于增强空间感。

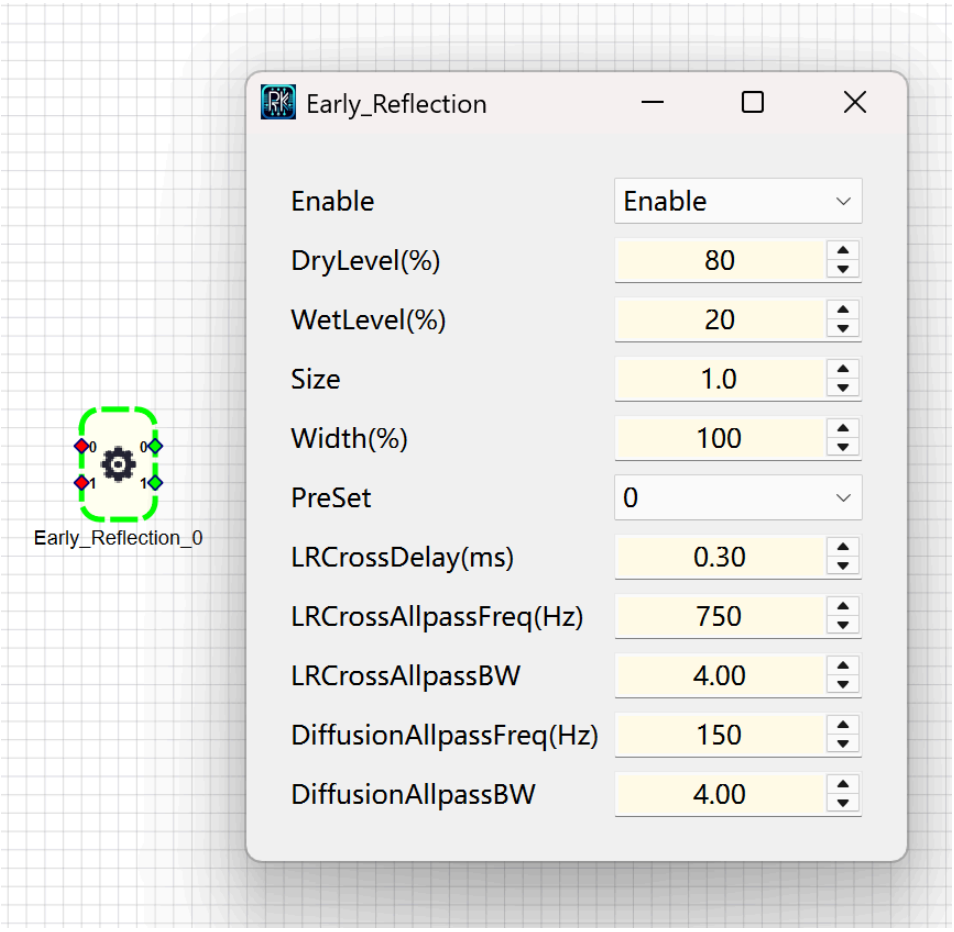


图 11.5.4-1 Early Reflection参数

参数数量：11

参数名	参数位	类型	功能	单位	建议取值	默认取值
Enable	0	float	模块使能开关	-	0/1	Enable
DryLevel(%)	1	float	干信号比例	%	0 ~ 100	80
WetLevel(%)	2	float	湿信号比例	%	0 ~ 100	20
Size	3	float	反射空间大小系数	-	0 ~ 2.0	1.0
Width(%)	4	float	立体声宽度	%	0 ~ 100	100
PreSet	5	float	预设模式，选择不同的反射模型。	-	0 ~ 32	0

参数名	参数位	类型	功能	单位	建议取值	默认取值
LRCrossDelay(ms)	6	float	左右声道交叉反馈延时	ms	0 ~ 10	0.30
LRCrossAllpassFreq(Hz)	7	float	交叉反馈全通滤波器频率	Hz	20 ~ 20k	750
LRCrossAllpassBW	8	float	交叉反馈全通滤波器带宽	-	0.1 ~ 10	4.00
DiffusionAllpassFreq(Hz)	9	float	扩散全通滤波器频率	Hz	20 ~ 20k	150
DiffusionAllpassBW	10	float	扩散全通滤波器带宽	-	0.1 ~ 10	4.00

11.5.5 Room Reverb

功能说明：房间混响。
这是一种频率相关的混响算法，允许用户独立调节低频和中高频的混响时间。通过设置分频点，可以模拟出不同材质房间的声学特性。

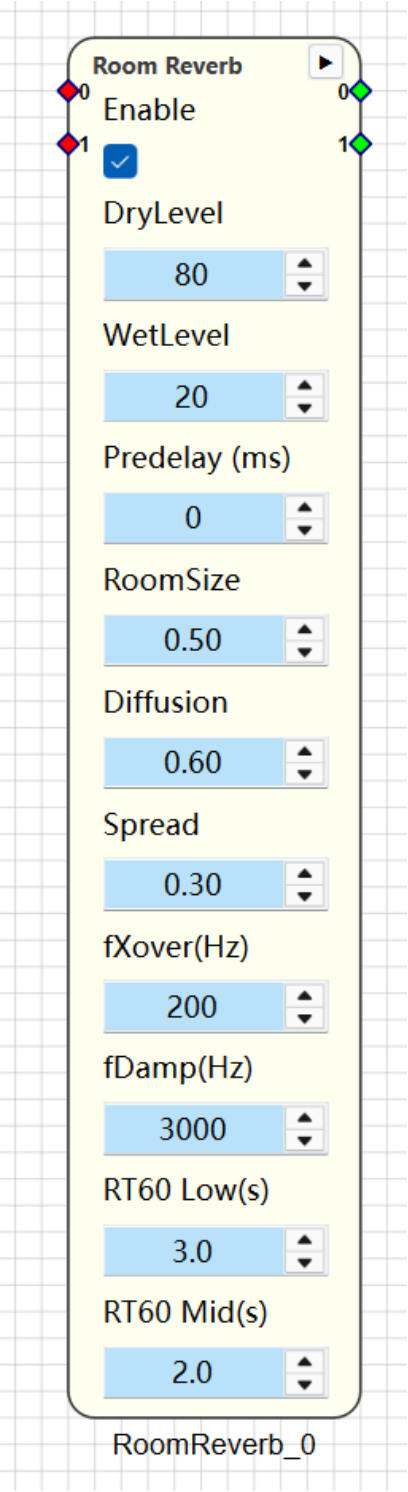


图 11.5.5-1 Room Reverb参数

参数数量：11

参数名	参数位	类型	功能	单位	建议取值	默认取值
Enable	0	float	使能开关	-	0/1	Check
DryLevel	1	float	干信号音量	-	0 ~ 100	80
WetLevel	2	float	湿信号音量	-	0 ~ 100	20
Predelay (ms)	3	float	预延时	ms	0 ~ 200	0
RoomSize	4	float	房间尺寸系数	-	0 ~ 1.0	0.50
Diffusion	5	float	扩散度	-	0 ~ 1.0	0.60

参数名	参数位	类型	功能	单位	建议取值	默认取值
Spread	6	float	空间展开度（立体声增强）	-	0 ~ 1.0	0.30
fXover(Hz)	7	float	低频/中频分频点。用于区分 RT60 Low 和 RT60 Mid 的作用范围。	Hz	20 ~ 1000	200
fDamp(Hz)	8	float	高频阻尼截止频率。高于此频率的声音衰减加速。	Hz	1000 ~ 20k	3000
RT60 Low(s)	9	float	低频混响时间（衰减60dB所需时间）。	s	0.1 ~ 10.0	3.0
RT60 Mid(s)	10	float	中频混响时间（衰减60dB所需时间）。	s	0.1 ~ 10.0	2.0

11.6 Chorus

功能说明：合唱效果器。
该模块通过将原始信号复制一份，并对其进行短暂且不断变化的延时，然后与原信号混合。这种处理模拟了多人同时演唱同一旋律时，由于音高和时间的微小差异而产生的厚实、宽广的声音效果。

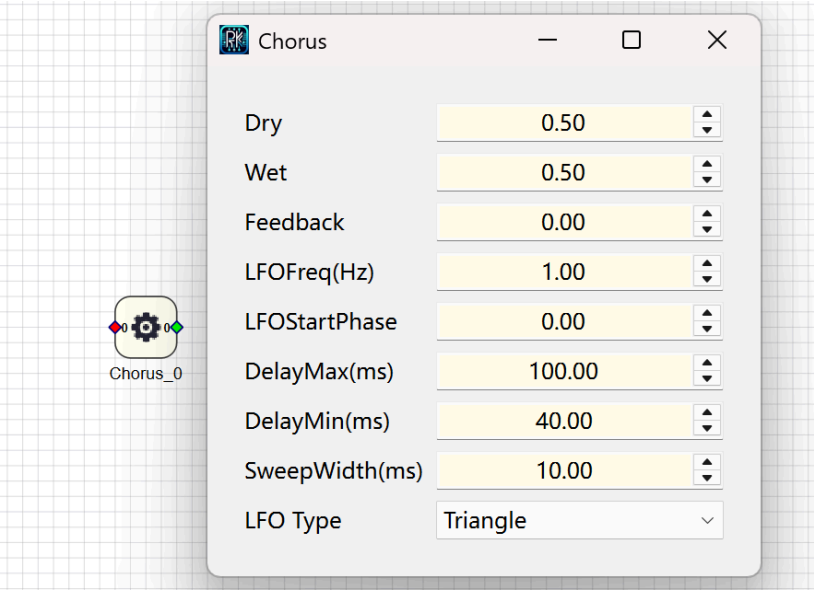


图 11.6-1 Chorus参数

参数数量：9

参数名	参数位	类型	功能	单位	建议取值	默认取值
Dry	0	float	干信号（直达声）比例	-	0 ~ 1.0	0.50
Wet	1	float	湿信号（效果声）比例	-	0 ~ 1.0	0.50
Feedback	2	float	反馈系数，将输出信号反馈回输入，增加金属感（Flanger特性），纯Chorus通常设为0。	-	0 ~ 0.99	0.00
LFOFreq(Hz)	3	float	低频振荡器频率，决定延时变化的快慢（合唱的颤动速度）。	Hz	0.1 ~ 5.0	1.00
LFOStartPhase	4	float	LFO的起始相位	-	0 ~ 1.0	0.00
DelayMax(ms)	5	float	延时缓冲区最大长度（内存分配限制）	ms	> DelayMin+Sweep	100.00
DelayMin(ms)	6	float	最小延时时间（基准延时）	ms	10 ~ 50	40.00
SweepWidth(ms)	7	float	扫频宽度（调制深度），延时时间将在 `DelayMin` 到 `DelayMin + SweepWidth` 之间变化。	ms	0 ~ 20	10.00
LFO Type	8	float	LFO波形形状 (Triangle/Sin等)	-	0~1	1

11.7 Transient Shaper

功能说明：瞬态整形器。
该模块用于重塑声音的动态包络，特别是针对声音的起音部分。

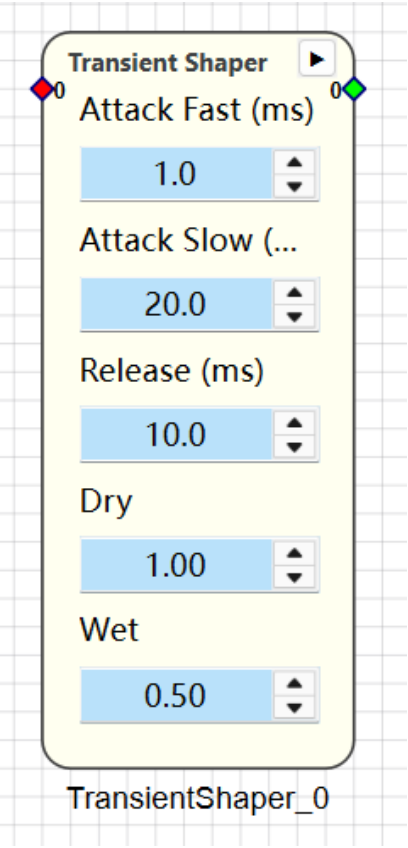


图 11.7-1 Transient Shaper参数

参数数量：5

参数名	参数位	类型	功能	单位	建议取值	默认取值
Attack Fast (ms)	0	float	快包络的建立时间，用于捕捉瞬间的电平峰值。	ms	0.1 ~ 10	1.0
Attack Slow (ms)	1	float	慢包络的建立时间，用于获取平均电平。	ms	10 ~ 100	20.0
Release (ms)	2	float	包络检测的释放时间	ms	10 ~ 500	10.0
Dry	3	float	干信号增益（原始信号）	-	0 ~ 2.0	1.00
Wet	4	float	湿信号增益(提取出的瞬态信号)，增加此值可增强打击感。	-	-2.0 ~ 2.0	0.50

11.8 Vibrato

功能说明：颤音效果器。
通过周期性地调节延时时间，从而产生音高的周期性波动效果。与 Chorus 不同，Vibrato 通常只输出湿信号，且延时时间非常短，主要侧重于音高的抖动而非空间的加厚。

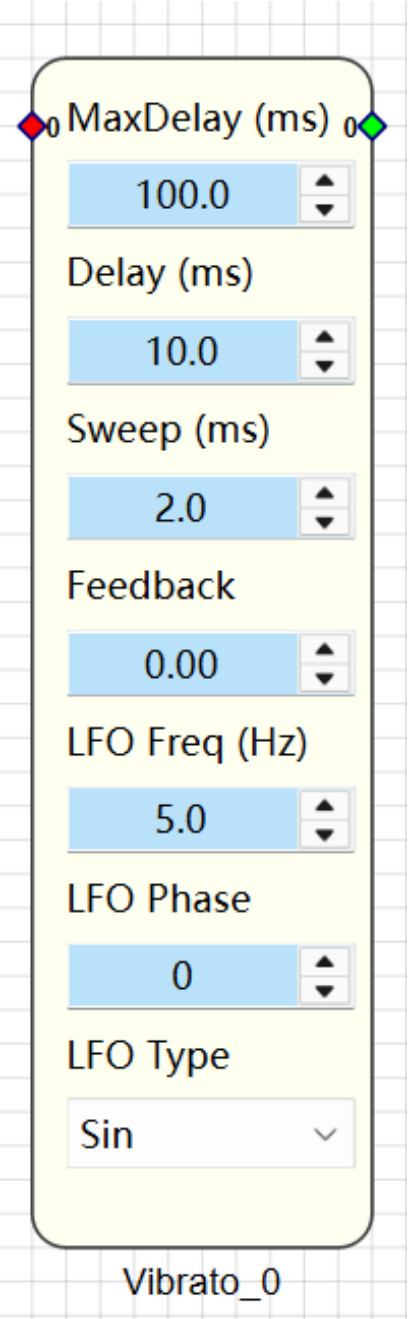


图 11.8-1 Vibrato参数

参数数量：7

参数名	参数位	类型	功能	单位	建议取值	默认取值
MaxDelay (ms)	0	float	最大延时内存分配	ms	> Delay+Sweep	100.0
Delay (ms)	1	float	基础延时时间	ms	1 ~ 20	10.0
Sweep (ms)	2	float	调制深度，决定音高偏离的程度。值越大音高波动越剧烈	ms	0.1 ~ 5.0	2.0
Feedback	3	float	反馈系数	-	0 ~ 0.99	0.00
LFO Freq (Hz)	4	float	颤音速度	Hz	1.0 ~ 10.0	5.0
LFO Phase	5	float	LFO 起始相位	-	0 ~ 1.0	0
LFO Type	6	float	LFO 波形形状 (Sin/Triangle等)	-		Sin

11.9 Robot Voice

功能说明：机器人音效（变声器）。
通过WOLA，生成类似于机器人声的效果。

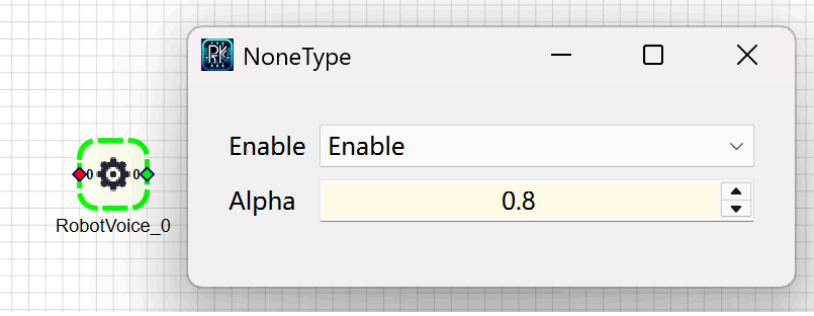


图 11.9-1 Robot Voice参数

参数数量：2

参数名	参数位	类型	功能	单位	建议取值	默认取值
Enable	0	float	模块使能开关。	-	0/1	1
Alpha	1	float	效果系数,通常用于控制变调的程度、颗粒大小或调制的频率。值越大，声音的“机械感”或断裂感可能越强。	-	0 ~ 1.0	0.8

12 Cores

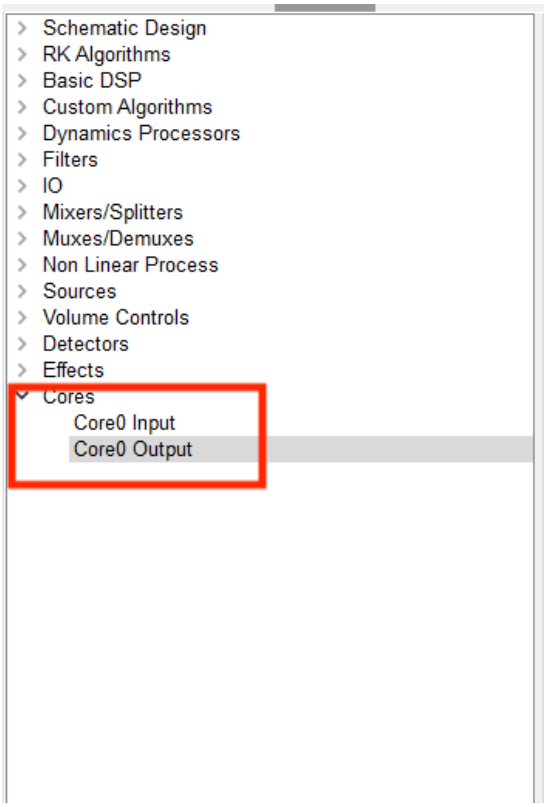


图 12-1 Cores位于工具的位置

本模块用于DSP芯片不同处理核的数据交互，每个核心都可以有一个或者多个输入和输出。

比如我们要实现核心0的输出做核心1的输入。

我们在核心0的编辑页面拖出Core0_output 控件

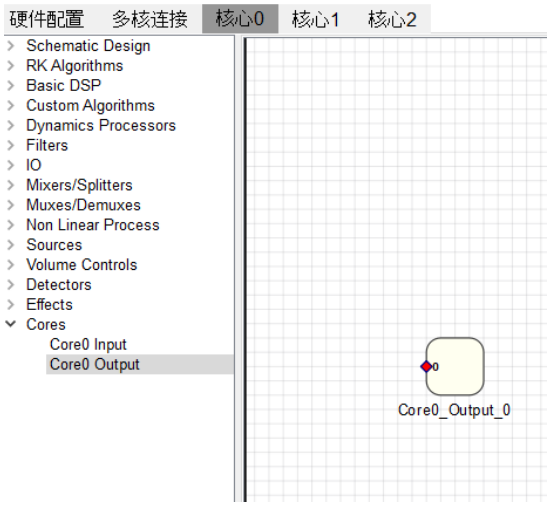


图 12-2 核心0编辑界面

点击左上角核心1进入核心1的图纸编辑界面，在核心1的编辑页面拖出Core1_input 控件

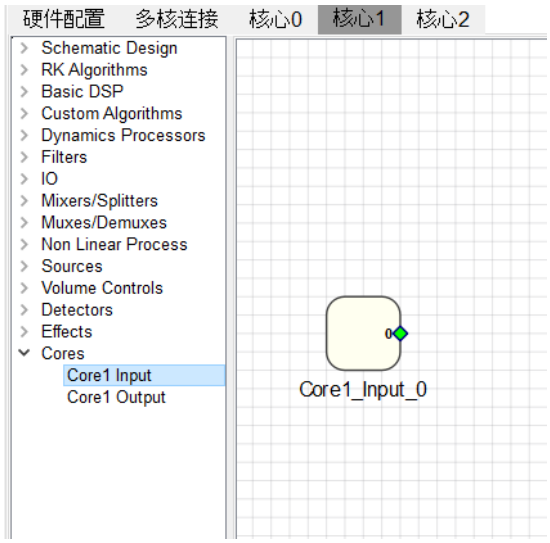


图 12-3 核心1编辑界面

点击左上角的多核连接，将两个控件的输入和输出连接。

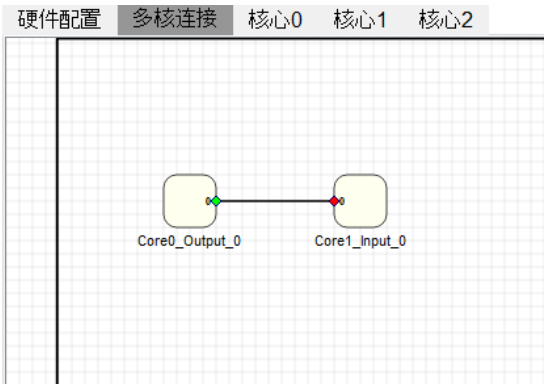


图 12-4 多核连接编辑界面

这样我们就实现了核心0的输出连接到核心1的输入。

12.1 Core0 Input

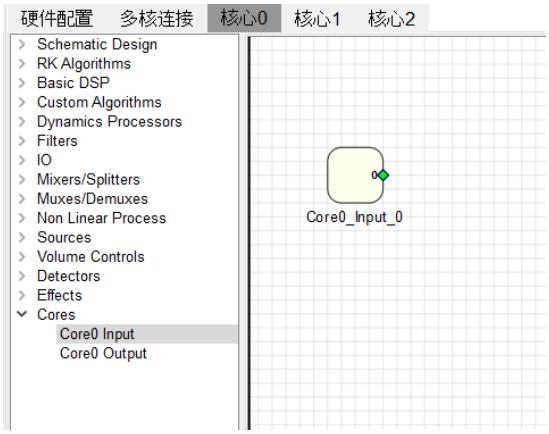


图 12.1-1 Core0 Input控件

在本核心编辑界面，input控件有一个输出口，可输出给本核心的其他控件输入端。

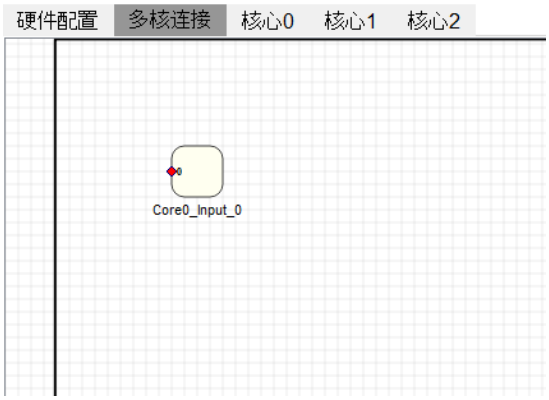


图 12.1-2 Core0 Input控件

在多核连接编辑界面，input控件有一个输入口，可接入其他核的信号输出端。

12.2 Core0 Output

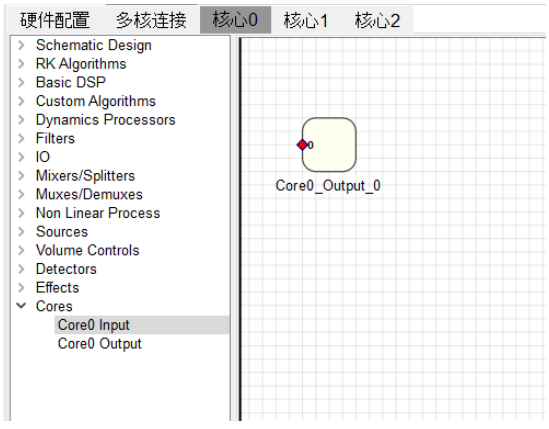


图 12.2-1 Core0 Output控件

在本核心编辑界面，output控件有一个输入端，可接入本核心的其他控件输出端。

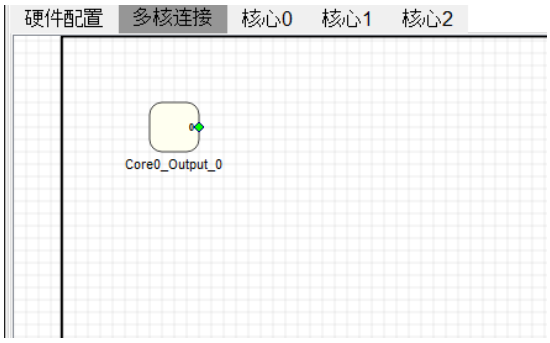


图 12.2-2 Core0 Output控件

在多核连接编辑界面，output控件有一个输出端，可输出给其他核的信号输入端。

13 Custom Algorithms

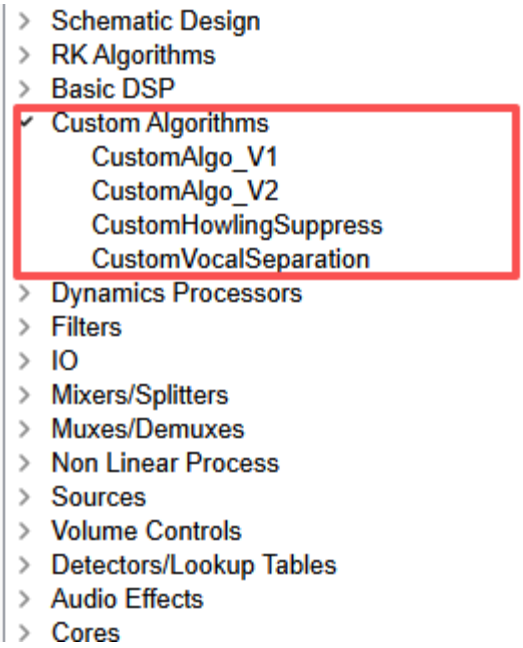


图 13-1 Custom Algorithms位于工具的位置

本模块主要使用第三方算法模版生成的自定义算法模块，CustomAlgo_V1和CustomAlgo_V2为两个示例算法，具体定制和集成方式，可以参考文档《Rockchip_RKStudio_CustomAlgo_Guide_CN.pdf》。

13.1 CustomHowlingSupporess

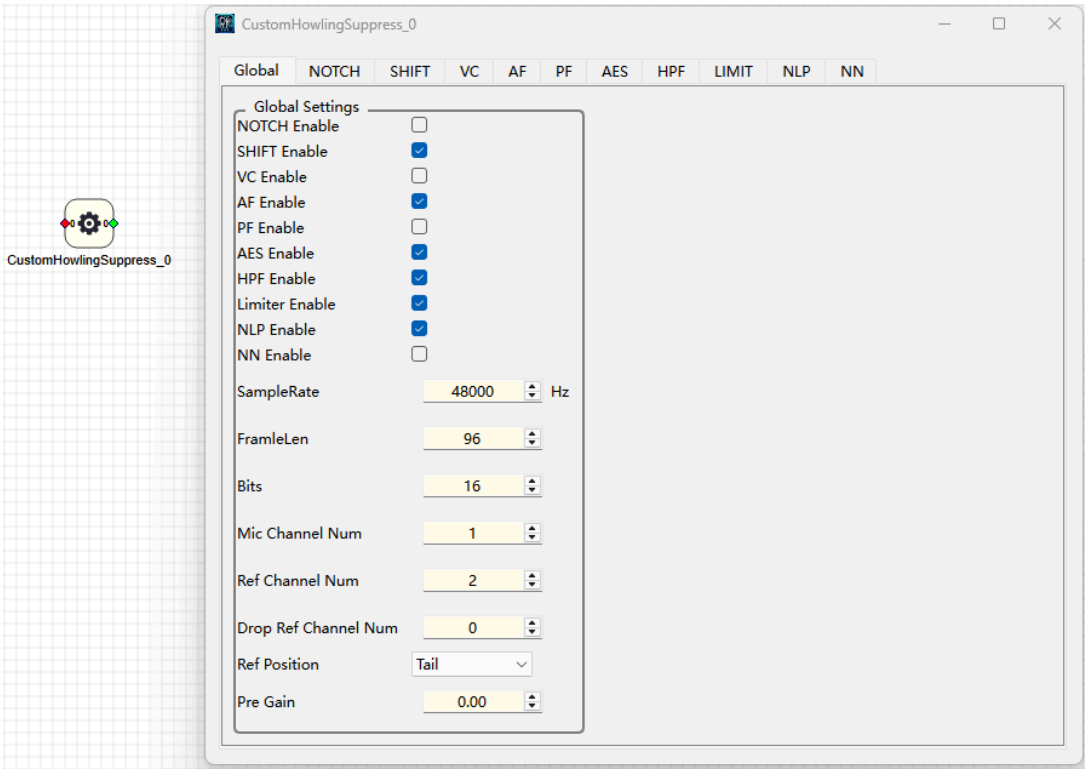


图 13.1-1 啸叫抑制模块算法

该模块实现啸叫抑制，适用于k歌，通话等场景。算法文档可以参考 《Rockchip_Developer_Guide_Howling_Suppresion_Tuning_CN.pdf》。

使用啸叫抑制算法，需要另外集成算法库，具体的使用方法，请联系相关工程师。

13.2 CustomVocalSeparation

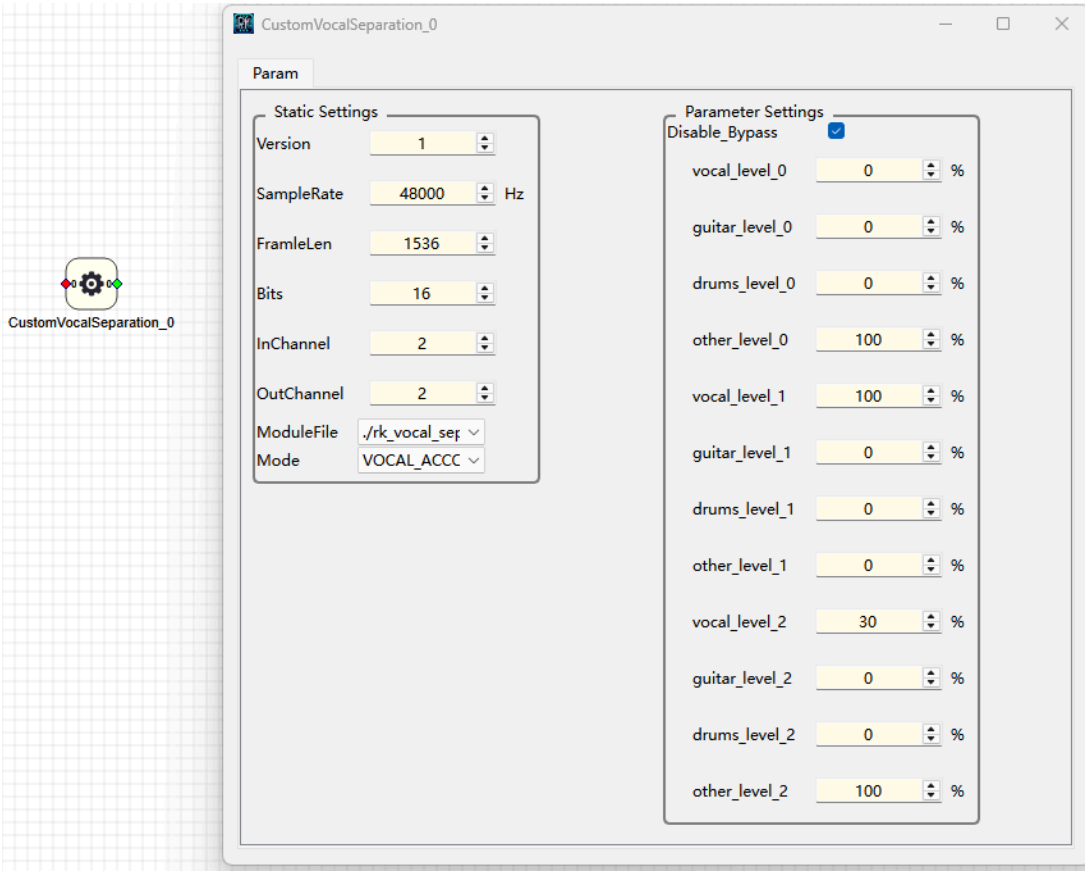


图 13.2-1 音乐声分离模块算法

该模块使用ai功能，实现音乐声分离，适用于k歌，虚拟环绕等场景。

使用音乐声分离算法，需要另外集成算法库，具体的使用方法，请联系相关工程师。

13.3 CustomAlgo_V1

功能说明：自定义算法模板 V1（基础版）。

这是一个示例模块，用于演示如何创建一个包含基础参数（下拉菜单、浮点数输入、整数输入）的自定义算法界面。开发者可参考此模板封装简单的 DSP 算法。

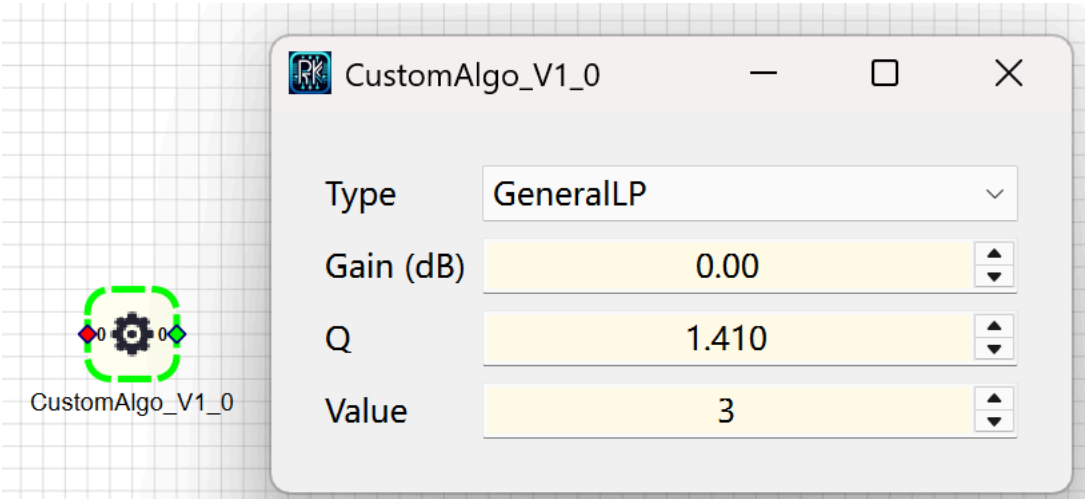


图 13.3-1 CustomAlgo_V1 控件和参数设置

13.4 CustomAlgo_V2

功能说明：自定义算法模板 V2（进阶版）。

这是一个更复杂的示例模块，展示了如何构建包含多标签页、选框、文件读取按钮以及复杂配置逻辑的算法界面。通常用于封装如 AEC、LMS 自适应滤波等复杂算法。

图 13.4-1 CustomAlgo_V2 控件

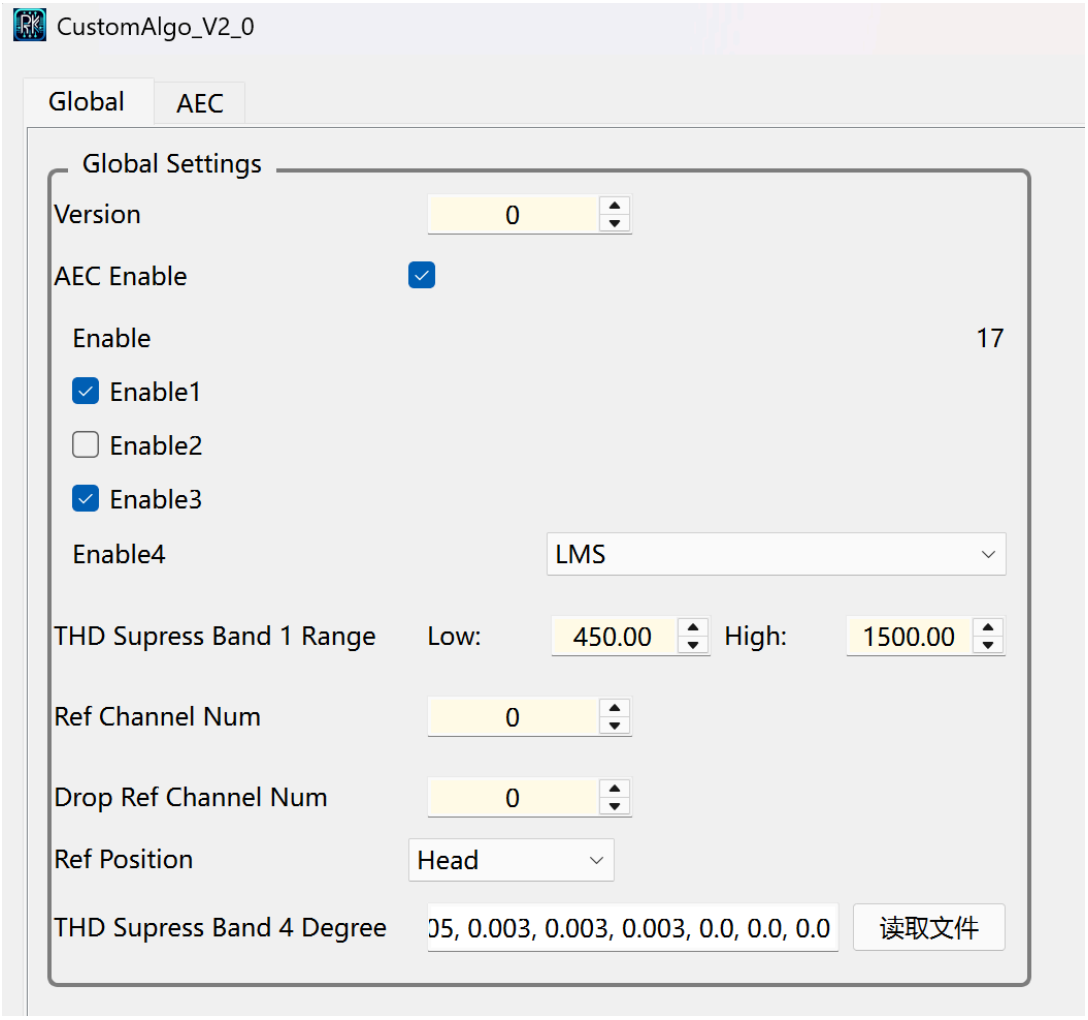


图 13.4-2 CustomAlgo_V2 参数设置

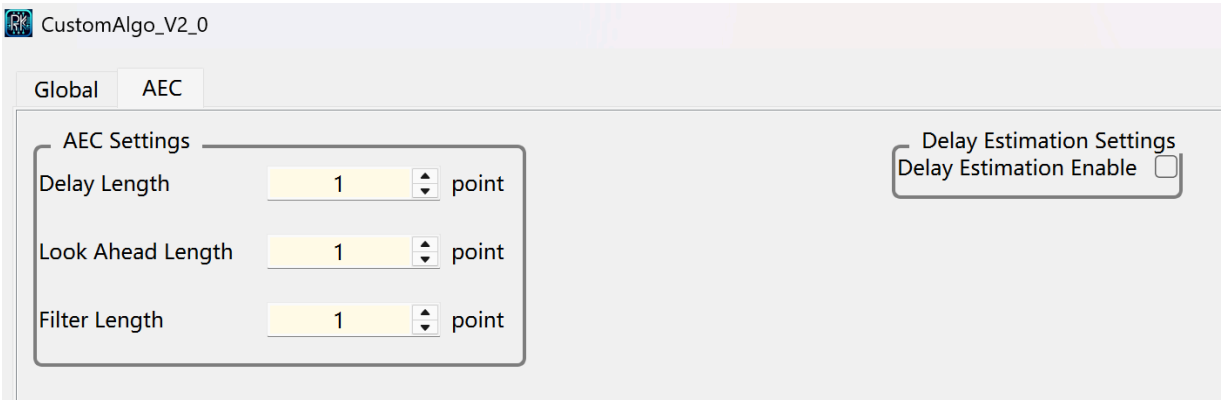


图 13.4-3 CustomAlgo_V2 参数设置

14 Counters

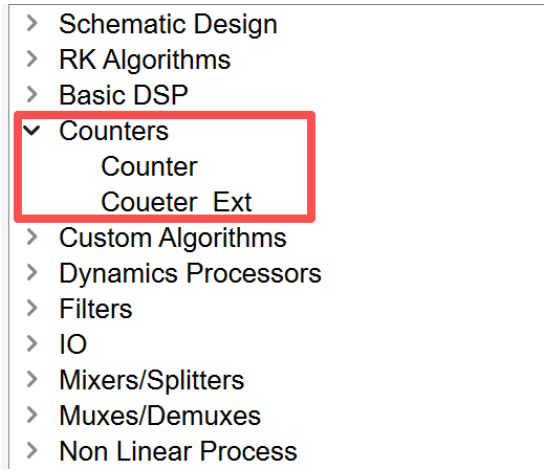


图 14-1 Counters位于工具中的位置

14.1 Counter

功能说明：基础计数器。该模块用于计算时间长度。

- 当 Enable被勾选（置1）时，模块开始从 0 计时，并且输出信号0。
- 当计时达到 MaxCount设定的时间（毫秒）后，计数停止（通常输出会保持最大值或触发某个逻辑状态，视具体 DSP 逻辑定义而定），这时候输出信号1。
- 当 Enable取消勾选（置0）时，计时器复位。

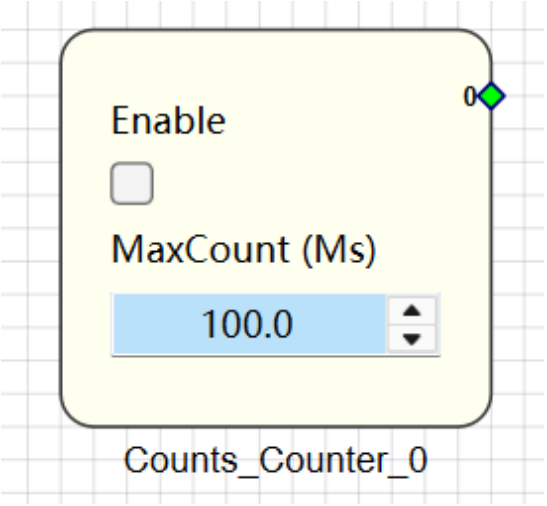


图 14.1-1 Counter 控件

14.2 Counter_Ext

功能说明：外部控制计时器。功能与标准 Counter 类似，区别在于取消了界面上的 Enable 按钮，转而由 Input 0端口接入的信号来控制计时状态，当输入为0时关闭计数功能，当输入为1时开启计算功能。

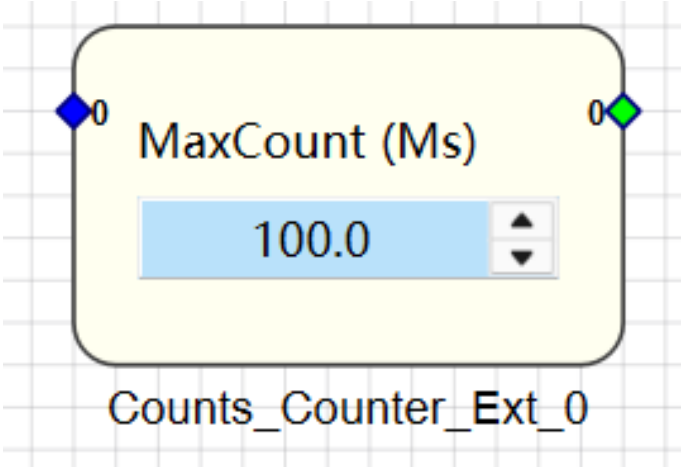


图 14.2-1 Counter_Ext 控件

15 Non Linear Process

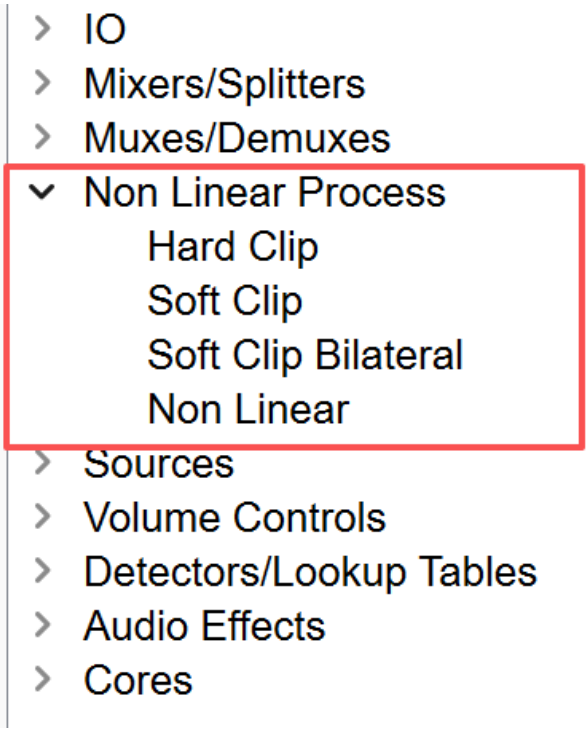


图 15-1 Non Linear Process位于工具的位置

15.1 Hard Clip

功能说明：Hard Clip 通过设定刚性阈值对音频信号进行强制截断。当输入信号的绝对值超过预设边界时，超出部分被直接舍弃，输出维持在边界水平。

- 技术逻辑

$$y = \begin{cases} L & \text{if } x > L \\ -L & \text{if } x < -L \\ x & \text{otherwise} \end{cases}$$

(其中 x 为输入，L 为限幅阈值)

- 特性描述
  - 波形特征：峰值处呈现完全平直的削峰状态，转折点呈现 90 度尖锐夹角。
  - 谐波分量：引入大量高次奇次谐波，失真感强烈，听感冷硬、刺耳。
- 应用场景

- 数字信号链末端的“最后防线”，防止样值溢出导致的严重杂音。
- 工业感、极端失真效果器的基础模块。

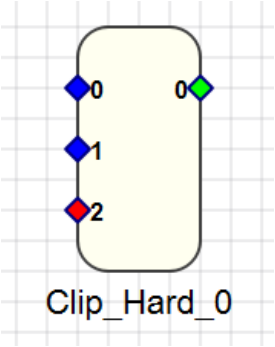


图 15.1-1 Hard Clip 控件

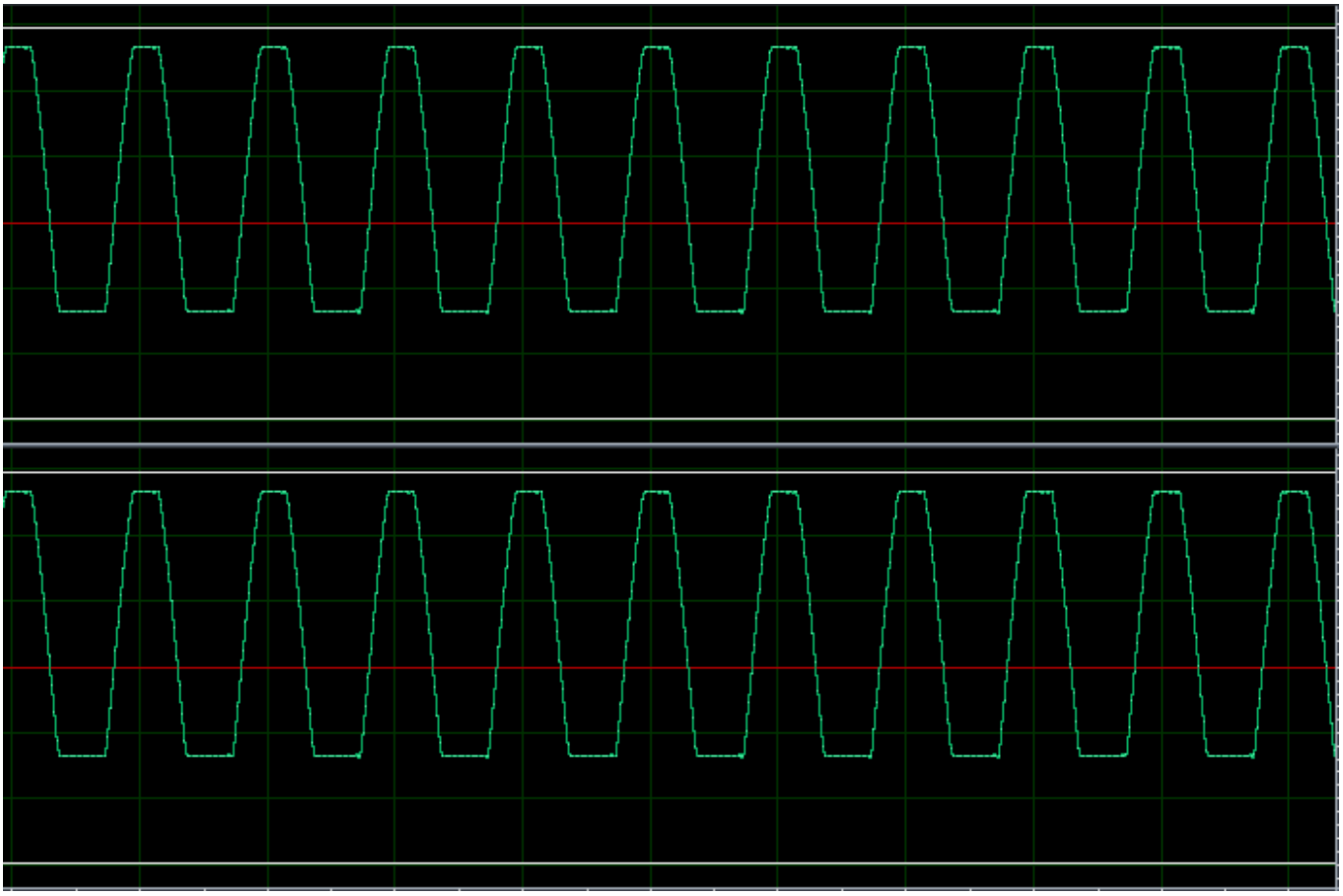


图 15.1-2 Hard Clip 波形图

15.2 Soft Clip

功能说明：Soft Clip 模拟了模拟电路的物理饱和特性。它在信号接近限幅值之前，通过特定的算法曲线进行渐进式增益压缩。

- 关键参数
  - Limit (dB)：最终输出电平的绝对边界。
  - Turn On (dB)：非线性处理起点。信号在该电平以上开始发生曲线畸变。
- 特性描述
  - 波形特征：峰值转折处呈现平滑的弧线过渡。
  - 谐波分量：主要产生低次谐波，具有温暖、柔和的听感。
- 应用场景
  - 提升信号的感知响度而不产生明显的削峰感。
  - 为数字音频“模拟化”。

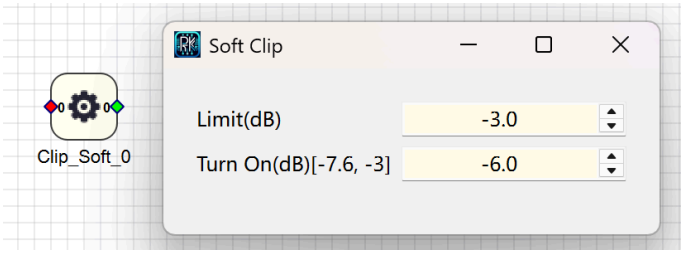


图 15.2-1 Soft Clip 控件

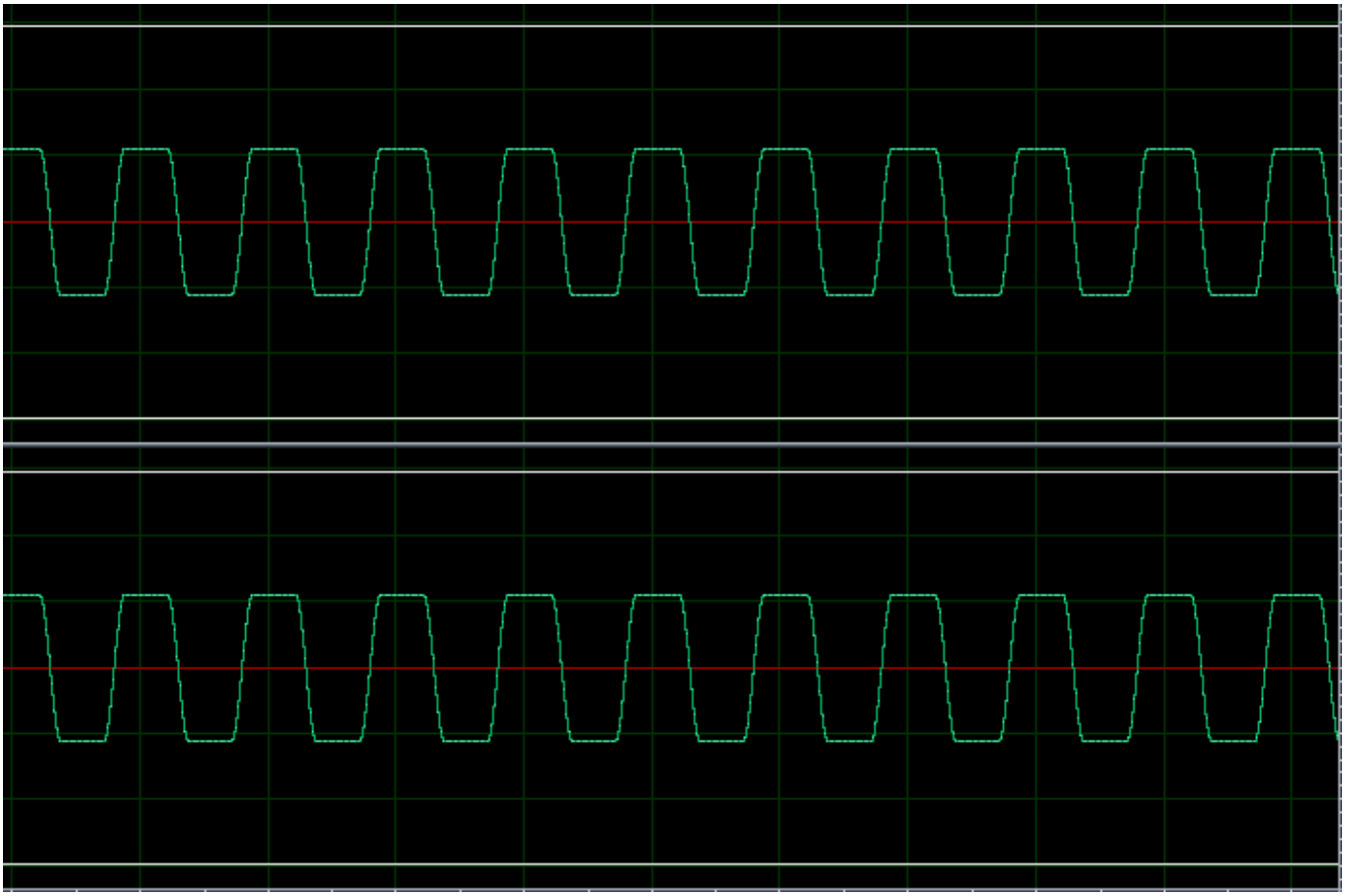


图 15.2-2 Soft Clip 波形图

15.3 Soft Clip Bilateral

功能说明：Bilateral 版本是软剪切功能的进阶形式，支持对音频信号的正半周和负半周进行独立的算法配置。

- 关键参数
  - Limit Up / Turn On Up：配置波形上边界的限幅与过渡参数。
  - Limit Down / Turn On Down：配置波形下边界的限幅与过渡参数。
- 特性描述
  - 核心优势：通过设置不对称的参数，可产生非对称失真。这是模拟特定硬件音色的关键。
  - 谐波重组：通过控制正负半周的不对称度，可以精细控制偶次谐波与奇次谐波的分布比例。
- 应用场景
  - 高端音箱模拟、高保真动态处理器、特征音色算法开发。

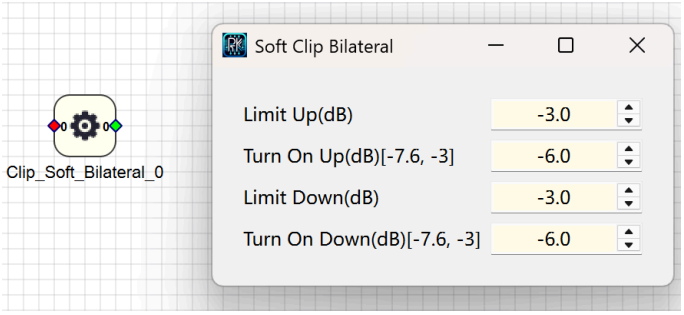


图 15.3-1 Soft Clip Bilateral 控件

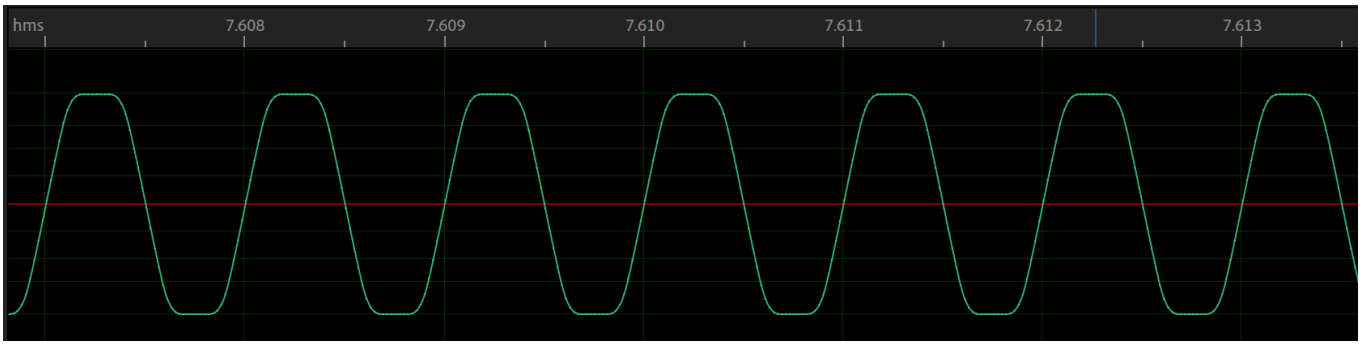


图 15.3-2 Soft Clip Bilateral 波形图

15.4 Non Linear

功能说明：Non Linear 模块通过预设的数学传递函数对输入信号进行全量程重塑。不同于仅作用于峰值的剪切算法，该模块会改变信号整体的传递关系。

- 常见模式
  - EXP (Exponential)：指数映射，用于增加信号的瞬态对比度或动态冲击。
- 应用场景
  - 波形整形（Wave-shaping）、音频动态包络重构、创意音效设计。

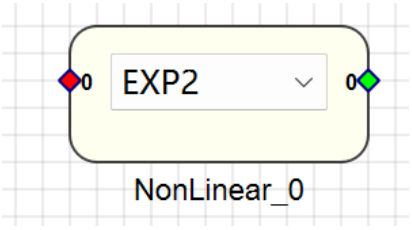


图 15.4-1 Non Linear 控件

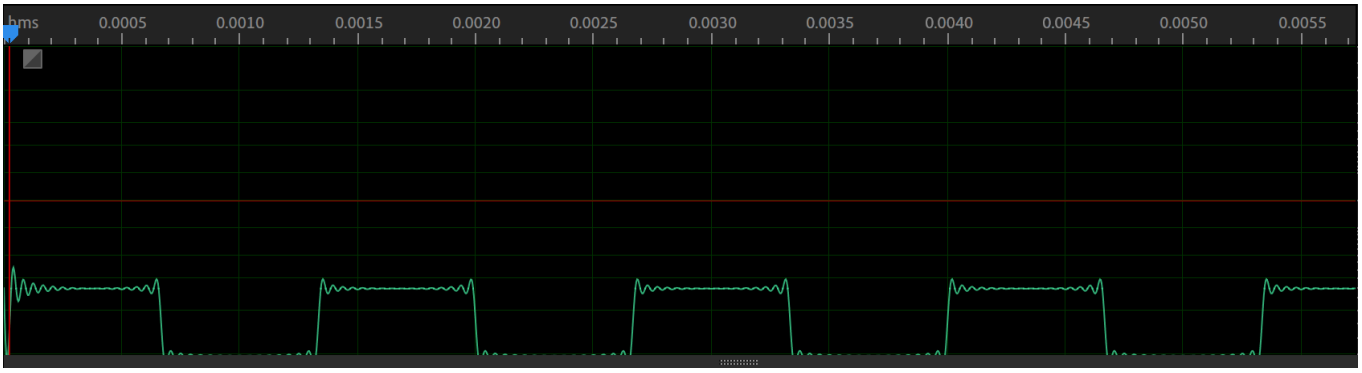


图 15.4-2 正弦波作为输入的输出波形图

附录1 Schematic Design

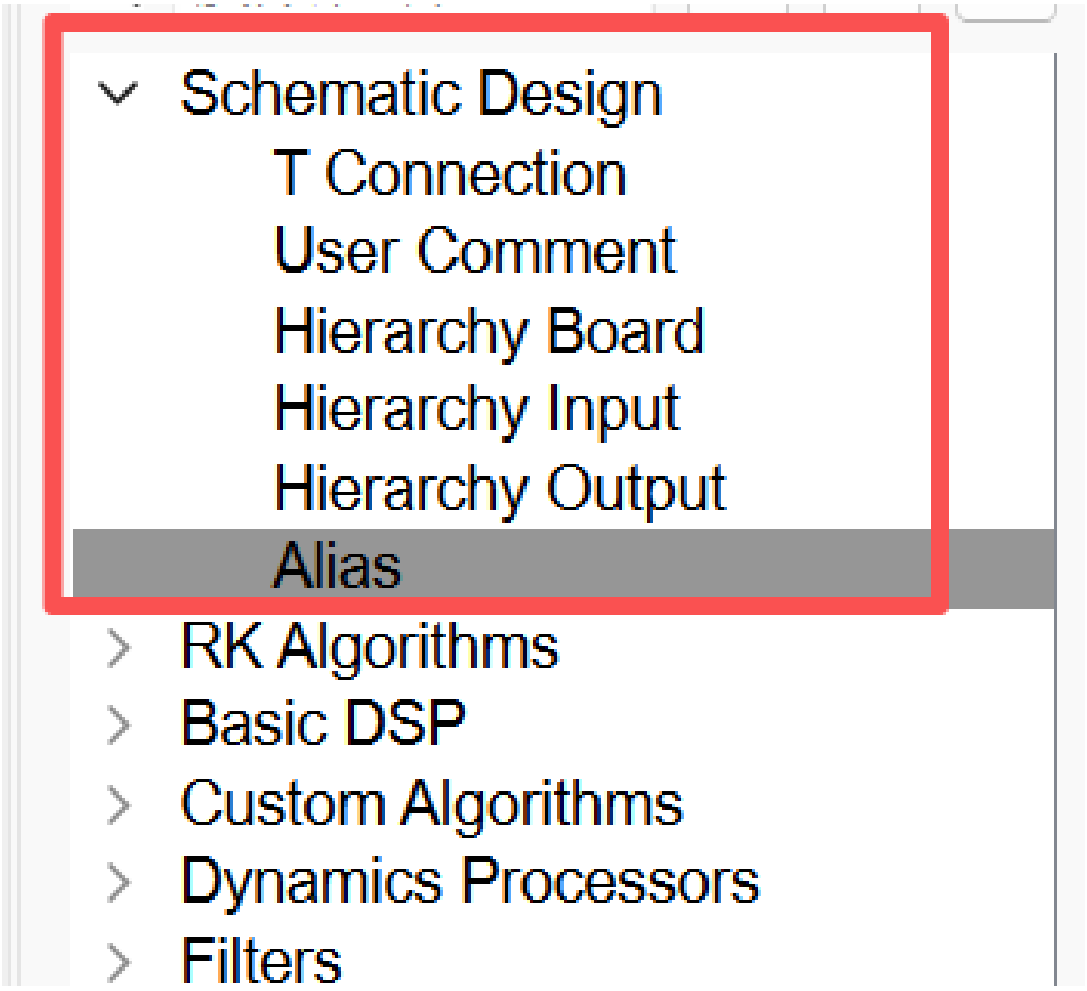


图 附录1-1 通用功能组件位于工具的位置

附录1.1 T Connection

功能说明：单通道输入，输出可以连接到多个组件，相当于将一个输入复制到多个输出，输出通道没有限制。

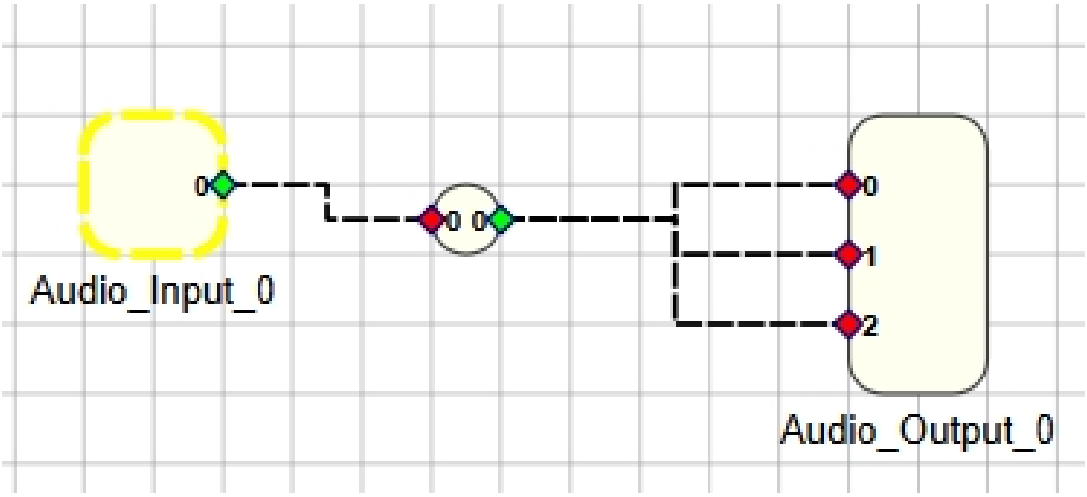


图 附录1.1-1 T Connection 组件连接

附录1.2 User Comment

功能说明：用户标签功能，可以对组件进行注释，便于理解。

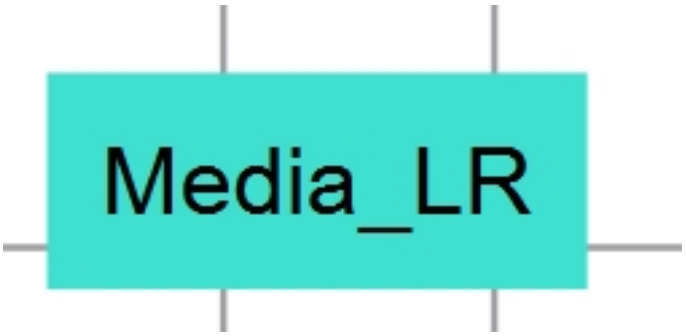


图 附录1.2-1 User Comment 组件

附录1.3 Hierarchy Board

功能说明：子图功能，多个组件可以放在子图中部署，便于管理和移植。子图支持加密/解密、导入/导出等功能。双击会跳转到具体的布局里面。

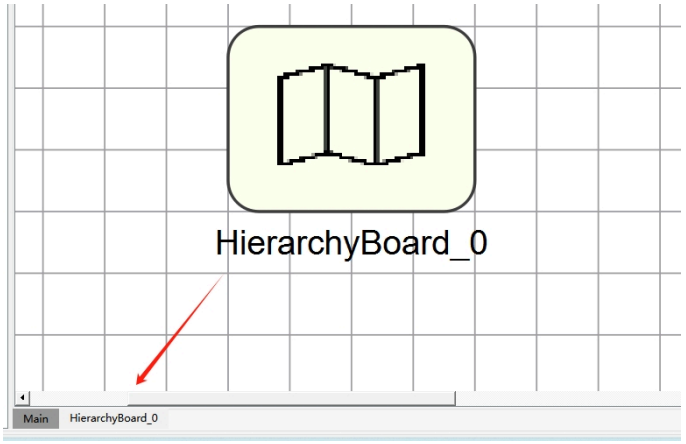


图 附录1.3-1 Hierarchy Board 子图功能

附录1.4 Hierarchy Input

功能说明：子图输入，子图的输入对应到主图上的输入，当增加输入通道数时，主图的输入也会增加。双击会跳转到对应的子图模块。

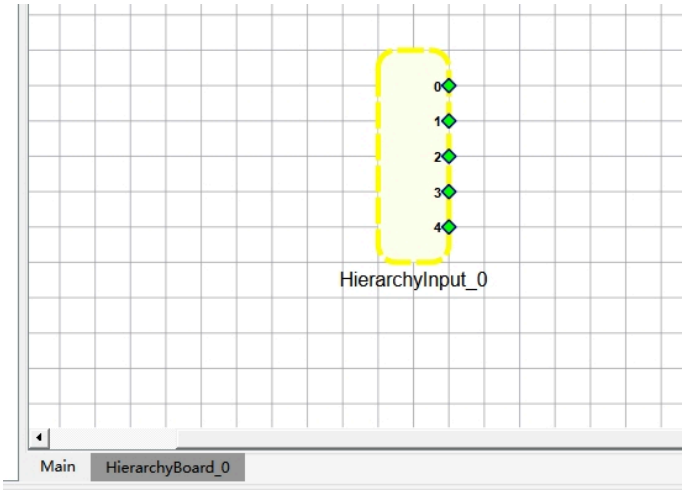


图 附录1.4-1 Hierarchy Input

附录1.5 Hierarchy Output

功能说明：子图输出，子图的输出对应到主图上的输出，当增加输出通道数时，主图的输出也会增加。双击会跳转到对应的子图模块。

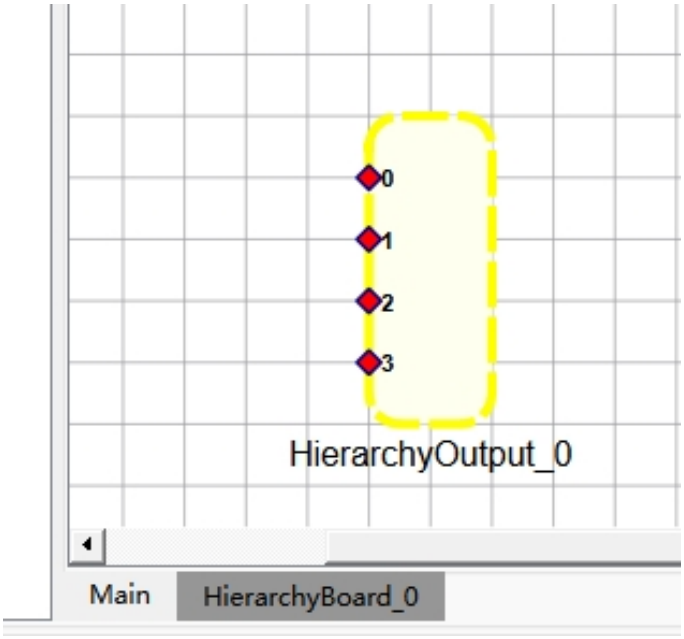


图 附录1.5-1 Hierarchy Output

附录1.6 Alias

功能说明：飞线连接，连接时隐藏了实线，更方便布置插件和查看连接情况。连接方式为一对多，输入只有一个，输出可以通过右键菜单任意增加。

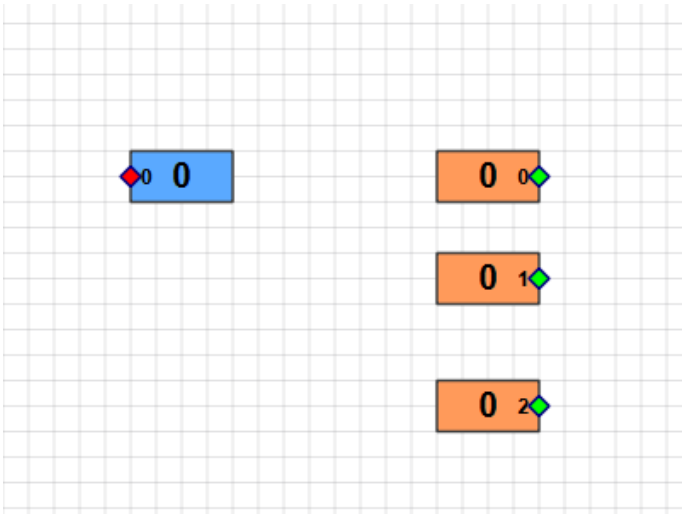


图 附录1.6-1 Alias

附录2 转换公式

附录2.1 参数转换说明

在软件的模块设置的参数（称为工具参数），如低频/高频截止频率，增益等，会经过程序处理转换为对应的参数传入算法（称为下发参数），具体参数内容打开RK_Tool 图纸工程文件目录下的xxx_param.json文件查看；
以下以RK Algorithms -> Loudness -> Loudness(L&H)EXT 模块为例：

```
"state_num": 7,
"state_names": [
    "BassFreq",
    "BassRatio",
    "TrebleFreq",
    "TrebleRatio",
    "FilterType",
    "PostGain",
    "SlewRate"
],
"state_values": [
    1000.0,
    50.0,
    7000.0,
    50.0,
    0,
    0,
    12.0
],
```


图 附录2.1-1 工具参数， 工具界面设置的参数

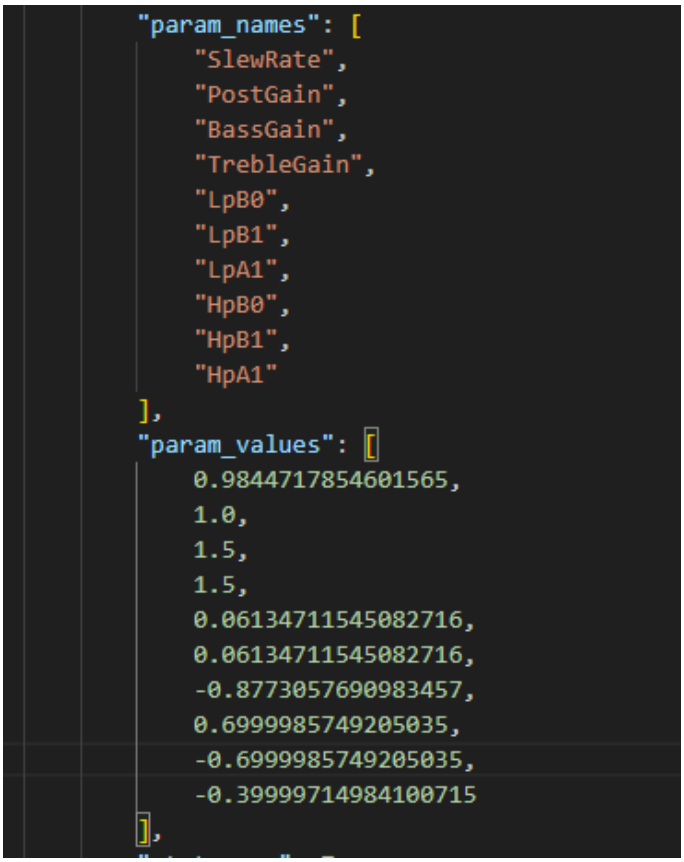


图 附录2.1-2 下发参数，参数转换成下发参数传入算法

附录2.2 IIR1st计算公式及说明

- **转换逻辑**：
  - 当Enable=0时，b0=1, b1=0, a1=0
  - 当Polarity=1时，b0，b1在下面公式计算的基础上取反
    以下列出五种滤波器类型的下发参数的计算方式为：

输入说明：

```
#       boost：对应工具设置的滤波器增益Boost(dB)值
    #       fc：    对应工具设置的截止频率Fc(Hz)值
    #       gain：  对应工具设置的整体增益Gain(dB)值
    #       Fs：  采样频率，在此处默认为48k，在计算归一化频率时候用到
```

```
# LP1st 一阶低通滤波器
    def calculate_lp1st_coeffs(fc: float, gain=0, Fs=48000):
        w0 = 2*np.pi*fc/Fs
        a1 = -pow(np.e, -w0)
        b0 = 0.5 * (1 + a1)
        b1 = b0

        gain_linear = 10**(gain/20)
        return [b0*gain_linear, b1*gain_linear], [1.0, a1]

    # HP1st     一阶高通滤波器
    def calculate_hp1st_coeffs(fc: float, gain=0, Fs=48000):
        w0 = 2*np.pi*fc/Fs
        a1 = -pow(np.e, -w0)
        b0 = 0.5 * (1 - a1)
        b1 = -b0

        gain_linear = 10**(gain/20)
        return [b0*gain_linear, b1*gain_linear], [1.0, a1]

    #AP1st      一阶全通滤波器
    def calculate_ap1st_coeffs(fc: float, gain=0, Fs=48000):
        w0 = 2*np.pi*fc/Fs
        a1 = -pow(np.e, -w0)
        b0 = a1
        b1 = 1

        gain_linear = 10**(gain/20)
        return [b0*gain_linear, b1*gain_linear], [1.0, a1]

    #LS1st      一阶低架滤波器
    def calculate_ls1st_coeffs(fc: float, boost: float, gain=0, Fs=48000):
        w0 = 2*np.pi*fc/Fs
        A = pow(10.0, boost / 20.0)
        a0 = np.tan(w0/2.0) + np.sqrt(A)
        a1 = (np.tan(w0/2.0) - np.sqrt(A))
        b0 = (A*np.tan(w0/2.0) + np.sqrt(A))
        b1 = (A*np.tan(w0/2.0) - np.sqrt(A))

        a0_invert = 1 / a0
        gain_linear = 10**(gain/20)
        return [b0*a0_invert*gain_linear, b1*a0_invert*gain_linear], [1.0, a1*a0_invert]

    #HS1st      一阶高架滤波器
    def calculate_hs1st_coeffs(fc: float, boost: float, gain=0, Fs=48000):
        w0 = 2*np.pi*fc/Fs
        A = pow(10.0, boost / 20.0)
        a0 = np.sqrt(A)*np.tan(w0/2.0)+1
        a1 = (np.sqrt(A)*np.tan(w0/2.0)-1) / a0
        b0 = (np.sqrt(A)*np.tan(w0/2.0)+A) / a0
        b1 = (np.sqrt(A)*np.tan(w0/2.0)-A) / a0

        a0_invert = 1 / a0
        gain_linear = 10**(gain/20)
        return [b0*a0_invert*gain_linear, b1*a0_invert*gain_linear], [1.0, a1*a0_invert]
```

附录2.3 IIR2nd计算公式及说明

- **转换逻辑**:
  - 当Enable=0时，b0=1, b1=0,b2=0, a1=0,a2=0
  - 当Polarity=1时，b0，b1,b2在下面公式计算的基础上取反
    以下列出16种滤波器类型的下发参数的计算方式为：

输入说明：

```
#       fc：     对应工具设置的截止频率Fc(Hz)值
    #       boost：对应工具设置的滤波器增益Boost(dB)值
    #       Q：     对应工具设置的滤波器品质因数Q值
    #       gain:   对应工具设置的整体增益Gain(dB)值
    #       Fs：  采样频率，在此处默认为48k，在计算归一化频率时候用到
```

```
# Parametric 参数滤波器
    def calculate_param_coeffs(fc: float, boost: float, Q: float, gain=0, Fs=48000):
        w0 = 2 * np.pi * fc / Fs
        A = 10 ** (boost / 40.0)
        alpha = np.sin(w0) / (2.0 * Q)

        a0 = 1 + alpha / A
        a1 = -2 * np.cos(w0)
        a2 = 1 - alpha / A
        b0 = 1 + alpha * A
        b1 = -2 * np.cos(w0)
        b2 = 1 - alpha * A

        a0_invert = 1 / a0
        gain_linear = 10 ** (gain/20)

        return [b0*a0_invert*gain_linear, b1*a0_invert*gain_linear, b2*a0_invert*gain_linear], [1.0, a1*a0_invert, a2*a0_invert]

    # LowShelf  低架滤波器
    def calculate_lowshelf_coeffs(fc: float, boost: float, Q: float, gain=0, Fs=48000):
        w0 = 2 * np.pi * fc / Fs
        A = 10 ** (boost / 40.0)
        tmp = (A + 1/A)*(1/(Q + 1e-8) - 1) + 2
        tmp = max(tmp, 0)
        alpha = 0.5*np.sin(w0)*np.sqrt(tmp)

        a0 = (A + 1) + (A - 1)*np.cos(w0) + 2*np.sqrt(A)*alpha
        a1 = -2*((A - 1) + (A + 1)*np.cos(w0))
        a2 = (A + 1) + (A - 1)*np.cos(w0) - 2*np.sqrt(A)*alpha
        b0 = A*((A + 1) - (A - 1)*np.cos(w0) + 2*np.sqrt(A)*alpha)
        b1 = 2*A*((A - 1) - (A + 1)*np.cos(w0))
        b2 = A*((A + 1) - (A - 1)*np.cos(w0) - 2*np.sqrt(A)*alpha)

        a0_invert = 1 / a0
        gain_linear = 10 ** (gain/20)

        return [b0*a0_invert*gain_linear, b1*a0_invert*gain_linear, b2*a0_invert*gain_linear], [1.0, a1*a0_invert, a2*a0_invert]

    # HighShelf 高架滤波器
    def calculate_highshelf_coeffs(fc: float, boost: float, Q: float, gain=0, Fs=48000):
        w0 = 2 * np.pi * fc / Fs
        A = 10 ** (boost / 40.0)
        tmp = (A + 1/A)*(1/(Q + 1e-8) - 1) + 2
        tmp = max(tmp, 0)
        alpha = 0.5*np.sin(w0)*np.sqrt(tmp)

        a0 = (A + 1) - (A - 1)*np.cos(w0) + 2*np.sqrt(A)*alpha
        a1 = 2*((A - 1) - (A + 1)*np.cos(w0))
        a2 = (A + 1) - (A - 1)*np.cos(w0) - 2*np.sqrt(A)*alpha
        b0 = A*((A + 1) + (A - 1)*np.cos(w0) + 2*np.sqrt(A)*alpha)
        b1 = -2*A*((A - 1) + (A + 1)*np.cos(w0))
        b2 = A*((A + 1) + (A - 1)*np.cos(w0) - 2*np.sqrt(A)*alpha)

        a0_invert = 1 / a0
        gain_linear = 10 ** (gain/20)

        return [b0*a0_invert*gain_linear, b1*a0_invert*gain_linear, b2*a0_invert*gain_linear], [1.0, a1*a0_invert, a2*a0_invert]

    # GeneralLP 低通滤波器
    def calculate_genlp_coeffs(fc: float, Q: float, gain=0, Fs=48000):
        w0 = 2 * np.pi * fc / Fs
        alpha = np.sin(w0) / (2 * Q)

        a0 = 1 + alpha
        a1 = -2 * np.cos(w0)
```

```
    a2 = 1 - alpha
    b0 = 0.5 * (1 - np.cos(w0))
    b1 = 1 - np.cos(w0)
    b2 = 0.5 * (1 - np.cos(w0))

    a0_invert = 1 / a0
    gain_linear = 10 ** (gain/20)

    return [b0*a0_invert*gain_linear, b1*a0_invert*gain_linear, b2*a0_invert*gain_linear], [1.0, a1*a0_invert, a2*a0_invert]

# GeneralHP 高通滤波器
def calculate_genhp_coeffs(fc: float, Q: float, gain=0, Fs=48000):
    w0 = 2 * np.pi * fc / Fs
    alpha = np.sin(w0) / (2 * Q)

    a0 = 1 + alpha
    a1 = -2 * np.cos(w0)
    a2 = 1 - alpha
    b0 = 0.5 * (1 + np.cos(w0))
    b1 = -(1 + np.cos(w0))
    b2 = 0.5 * (1 + np.cos(w0))

    a0_invert = 1 / a0
    gain_linear = 10 ** (gain/20)

    return [b0*a0_invert*gain_linear, b1*a0_invert*gain_linear, b2*a0_invert*gain_linear], [1.0, a1*a0_invert, a2*a0_invert]

# GeneralBP 带通滤波器
def calculate_genbp_coeffs(fc: float, Q: float, gain=0, Fs=48000):
    w0 = 2 * np.pi * fc / Fs
    BW = Q_to_BW(Q)
    alpha = np.sin(w0) * np.sinh(0.5*np.log(2)*BW*w0/np.sin(w0))

    a0 = 1 + alpha
    a1 = -2 * np.cos(w0)
    a2 = 1 - alpha
    b0 = alpha
    b1 = 0
    b2 = -alpha

    a0_invert = 1 / a0
    gain_linear = 10 ** (gain/20)

    return [b0*a0_invert*gain_linear, b1*a0_invert*gain_linear, b2*a0_invert*gain_linear], [1.0, a1*a0_invert, a2*a0_invert]

# GeneralBS 带阻滤波器
def calculate_genbs_coeffs(fc: float, Q: float, gain=0, Fs=48000):
    w0 = 2 * np.pi * fc / Fs
    BW = Q_to_BW(Q)
    alpha = np.sin(w0) * np.sinh(0.5*np.log(2)*BW*w0/(np.sin(w0)+1e-8))

    a0 = 1 + alpha
    a1 = -2 * np.cos(w0)
    a2 = 1 - alpha
    b0 = 1
    b1 = -2 * np.cos(w0)
    b2 = 1

    a0_invert = 1 / a0
    gain_linear = 10 ** (gain/20)

    return [b0*a0_invert*gain_linear, b1*a0_invert*gain_linear, b2*a0_invert*gain_linear], [1.0, a1*a0_invert, a2*a0_invert]

# AFP       全通滤波器
def calculate_apf_coeffs(fc: float, Q: float, gain=0, Fs=48000):
```

```
    w0 = 2 * np.pi * fc / Fs
    tmp = fc / Q
    K = np.pi*tmp/Fs
    K = K if K <= 0.95*np.pi/2 else 0.95*np.pi/2
    beta = 1 - 2/(np.tan(K) + 1)
    gamma = -np.cos(w0)

    a1 = gamma*(1 - beta)
    a2 = -beta
    b0 = -beta
    b1 = gamma*(1 - beta)
    b2 = 1

    gain_linear = 10 ** (gain/20)

    return [b0*gain_linear, b1*gain_linear, b2*gain_linear], [1.0, a1, a2]

# Peaking   峰值滤波器
def calculate_peak_coeffs(fc: float, Q: float, gain=0, Fs=48000):
    w0 = 2 * np.pi * fc / Fs
    beta = 0.5 * (1 - np.tan(w0/(2*Q))) / (1 + np.tan(w0/(2*Q)))
    gamma = (0.5 + beta) * np.cos(w0)

    a1 = -2 * gamma
    a2 = 2 * beta
    b0 = 0.5 - beta
    b1 = 0
    b2 = -0.5 + beta

    gain_linear = 10 ** (gain/20)

    return [b0*gain_linear, b1*gain_linear, b2*gain_linear], [1.0, a1, a2]

# Notch     陷波滤波器
def calculate_notch_coeffs(fc: float, Q: float, gain=0, Fs=48000):
    w0 = 2 * np.pi * fc / Fs
    alpha = np.sin(w0) / (2 * Q)

    a0 = 1 + alpha
    a1 = -2 * np.cos(w0)
    a2 = 1 - alpha
    b0 = 1
    b1 = -2 * np.cos(w0)
    b2 = 1

    a0_invert = 1 / a0
    gain_linear = 10 ** (gain/20)

    return [b0*a0_invert*gain_linear, b1*a0_invert*gain_linear, b2*a0_invert*gain_linear], [1.0, a1*a0_invert, a2*a0_invert]

# ChebyshevLP       切比雪夫低通滤波器
def calculate_cheby1lp_coeffs(fc: float, Q: float, gain=0, Fs=48000):
    gain_linear = 10 ** (gain / 20)
    b, a = cheby1(2, Q, fc, "lowpass", analog=False, fs=Fs)
    b = b * gain_linear
    b = b.tolist()
    a = a.tolist()
    return b, a

# ChebyshevHP       切比雪夫高通滤波器
def calculate_cheby1hp_coeffs(fc: float, Q: float, gain=0, Fs=48000):
    gain_linear = 10 ** (gain / 20)
    b, a = cheby1(2, Q, fc, "highpass", analog=False, fs=Fs)
    b = b * gain_linear
    b = b.tolist()
```

```
    a = a.tolist()
    return b, a

# BesselLP  贝塞尔低通滤波器
def calculate_besselp_coeffs(fc: float, gain=0, Fs=48000):
    gain_linear = 10 ** (gain / 20)
    b, a = bessel(2, fc, "lowpass", analog=False, fs=Fs)
    b = b * gain_linear
    b = b.tolist()
    a = a.tolist()
    return b, a

# BesselHP  贝塞尔高通滤波器
def calculate_besselhp_coeffs(fc: float, gain=0, Fs=48000):
    gain_linear = 10 ** (gain / 20)
    b, a = bessel(2, fc, "highpass", analog=False, fs=Fs)
    b = b * gain_linear
    b = b.tolist()
    a = a.tolist()
    return b, a

# ButterLP  巴特沃斯低通滤波器
def calculate_butterlp_coeffs(fc: float, gain=0, Fs=48000):
    gain_linear = 10 ** (gain / 20)
    b, a = butter(2, fc, "lowpass", analog=False, fs=Fs)
    b = b * gain_linear
    b = b.tolist()
    a = a.tolist()
    return b, a

# ButterHP  巴特沃斯高通滤波器
def calculate_butterhp_coeffs(fc: float, gain=0, Fs=48000):
    gain_linear = 10 ** (gain / 20)
    b, a = butter(2, fc, "highpass", analog=False, fs=Fs)
    b = b * gain_linear
    b = b.tolist()
    a = a.tolist()
    return b, a
```

附录2.4 IIR_Nnd计算公式及说明

- **转换逻辑**：
  - 当Enable=0时，a0=1，b0=1，其他参数都为0
  - 当Polarity=1时，滤波器分子参数b在下面公式计算的基础上取反

输入说明：

```
#   enable:       对应工具参数的enable 使能
#   polarity:     对应工具参数的polarity
#   filter_type:  对应工具参数的filter_type滤波器类型
#   order:        对应工具参数的order 阶数
#   fc:           对应工具参数的fc截止频率
#   rpldb:        对应工具参数的Rpl (dB) 通道内最大增益
#   Fs :          采样频率，在此处默认为48k，在计算归一化频率时候用到
```

```python
def calculate_filter_high_coeffs(enable: float,
                                 polarity: float,
                                 filter_type: float,
                                 order: float,
                                 fc: float,
                                 rpldb: float,
                                 Fs=48000):
    if not enable:
        num_sos = (order+1)//2
        return num_sos, [1, 0, 0]*num_sos, [1.0, 0, 0]*num_sos

    if filter_type == 0:  # "Butter LP 巴特沃斯低通滤波器 "
        sos = butter(order, fc, btype='low', analog=False, output='sos', fs=Fs)
        b, a = sos_to_ba(sos)
    elif filter_type == 1:  # "Butter HP 巴特沃斯高通滤波器 "
        sos = butter(order, fc, btype='high', analog=False, output='sos', fs=Fs)
        b, a = sos_to_ba(sos)
    elif filter_type == 2:  # "Bessel LP  贝塞尔低通滤波器 "
        sos = bessel(order, fc, btype='low', analog=False, output='sos', fs=Fs)
        b, a = sos_to_ba(sos)
    elif filter_type == 3:  # "Bessel HP 贝塞尔高通滤波器 "
        sos = bessel(order, fc, btype='high', analog=False, output='sos', fs=Fs)
        b, a = sos_to_ba(sos)
    elif filter_type == 4:  # "Cheby1 LP 第一类切比雪夫低通滤波器 "
        sos = cheby1(order, rpldb, fc, btype='low', analog=False, output='sos', fs=Fs)
        b, a = sos_to_ba(sos)
    elif filter_type == 5:  # "Cheby1 HP 第一类切比雪夫高通滤波器 "
        sos = cheby1(order, rpldb, fc, btype='high', analog=False, output='sos', fs=Fs)
        b, a = sos_to_ba(sos)
    elif filter_type == 6:  # "Cheby2 LP 第二类切比雪夫低通滤波器 "
        sos = cheby2(order, rpldb, fc, btype='low', analog=False, output='sos', fs=Fs)
        b, a = sos_to_ba(sos)
    elif filter_type == 7:  # "Cheby2 HP 第二类切比雪夫高通滤波器 "
        sos = cheby2(order, rpldb, fc, btype='high', analog=False, output='sos', fs=Fs)
        b, a = sos_to_ba(sos)
    else:
        raise ValueError("Unsupported filter type")

    if polarity == 1:
        b[0:3] = -b[0:3]
    b = b.tolist()
    a = a.tolist()
    return (order+1)//2, b, a

def sos_to_ba(sos):
    """
    将sos格式转换为ba格式
    :param sos:
    :return:
    """
    b_list = []
    a_list = []
    num_sos = len(sos)

    gain_all = sos[0][0]
    gain = gain_all**(1/num_sos)
    sos[0][:3] = sos[0][:3]/gain_all
    for section in sos:
        b_section = section[:3]*gain  # 提取当前二阶段的b系数
        a_section = np.concatenate(([1], section[4:]))  # 提取当前二阶段的a系数，包含a0=1

        # 将当前二阶段的b和a系数添加到列表中
        b_list.extend(b_section)
        a_list.extend(a_section)
```

```
    # 将列表转换为numpy数组
    b = np.array(b_list)
    a = np.array(a_list)

    return b, a
```

附录2.5 Graphical EQ 滤波器参数计算公式及说明

graphical EQ 有多个滤波器，每一个滤波器都进行一次变换；

- **转换逻辑**：
  - 当bypass=1时，b0=1, b1=0, a1=0
  - 以下列出下发参数的计算方式为：
    输入说明：

```
#       fc：    对应工具设置的截止频率Fc(Hz)值
#       Q ：    对应工具设置的品质因数Q值
#       boost：对应工具设置的滤波器增益Boost(dB)值
#       gain：  对应工具设置的整体增益Gain(dB)值
#       filter_tpye：对应工具选择的滤波器类型的数值
#       bypass ：   对应工具的bypass 失能开关
#       Fs：  采样频率，在此处默认为48k，在计算归一化频率时候用到
```

```
def calculate_filter_coefficient(fc: float, Q: float, boost: float, gain: float = 0, filter_type: float = 0, bypass: float = 0, Fs=48000):
    # 系数范围检查
    # fc
    fc = fc if fc > 20 else 20
    fc = fc if fc < Fs / 2 else Fs / 2
    fc = fc

    if bypass:
        return [1, 0, 0], [1, 0, 0]

    gain_linear = 10**(gain/20)

    if 0 == filter_type:  # Parametric
        b, a = calculate_param_coeffs(fc, boost, Q, gain, Fs)
    elif 1 == filter_type:  # LowShelf
        b, a = calculate_lowshelf_coeffs(fc, boost, Q, gain, Fs)
    elif 2 == filter_type:  # HighShelf
        b, a = calculate_highshelf_coeffs(fc, boost, Q, gain, Fs)
    elif 3 == filter_type:  # GeneralLP
        b, a = calculate_genlp_coeffs(fc, Q, gain, Fs)
    elif 4 == filter_type:  # GeneralHP
        b, a = calculate_genhp_coeffs(fc, Q, gain, Fs)
    elif 5 == filter_type:  # ButterLP
        b, a = butter(2, fc, btype='low', analog=False, output='ba', fs=Fs)
        b = b*gain_linear
        b = b.tolist()
        a = a.tolist()
    elif 6 == filter_type:  # ButterHP
        b, a = butter(2, fc, btype='high', analog=False, output='ba', fs=Fs)
        b = b*gain_linear
        b = b.tolist()
        a = a.tolist()
    elif 7 == filter_type:  # BesselLP
        b, a = bessel(2, fc, btype='low', analog=False, output='ba', fs=Fs)
        b = b*gain_linear
        b = b.tolist()
        a = a.tolist()
    elif 8 == filter_type:  # BesselHP
        b, a = bessel(2, fc, btype='high', analog=False, output='ba', fs=Fs)
        b = b*gain_linear
        b = b.tolist()
        a = a.tolist()
    elif 9 == filter_type:  # Allpass
        b, a = calculate_apf_coeffs(fc, Q, gain, Fs)
    return b, a
```

附录2.6 Crossover 控件计算公式及说明

- Crossover_2_Way分离出低频、高频分量；控件一共有两个滤波器，低频支路一个低通滤波器；高频支路一个高通滤波器
- Crossover_3_Way低频、中频、高频分量；控件一共有四个滤波器，低频支路一个低通滤波器；中频支路使用一个高通滤波器和一个低通滤波器串联；高频支路一个高通滤波器

低通滤波器和高通滤波器使用不同的处理函数进行转换

- **转换逻辑**：
  - 当bypass=1时，b0=1, b1=0, a1=0
    以下列出下发参数的计算方式为：
    输入说明：

```
#       fc：    对应工具设置的截止频率Fc(Hz)值
#       gain:   对应工具设置的整体增益Gain(dB)值
#       filter_tpye: 对应工具选择的滤波器类型的数值
#       Fs:   采样频率，在此处默认为48k，在计算归一化频率时候用到
```

```
def calculate_crossover_lp_coefficient(fc: float, gain: float, filter_type: float, Fs=48000):
    # 系数范围检查
    # fc
    fc = fc if fc > 20 else 20
    fc = fc if fc < Fs / 2 else Fs / 2
    fc = fc
    # boost
    gain_linear = pow(10, gain / 20.0)

    a0 = 1
    a1 = 0
    a2 = 0
    b0 = 0
    b1 = 0
    b2 = 0

    omega_c = np.pi * fc
    theta_c = omega_c / Fs

    b = np.array([1.0, 0.0, 0.0])
    a = np.array([1.0, 0.0, 0.0])

    if CROSSOVER_FILTER_TYPE.Bypass == filter_type:
        b = None
        a = None
    elif CROSSOVER_FILTER_TYPE.LR12 == filter_type:
        k = omega_c / np.tan(theta_c)
        den = k * k + omega_c * omega_c + 2 * k * omega_c
        b1_Num = -2 * k * k + 2 * omega_c * omega_c
        b2_Num = -2 * k * omega_c + k * k + omega_c * omega_c

        b0 = omega_c * omega_c / den
        b1 = 2 * b0
        b2 = b0
        a1 = b1_Num / den
        a2 = b2_Num / den

        b = np.array([b0, b1, b2])
        a = np.array([1.0, a1, a2])
        b[:3] = b[:3] * gain_linear

    elif CROSSOVER_FILTER_TYPE.Butter12 == filter_type or CROSSOVER_FILTER_TYPE.LR24 == filter_type:
        k = 1.0 / np.tan(theta_c)

        b0 = 1.0 / (1.0 + RK_GLOBAL.SQRT_TWO * k + k * k)
        b1 = 2 * b0
        b2 = b0
        a1 = 2 * b0 * (1 - k * k)
        a2 = b0 * (1 - RK_GLOBAL.SQRT_TWO * k + k * k)

        b = np.array([b0, b1, b2])
        a = np.array([a0, a1, a2])
        if CROSSOVER_FILTER_TYPE.LR24 == filter_type:
            b = np.append(b, [b0, b1, b2])
            a = np.append(a, [a0, a1, a2])
        b[:3] = b[:3] * gain_linear

    elif CROSSOVER_FILTER_TYPE.Bessel12 == filter_type:
        k = 1.0 / np.tan(theta_c)

        b0 = 1.0 / (1.0 + RK_GLOBAL.SQRT_THREE * k + k * k)
        b1 = 2 * b0
        b2 = b0
        a1 = 2 * b0 * (1 - k * k)
        a2 = b0 * (1 - RK_GLOBAL.SQRT_THREE * k + k * k)
```

```
    b = np.array([b0, b1, b2])
    a = np.array([a0, a1, a2])
    b[:3] = b[:3] * gain_linear

elif CROSSOVER_FILTER_TYPE.Butter18 == filter_type or CROSSOVER_FILTER_TYPE.LR36 == filter_type:
    sos = butter(3, fc, btype='low', analog=False, output='sos', fs=Fs)

    b, a = sos_to_ba(sos)

    if CROSSOVER_FILTER_TYPE.LR36 == filter_type:
        b = np.append(b, b)
        a = np.append(a, a)
    b[:3] = b[:3] * gain_linear

elif CROSSOVER_FILTER_TYPE.Bessel18 == filter_type:
    sos = bessel(3, fc, btype='low', analog=False, output='sos', fs=Fs)

    b, a = sos_to_ba(sos)

    b[:3] = b[:3] * gain_linear

elif CROSSOVER_FILTER_TYPE.Butter24 == filter_type or CROSSOVER_FILTER_TYPE.LR48 == filter_type:
    sos = butter(4, fc, btype='low', analog=False, output='sos', fs=Fs)

    b, a = sos_to_ba(sos)

    if CROSSOVER_FILTER_TYPE.LR48 == filter_type:
        b = np.append(b, b)
        a = np.append(a, a)
    b[:3] = b[:3] * gain_linear

elif CROSSOVER_FILTER_TYPE.Bessel24 == filter_type:
    sos = bessel(4, fc, btype='low', analog=False, output='sos', fs=Fs)

    b, a = sos_to_ba(sos)

    b[:3] = b[:3] * gain_linear

elif CROSSOVER_FILTER_TYPE.Butter36 == filter_type:
    sos = butter(6, fc, btype='low', analog=False, output='sos', fs=Fs)

    b, a = sos_to_ba(sos)

    b[:3] = b[:3] * gain_linear

elif CROSSOVER_FILTER_TYPE.Bessel36 == filter_type:
    sos = bessel(6, fc, btype='low', analog=False, output='sos', fs=Fs)

    b, a = sos_to_ba(sos)

    b[:3] = b[:3] * gain_linear

elif CROSSOVER_FILTER_TYPE.Butter48 == filter_type:
    sos = butter(8, fc, btype='low', analog=False, output='sos', fs=Fs)

    b, a = sos_to_ba(sos)

    b[:3] = b[:3] * gain_linear

elif CROSSOVER_FILTER_TYPE.Bessel48 == filter_type:
    sos = bessel(8, fc, btype='low', analog=False, output='sos', fs=Fs)

    b, a = sos_to_ba(sos)
```

```
        b[:3] = b[:3] * gain_linear

    return b, a


def calculate_crossover_hp_coefficient(fc: float, gain: float, filter_type: float, Fs=48000):
    # 系数范围检查
    # fc
    fc = fc if fc > 20 else 20
    fc = fc if fc < Fs / 2 else Fs / 2
    fc = fc
    # boost
    gain_linear = pow(10, gain / 20.0)

    a0 = 1
    a1 = 0
    a2 = 0
    b0 = 0
    b1 = 0
    b2 = 0

    omega_c = np.pi * fc
    theta_c = omega_c / Fs

    b = np.array([1.0, 0.0, 0.0])
    a = np.array([1.0, 0.0, 0.0])
    if CROSSOVER_FILTER_TYPE.Bypass == filter_type:
        b = None
        a = None
    elif CROSSOVER_FILTER_TYPE.LR12 == filter_type:
        k = omega_c / np.tan(theta_c)
        den = k * k + omega_c * omega_c + 2 * k * omega_c
        b1_Num = -2 * k * k + 2 * omega_c * omega_c
        b2_Num = -2 * k * omega_c + k * k + omega_c * omega_c

        b0 = k * k / den
        b1 = -2 * b0
        b2 = b0
        a1 = b1_Num / den
        a2 = b2_Num / den

        b = np.array([b0, b1, b2])
        a = np.array([a0, a1, a2])
        b[:3] = b[:3] * gain_linear

    elif CROSSOVER_FILTER_TYPE.Butter12 == filter_type or CROSSOVER_FILTER_TYPE.LR24 == filter_type:
        k = np.tan(theta_c)

        b0 = 1.0 / (1.0 + RK_GLOBAL.SQRT_TWO * k + k * k)
        b1 = -2 * b0
        b2 = b0
        a1 = 2 * b0 * (k * k - 1)
        a2 = b0 * (1 - RK_GLOBAL.SQRT_TWO * k + k * k)

        b = np.array([b0, b1, b2])
        a = np.array([a0, a1, a2])
        if CROSSOVER_FILTER_TYPE.LR24 == filter_type:
            b = np.append(b, [b0, b1, b2])
            a = np.append(a, [a0, a1, a2])
        b[:3] = b[:3] * gain_linear

    elif CROSSOVER_FILTER_TYPE.Bessel12 == filter_type:
        k = np.tan(theta_c)

        b0 = 1.0 / (1.0 + RK_GLOBAL.SQRT_THREE * k + k * k)
```

```
    b1 = -2 * b0
    b2 = b0
    a1 = 2 * b0 * (k * k - 1)
    a2 = b0 * (1 - RK_GLOBAL.SQRT_THREE * k + k * k)

    b = np.array([b0, b1, b2])
    a = np.array([a0, a1, a2])
    b[:3] = b[:3] * gain_linear

elif CROSSOVER_FILTER_TYPE.Butter18 == filter_type or CROSSOVER_FILTER_TYPE.LR36 == filter_type:
    sos = butter(3, fc, btype='high', analog=False, output='sos', fs=Fs)

    b, a = sos_to_ba(sos)

    if CROSSOVER_FILTER_TYPE.LR36 == filter_type:
        b = np.append(b, b)
        a = np.append(a, a)
    b[:3] = b[:3] * gain_linear

elif CROSSOVER_FILTER_TYPE.Bessel18 == filter_type:
    sos = bessel(3, fc, btype='high', analog=False, output='sos', fs=Fs)

    b, a = sos_to_ba(sos)

    b[:3] = b[:3] * gain_linear

elif CROSSOVER_FILTER_TYPE.Butter24 == filter_type or CROSSOVER_FILTER_TYPE.LR48 == filter_type:
    sos = butter(4, fc, btype='high', analog=False, output='sos', fs=Fs)

    b, a = sos_to_ba(sos)

    if CROSSOVER_FILTER_TYPE.LR48 == filter_type:
        b = np.append(b, b)
        a = np.append(a, a)
    b[:3] = b[:3] * gain_linear

elif CROSSOVER_FILTER_TYPE.Bessel24 == filter_type:
    sos = bessel(4, fc, btype='high', analog=False, output='sos', fs=Fs)

    b, a = sos_to_ba(sos)

    b[:3] = b[:3] * gain_linear
elif CROSSOVER_FILTER_TYPE.Butter36 == filter_type:
    sos = butter(6, fc, btype='high', analog=False, output='sos', fs=Fs)

    b, a = sos_to_ba(sos)

    b[:3] = b[:3] * gain_linear
elif CROSSOVER_FILTER_TYPE.Bessel36 == filter_type:
    sos = bessel(6, fc, btype='high', analog=False, output='sos', fs=Fs)

    b, a = sos_to_ba(sos)

    b[:3] = b[:3] * gain_linear
elif CROSSOVER_FILTER_TYPE.Butter48 == filter_type:
    sos = butter(8, fc, btype='high', analog=False, output='sos', fs=Fs)

    b, a = sos_to_ba(sos)

    b[:3] = b[:3] * gain_linear
elif CROSSOVER_FILTER_TYPE.Bessel48 == filter_type:
    sos = bessel(8, fc, btype='high', analog=False, output='sos', fs=Fs)

    b, a = sos_to_ba(sos)
```

```
        b[:3] = b[:3] * gain_linear
    return b, a
```

附录2.7 使用python库函数计算fir滤波器参数

py的scipy的signal函数库里有firwin()和firwin2()两个函数计算fir滤波器的参数

- 其中firwin()的设计方法比较简单，只需要基本的采样频率、截止频率、滤波器类型、滤波器长度就能设计；
- firwin2()比前者更加灵活，不仅可以实现前者的基本功能，还可以设置对应频点的增益；

以下我们使用firwin()函数设计一个10阶（对应滤波器长度为11）、截止频率为10khz的低通滤波器：

```
from scipy.signal  import firwin

    # 输入参数说明：
    # numtaps 为滤波器长度，即滤波器参数个数
    # fs为采样频率，cutoff为截止频率
    # window 为选择对应的窗函数，默认为hamming窗
    # pass_zero可以指定对应的滤波器类型
    # 其他参数可以参考py库对应的源码

    coeffs=firwin(numtaps=11,fs=48000,cutoff=10000,window="hamming",pass_zero="lowpass")
    print("filter coeffs:" ,coeffs)
```

输出结果为：

```
filter coeff: [ 0.00131037 -0.01149941 -0.02967312  0.05396302  0.27879614  0.414206
  0.27879614  0.05396302 -0.02967312 -0.01149941  0.00131037]
```

得到对应的fir滤波器参数之后可以使用4.1 FIR 控件

附录2.8 Ratio增益转换为线性增益

参数中的设定的ratio增益会转换成线性增益，转换代码如下：

```
def ratio_to_linear(ratio):
    return 3 * ratio / 100
```

附录2.9 SlewRate参数转换为平滑系数

参数中的 SlewRate会转换为平滑系数，与采样率和帧长有关，转换代码如下：

```
def slew_alpha_get(slew_rate, fs=48000, block_size=64):
    if slew_rate == 0:
        return 0
    tc = 0.0416 / block_size * math.pow(2, slew_rate - 1) / 1000
    alpha = math.exp(-1 / (tc * fs))
    return alpha
```

附录2.10 与延时和控制时间相关的参数转换

会将实际的时间(ms)，转换为时间的点数， 常用于hold time、look ahead time、delay time等参数的转换：

```
def ms_to_samples(ms, fs=48000):
    return float(round(ms * fs / 1000))
```

附录2.11 与平滑时间系数相关的参数转换

平滑时间系数(ms)，转换为平滑系数，与采样率和帧长有关，常用于rms_tc、attack_time、release_time、decay_time等参数的转换：

```
def smooth_coeff_cal(smooth_time, fs=48000, block_size=64):
    if smooth_time == 0:
        return 1
    else:
        return 1 - math.exp(-1000 * block_size / (smooth_time * fs))
```

附录2.12 与增益变化系数相关的参数转换

db变化系数(dbps、db/sec)，转换为平滑系数，与采样率和帧长有关，常用于RampUp、RmapDown、ThldAttack、ThldRelease等参数的转换：

```
def dbPs_to_linear(dbPs, fs=48000, block_size=64):
    return abs(1 - pow(10, dbPs * block_size / 10 / fs))
```

附录2.13 增益相关的参数转换

将dB值转换为线性值，常用于增益相关的参数的转换：
需要区分是rms还是peak，一般rms相关的动态插件都是用rms，比如compress rms, limit rms等，其他的动态处理插件，以及实际作用到输出信号的增益，都是用的peak，比如mixer、gain中的相关增益。

```
def db_to_linear(db, en_rms=False):
    if db <= -200:
        return 0
    if en_rms:
        return 10**(db/10)
    else:
        return 10**(db/20)
```

附录3 术语说明

附录3.1 biquads：

在语音信号处理中，biquads一般为二阶的IIR滤波器，工程应用上的高阶IIR滤波器很容易变得不稳定，二阶IIR滤波器比较稳定，通常上使用二阶滤波器级联组成高阶滤波器。

iir二阶滤波器（biquads）的参数为5个（b0,b1,b2,a1,a2）(a0固定为1，所以在在参数设置没有写出)

附录3.2 attack_time：

响应时间/启动时间/上升速度等，是指信号达到阈值触发到作用的时间，举个例子，我们设定一个压缩器，门限值为-30dB，设定增益为-6dB，设定attack_time为100ms，则一个信号的幅度缓慢增大，到-30dB阈值时触发压缩器，压缩器开始工作，但是增益不会马上变化为-6dB，而是在100ms内从0dB缓慢变化到-6dB，此时达到工作点。从触发阈值到达到工作点的这段时间我们称为attack_time。也叫响应时间，启动时间等。

附录3.3 release_time：

释放时间/衰减时间/下降速度等，一般和attack_time成对出现。是指信号从工作点到释放点的时间。举个例子，设定release_time为80ms，回到上方attck_time设定的情景，当信号触发阈值慢慢到达工作点之后，信号又逐渐下降到门限值（-30dB）以下，此时增益不是立马从-6dB恢复到0dB，而是在80ms内缓慢从-6dB变化到0dB，此时达到释放点。我们称从脱离阈值作用范围到完全释放点的时间为release_time，也叫释放时间，衰减时间等

附录3.4 品质因数Q：

数字滤波器的品质因数Q，描述了滤波器在中心频率附近的带宽与中心频率之间的关系。具体来说，Q值越高，滤波器的带宽越窄，频率选择性越好；Q值越低，带宽越宽，频率选择性越差。

以下以峰值滤波器为例

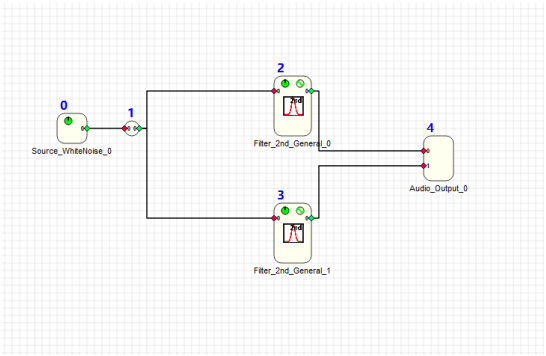


图 附录3.4-1 控件接线图

上方为标号为2的滤波器设置中心频率为4000hz，Q为15的peaking滤波器； 下方标号为3的滤波器设置中心频率为4000hz，Q为1.41的peaking滤波器；

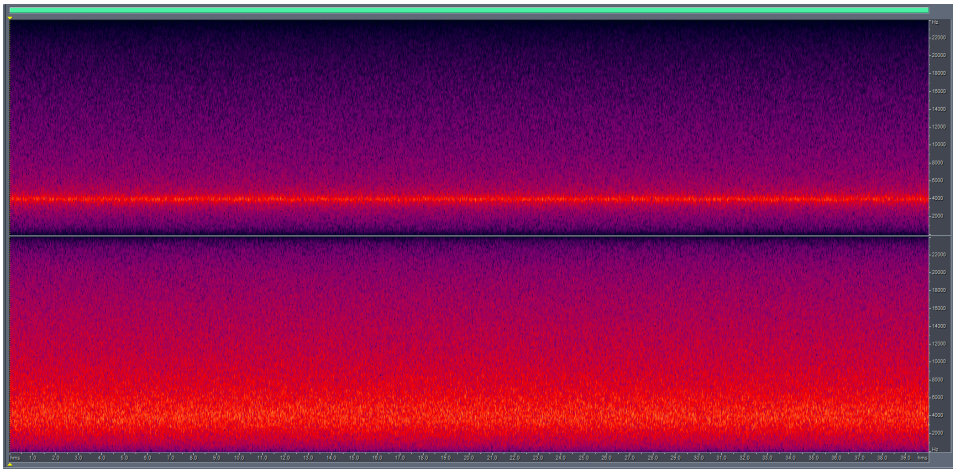


图 附录3.4-2 输出频域图

由输出结果可见，Q值越大，作用越集中，频段带宽越窄。

附录3.5 Boost（滤波器增益） 与 Gain（输出整体增益）：

首先先解释Gain参数，Gain为输出整体增益，所有滤波器可设置gian参数，对全频段都一致有效，以下我们以Paramtric参数滤波器为例，该滤波器计算函数代码为：

```
def calculate_param_coeffs(fc: float, boost: float, Q: float, gain=0, Fs=48000):
    w0 = 2 * np.pi * fc / Fs
    A = 10 ** (boost / 40.0)
    alpha = np.sin(w0) / (2.0 * Q)

    a0 = 1 + alpha / A
    a1 = -2 * np.cos(w0)
    a2 = 1 - alpha / A
    b0 = 1 + alpha * A
    b1 = -2 * np.cos(w0)
    b2 = 1 - alpha * A
    a0_invert = 1 / a0

# 由下方代码可看出，gain (dB)增益转换为 线性 gain_linear 增益后，
# 直接乘以滤波器的分子系数得到新的系数输出
    gain_linear = 10 ** (gain/20)
    return [b0*a0_invert*gain_linear, b1*a0_invert*gain_linear, b2*a0_invert*gain_linear], [1.0, a1*a0_invert, a2*a0_invert]
```

以下为列出parametric filter 二阶参数滤波器的幅频响应图，直观体验gain增益的作用，对整体的曲线上下平移(蓝线对应Gain=0dB，黄线对应Gain=1dB)：

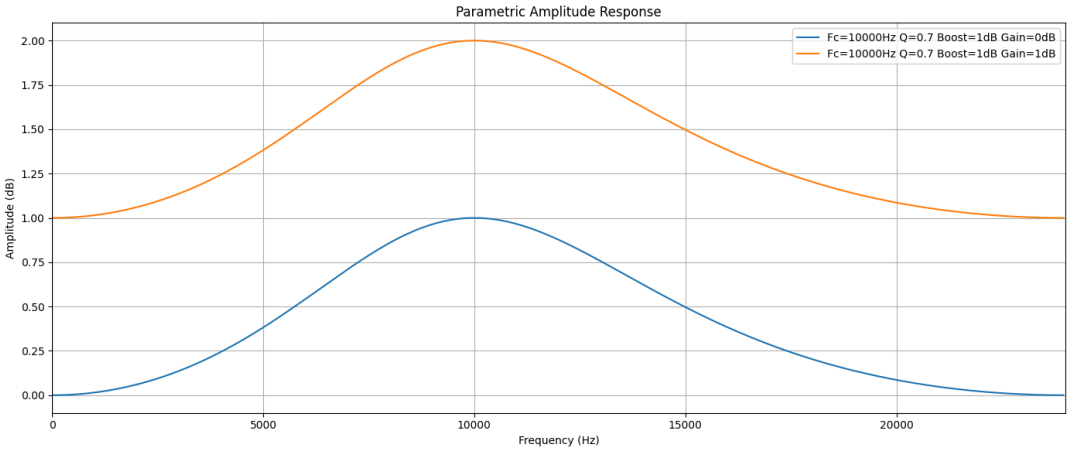


图 附录3.5-1 可以看到，Gain参数对滤波器的频响曲线有整体上下平移的作用

Boost为滤波器增益，仅有以下以下几个滤波器有这个参数，其他滤波器都没有Boost参数。

Type	说明
Parametric	参数滤波器
LowShelf	低架滤波器
HighShelf	高架滤波器

对 Parametric 参数滤波器 Boost增益决定的是中心频率Fc的增益如上图的蓝色曲线，设定的Fc为10khz Boost为1dB，对应的幅频响应曲线在10khz的幅值增益为1dB。

而对于 LowShelf (低架滤波器)和HighShelf（高架滤波器）Boost设定为增益的最大值，在Fc的增益为1/2 boost（dB）

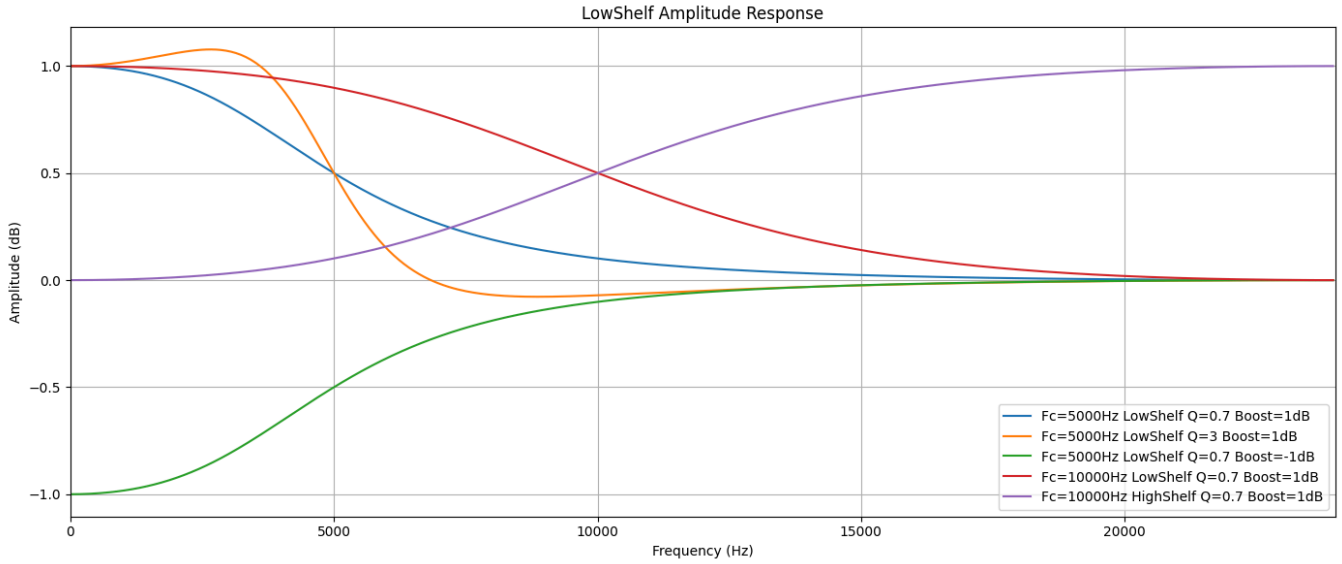


图 附录3.5-2 LowShelf和HighShelf滤波器理论频响曲线

附录3. 6 day & wet 干声和湿声：

在数字音频处理中，把未经处理的、原始的音频信号称为干声（dry）；而把经过音频处理系统（如混响，延时，调制等）处理的声音称为湿声（wet）。

附录3.7 响度（Loudness）和声音强度（Sound intensity）

响度和声音强度都反映声音的大小，但两者却不是完全相同。

- 声音强度：客观的、物理的衡量声音的大小；表示单位时间内通过单位面积的声能。它通常以 瓦特每平方米（W/m²） 为单位
- 响度：人主观感知的声音大小；表示声音听起来有多“响”。它通常以 方（Phon） 或 宋（Sone） 为单位。

人的耳朵对不同频率的声音的敏感度不同，即要让人感知到相同的声音大小（响度），不同频率的声音强度不一样，如图附录3.7-1所示，横坐标为声音频率，纵坐标为声音强度，图中红色和蓝色曲线为达到设定响度时不同频率声音需要多大的声音强度；

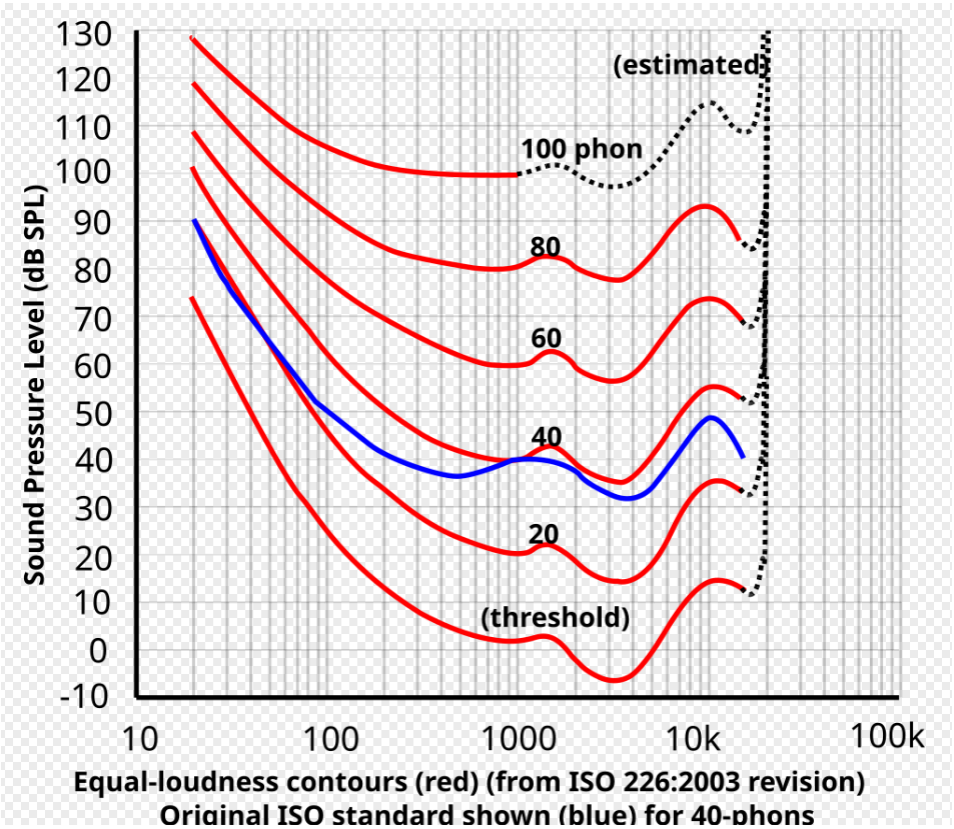


图 附录3.7-1 不同频率的声音强度和响度的对应曲线

由图可知，人的耳朵对3KHZ频率左右的信号最为敏感（图中曲线凹陷处），该频率的声音只需要较低的声音强度就能达到设定的响度效果，而其他频率需要更高的声音强度才能达到相同的响度效果；